Architecture Diagram Collection

From first principles to expert practice

Built from the manuscript sources · 0 chapter(s) in this edition

Appendix D — Architecture Diagram Collection

Chapter 1 — What Is System Design?
Chapter 2 — How to Think Like a System Designer
Chapter 3 — The System Design Learning Method
Chapter 4 — Quantitative Foundations for System Design
Chapter 5 — Computer Architecture
Chapter 6 — Operating Systems
Chapter 7 — Networking Fundamentals
Chapter 8 — Web Application Fundamentals
Chapter 9 — Real-Time Communication
Chapter 10 — Security Fundamentals for System Designers
Chapter 11 — Relational Databases
Chapter 12 — NoSQL Databases
Chapter 13 — Storage Engine Internals
Chapter 14 — Database Scaling: Replication, Partitioning and Sharding
Chapter 15 — Object, Block and File Storage
Chapter 16 — Search Systems
Chapter 17 — Data Structures and Algorithms for Scale
Chapter 18 — Scalability
Chapter 19 — Load Balancing
Chapter 20 — Caching
Chapter 21 — Content Delivery Networks
Chapter 22 — API and Traffic Management
Chapter 23 — Performance Engineering
Chapter 24 — Distributed Systems Fundamentals
Chapter 25 — Consistency and Availability
Chapter 26 — Consensus and Coordination
Chapter 27 — Replication and Fault Tolerance
Chapter 28 — Distributed Transactions
Chapter 29 — Distributed Data Processing

Appendix D — Architecture Diagram Collection

> Generated by `build/build.py` from every Mermaid diagram in the manuscript. Sources are in `diagrams/chNN/*.mmd`. Each figure links back to the chapter that explains it.

Chapter 1 — What Is System Design?

Figure 1.1 — PhotoShare v0: a single-server design.

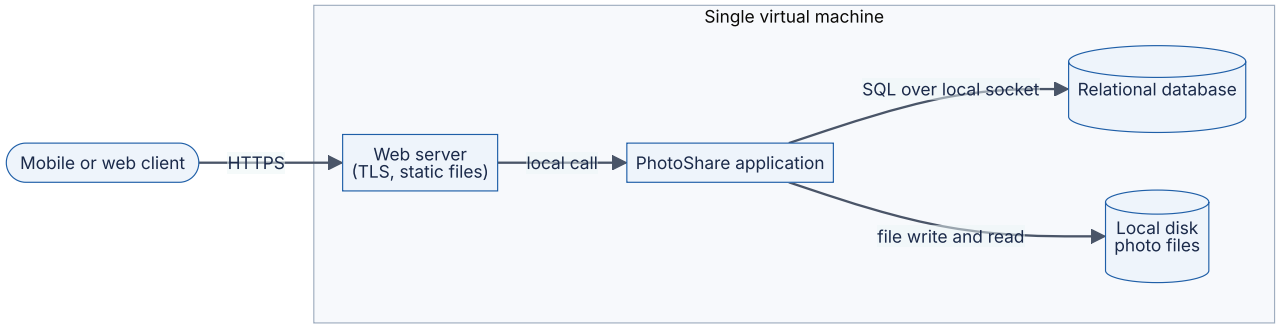


Figure 1.2 — High-level design of the PhotoShare upload path.

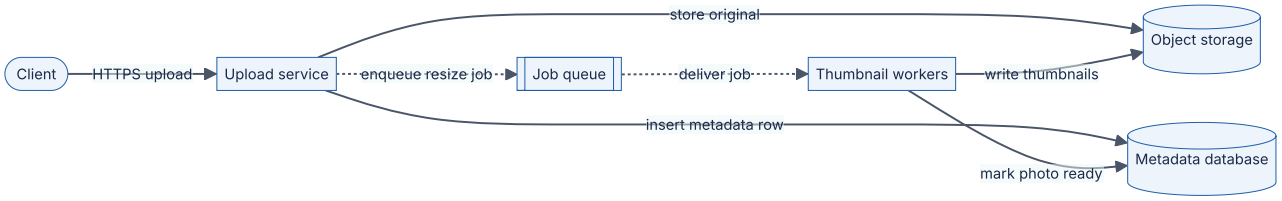


Figure 1.3 — Low-level design of the upload component (class view).

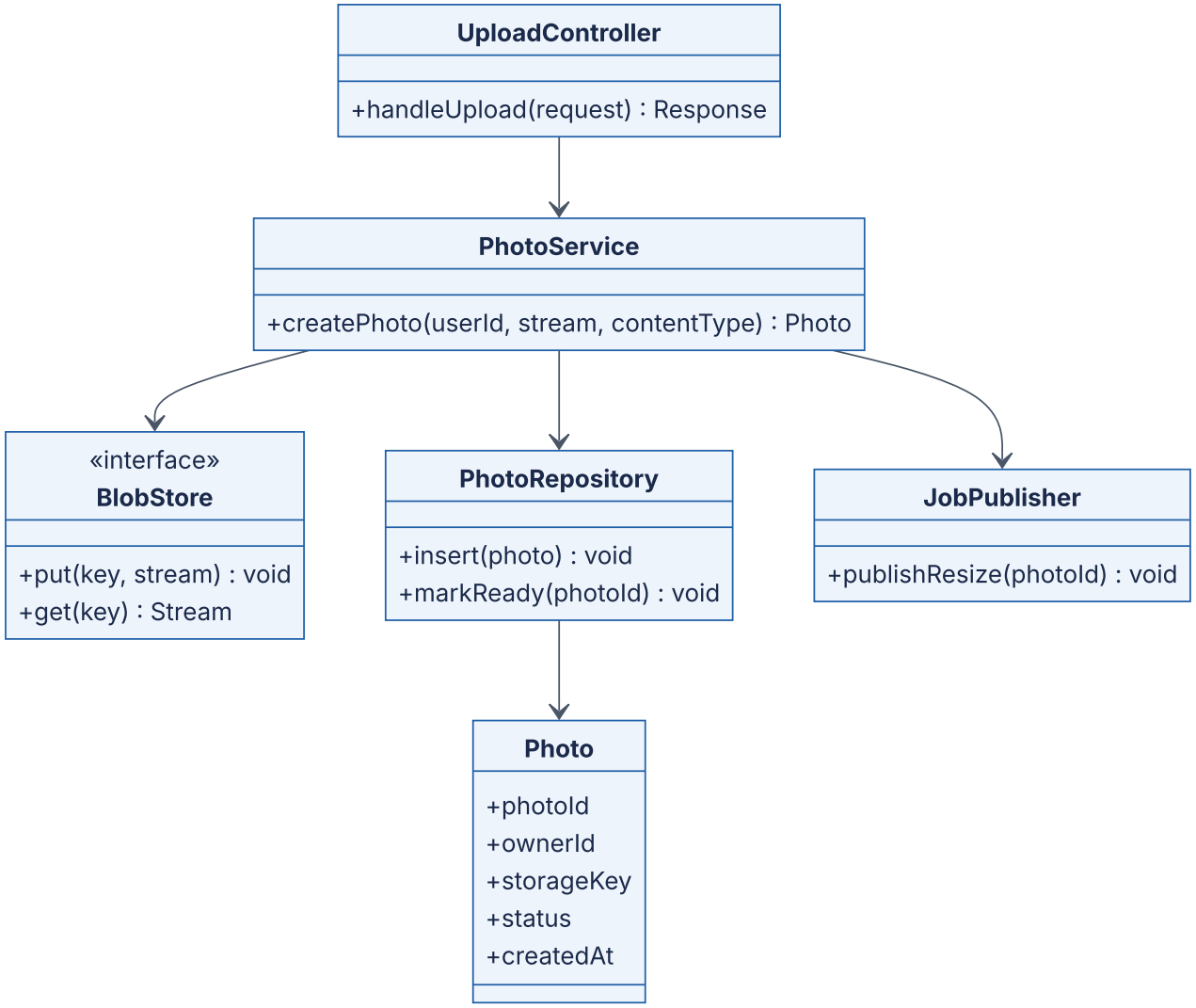


Figure 1.4 — ADR lifecycle.

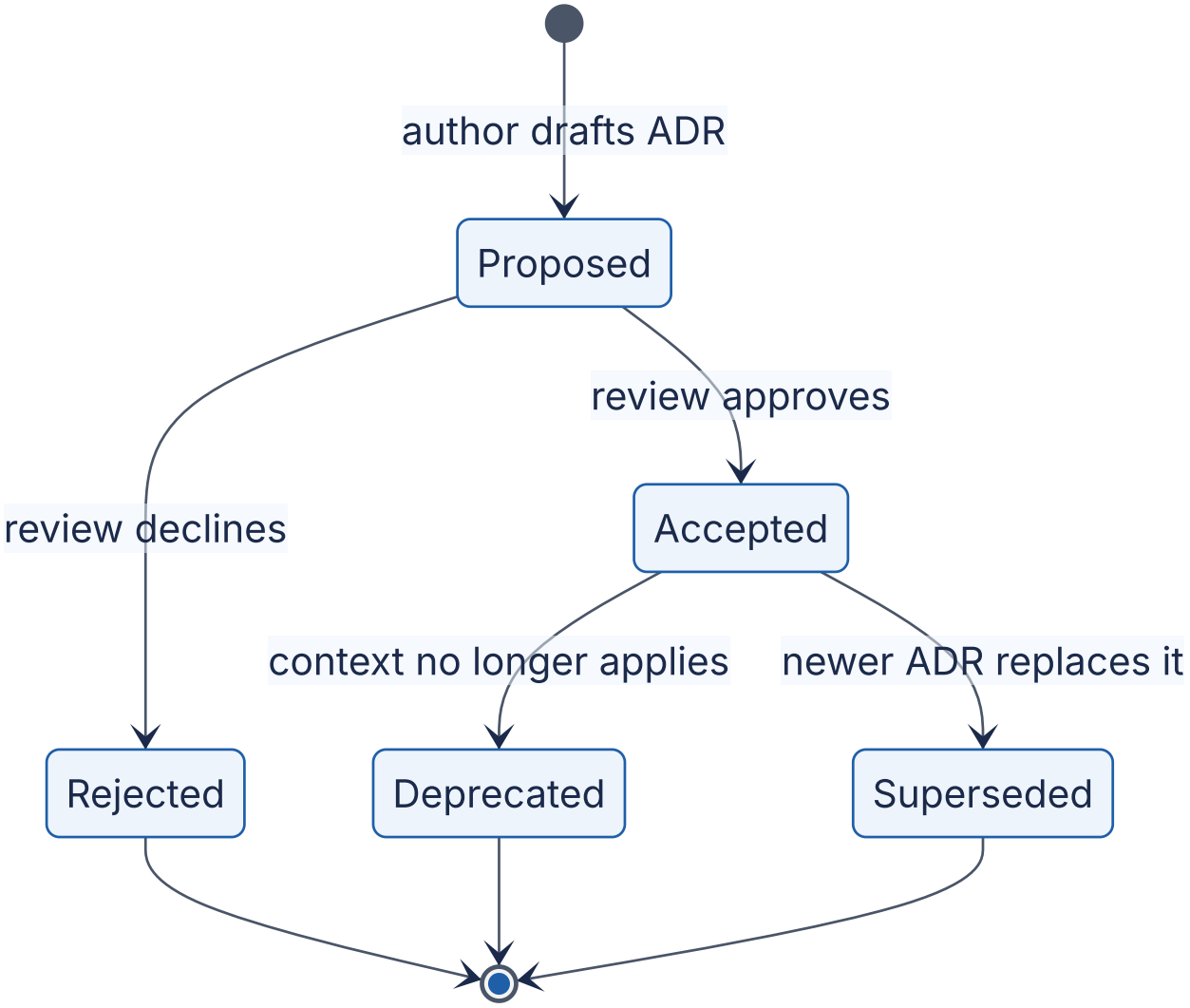


Figure 1.5 — PhotoShare v1: stateless application tier and managed storage.

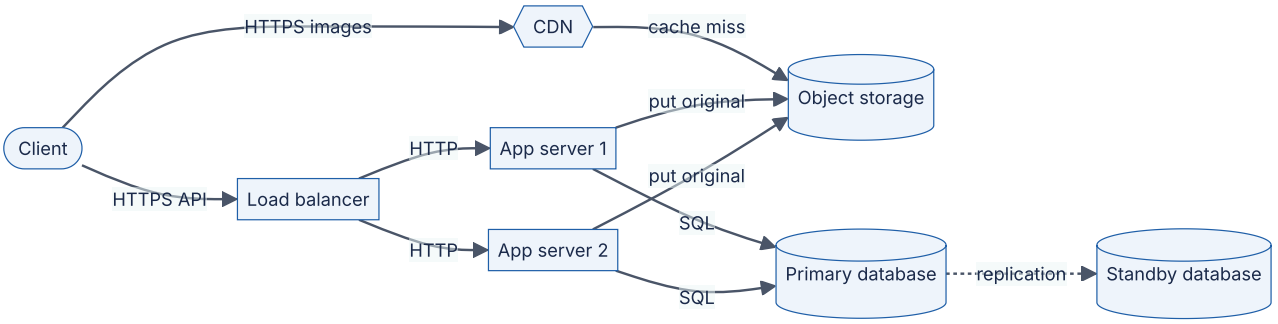


Figure 1.6 — PhotoShare v2: cache, read replicas and asynchronous processing.

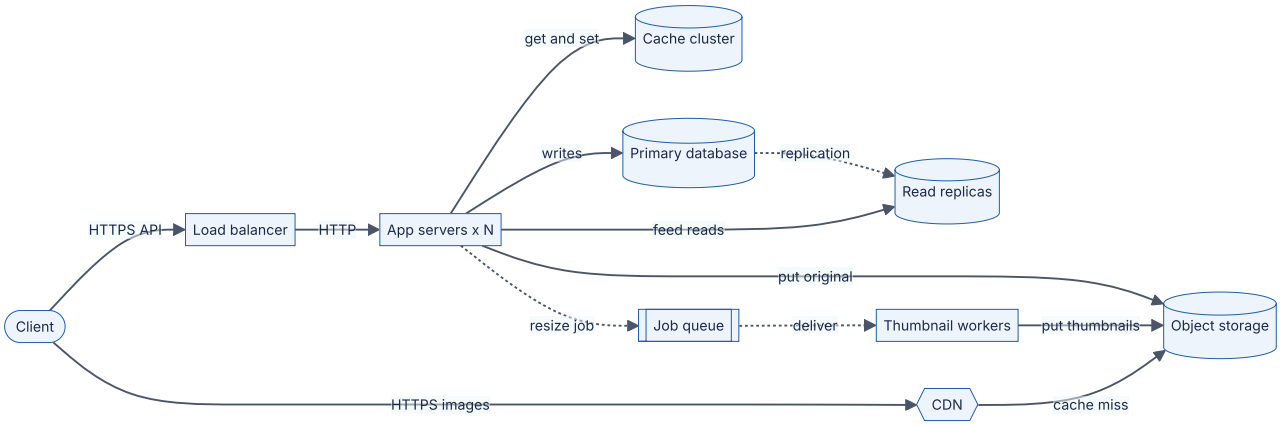


Figure 1.7 — PhotoShare v3: services, partitioned data and multi-zone deployment.

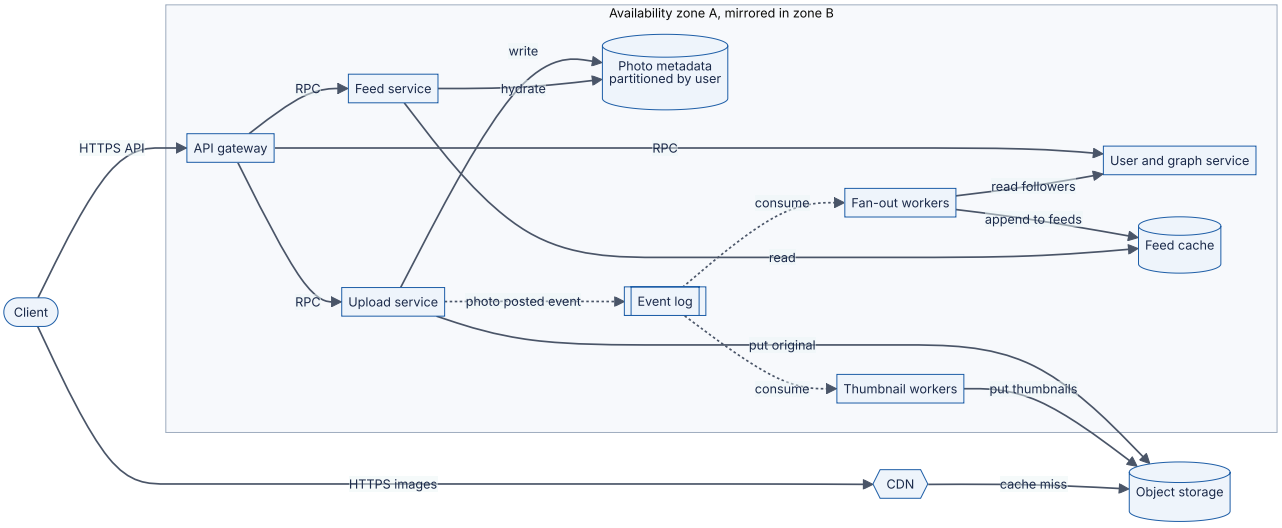


Figure 1.8 — The life of a design in a typical engineering organization.

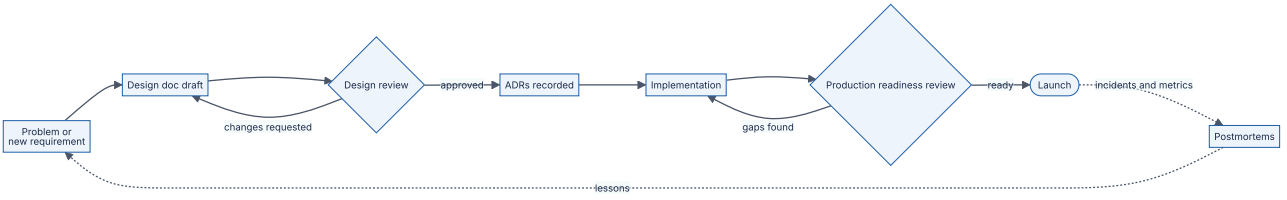
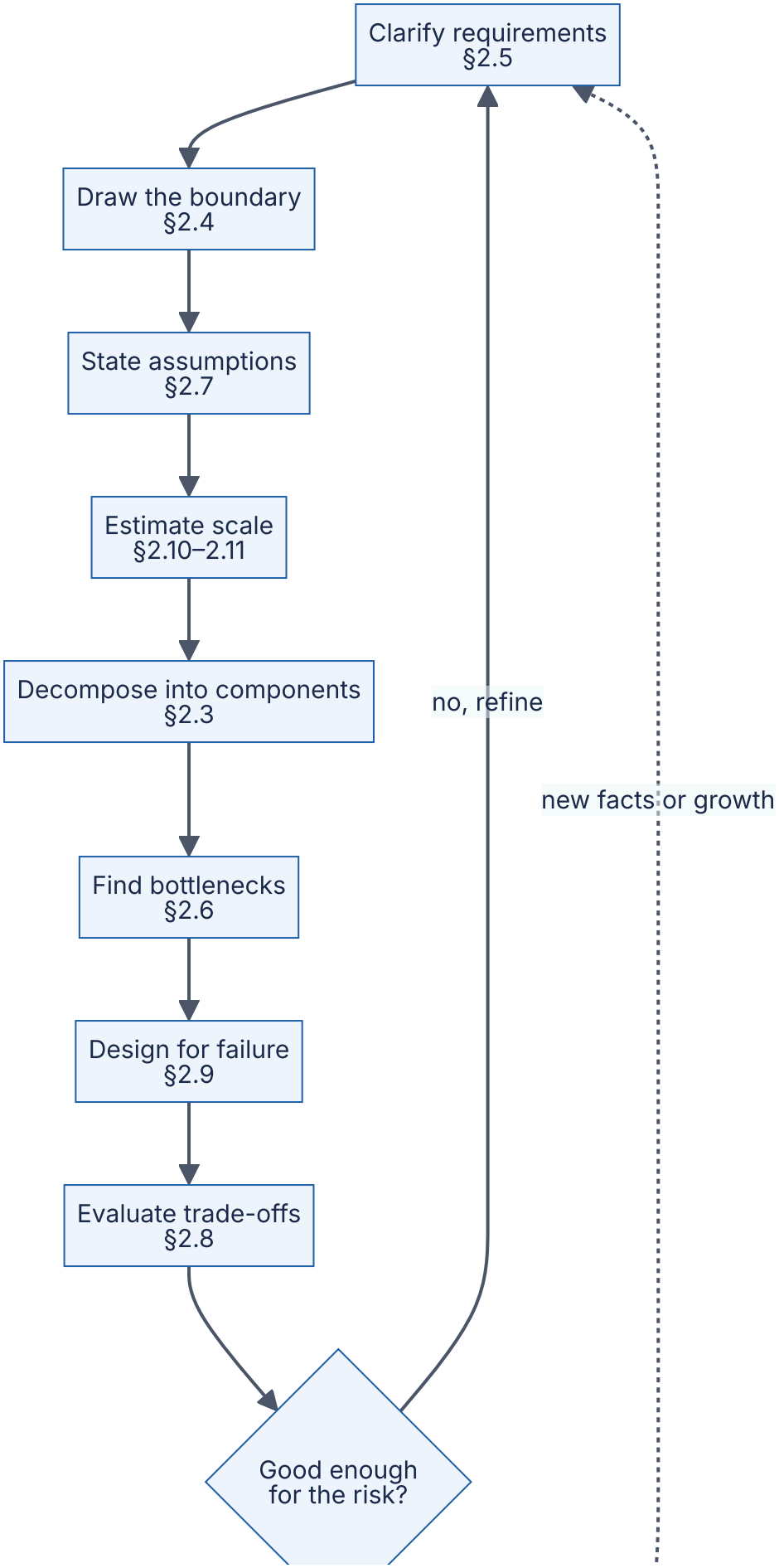


Figure 1.9 — A typical 45-minute system design interview.



Chapter 2 — How to Think Like a System Designer

Figure 2.1 — The system designer’s thinking loop.


Clarify requirements §2.5
Draw the boundary §2.4
State assumptions §2.7
Estimate scale §2.10–2.11
Decompose into components §2.3
Find bottlenecks §2.6
Design for failure §2.9
Evaluate trade-offs §2.8
Good enough for the risk?
no, refine
new facts or growth

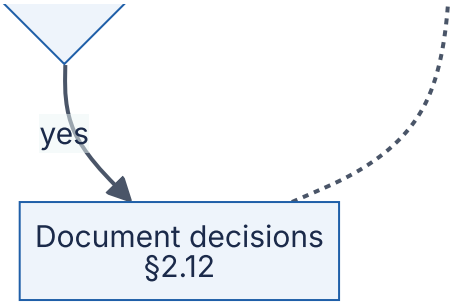


Figure 2.2 — QuickBite first-pass decomposition.

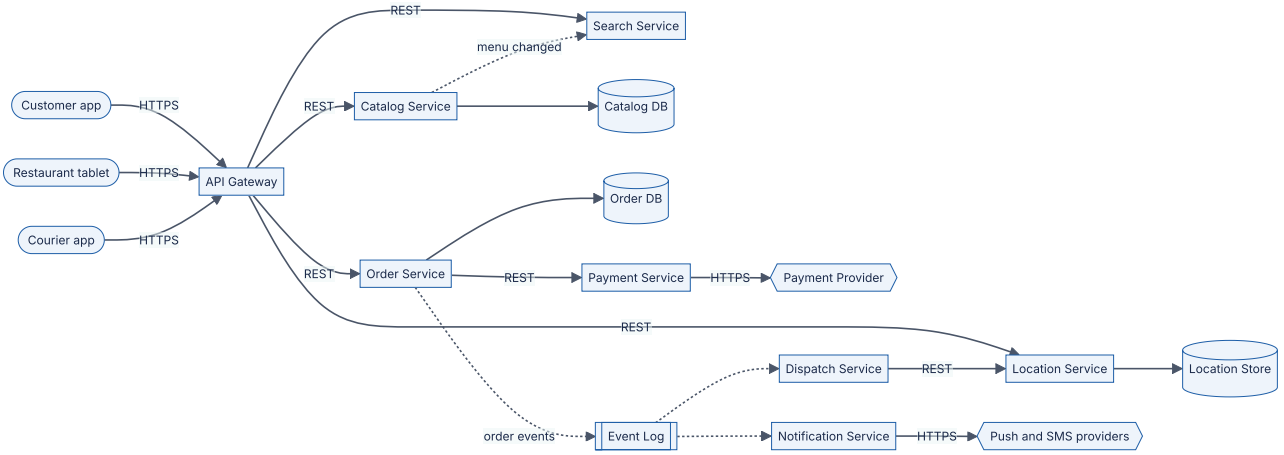


Figure 2.3 — QuickBite system-context diagram.

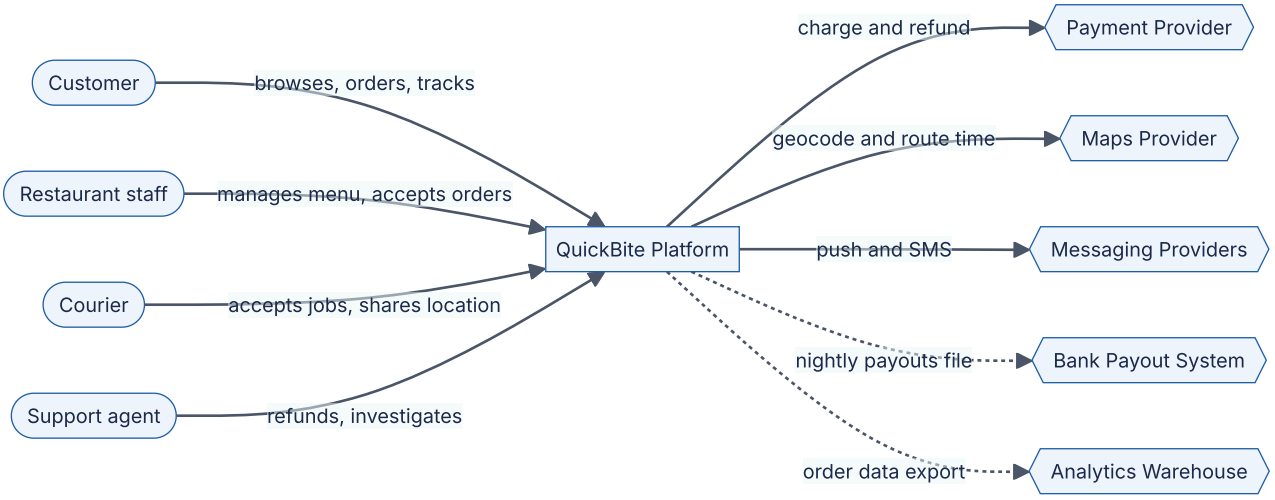


Figure 2.4 — Finding the bottleneck with the USE method.

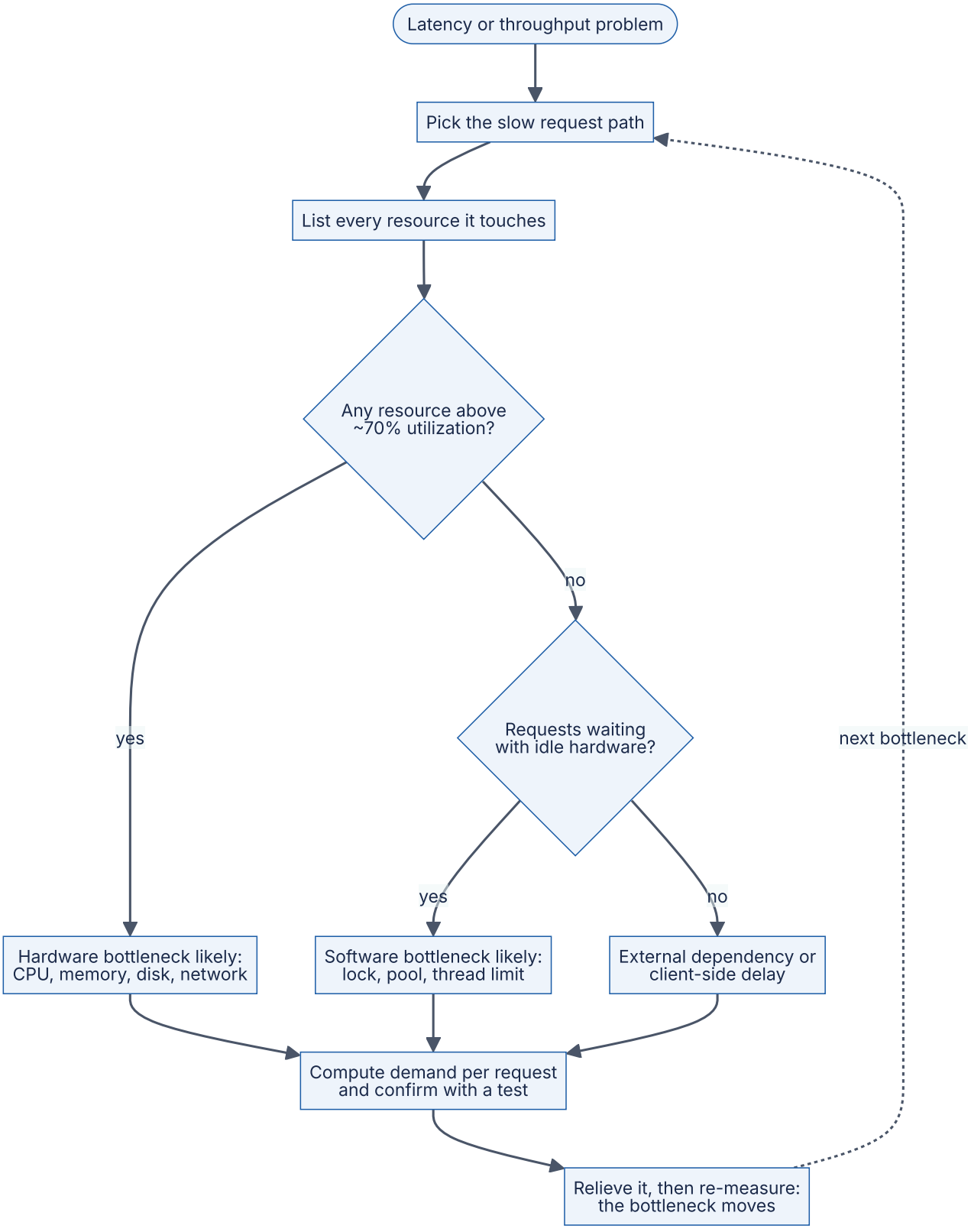


Figure 2.5 — Choosing the decision process by reversibility.

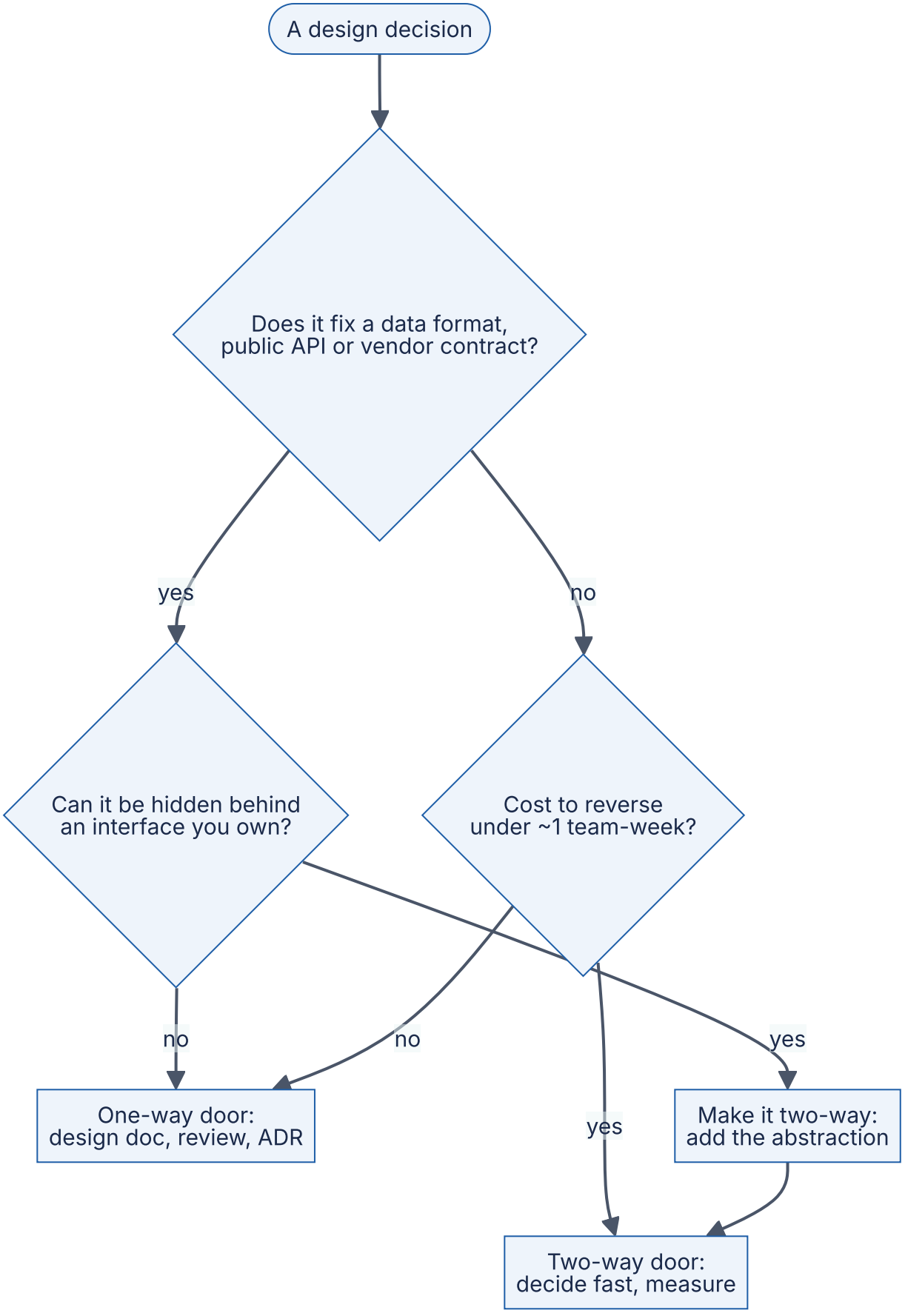


Figure 2.6 — QuickBite checkout designed for failure.

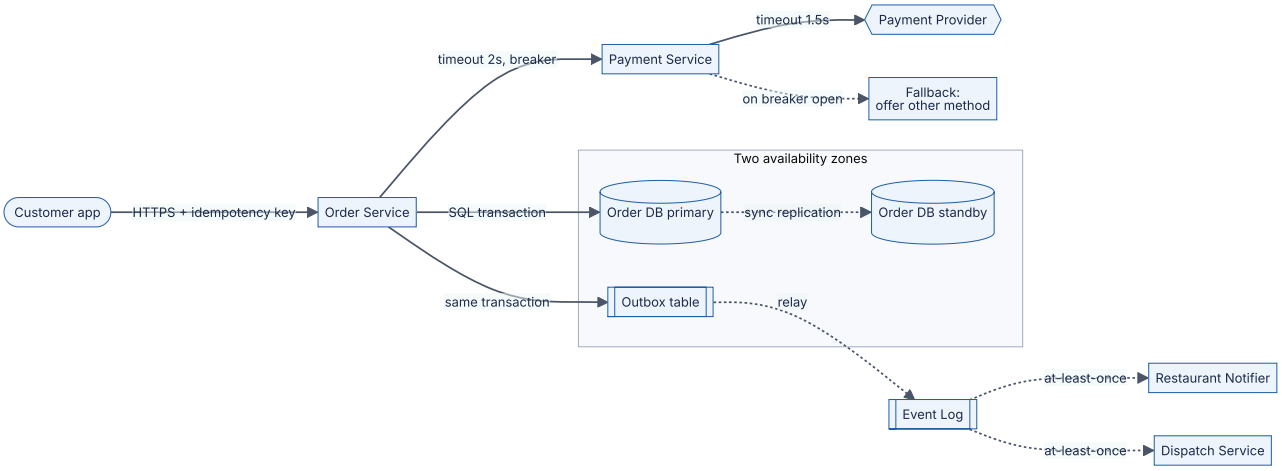


Figure 2.7 — QuickBite C4 container view, after the §2.8 decision.

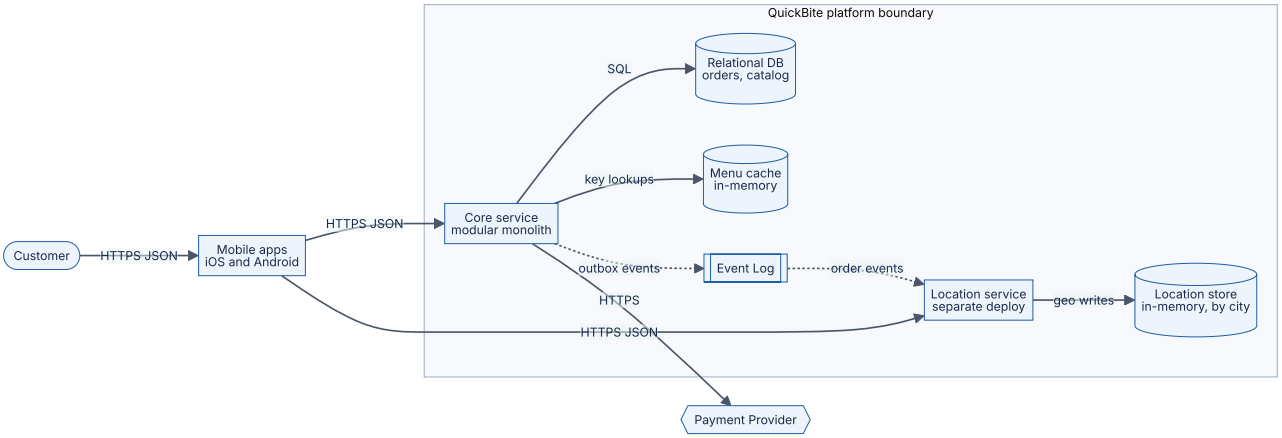
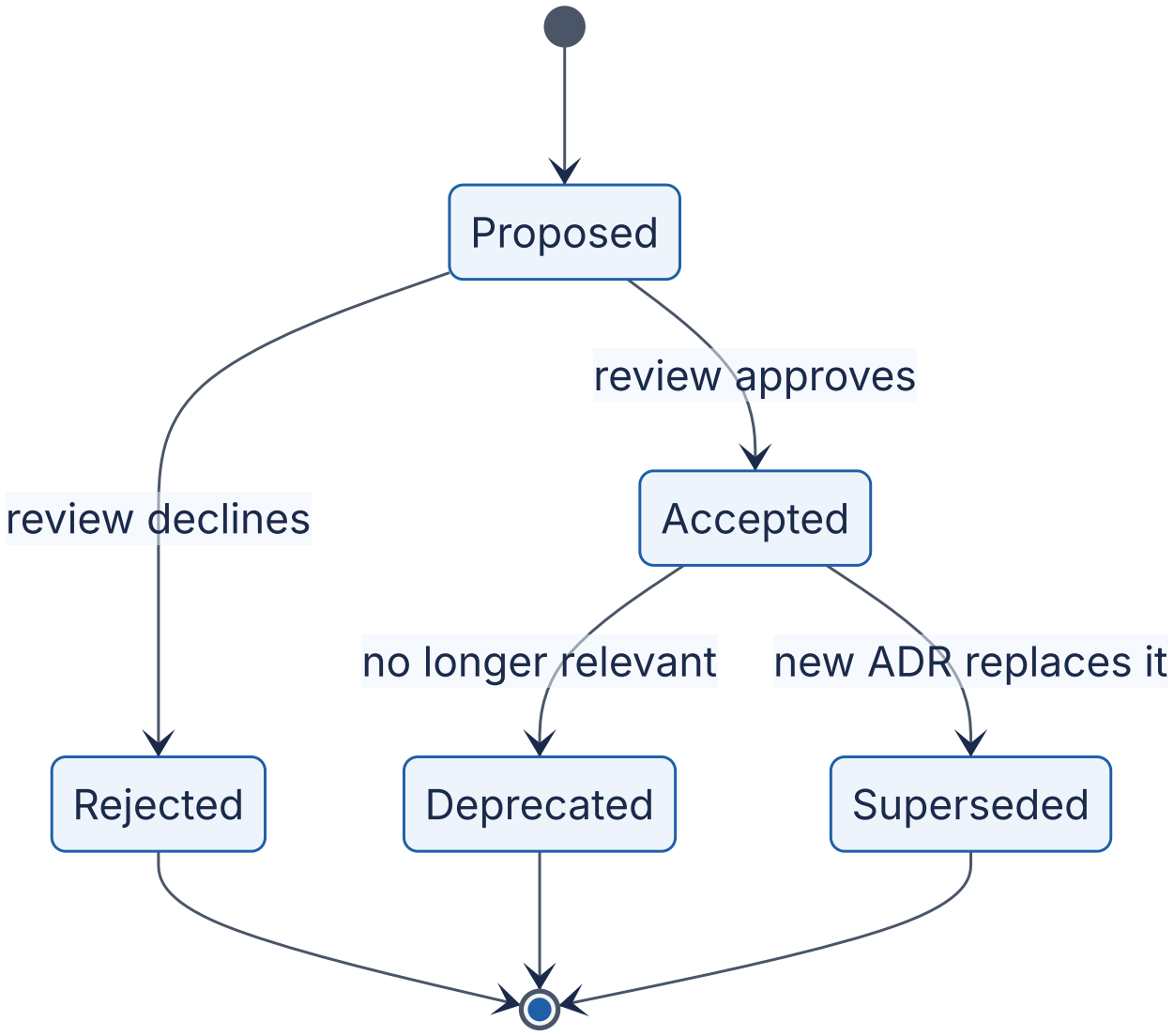
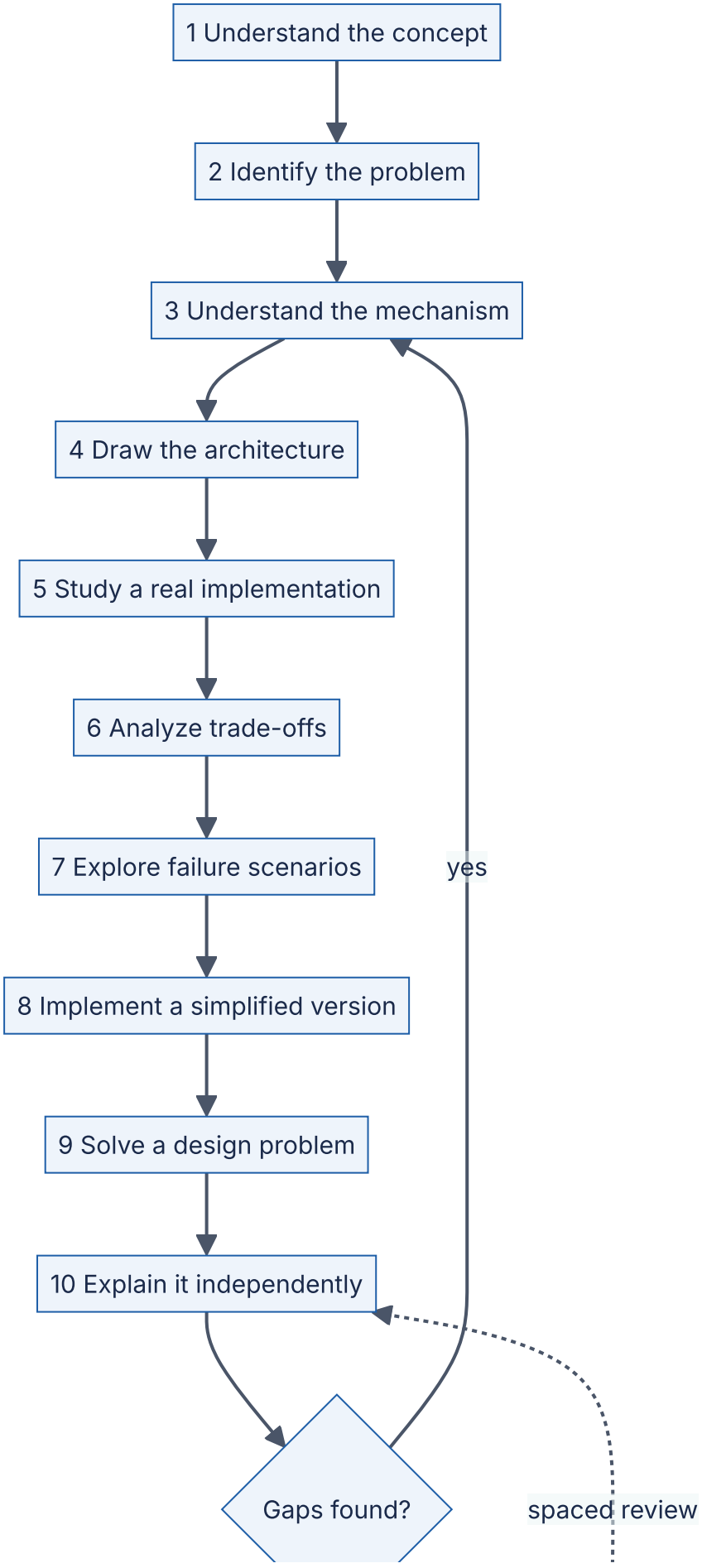


Figure 2.8 — ADR life cycle.



Chapter 3 — The System Design Learning Method

Figure 3.1 — The ten-step concept-mastery loop.


1 Understand the concept
2 Identify the problem
3 Understand the mechanism
4 Draw the architecture
5 Study a real implementation
6 Analyze trade-offs
7 Explore failure scenarios
8 Implement a simplified version
9 Solve a design problem
10 Explain it independently
Gaps found?
yes
spaced review

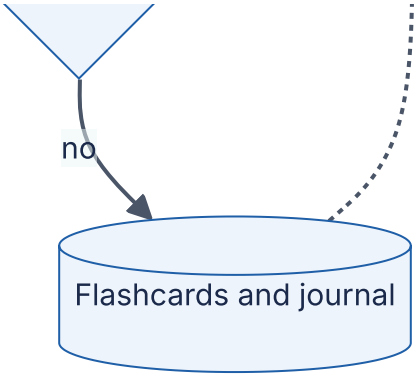


Figure 3.2 — Consistent hashing in a client-routed distributed cache.

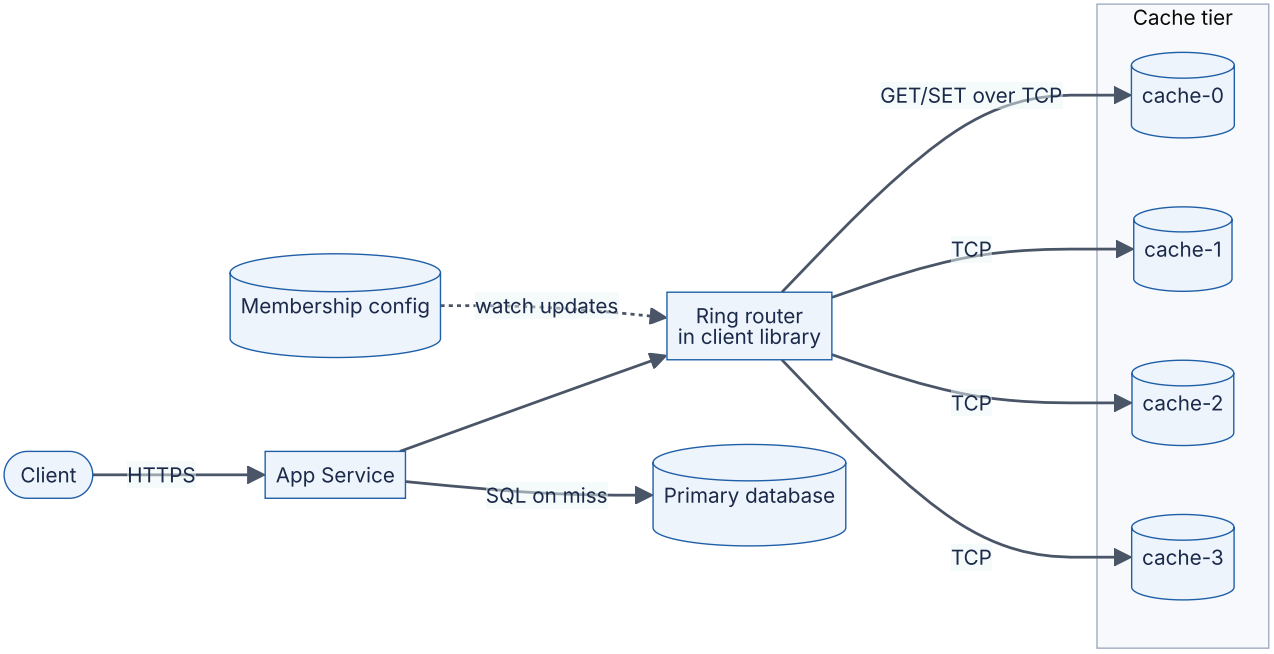


Figure 3.3 — Session read when the primary owner has failed.

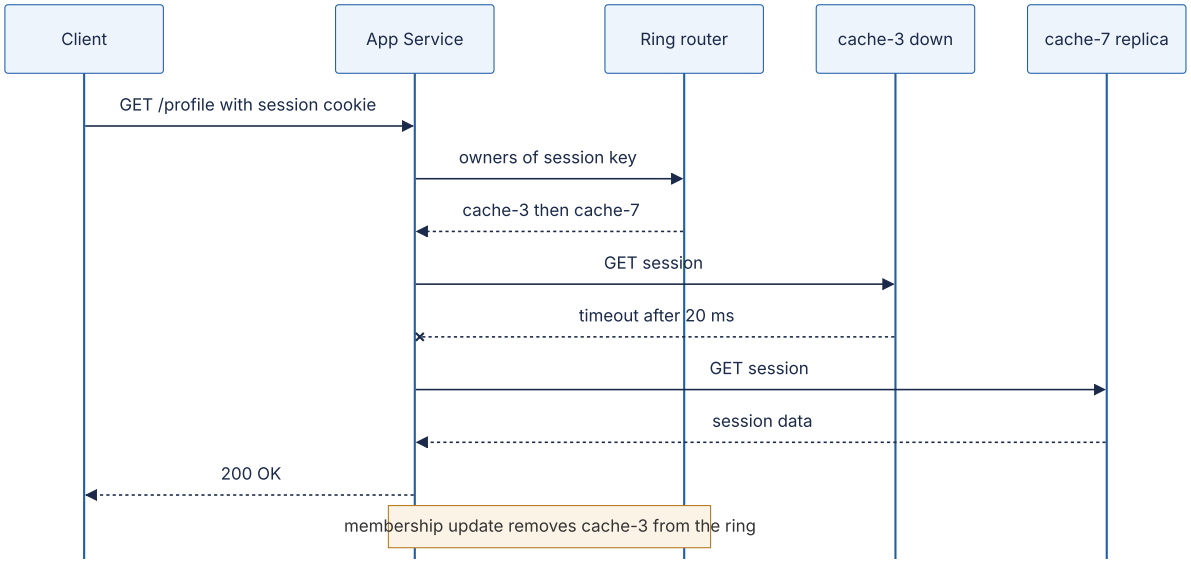


Figure 3.4 — A five-box Leitner schedule for flashcards.

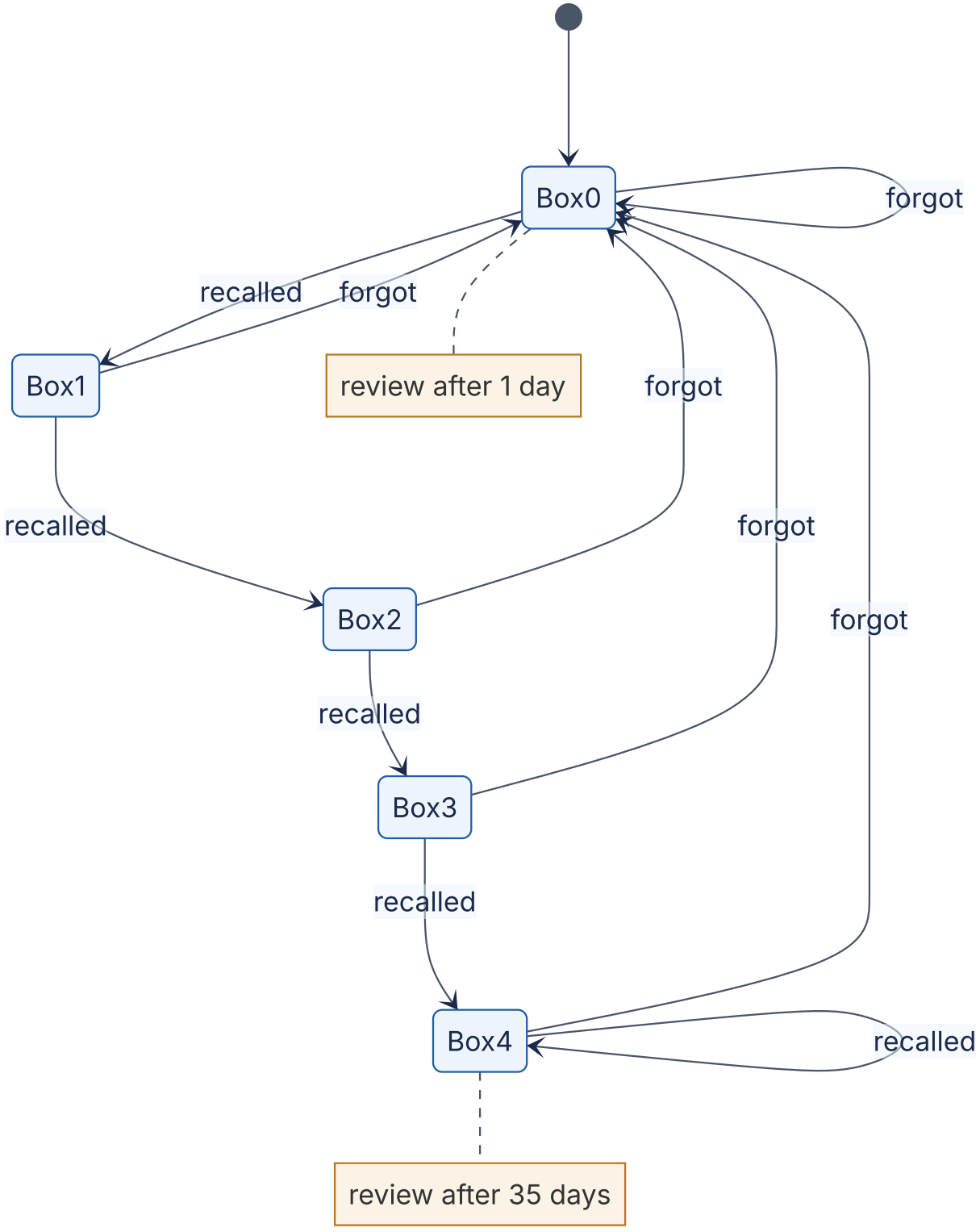
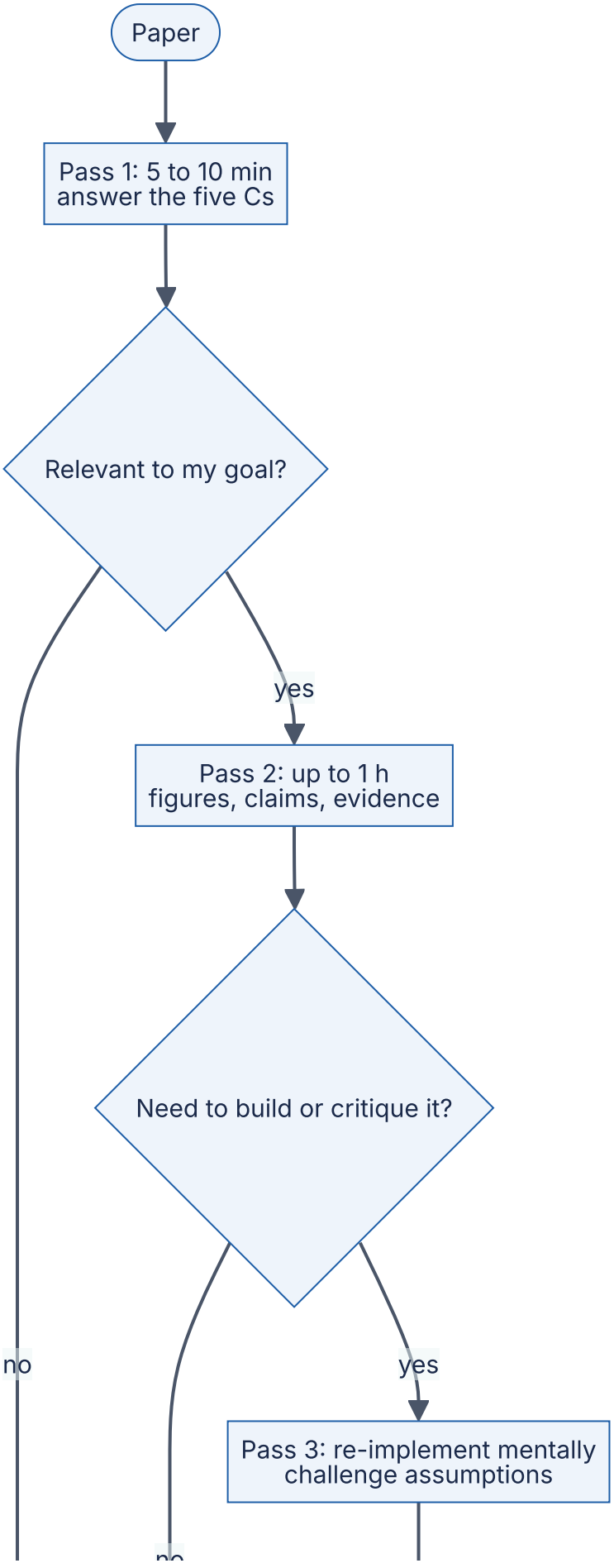


Figure 3.5 — The three-pass reading decision flow.


Paper
Pass 1: 5 to 10 min
answer the five Cs
Relevant to my goal?
yes
Pass 2: up to 1 h
figures, claims, evidence
Need to build or critique it?
no
yes
Pass 3: re-implement mentally
challenge assumptions

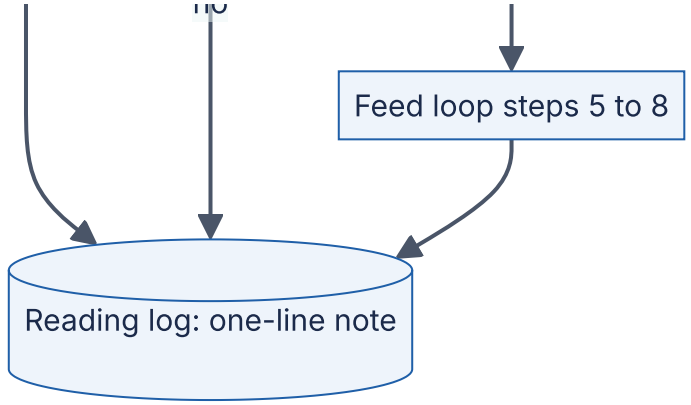


Figure 3.6 — The 12-month beginner path (months shown as M01–M12).

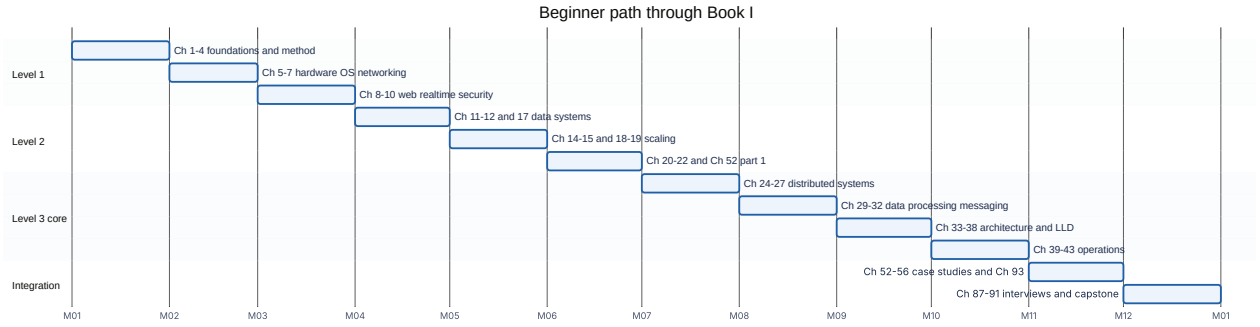


Figure 3.7 — The postmortem study loop.



Chapter 4 — Quantitative Foundations for System Design

Figure 4.1 — B-tree fan-out turns a billion keys into four levels.

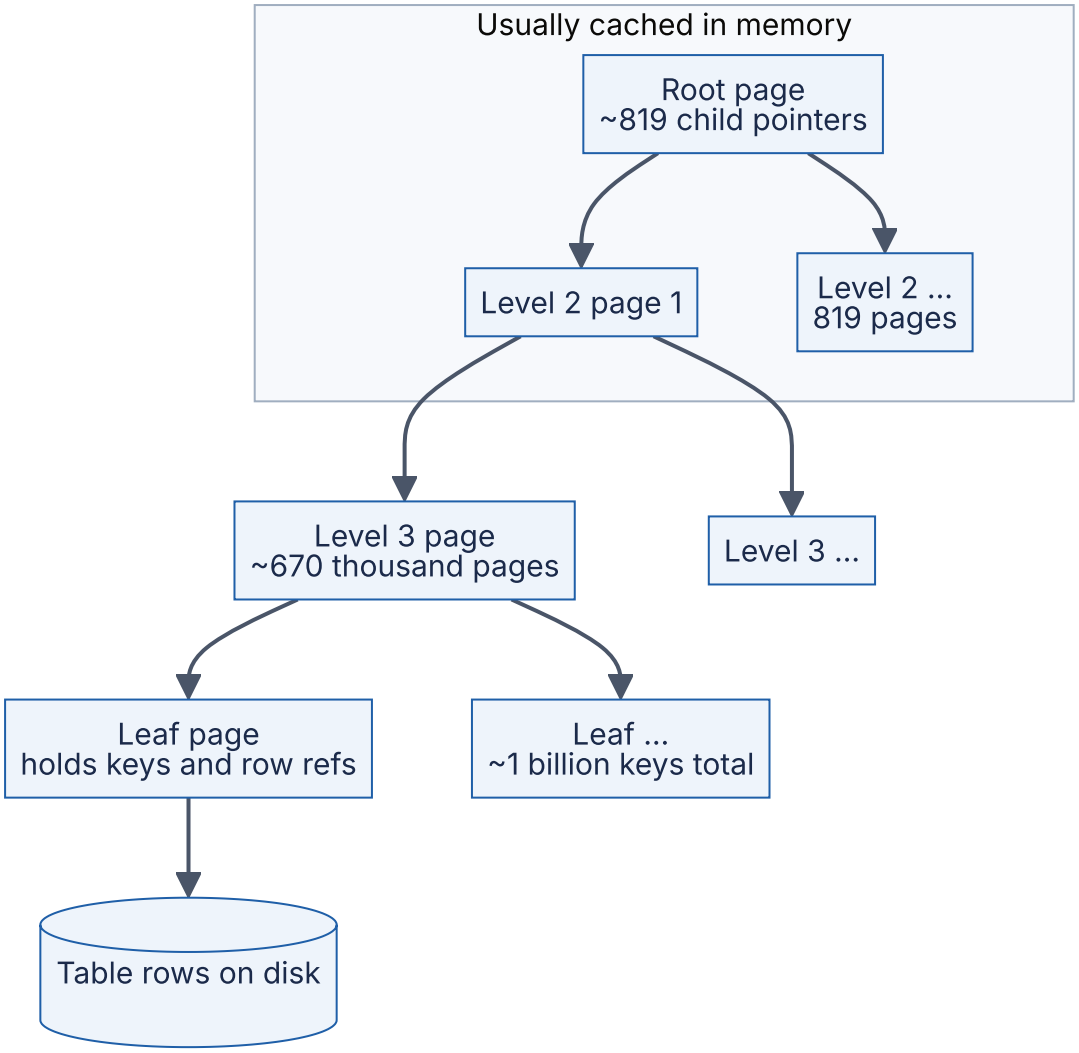


Figure 4.2 — Fan-out: the slowest leaf determines the response time.

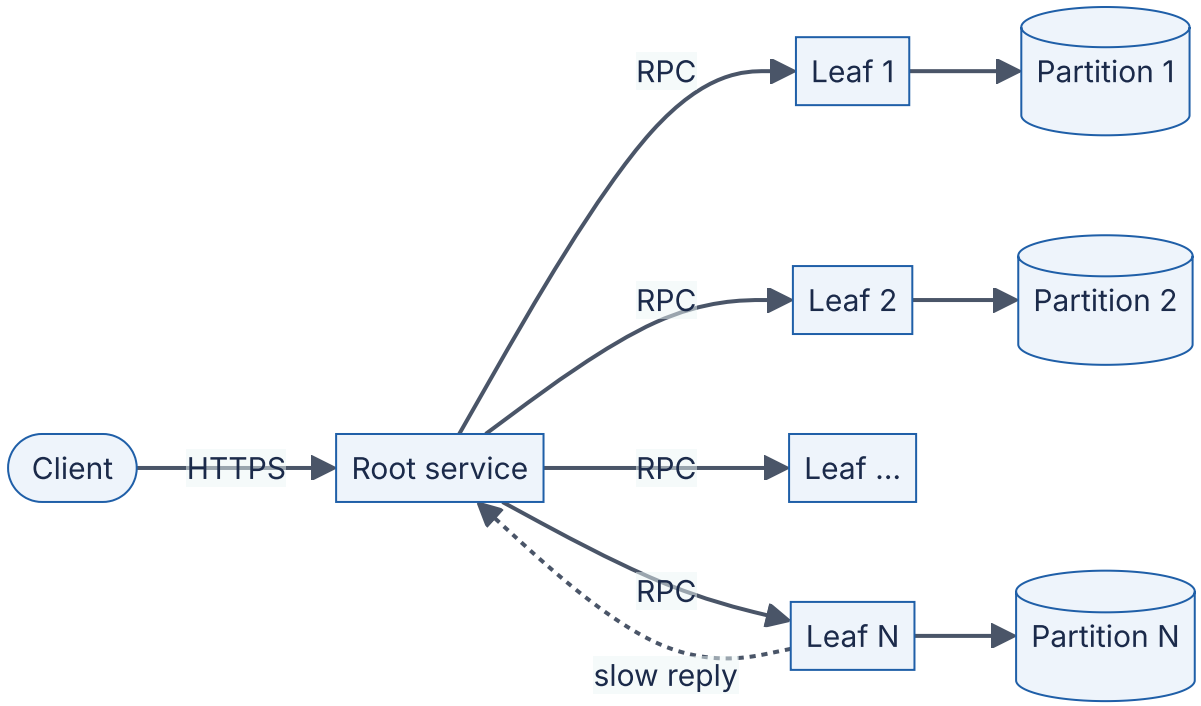
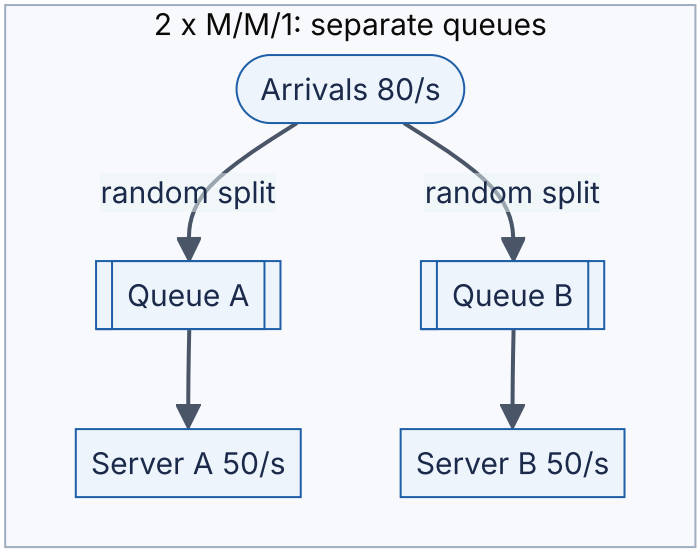


Figure 4.3 — The M/M/1 queue.



Figure 4.4 — One shared queue versus separate queues.



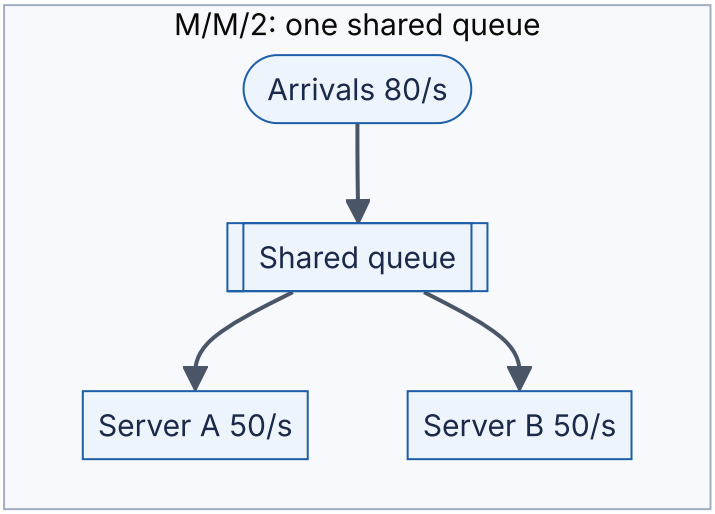


Figure 4.5 — Choosing the boundary for Little’s Law.

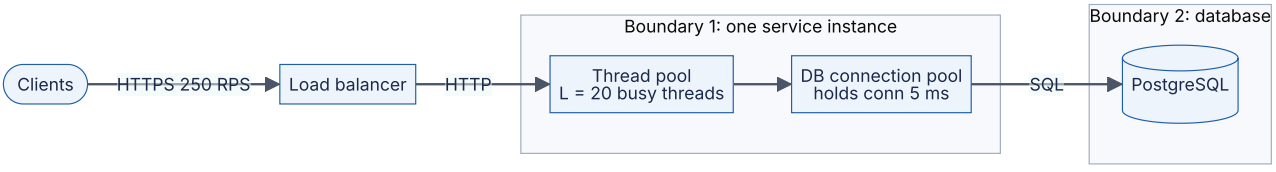


Figure 4.6 — Series, parallel and k-of-n availability.

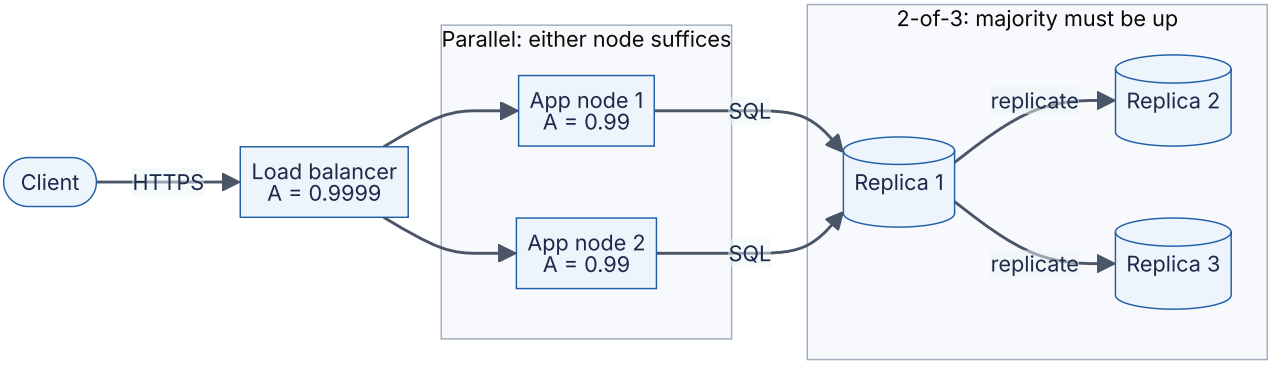


Figure 4.7 — The up-down cycle of a repairable component.

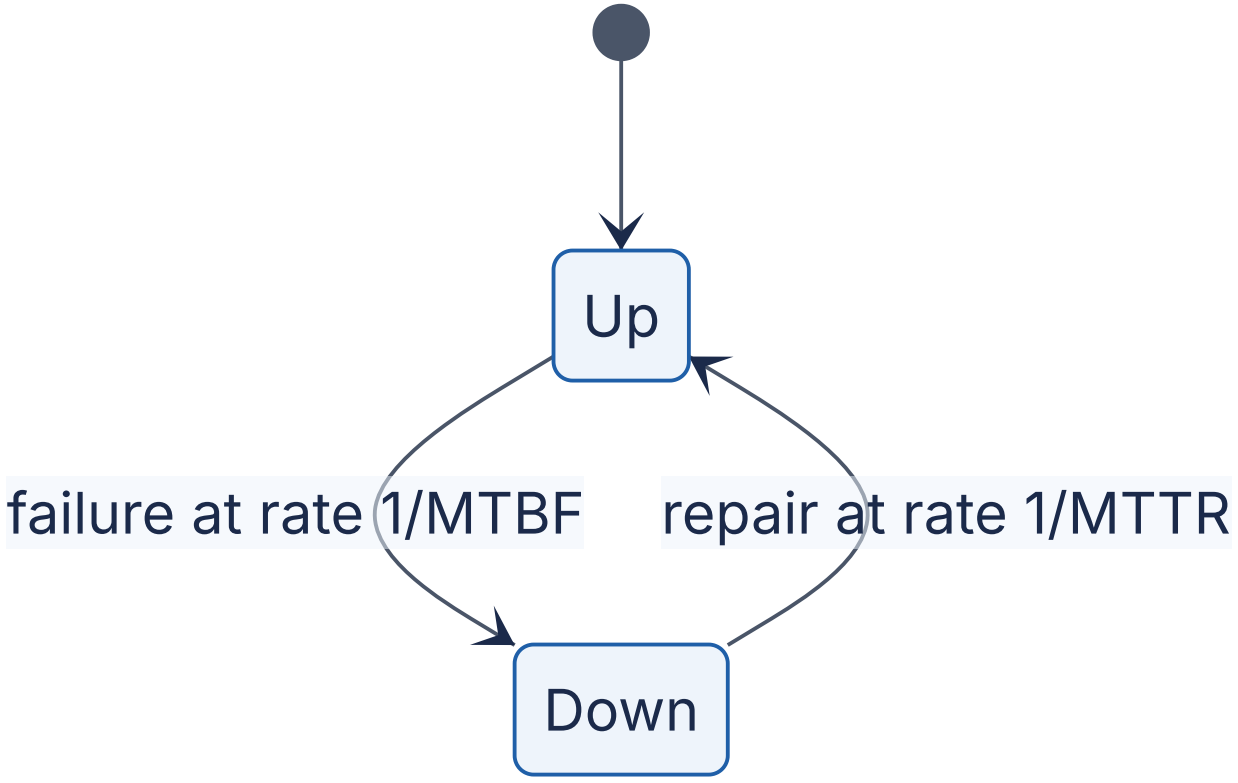


Figure 4.8 — The replica-loss race for three-way replication.

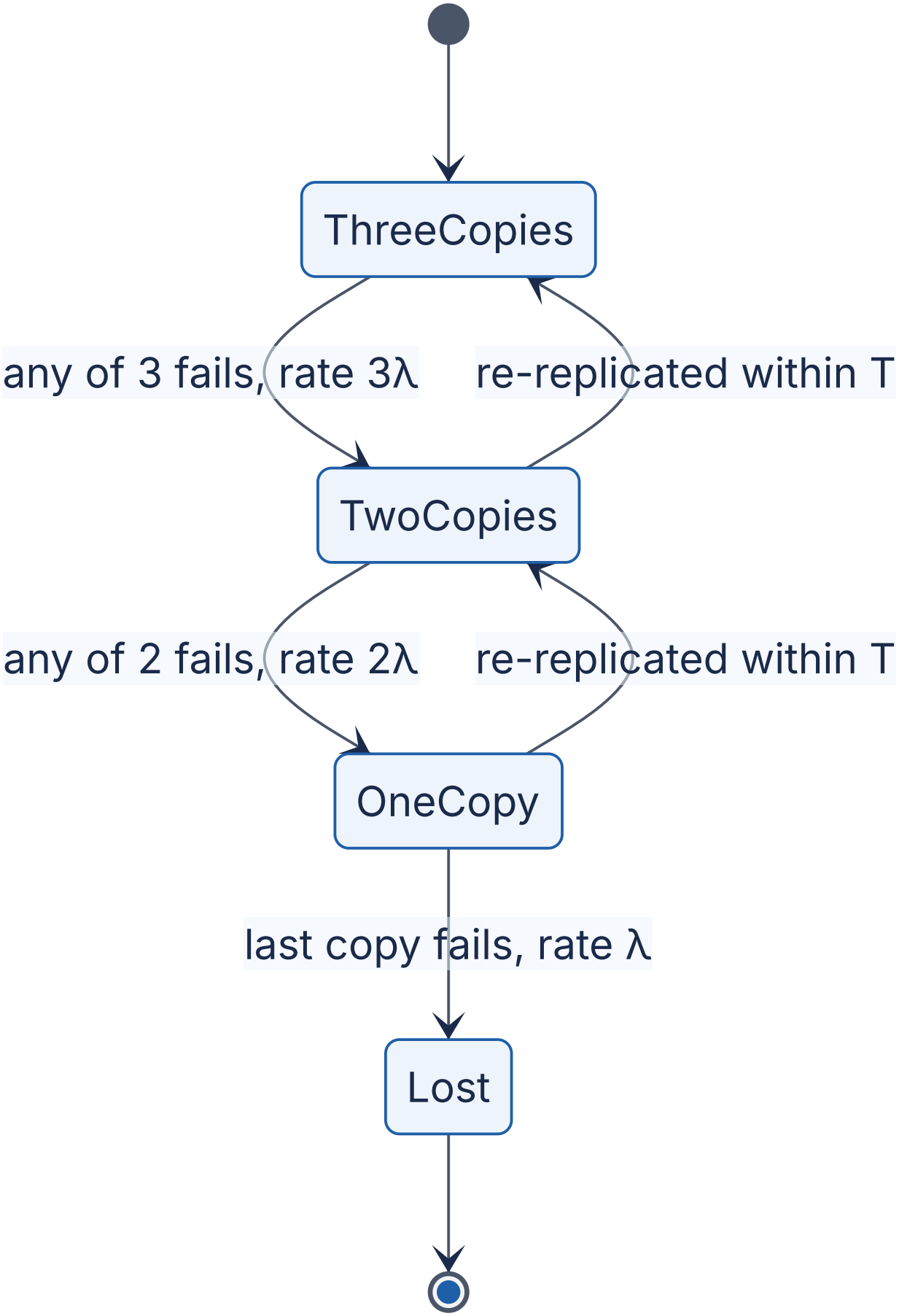
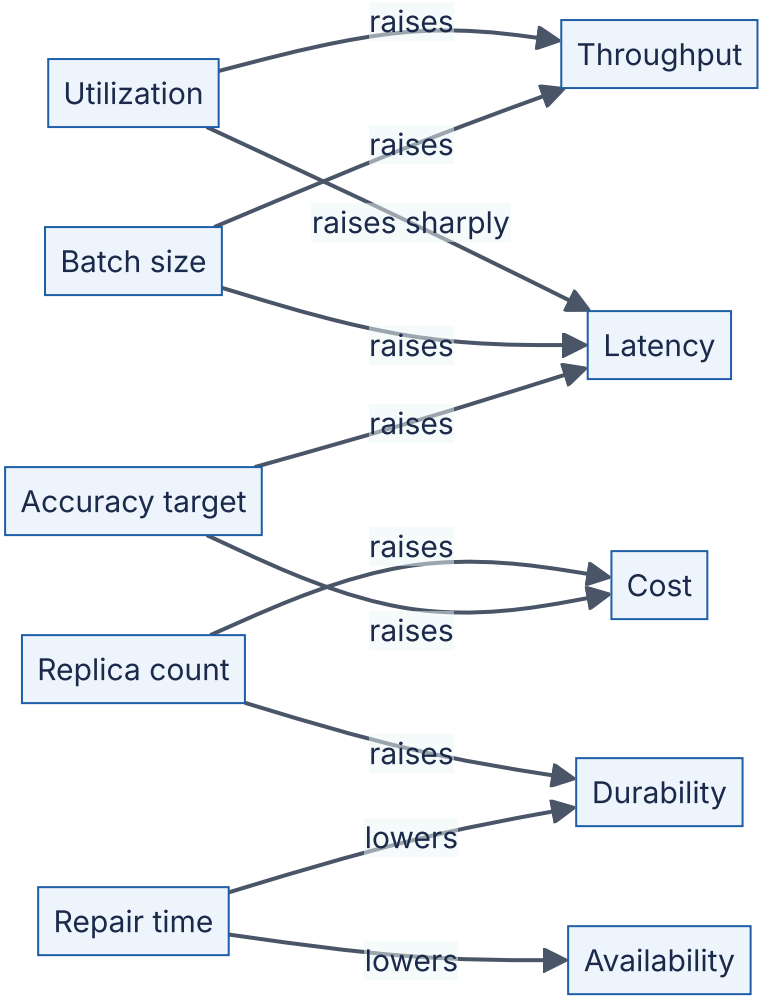

ThreeCopies
any of 3 fails, rate 3λ
re-replicated within T
TwoCopies
any of 2 fails, rate 2λ
re-replicated within T
OneCopy
last copy fails, rate λ
Lost

Figure 4.9 — Six recurring quantitative relationships.



Chapter 5 — Computer Architecture

Figure 5.1 — Inside a single-socket server.

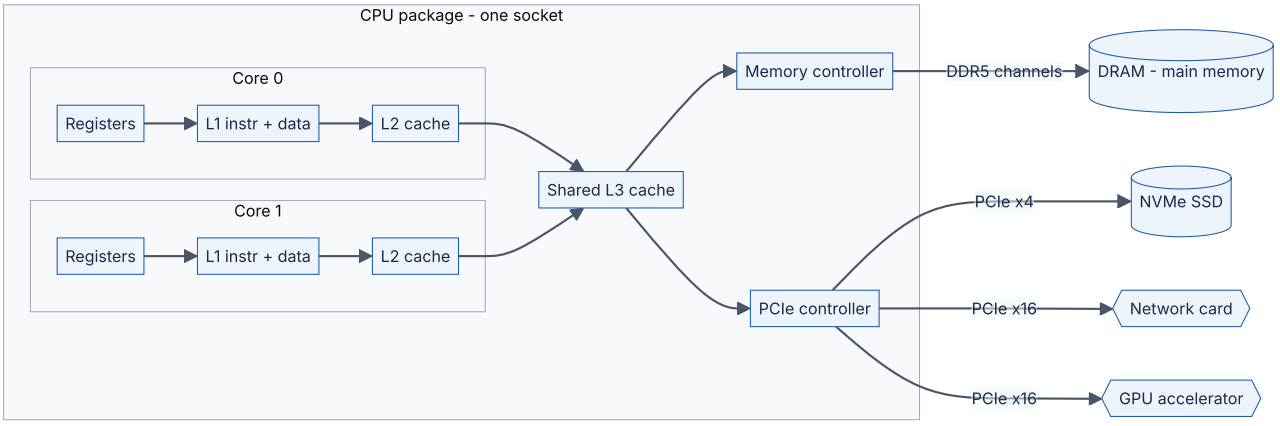


Figure 5.2 — A pipelined core with a branch predictor.

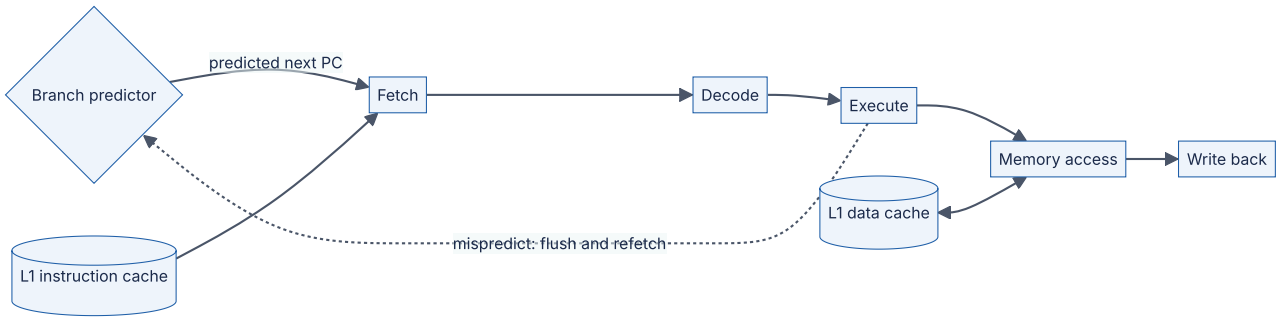


Figure 5.3 — What happens on a load instruction.

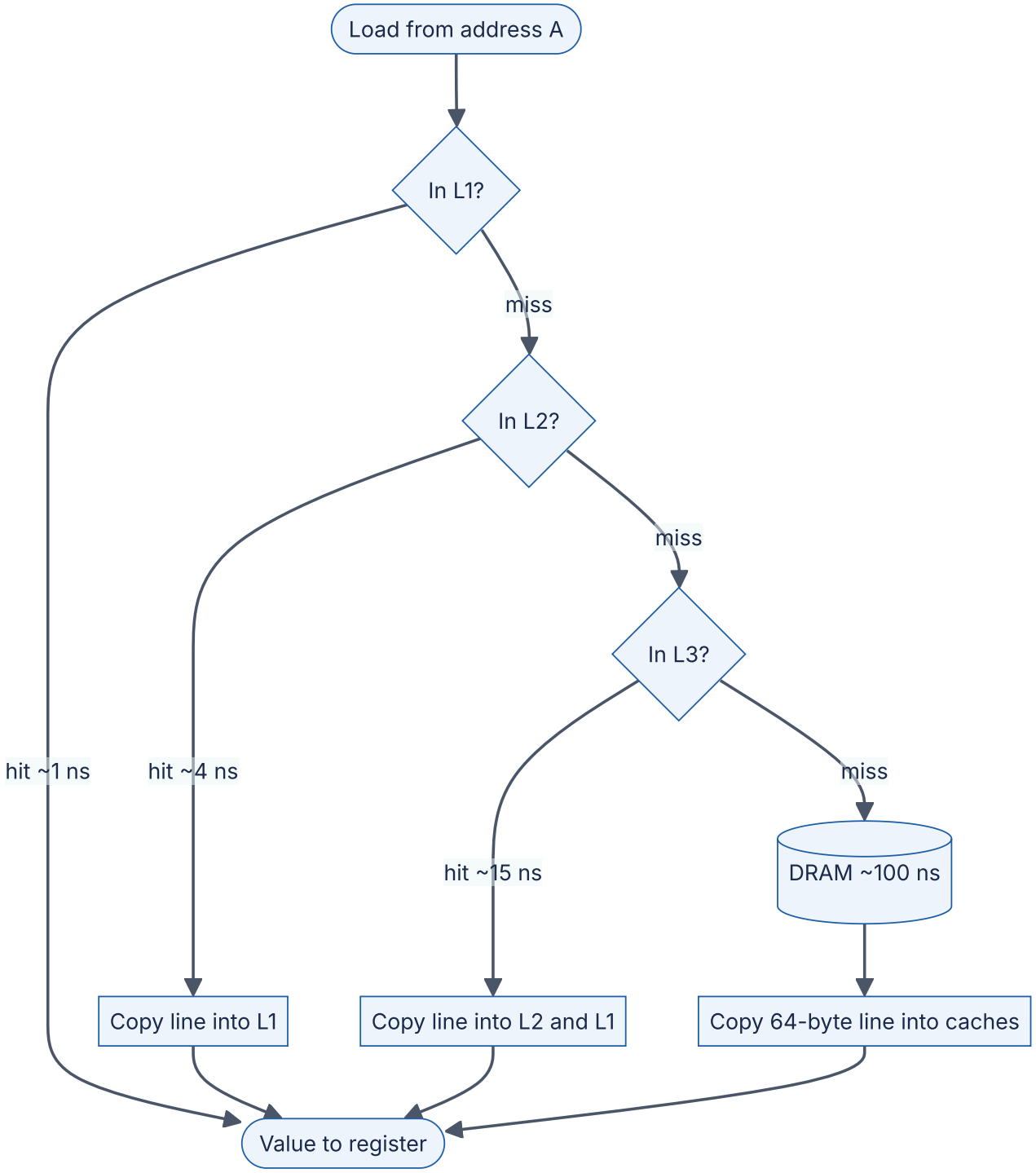


Figure 5.4 — MESI state transitions for one cache line in one core’s cache.

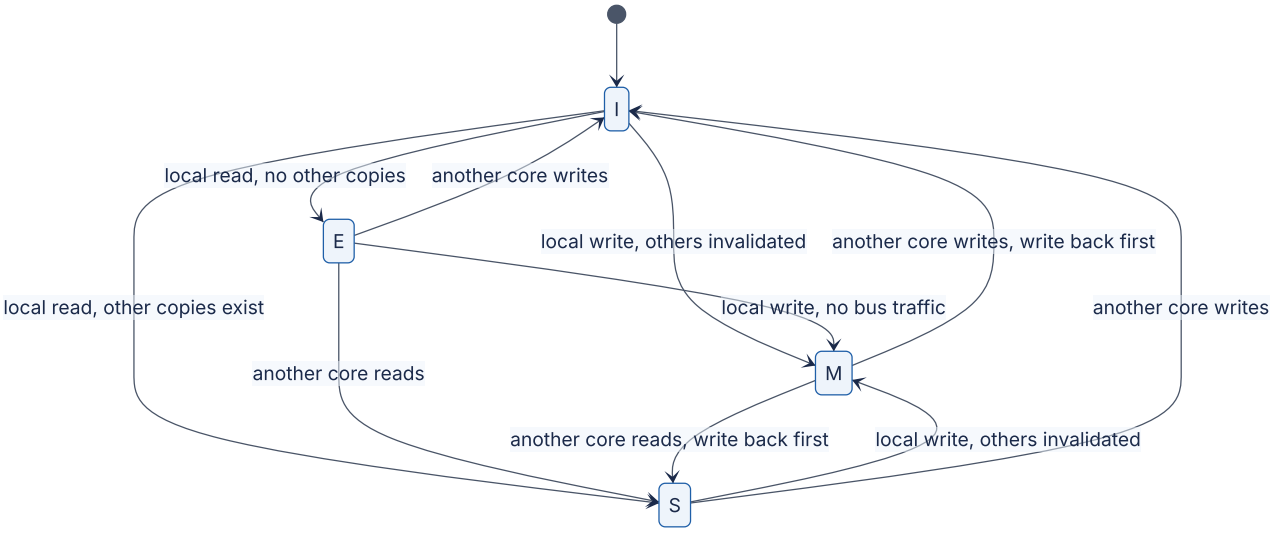


Figure 5.5 — The memory and storage hierarchy (as of 2024–2025, approximate).

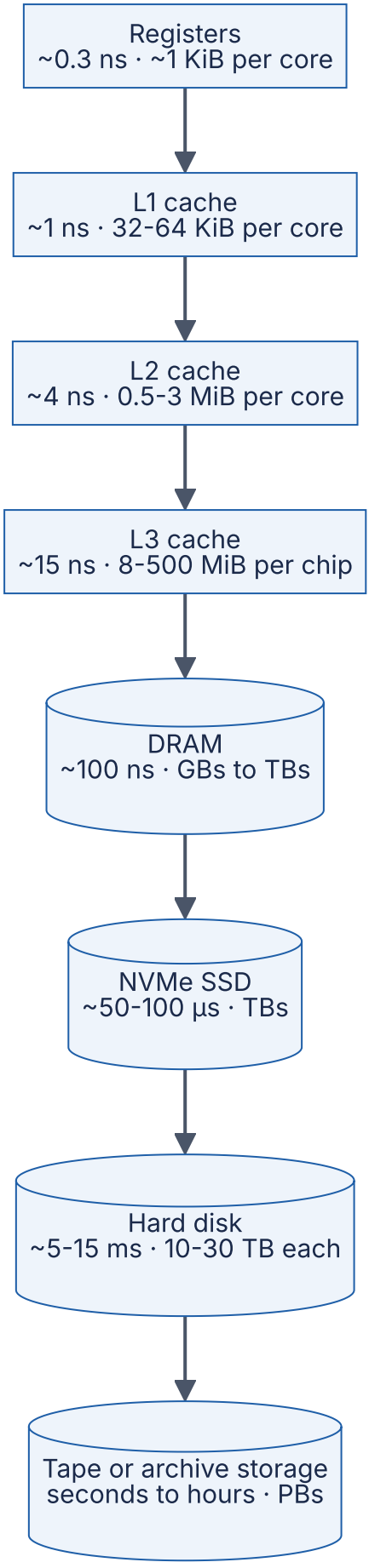

Registers
~0.3 ns · ~1 KiB per core
L1 cache
~1 ns · 32-64 KiB per core
L2 cache
~4 ns · 0.5-3 MiB per core
L3 cache
~15 ns · 8-500 MiB per chip
DRAM
~100 ns · GBs to TBs
NVMe SSD
~50-100 µs · TBs
Hard disk
~5-15 ms · 10-30 TB each
Tape or archive storage
seconds to hours · PBs

Figure 5.6 — How an SSD handles an overwrite and garbage collection.

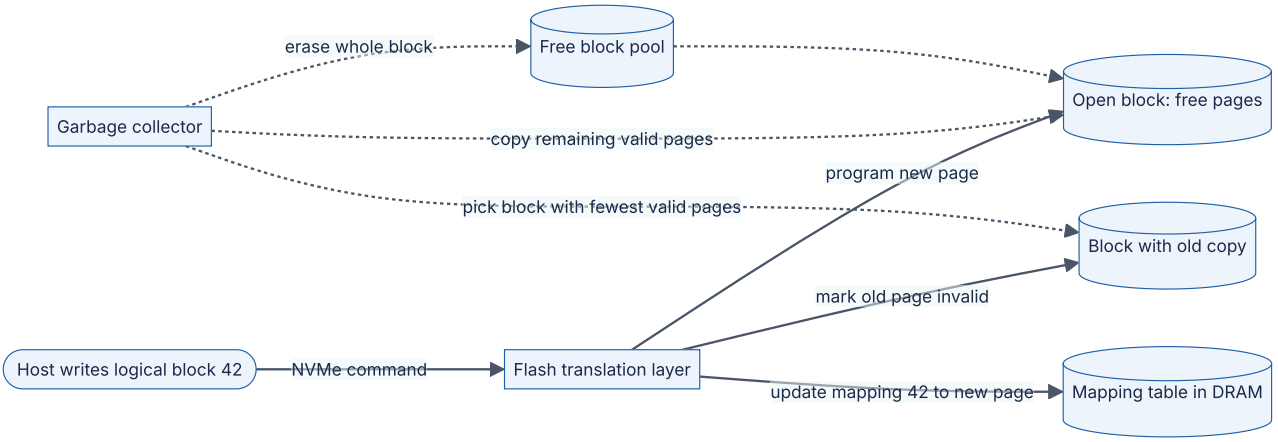


Figure 5.7 — Reading a block from an NVMe SSD with DMA and an interrupt.

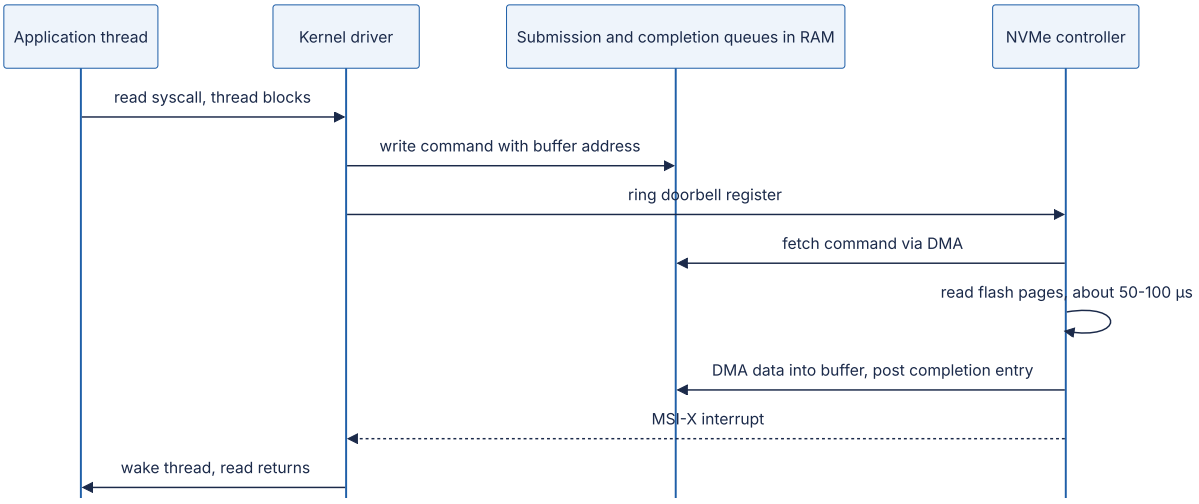


Figure 5.8 — Diagnosing what limits a workload.

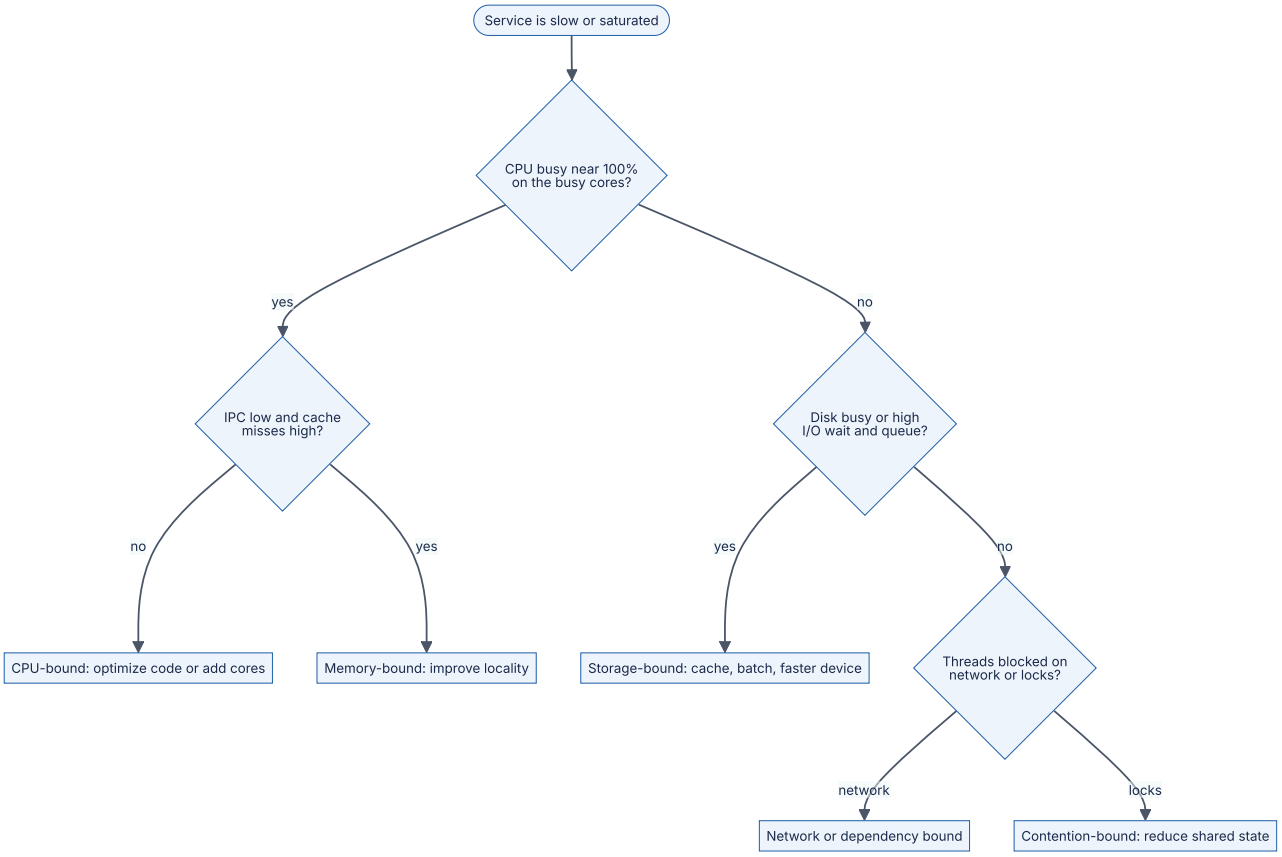


Figure 5.9 — A two-socket NUMA server.

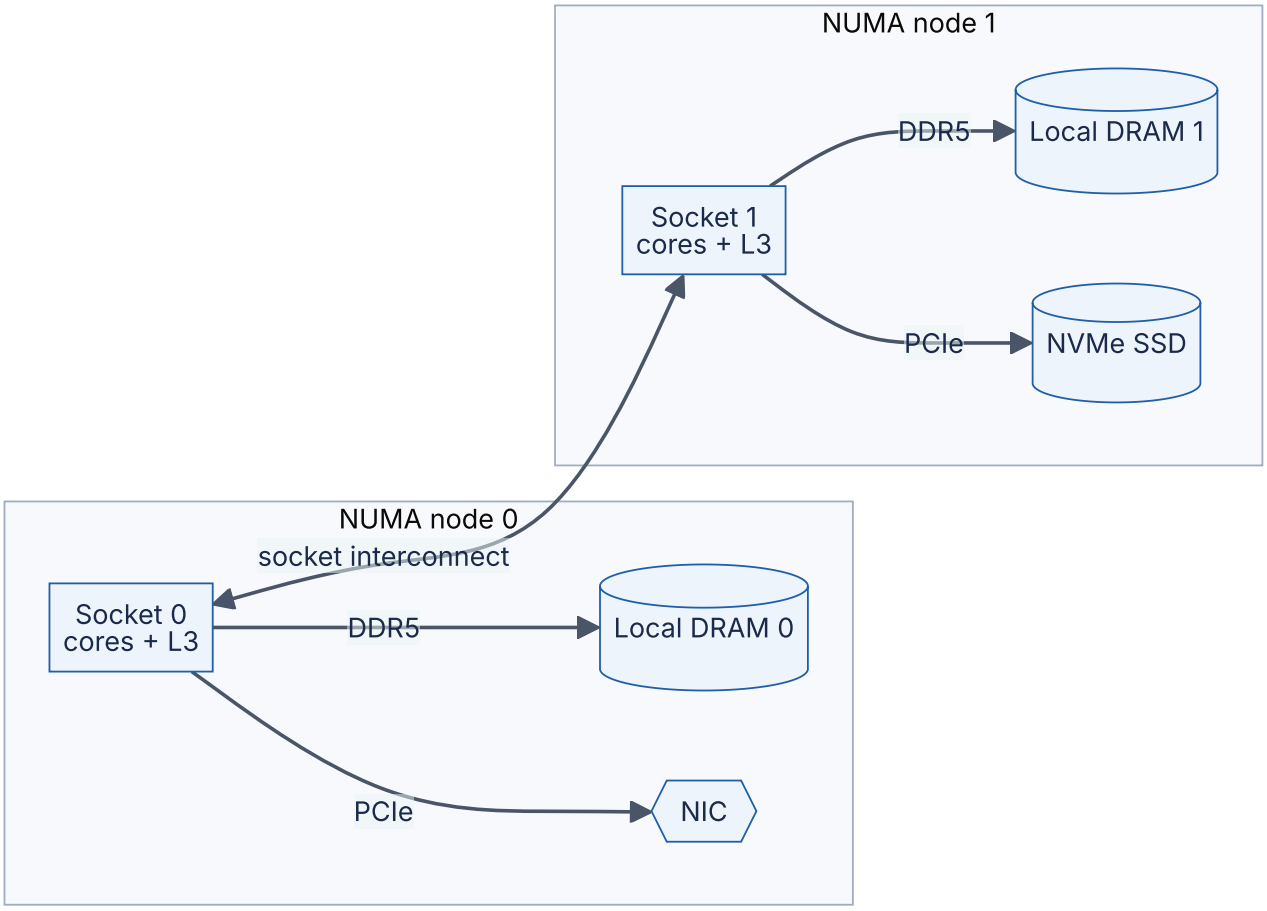
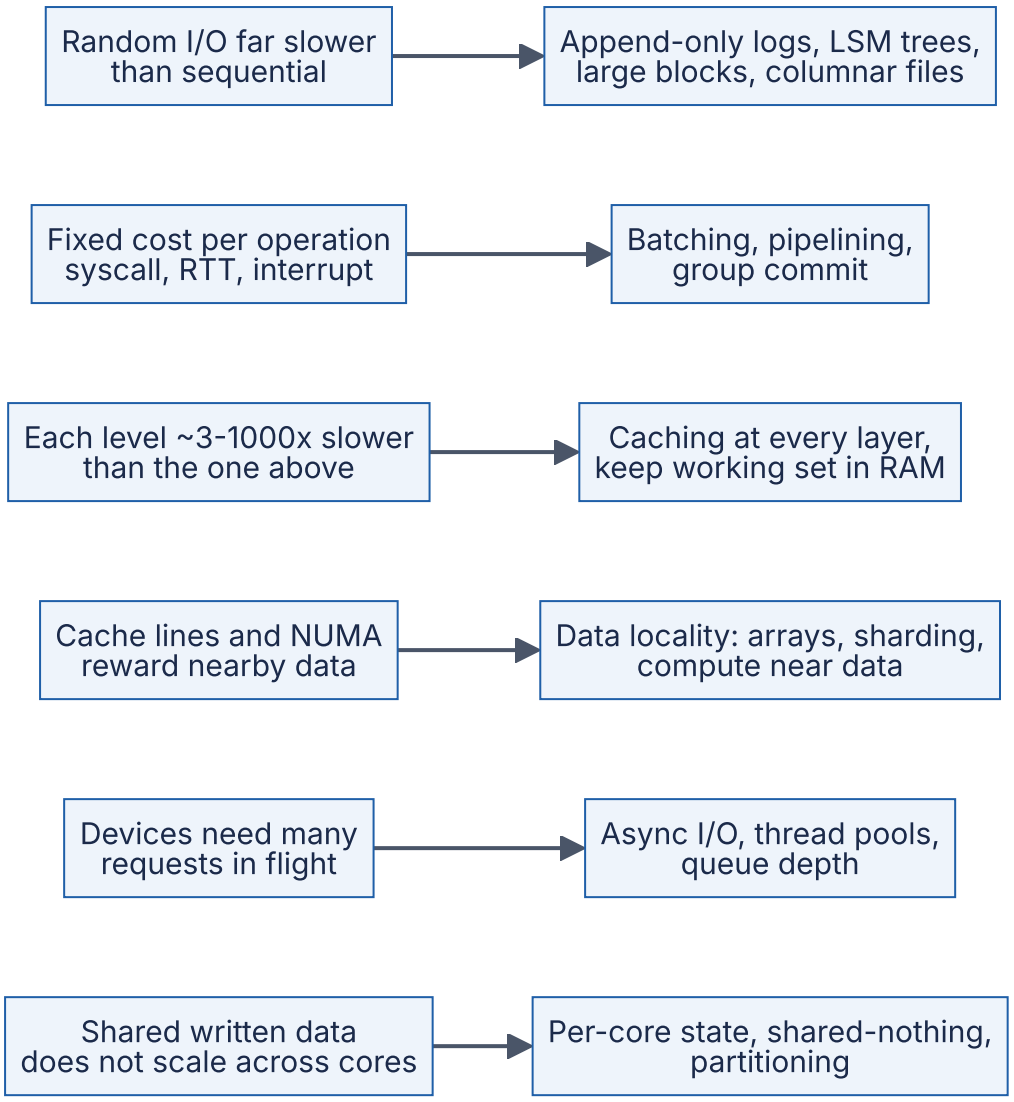


Figure 5.10 — From hardware facts to design decisions.



Chapter 6 — Operating Systems

Figure 6.1 — Layout of a process's virtual address space on Linux x86-64.

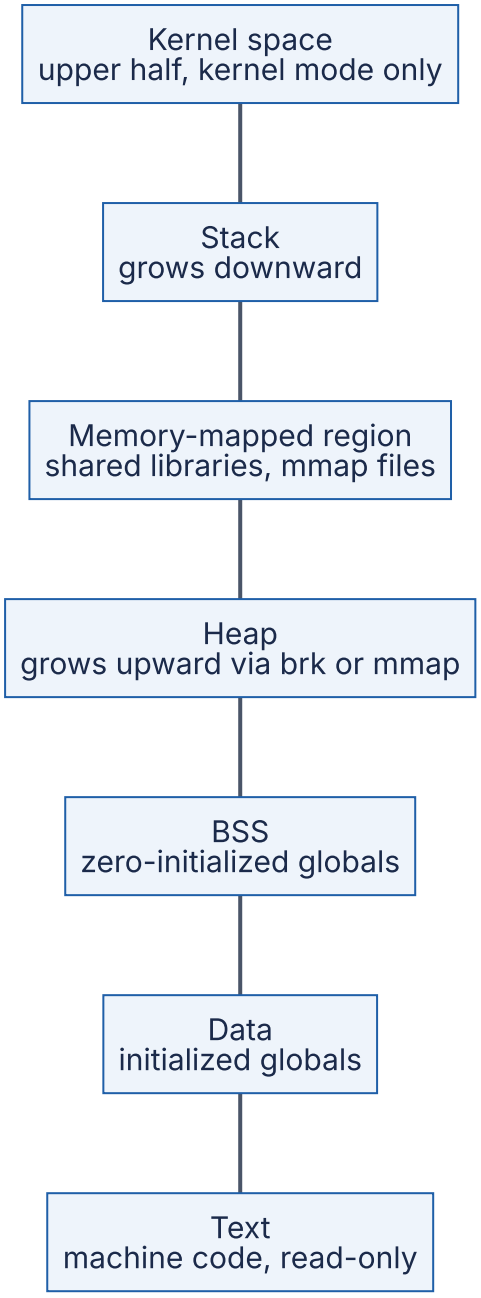


Figure 6.2 — Process state diagram.

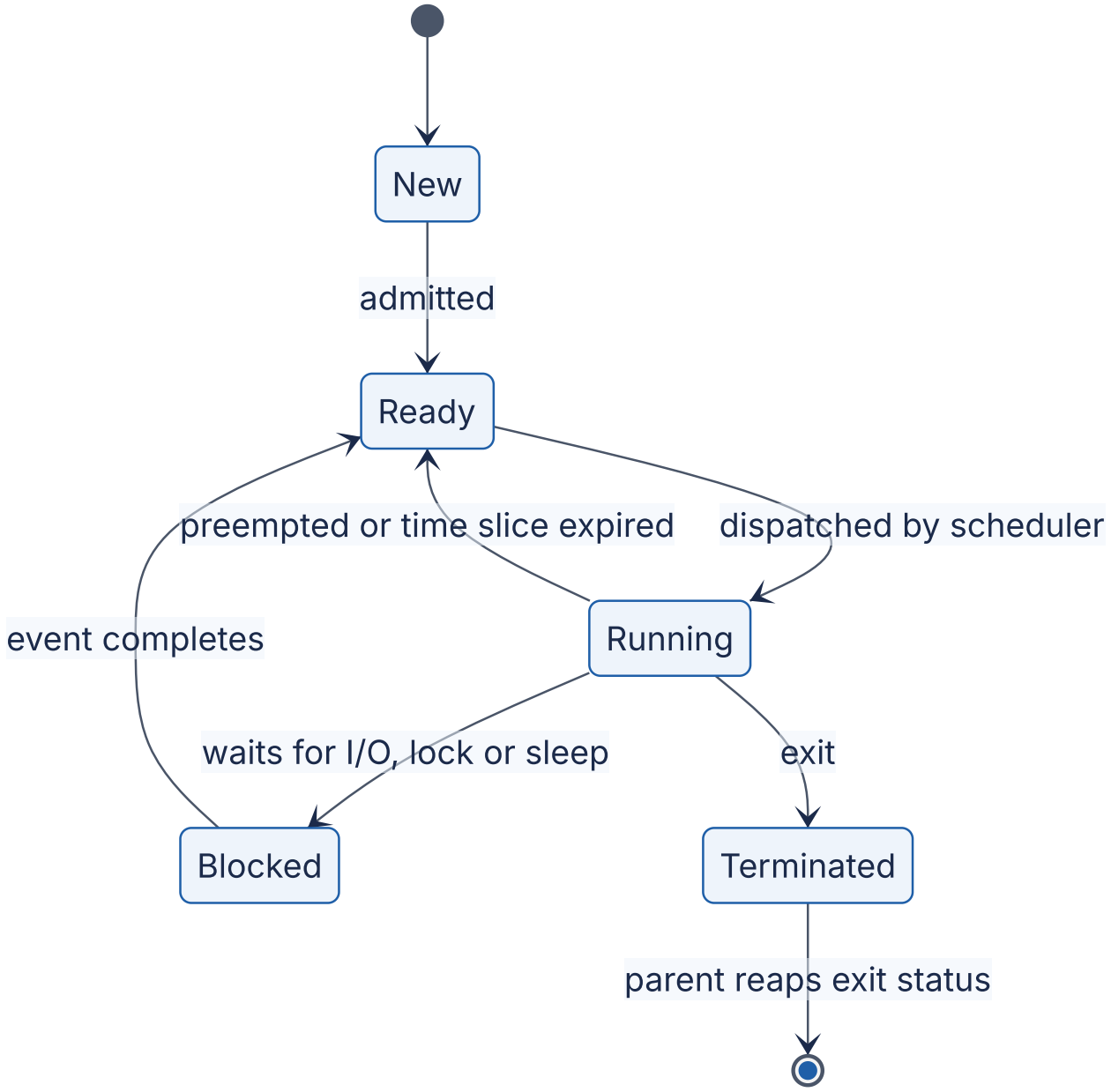


Figure 6.3 — A context switch between two threads triggered by a timer interrupt.

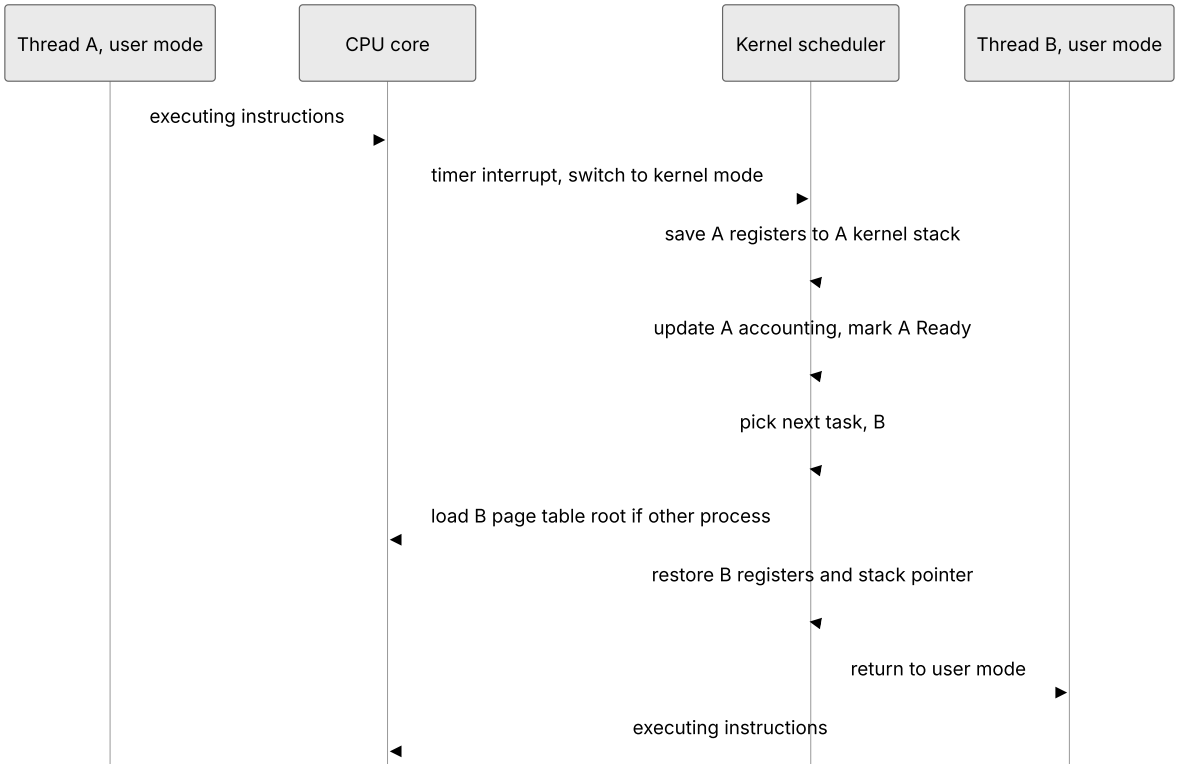


Figure 6.4 — Multi-level feedback queue decisions.

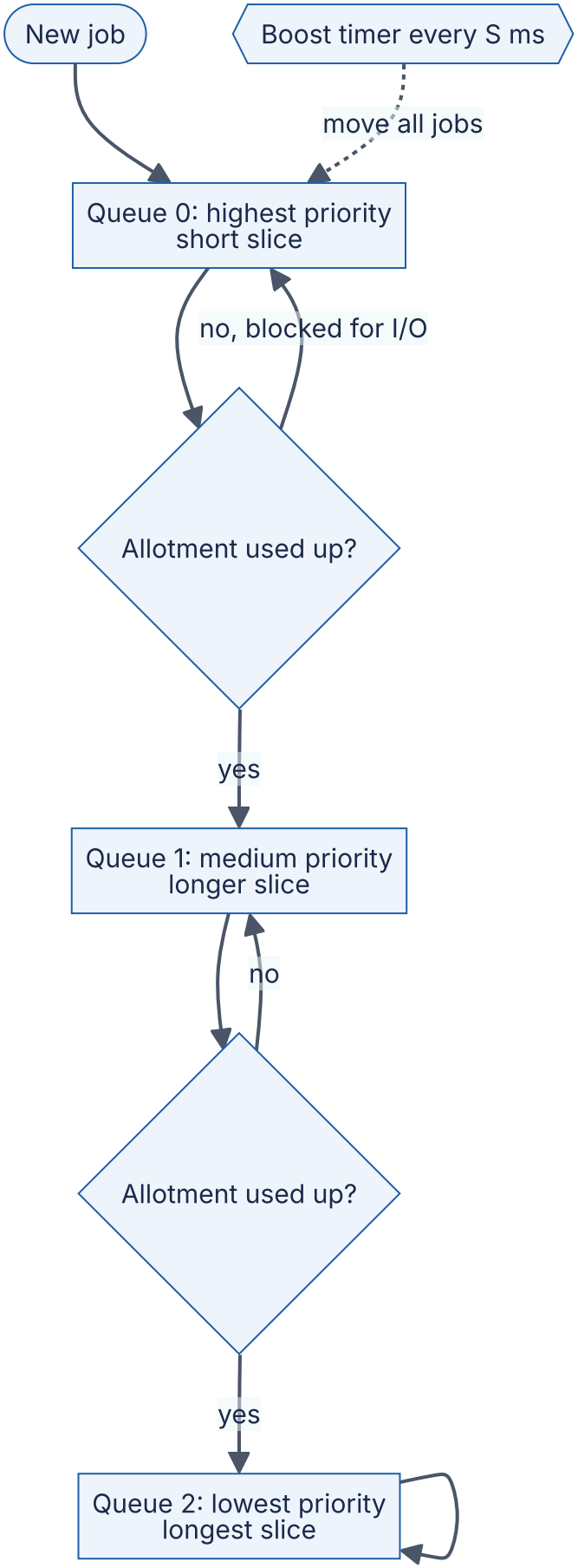
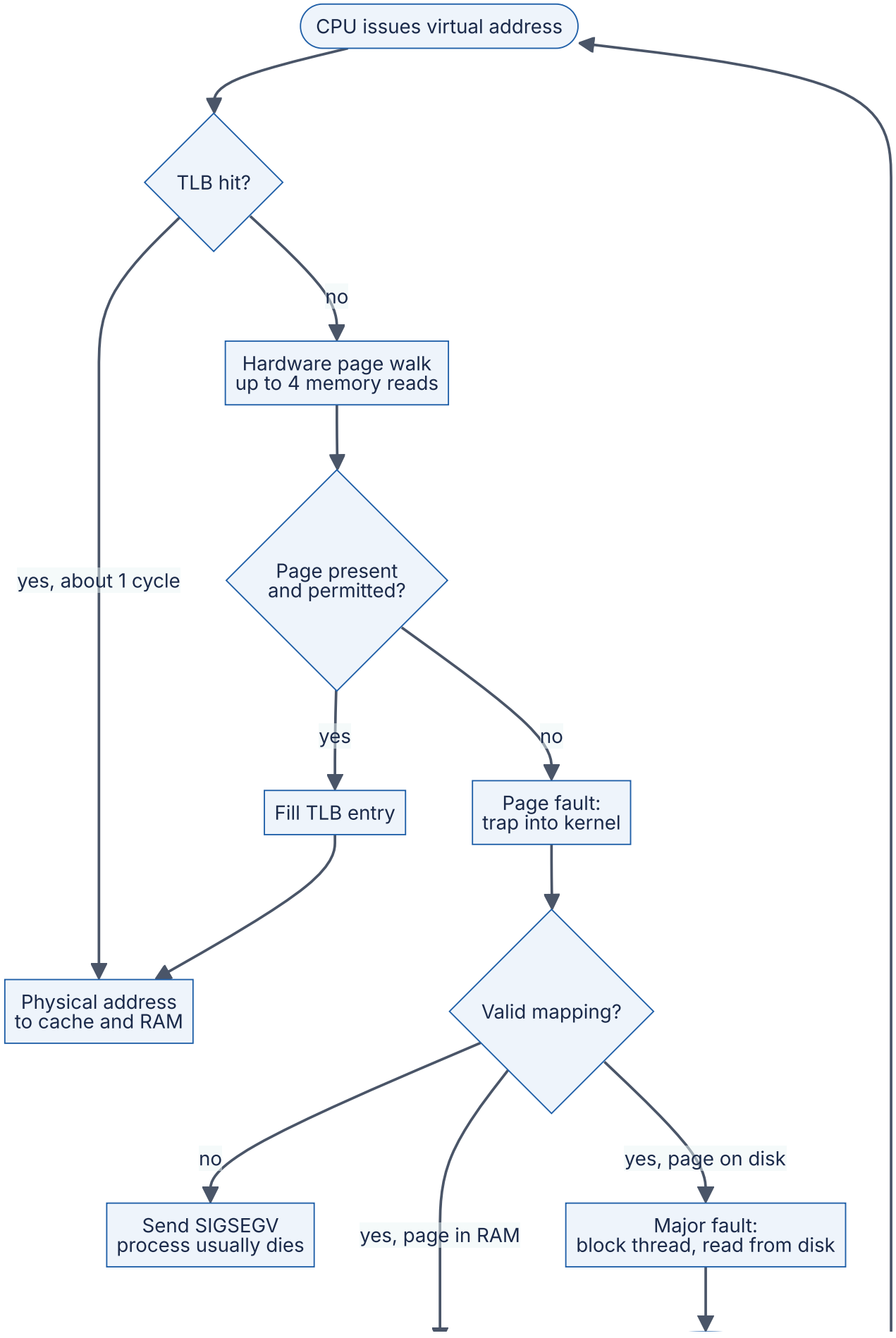

New job
Boost timer every S ms
move all jobs
Queue 0: highest priority
short slice
no, blocked for I/O
Allotment used up?
yes
Queue 1: medium priority
longer slice
no
Allotment used up?
yes
Queue 2: lowest priority
longest slice

Figure 6.5 — Translating a virtual address: TLB, page walk and page fault.


CPU issues virtual address
TLB hit?
yes, about 1 cycle
no
Hardware page walk
up to 4 memory reads
Page present
and permitted?
yes
Fill TLB entry
no
Page fault:
trap into kernel
Physical address
to cache and RAM
Valid mapping?
no
Send SIGSEGV
process usually dies
yes, page in RAM
yes, page on disk
Major fault:
block thread, read from disk

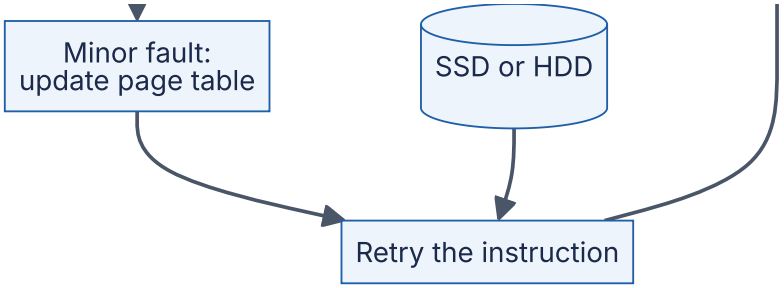


Figure 6.6 — A `read()` system call that must wait for the disk.

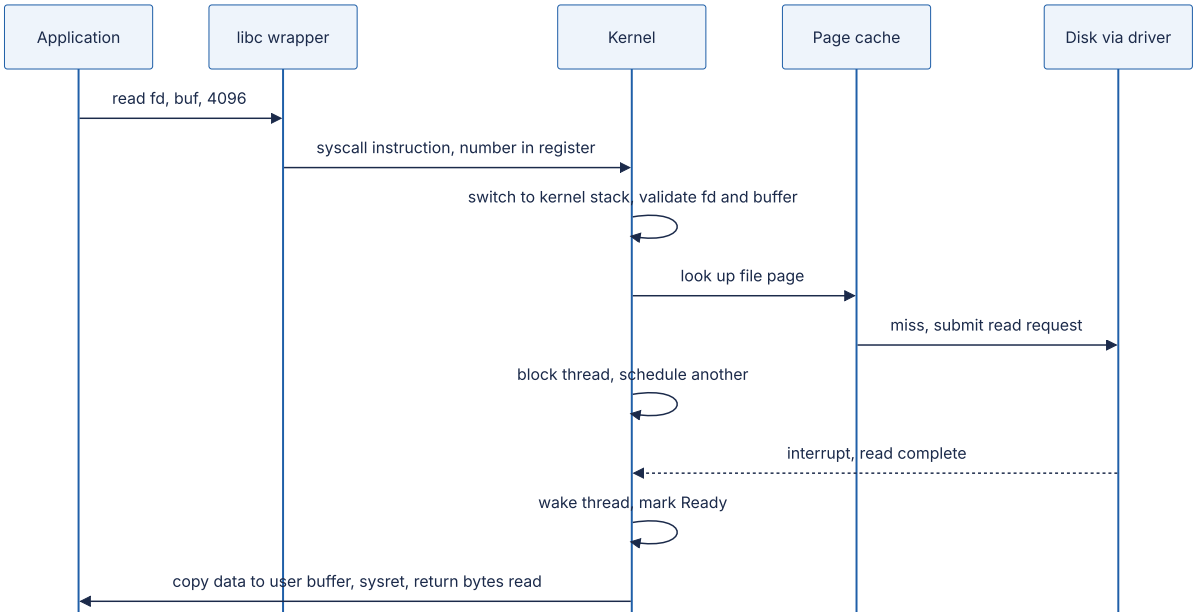


Figure 6.7 — Two threads lose an update to a shared counter.

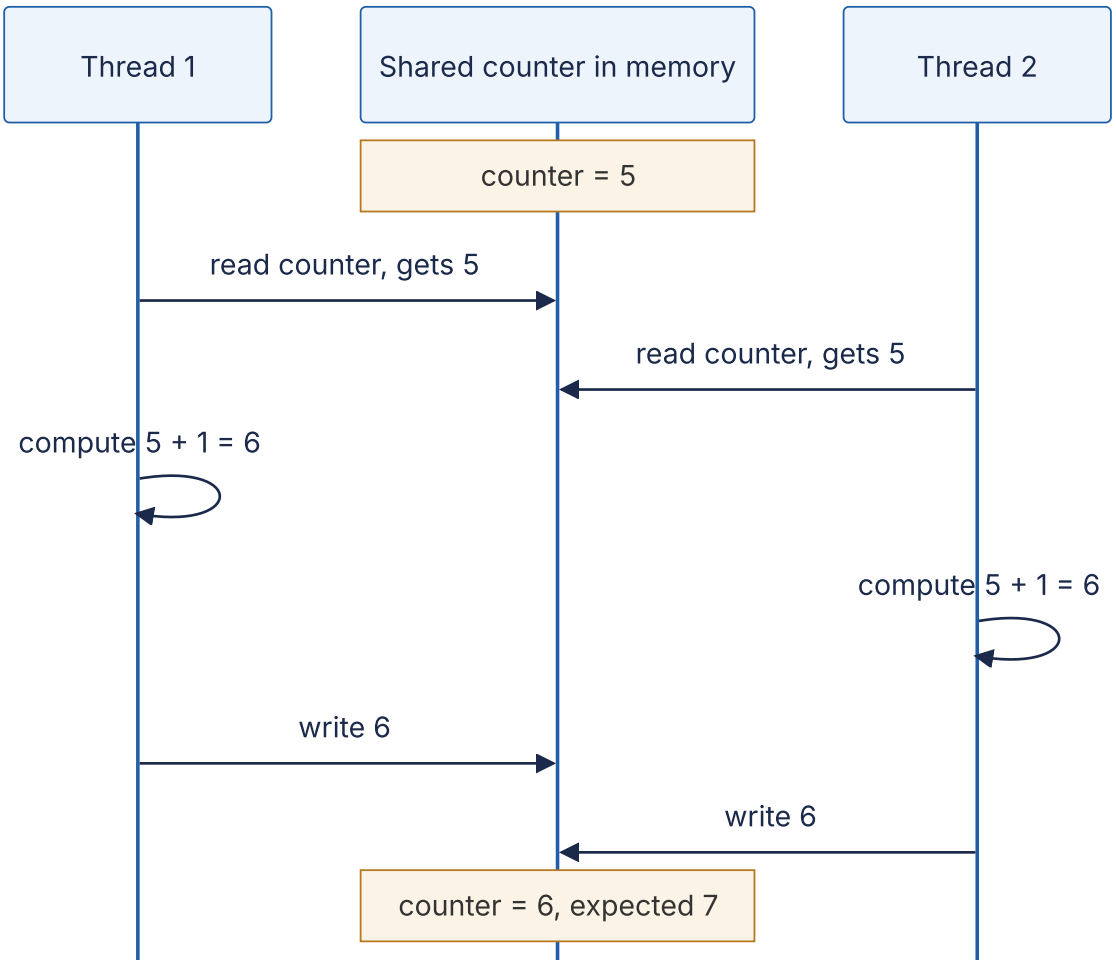


Figure 6.8 — Resource-allocation graph with a deadlock cycle.

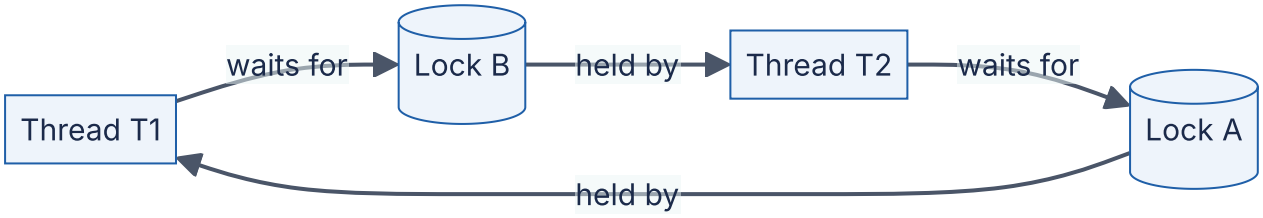


Figure 6.9 — Priority inversion on Mars Pathfinder.

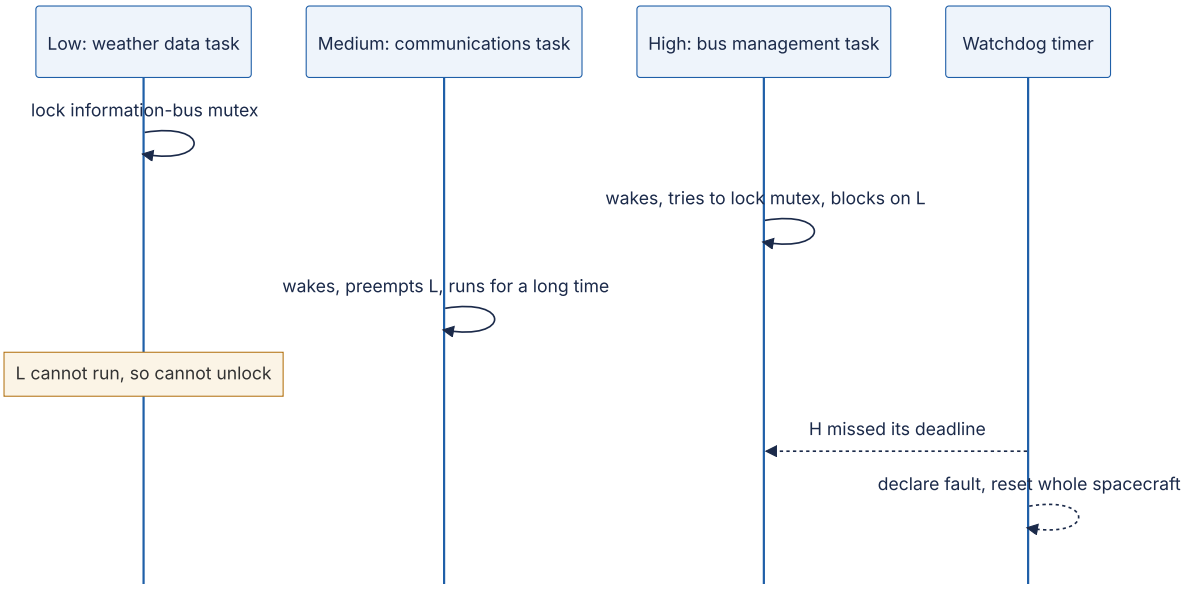


Figure 6.10 — An event loop on epoll.

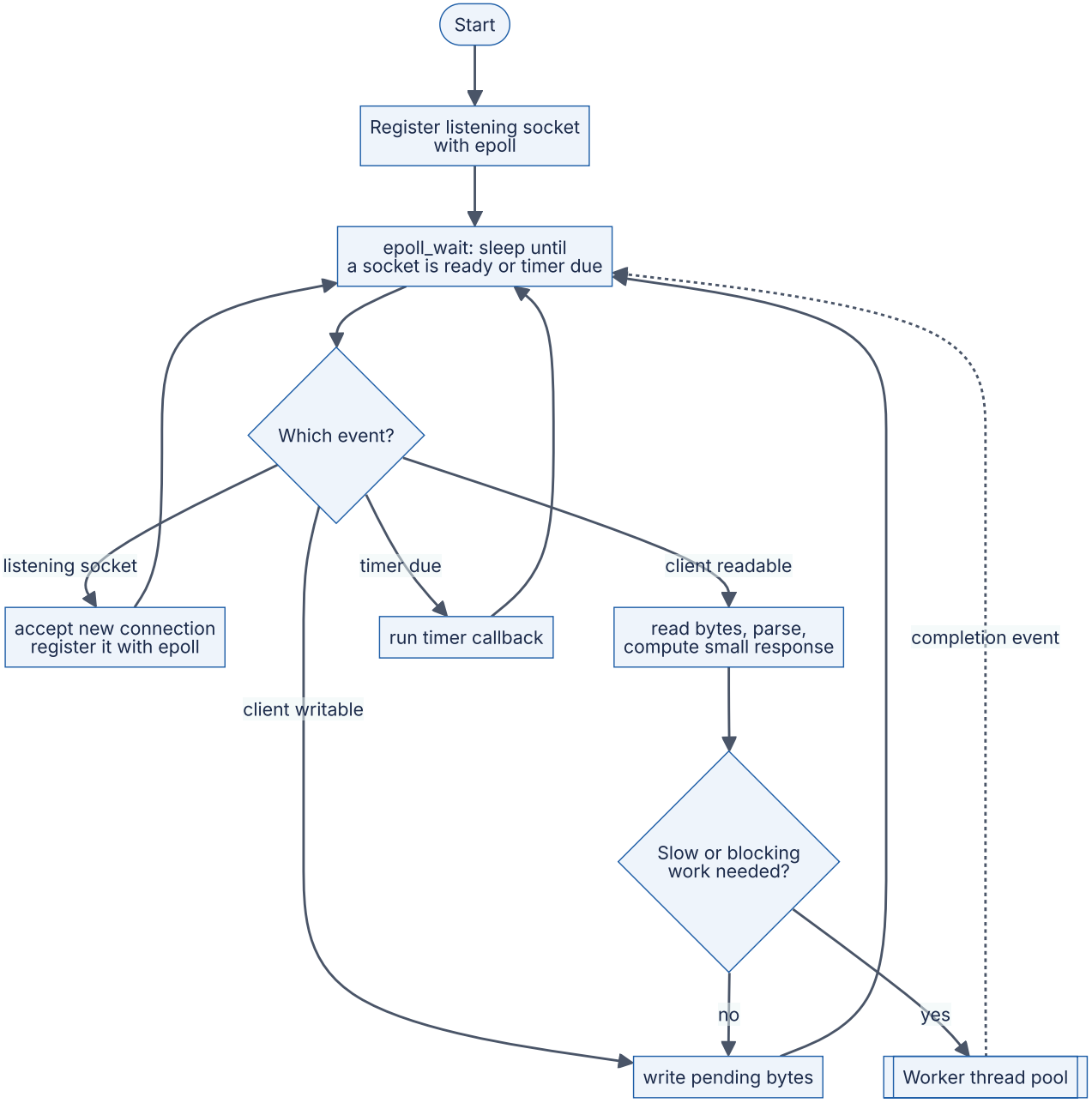


Figure 6.11 — io_uring submission and completion rings.

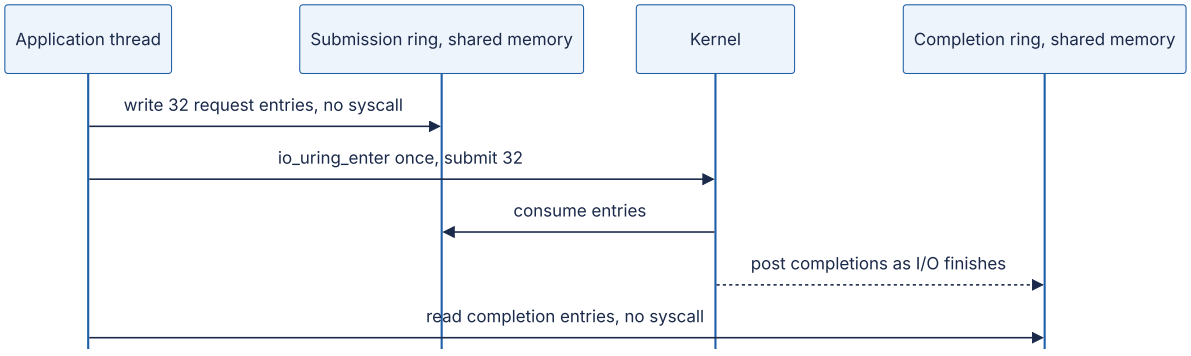
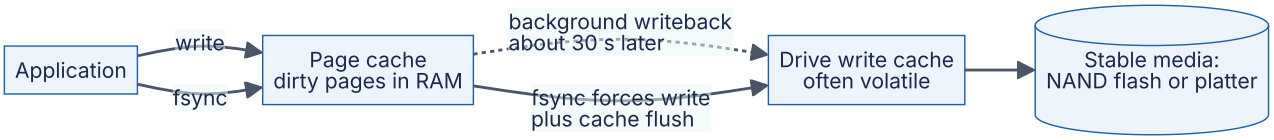


Figure 6.12 — The write path from application to stable storage.



Chapter 7 — Networking Fundamentals

Figure 7.1 — Encapsulation down the sender’s stack and decapsulation up the receiver’s.

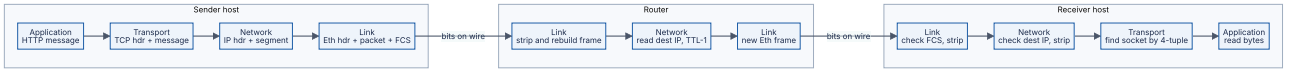


Figure 7.2 — BGP routing between autonomous systems, with an anycast prefix announced from two sites.

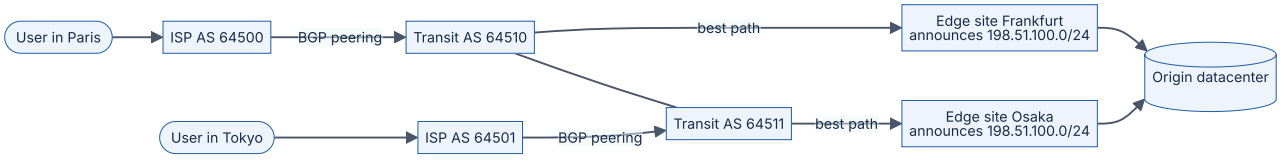


Figure 7.3 — DNS resolution of www.example.com with an empty cache.

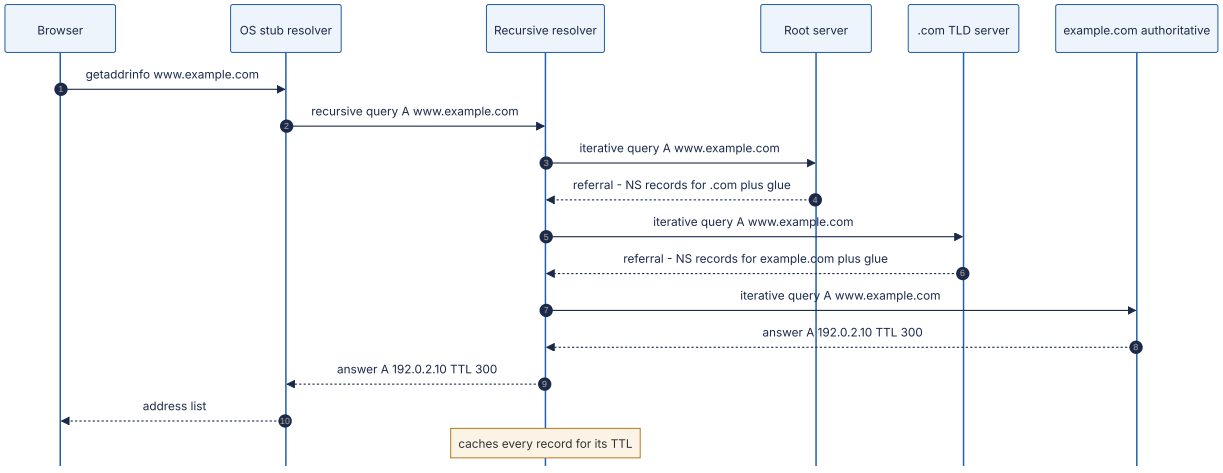


Figure 7.4 — TCP three-way handshake followed by the first request.

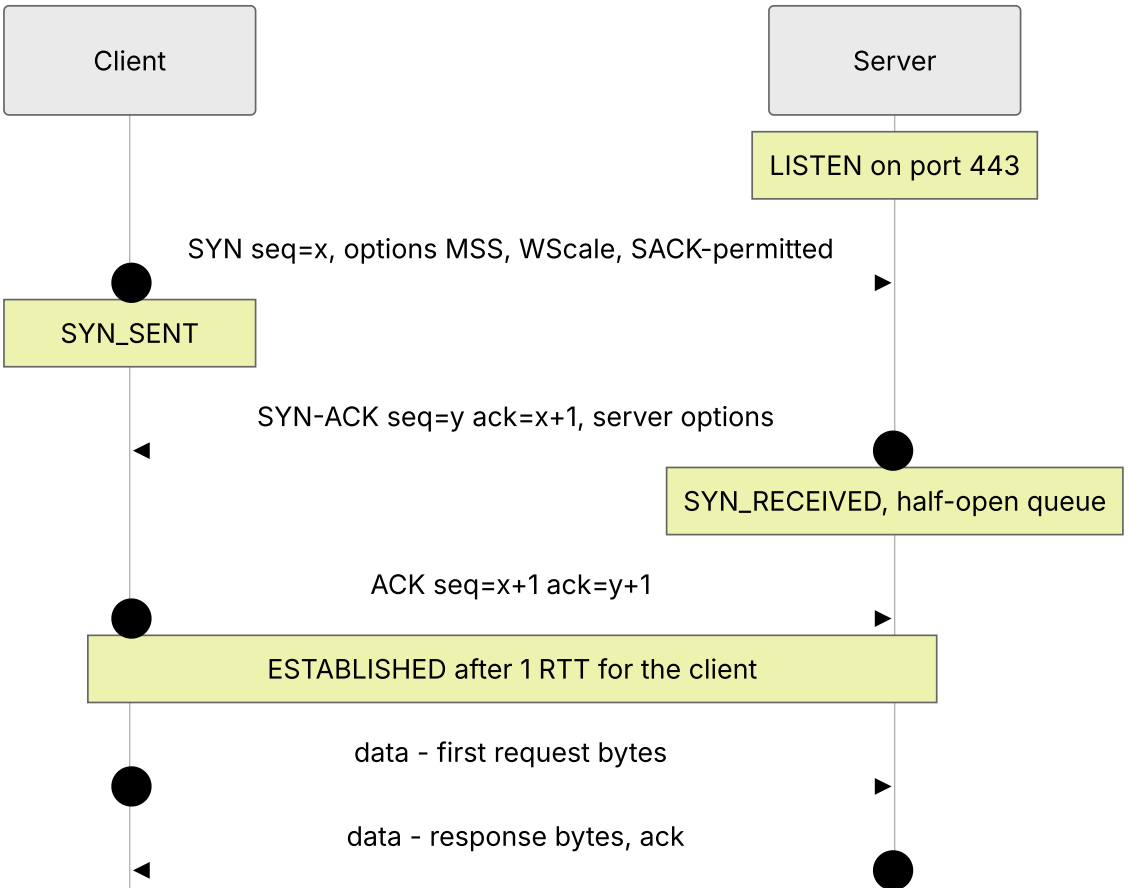


Figure 7.5 — TCP four-way close and the TIME_WAIT state.

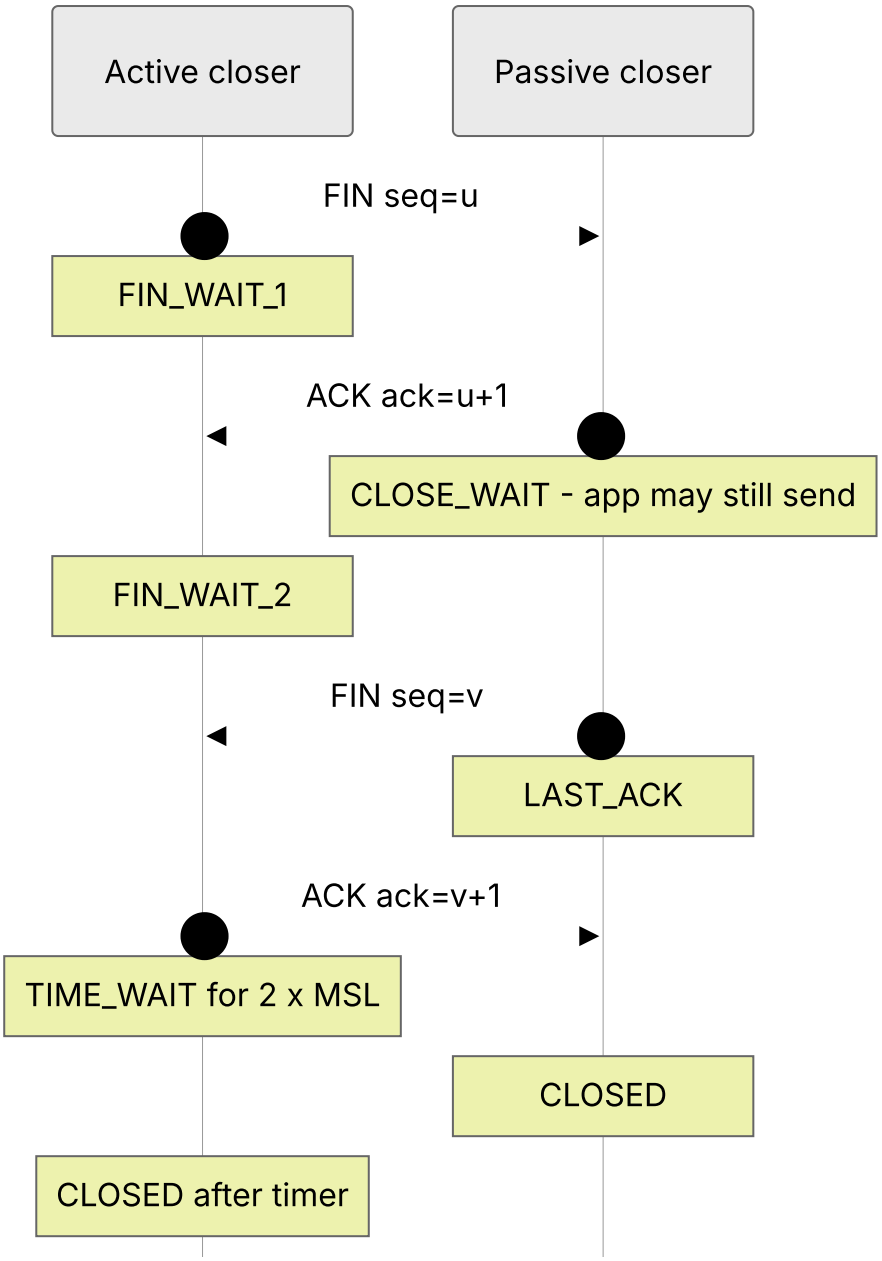


Figure 7.6 — Loss-based TCP congestion control states (Reno-style).

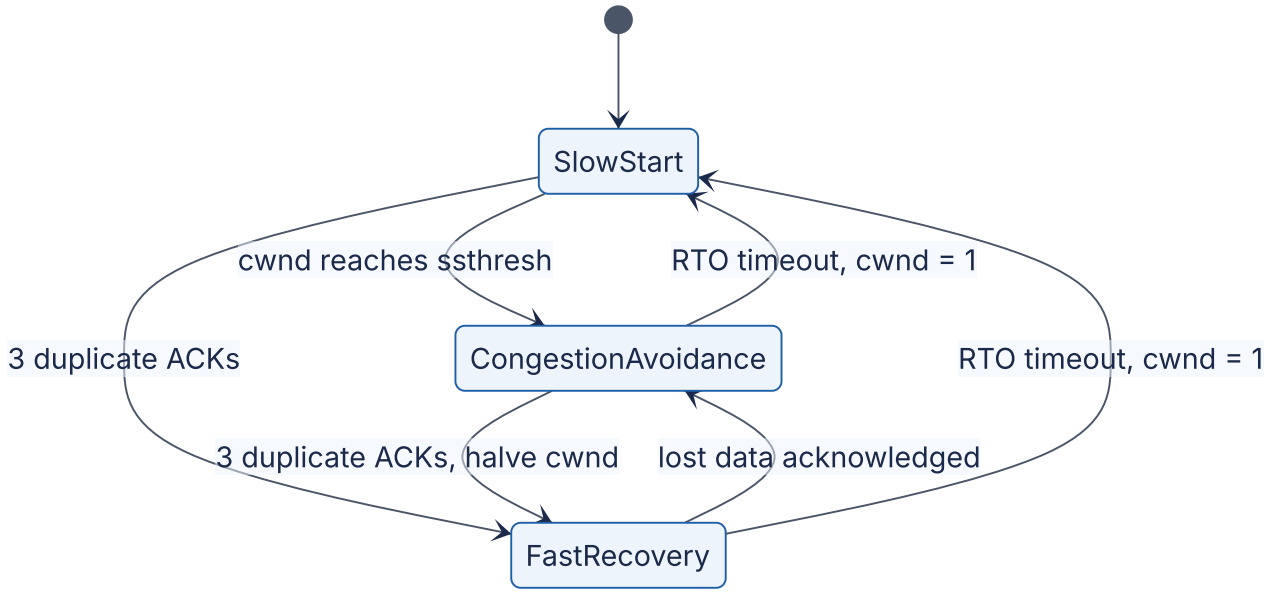


Figure 7.7 — Protocol stacks for the three HTTP versions.

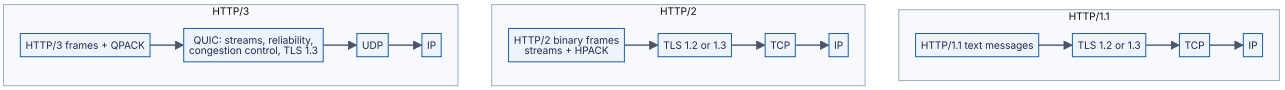


Figure 7.8 — TLS 1.3 full handshake (after the TCP handshake).

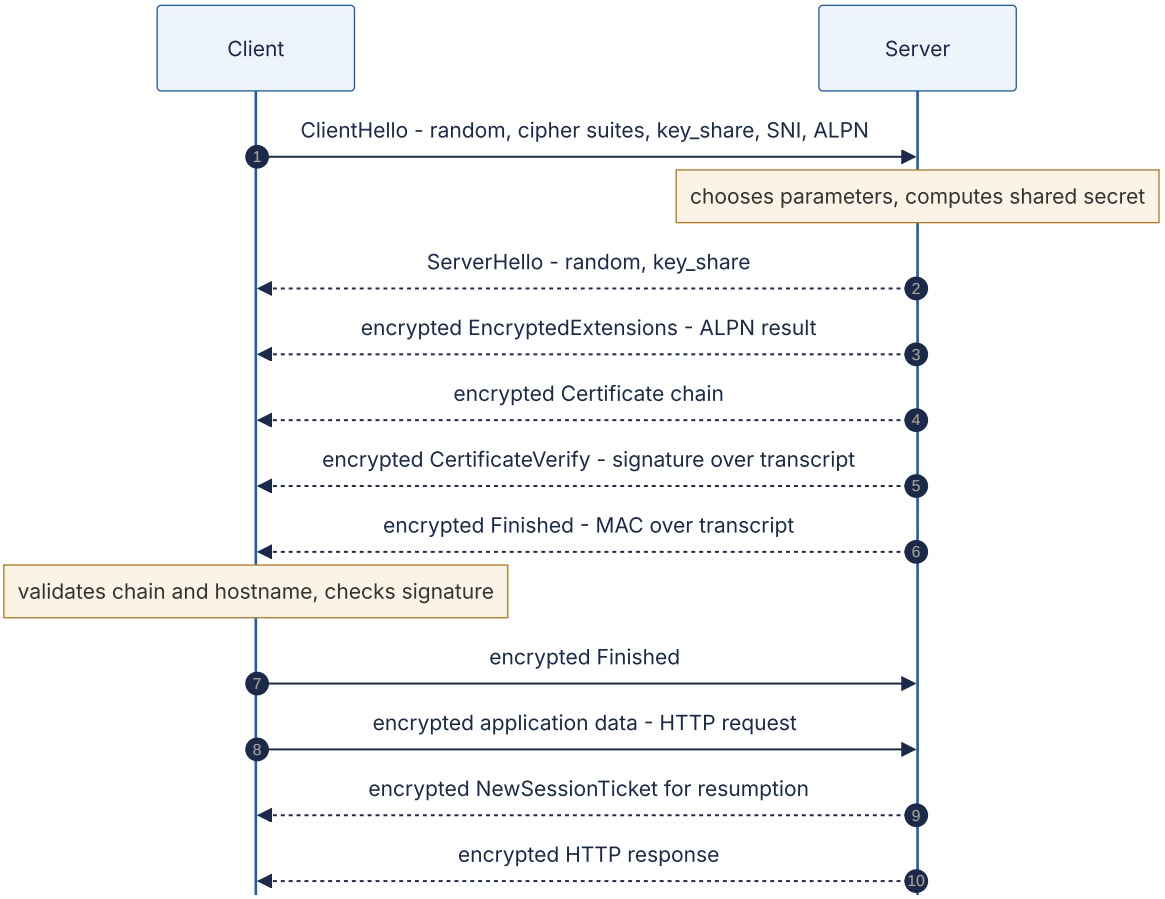


Figure 7.9 — WebSocket opening handshake and framing.

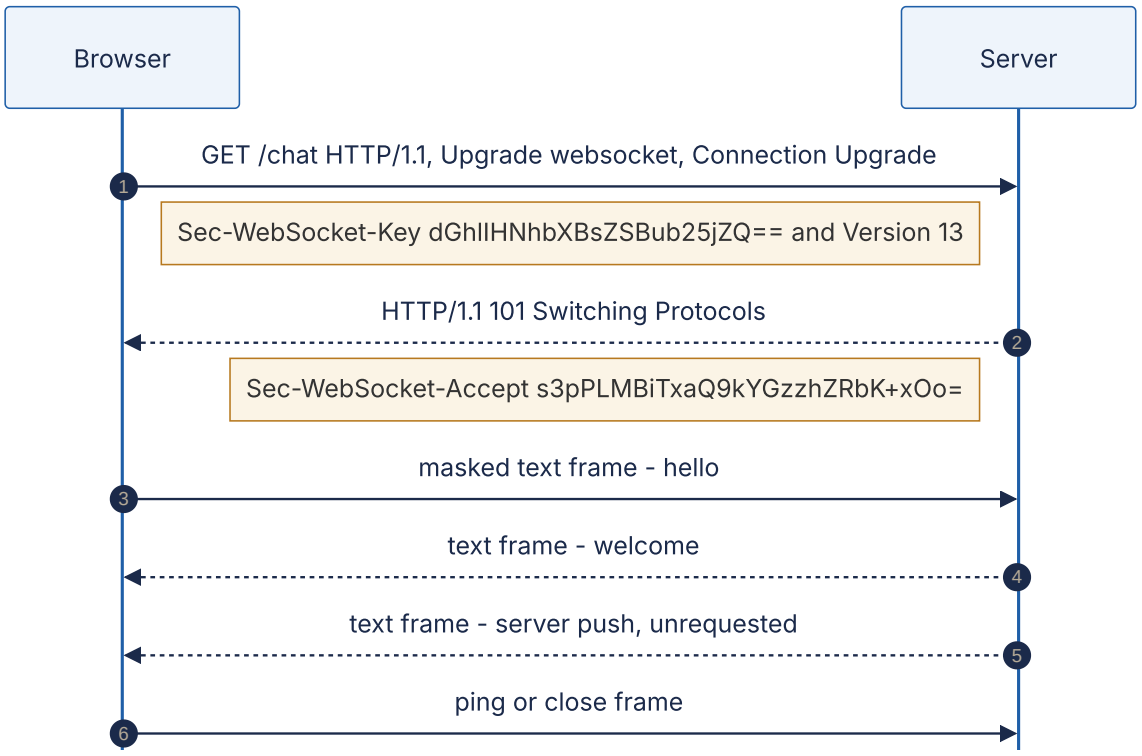


Figure 7.10 — Forward proxy versus reverse proxy.

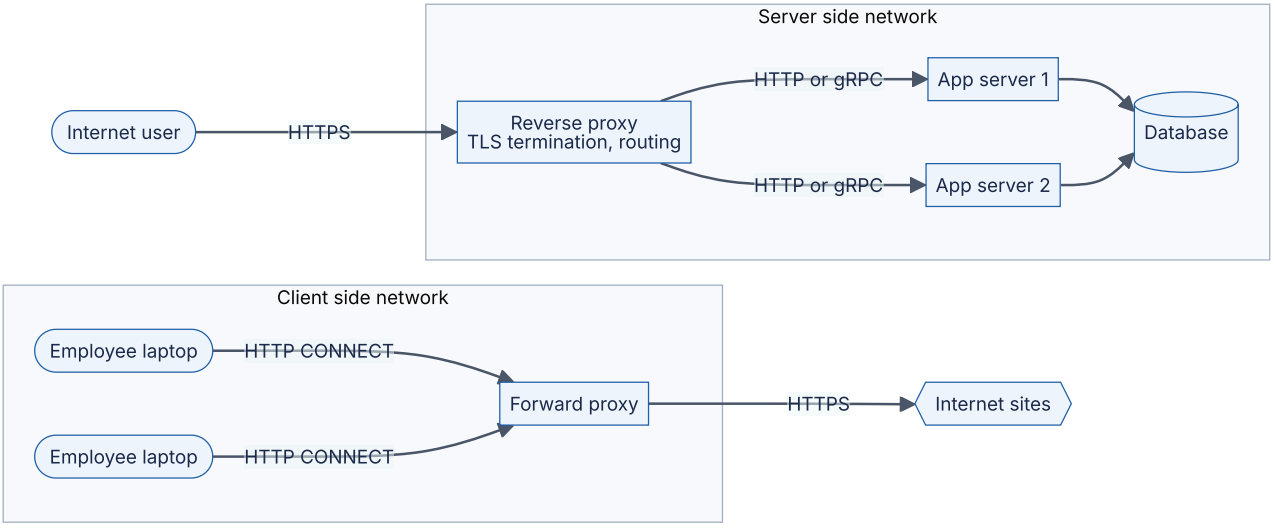


Figure 7.11 — Source NAT for an outbound connection.

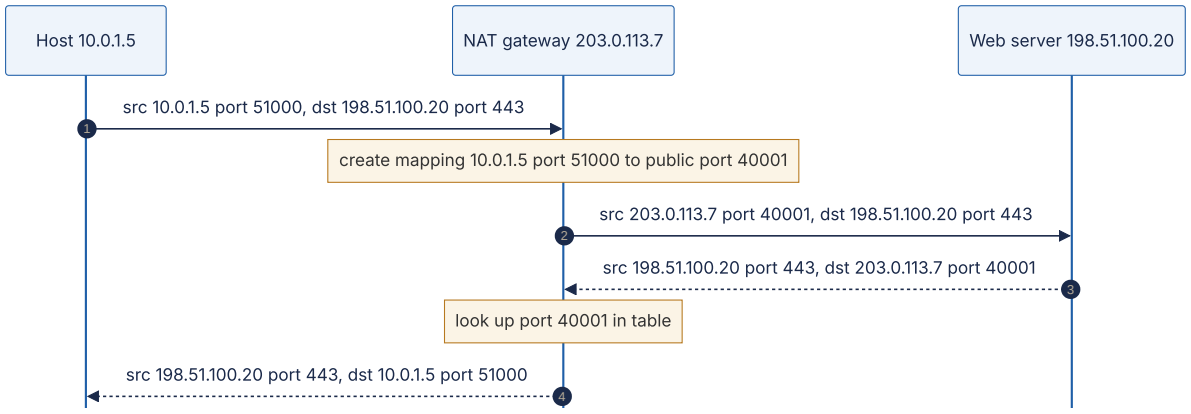


Figure 7.12 — A leaf-spine fabric with ECMP.

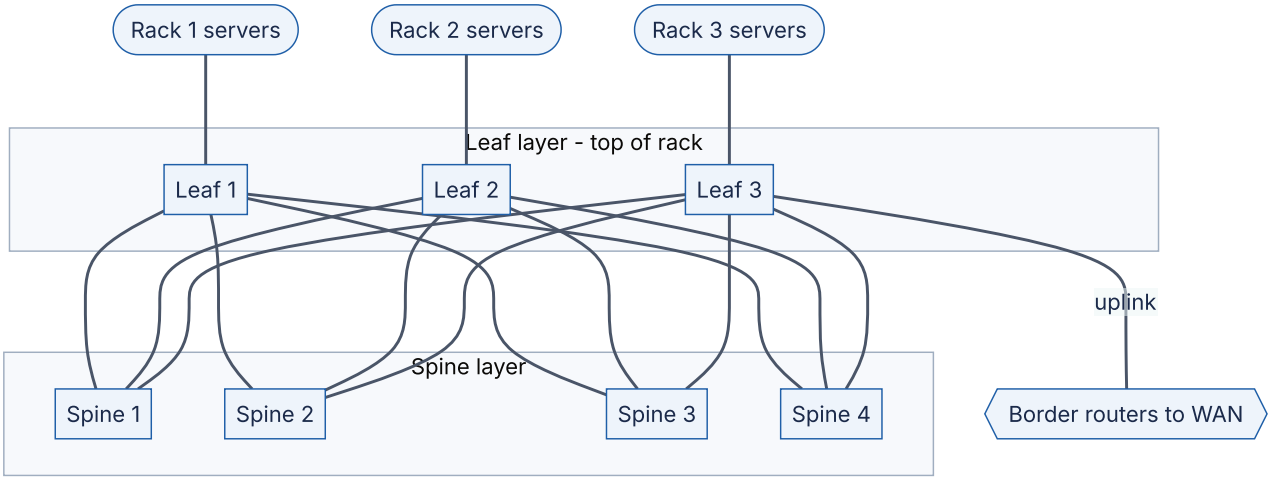
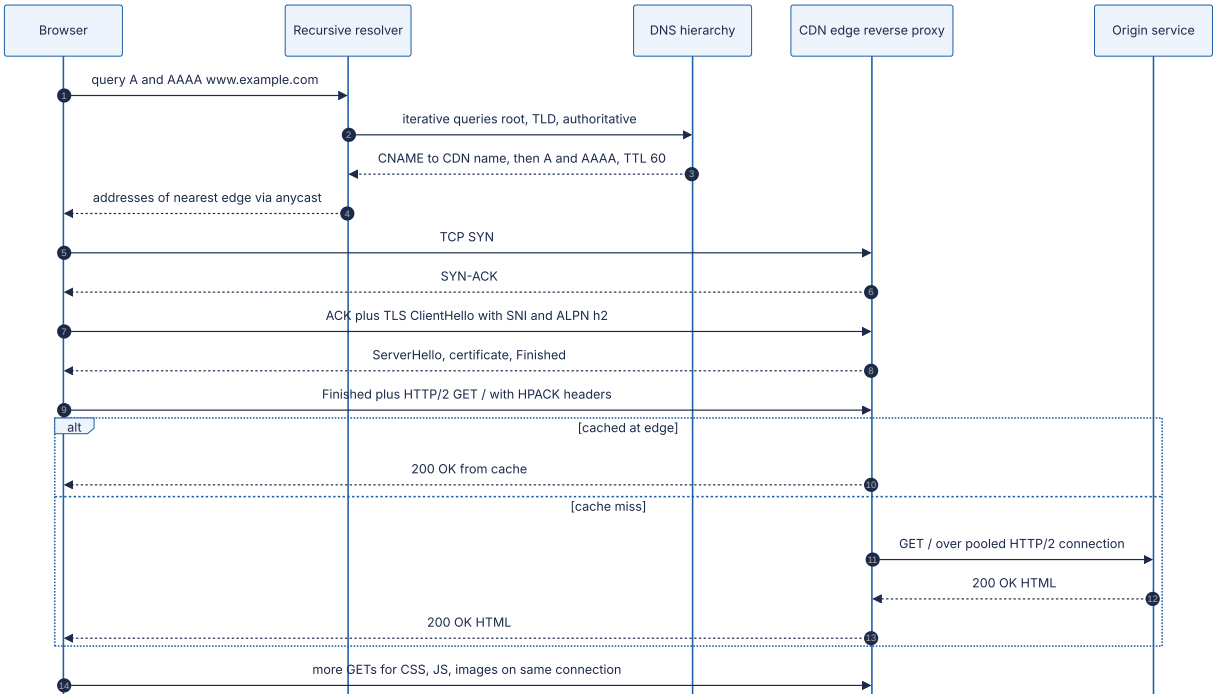


Figure 7.13 — Loading https://www.example.com/ end to end.



Chapter 8 — Web Application Fundamentals

Figure 8.1 — Three-tier client-server architecture.

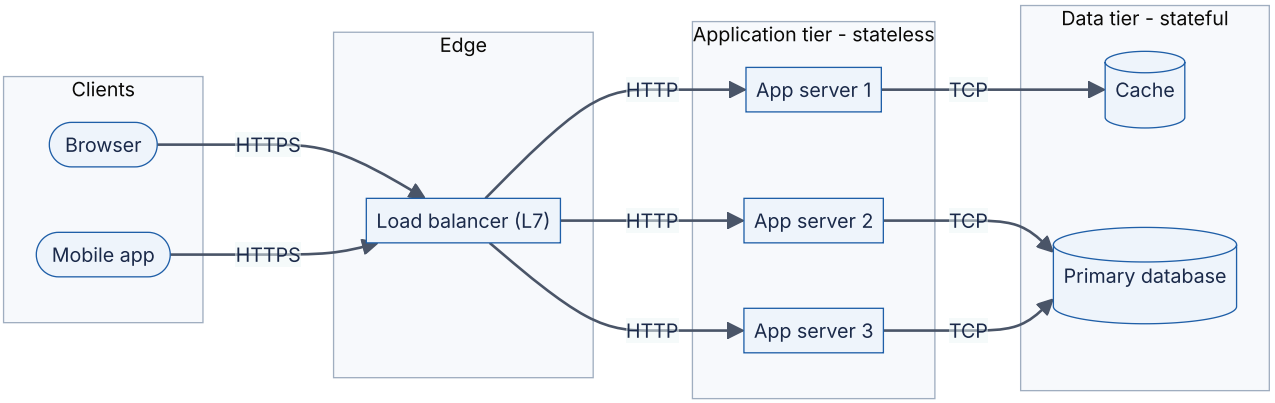


Figure 8.2 — Backend-for-frontend aggregation.

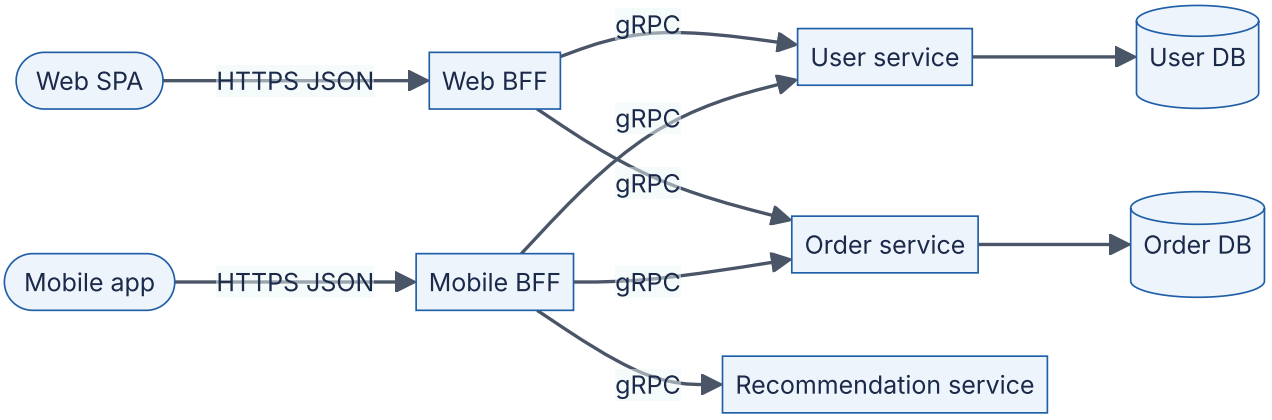


Figure 8.3 — End-to-end lifecycle of one HTTPS API request.

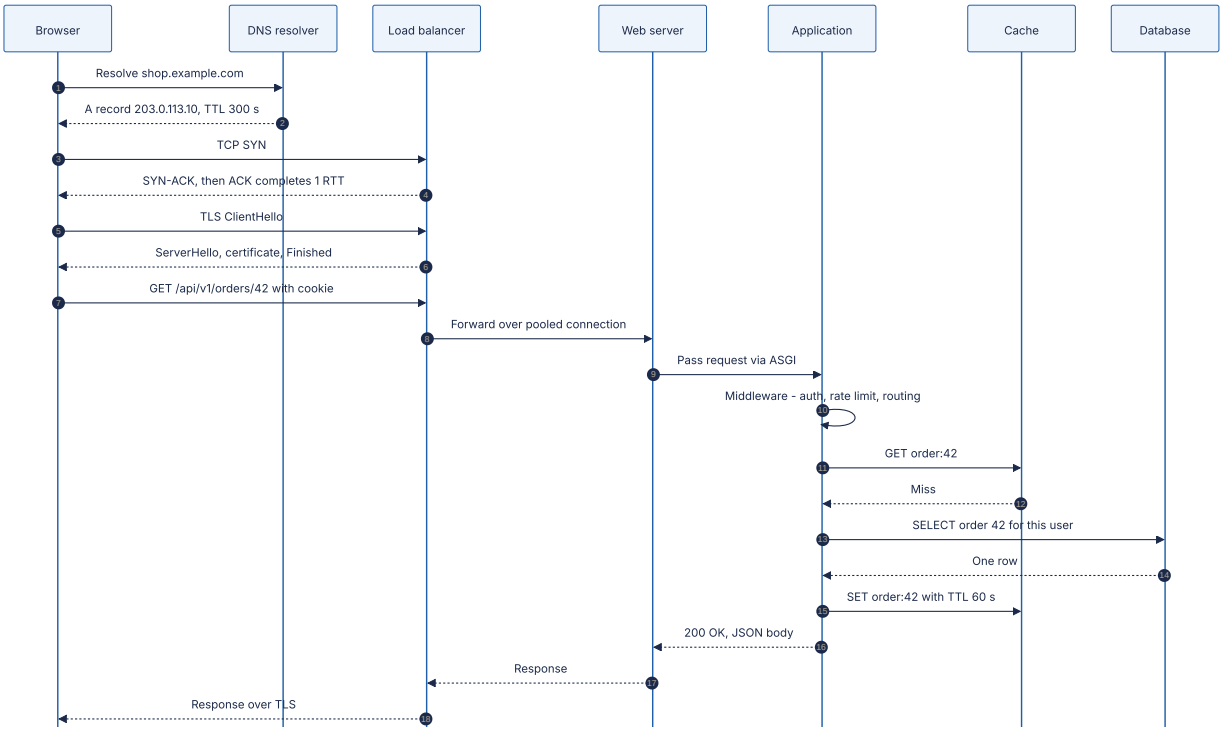


Figure 8.4 — N+1 resolution versus DataLoader batching.

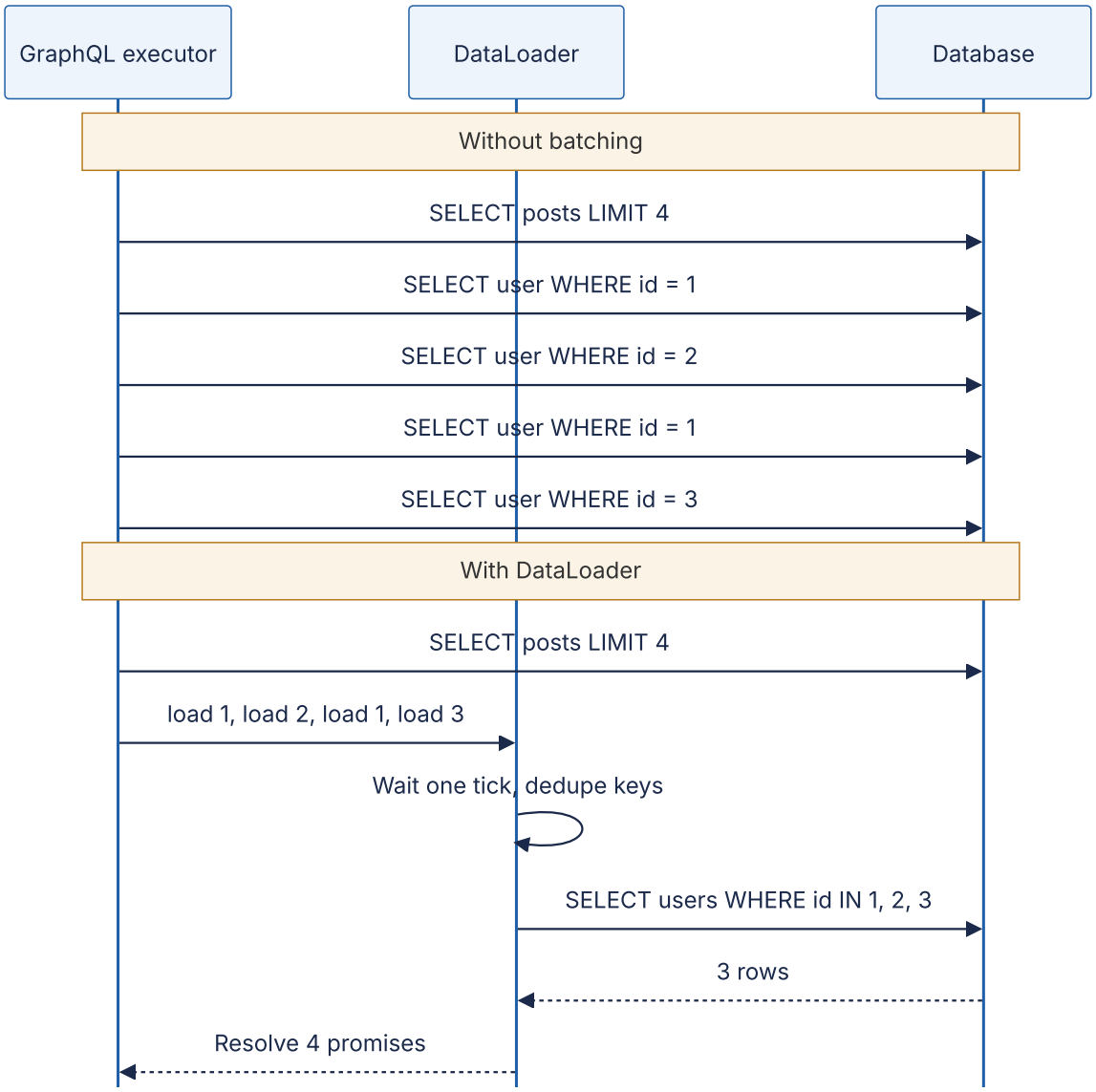


Figure 8.5 — gRPC call mapped onto an HTTP/2 connection.

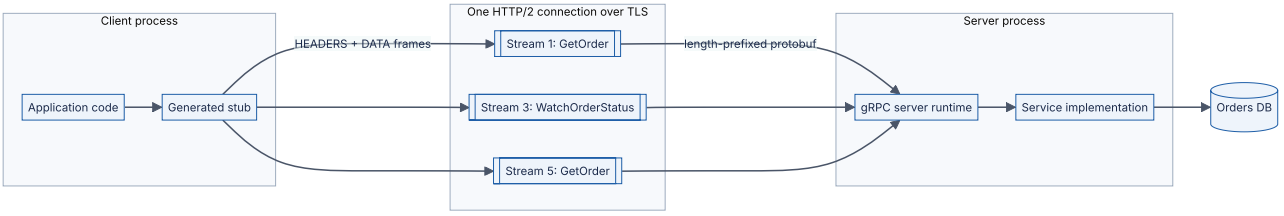


Figure 8.6 — Deadline propagation through a call chain.

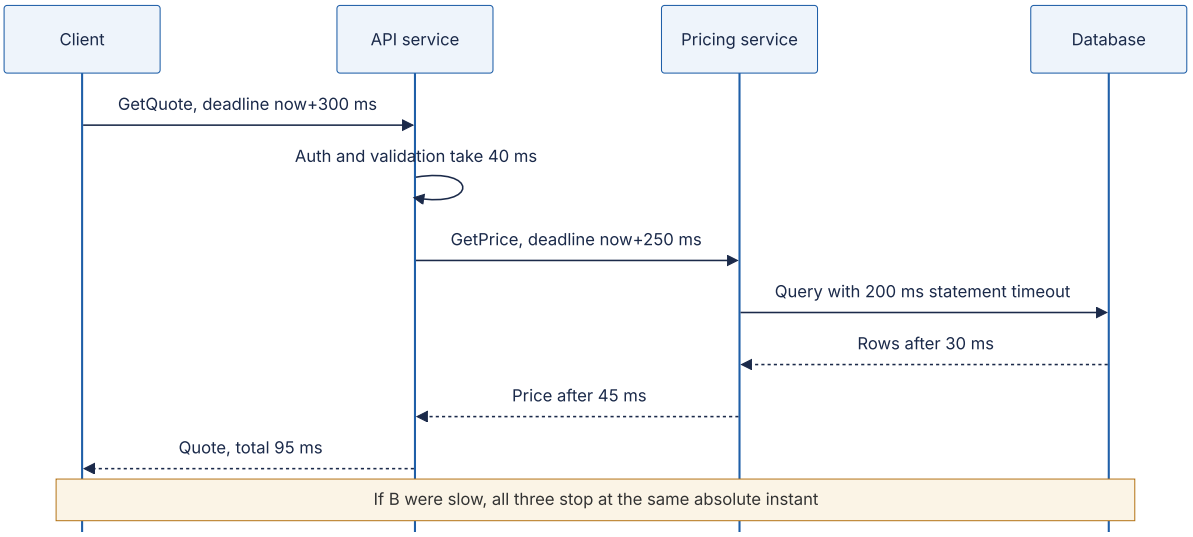


Figure 8.7 — Mixed versions during a rolling deployment.

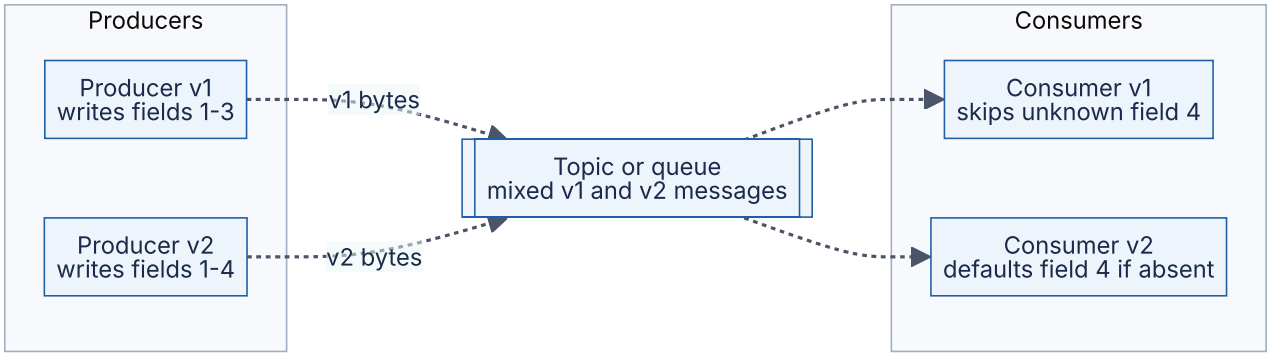


Figure 8.8 — Server-side session lookup.

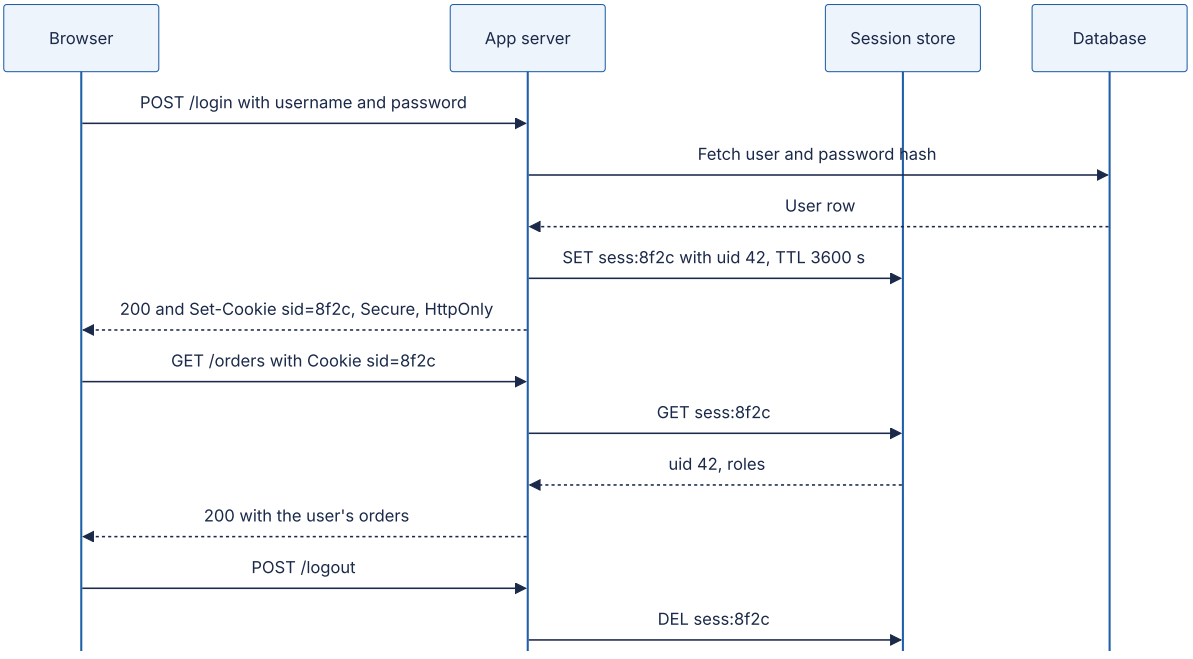


Figure 8.9 — Relationship graph for a Zanzibar-style check.

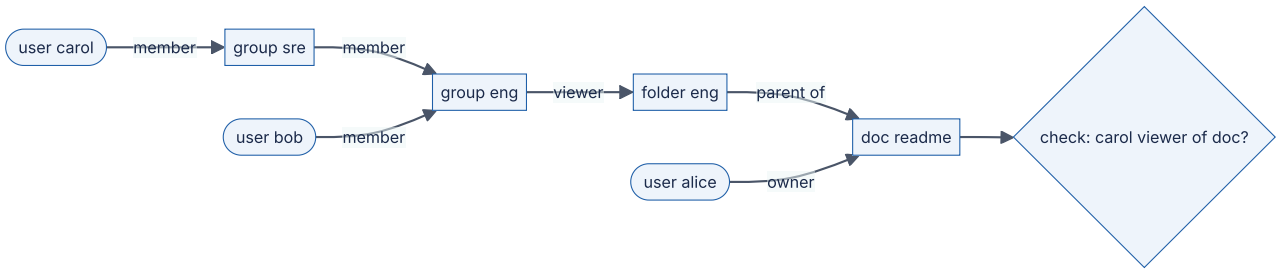
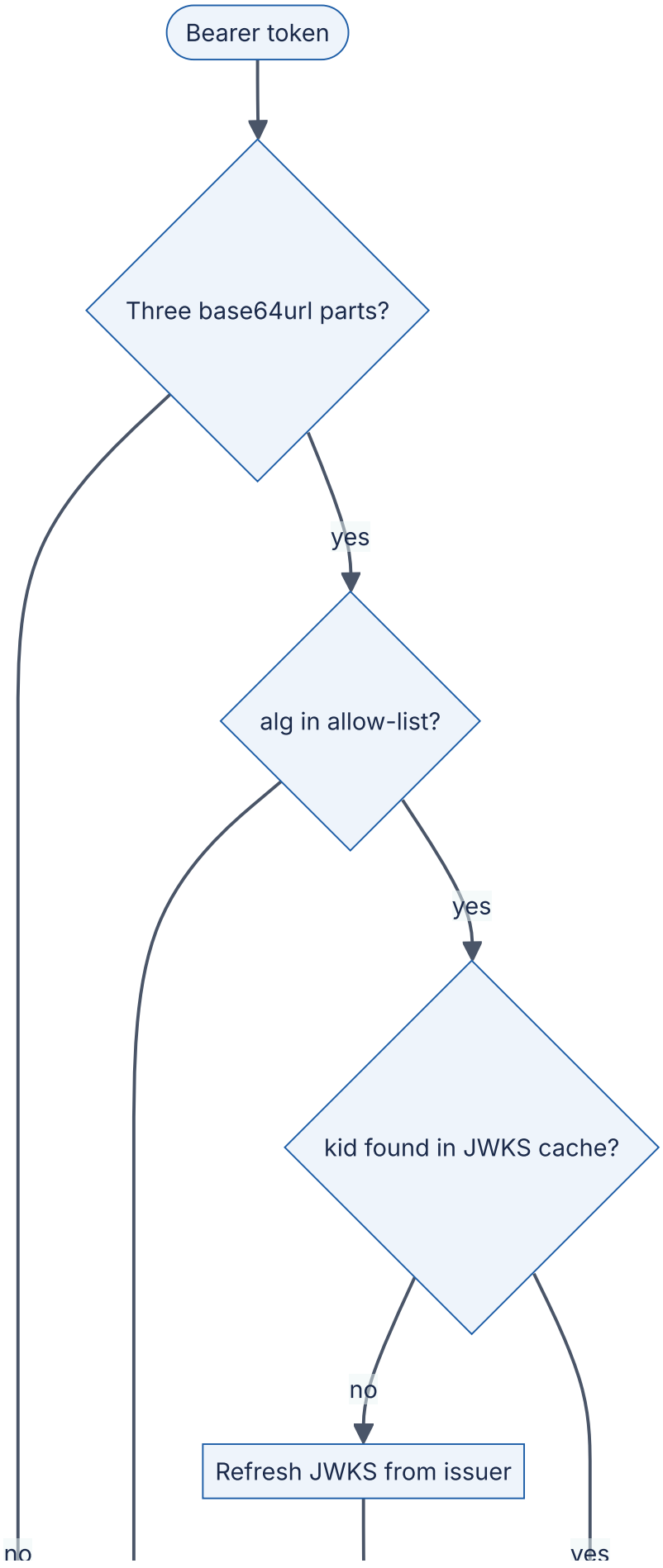


Figure 8.10 — JWT validation pipeline.


Bearer token
Three base64url parts?
yes
alg in allow-list?
yes
kid found in JWKS cache?
no
Refresh JWKS from issuer
no
yes

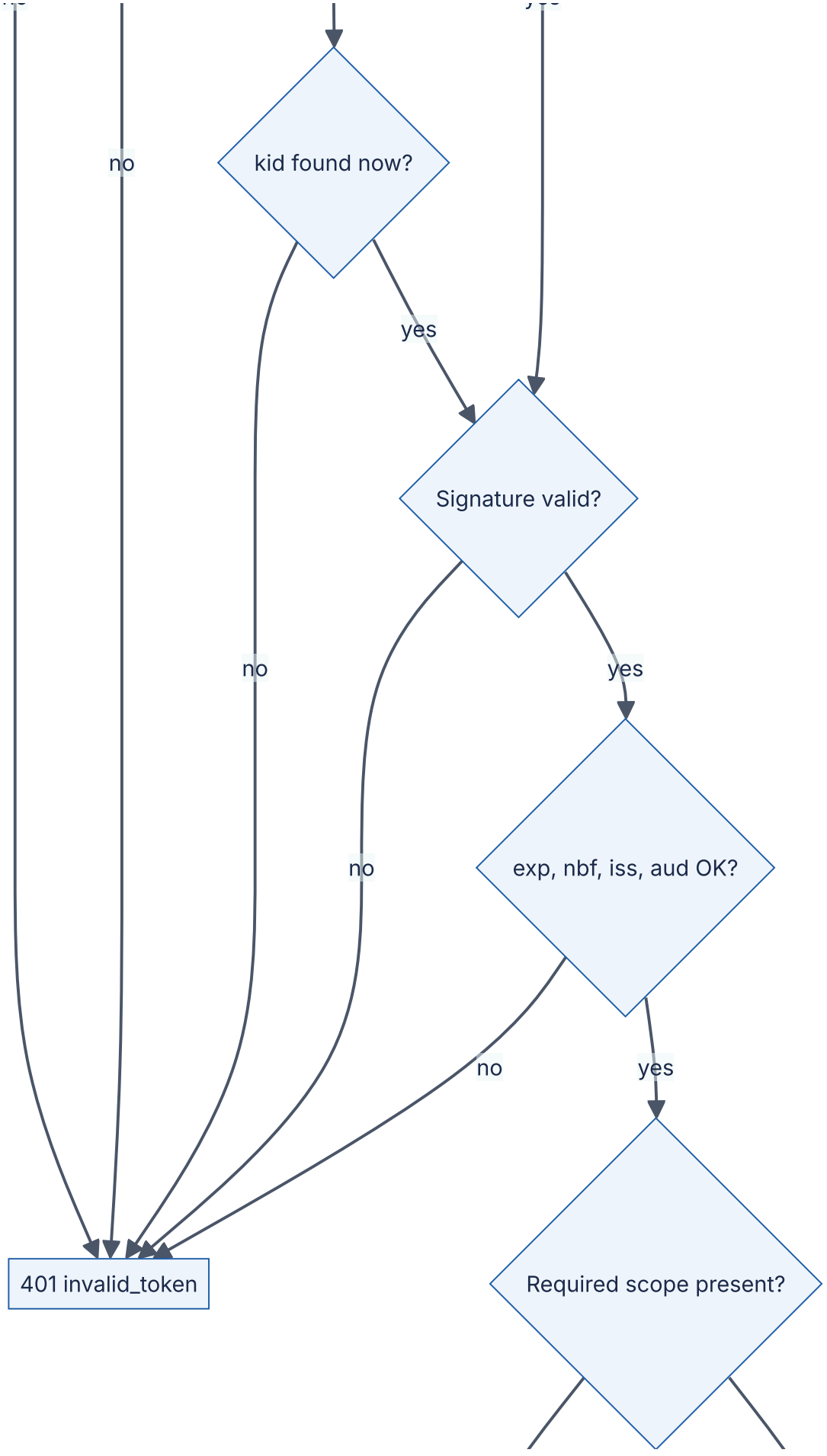

no
kid found now?
yes
Signature valid?
no
yes
no
exp, nbf, iss, aud OK?
no
yes
401 invalid_token
Required scope present?



Figure 8.11 — OAuth 2.0 authorization code flow with PKCE.

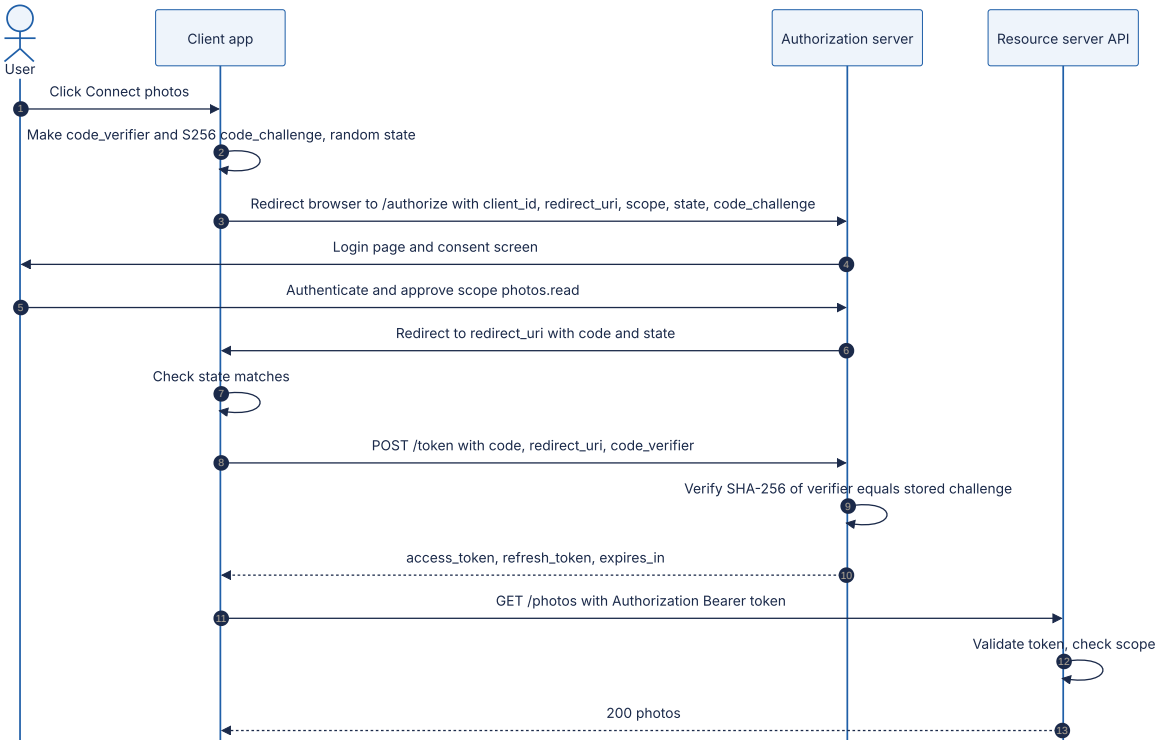
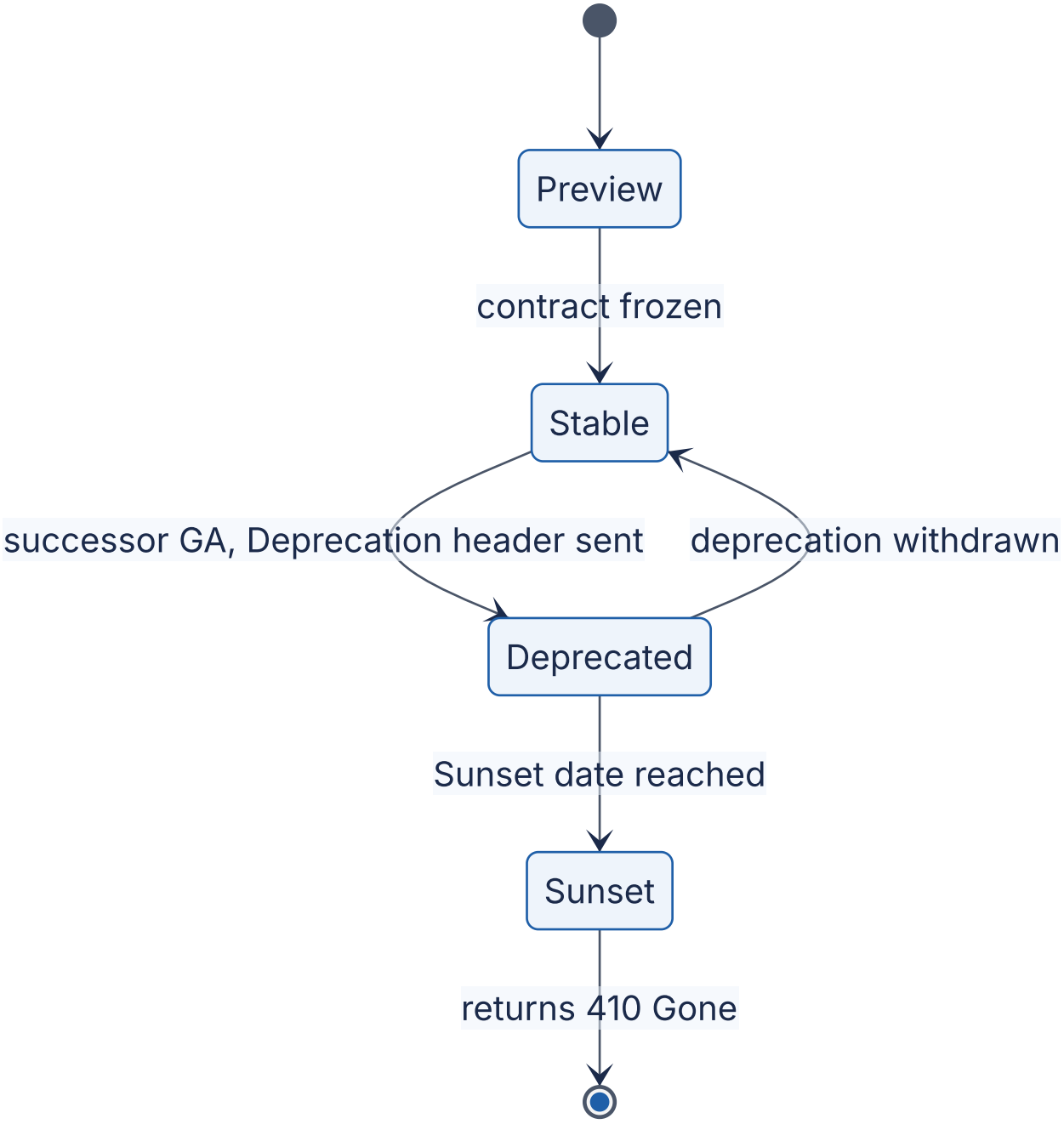


Figure 8.12 — Lifecycle of an API version.



Chapter 9 — Real-Time Communication

Figure 9.1 — Short polling.

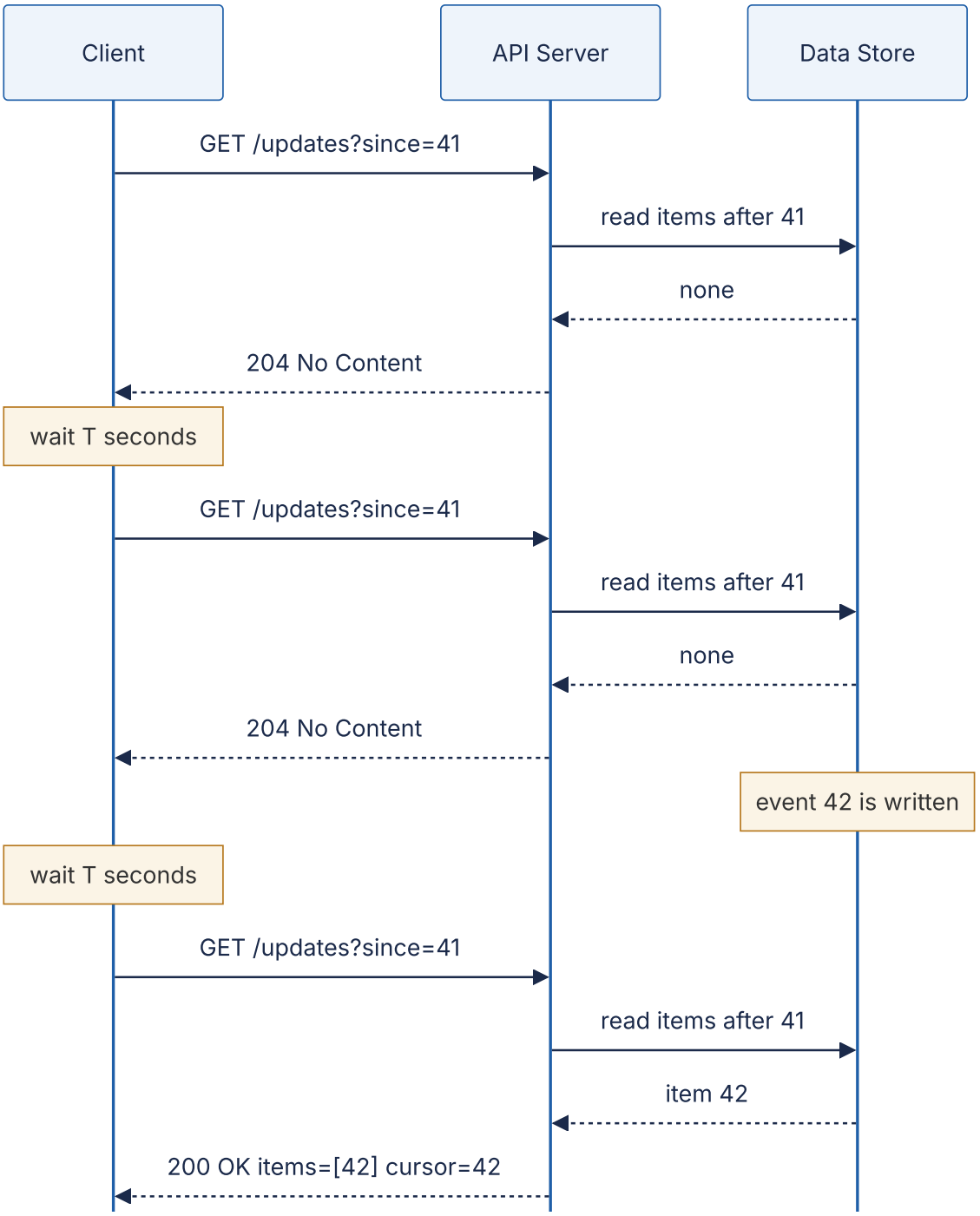


Figure 9.2 — Long polling with cursors and a hold timeout.

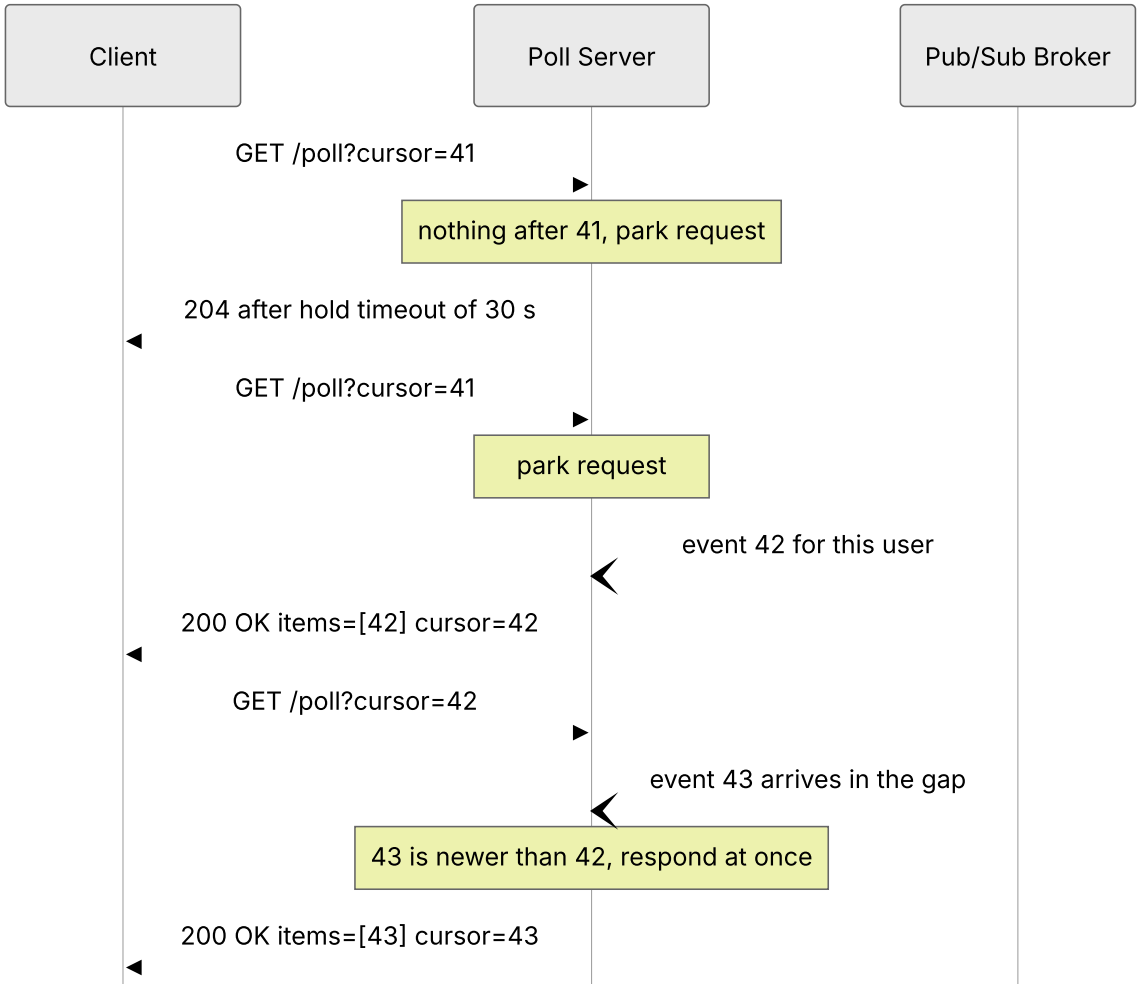


Figure 9.3 — SSE streaming, disconnection and resumption with `Last-Event-ID`.

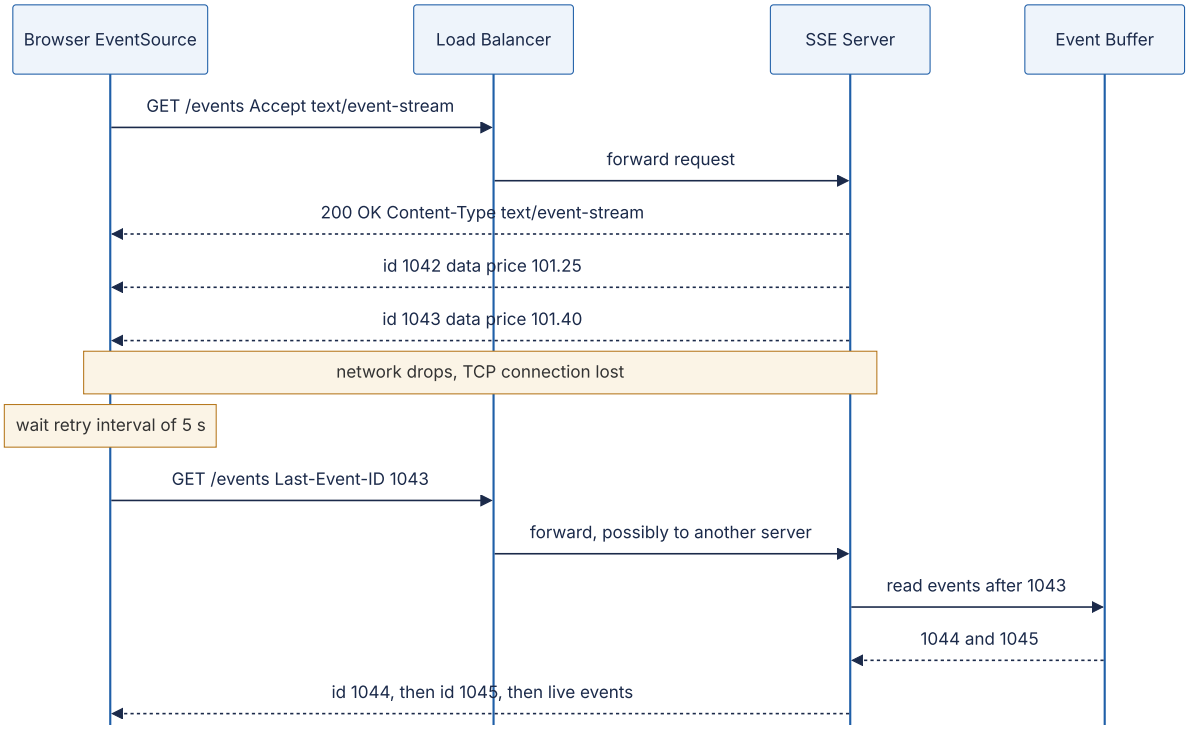


Figure 9.4 — WebSocket opening handshake, message exchange, heartbeat and closing handshake.

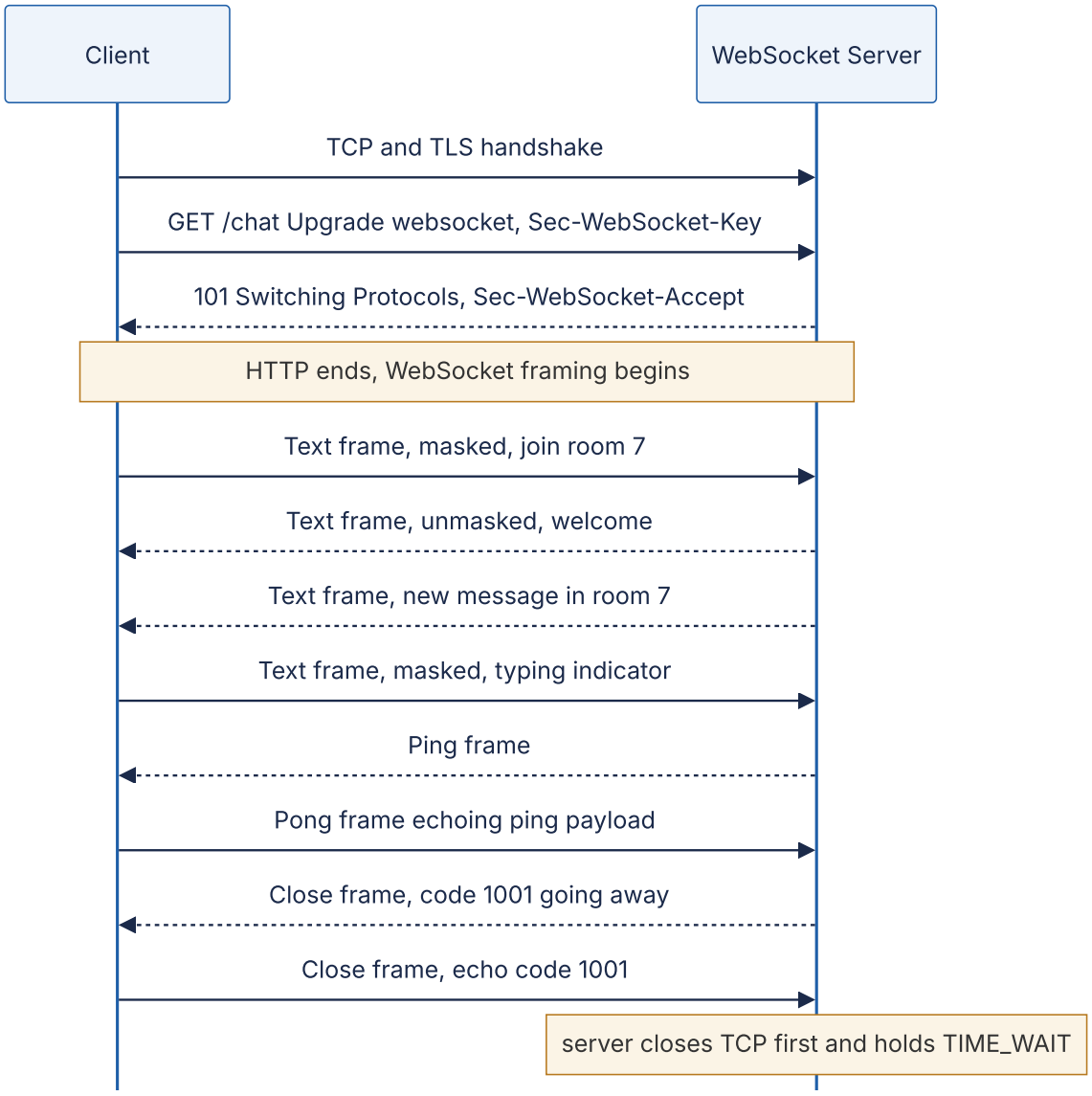


Figure 9.5 — Lifecycle of a WebSocket connection as seen by a gateway.

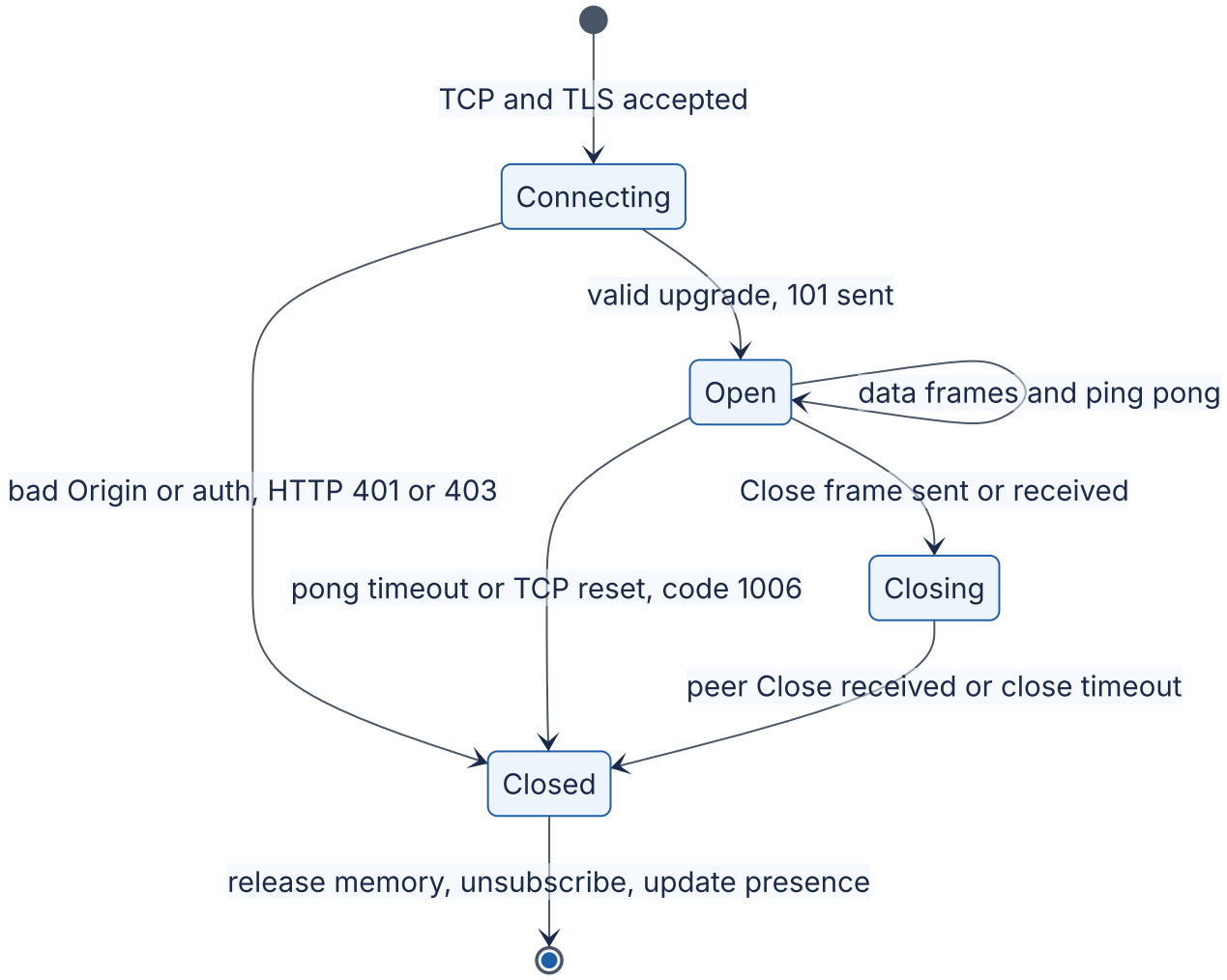


Figure 9.6 — WebRTC session establishment: signaling, STUN, ICE and TURN fallback.

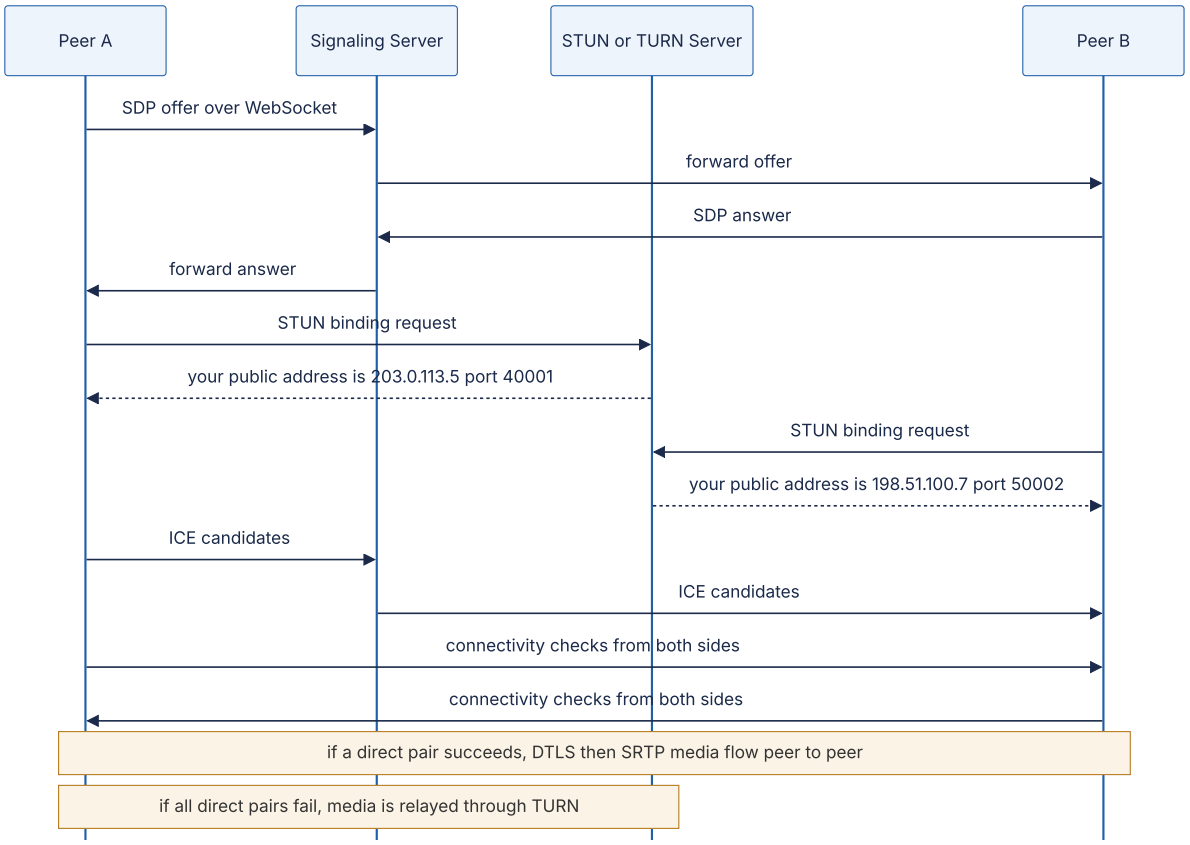
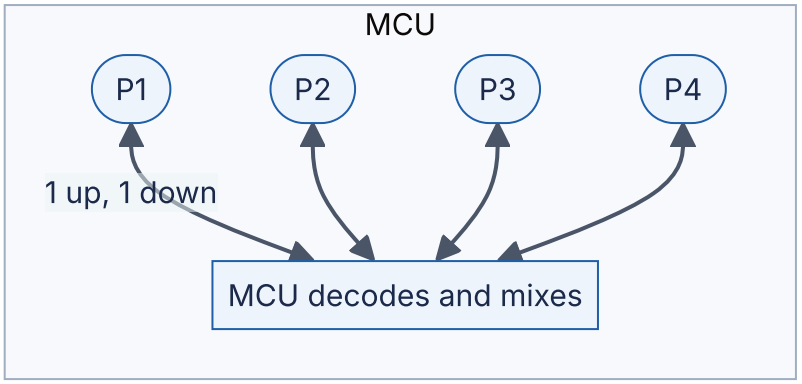
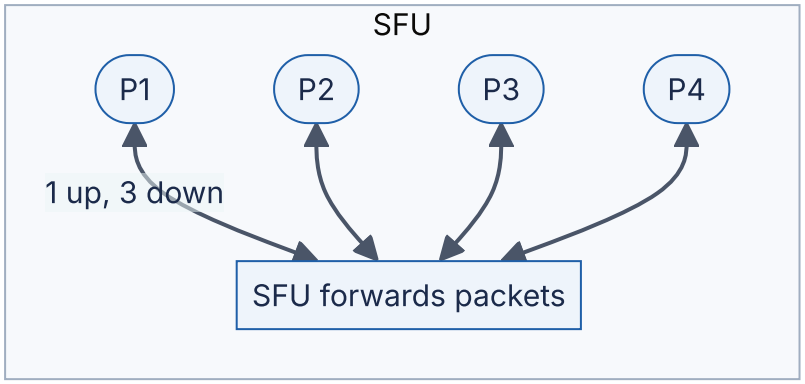


Figure 9.7 — Mesh, SFU and MCU topologies for a four-person call.





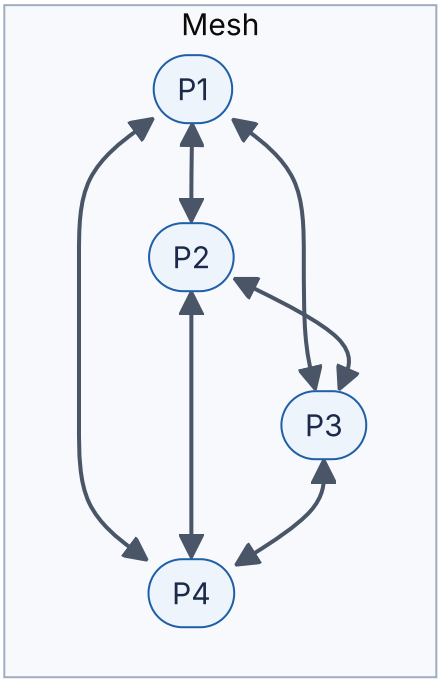


Figure 9.8 — Mobile push architecture.

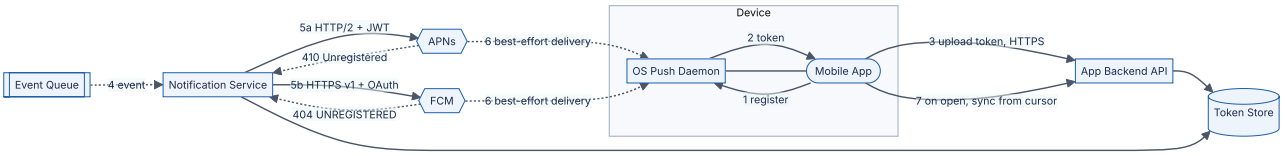


Figure 9.9 — Architecture of a connection gateway fleet.

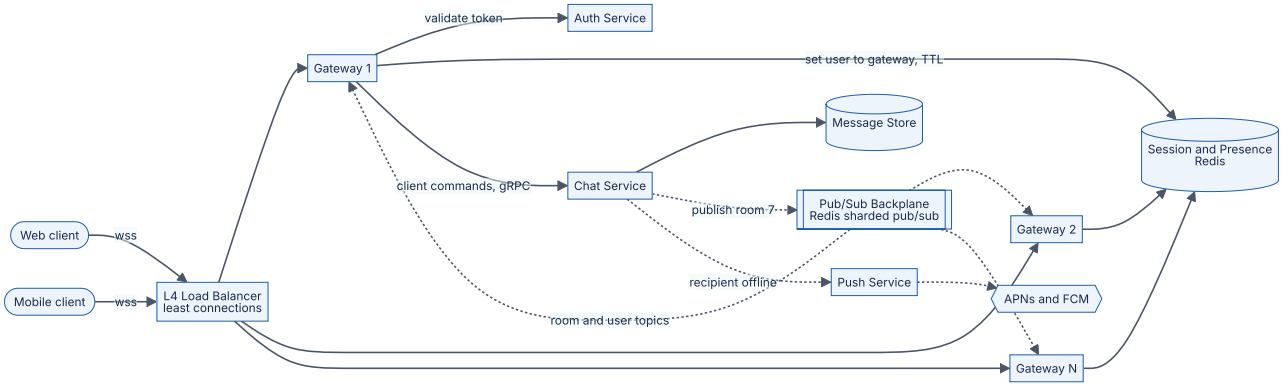
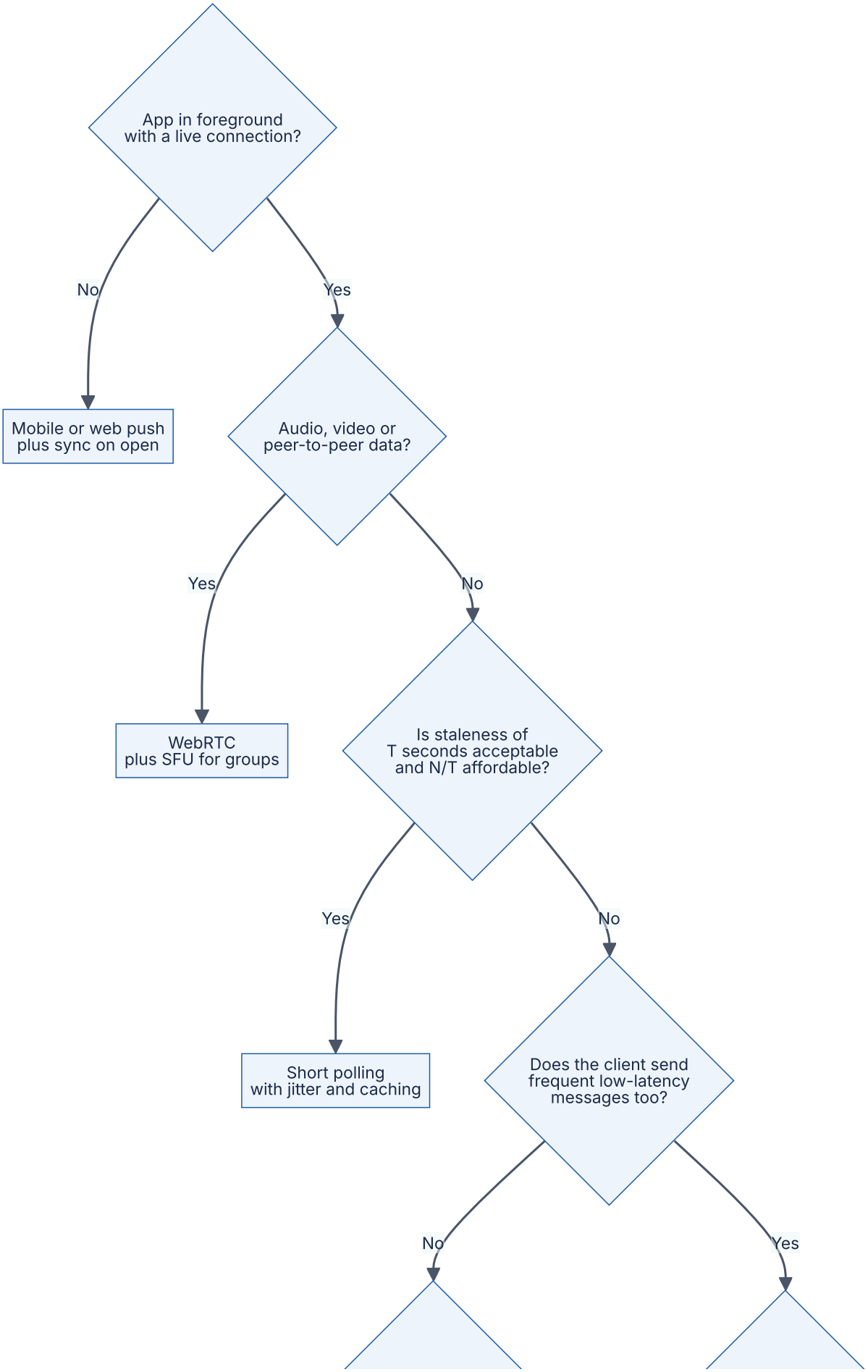
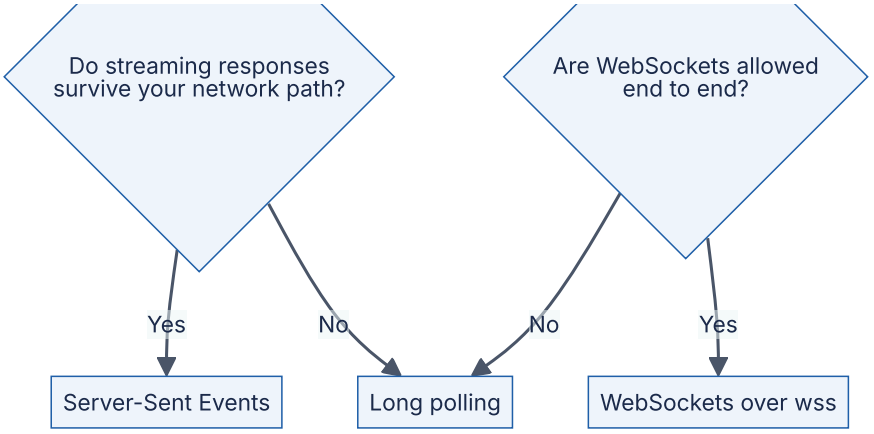


Figure 9.10 — Decision flowchart for choosing a real-time technique.


App in foreground with a live connection?
No
Yes
Mobile or web push plus sync on open
Audio, video or peer-to-peer data?
Yes
No
WebRTC plus SFU for groups
Is staleness of T seconds acceptable and N/T affordable?
Yes
No
Short polling with jitter and caching
Does the client send frequent low-latency messages too?
No
Yes



Chapter 10 — Security Fundamentals for System Designers

Figure 10.1 — Data-flow diagram of a photo-sharing web application with trust boundaries.

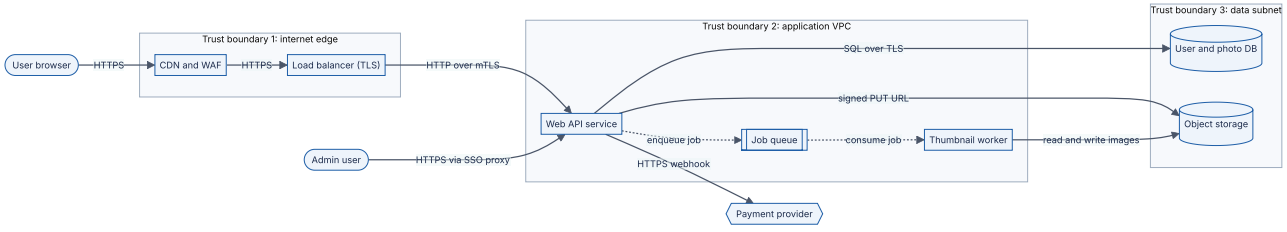


Figure 10.2 — Simplified TLS 1.3 handshake: ephemeral key exchange plus signature.

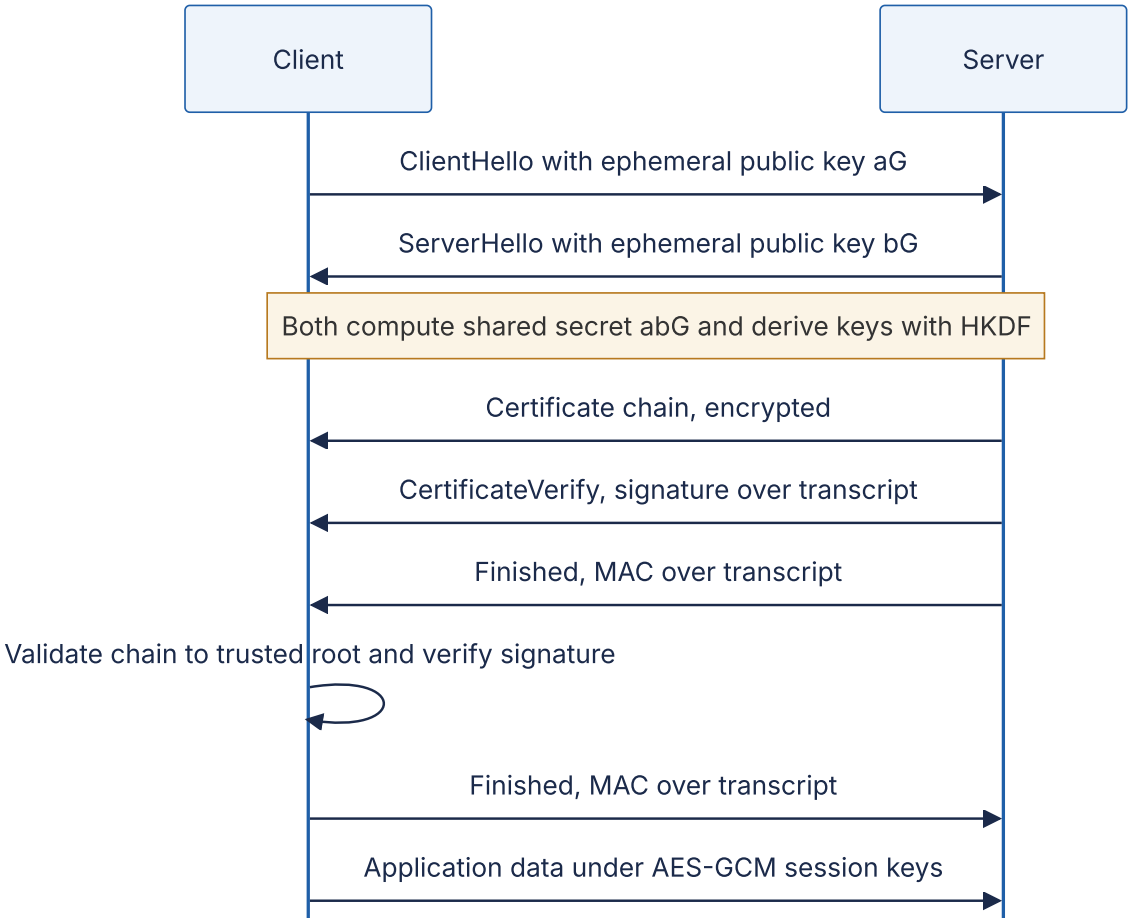


Figure 10.3 — HMAC request signing with replay protection.

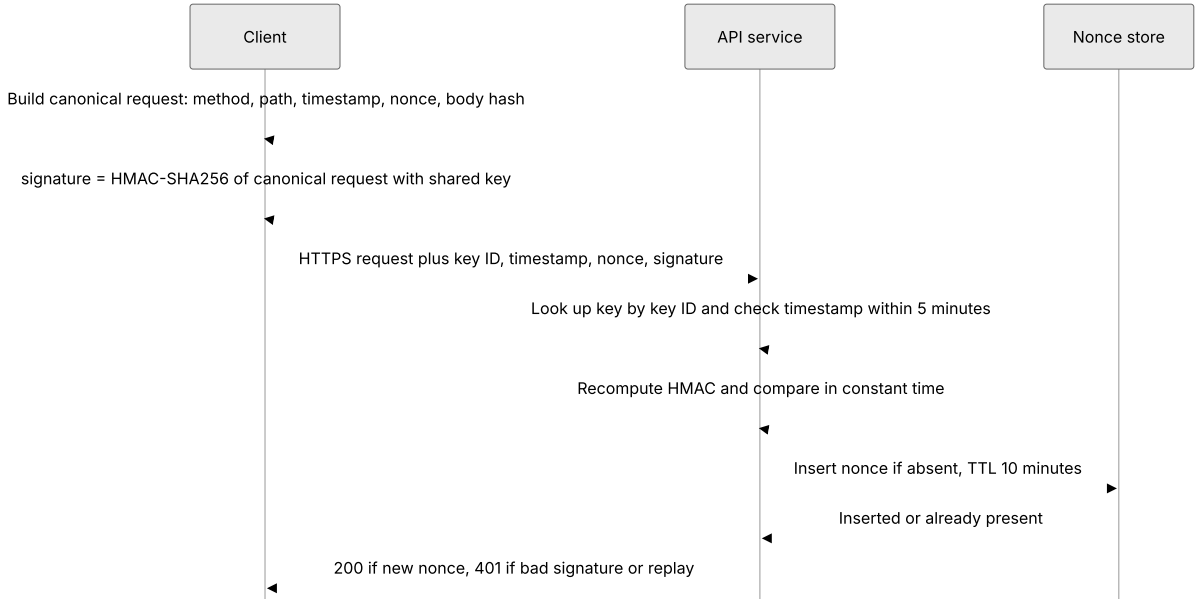


Figure 10.4 — Certificate chain of trust and Certificate Transparency.

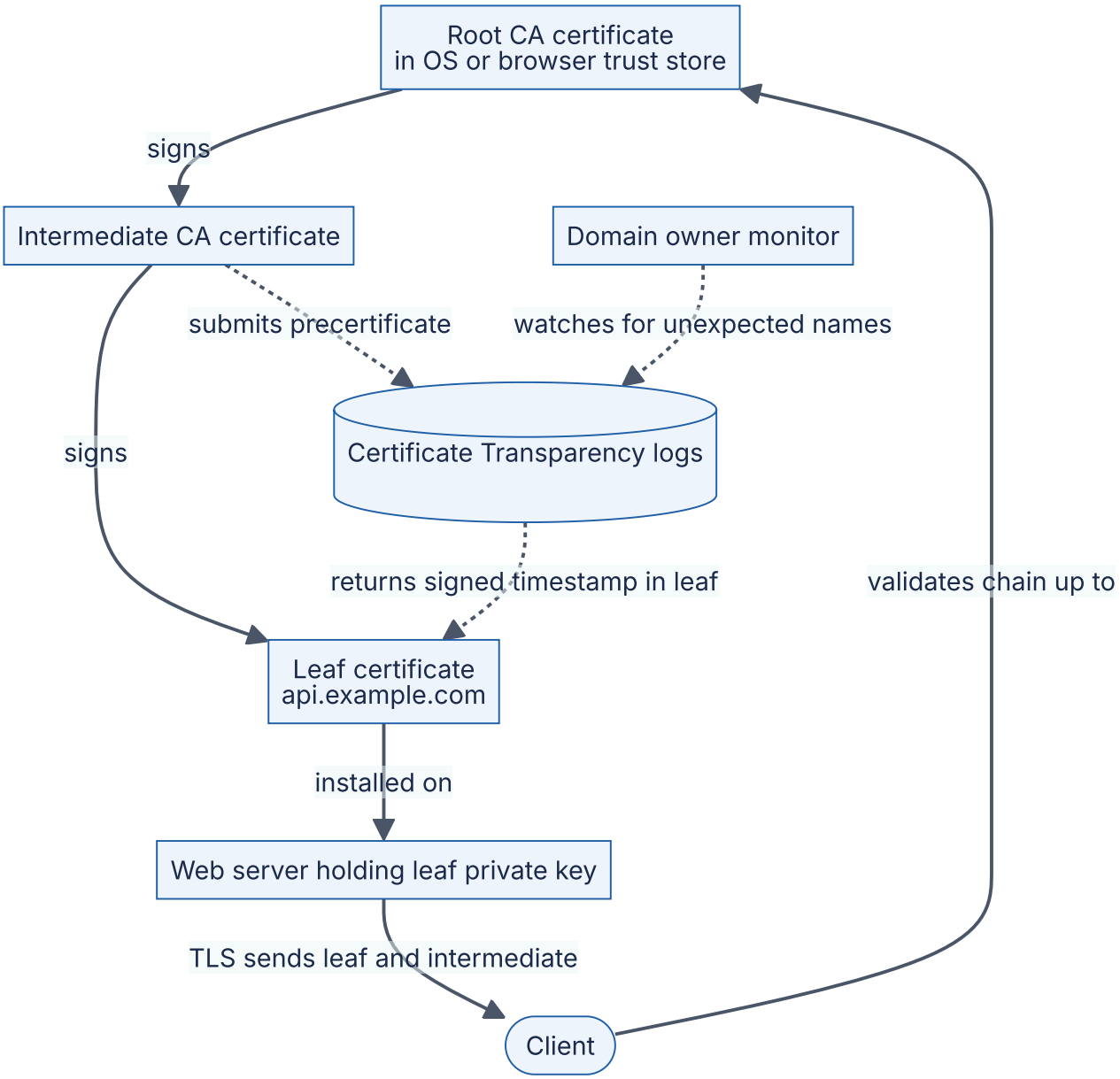


Figure 10.5 — Registration and login with salted Argon2id and a pepper.

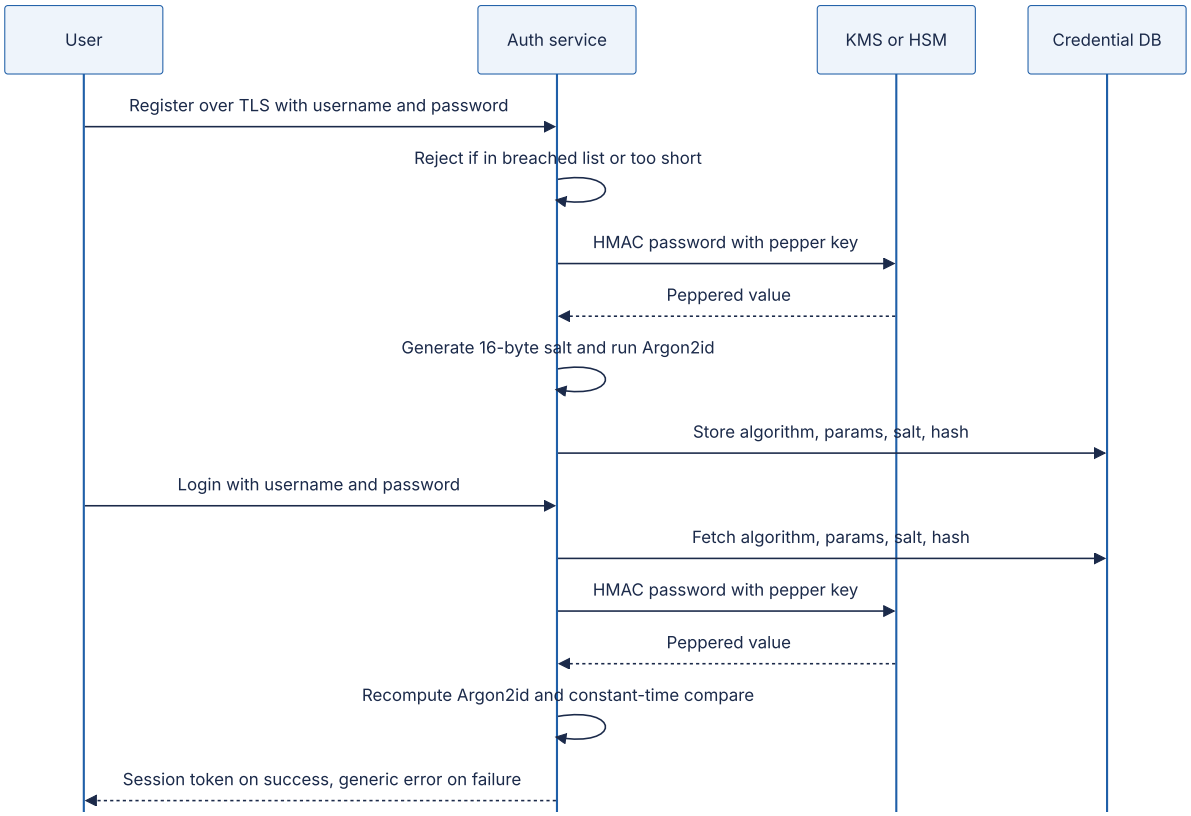


Figure 10.6 — A workload obtaining dynamic database credentials from a vault.

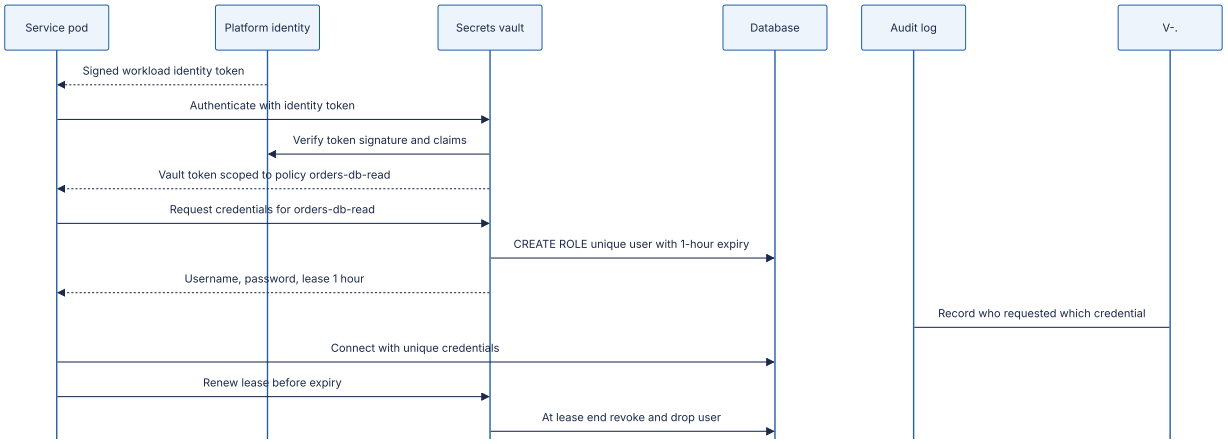


Figure 10.7 — Envelope encryption: write and read paths.

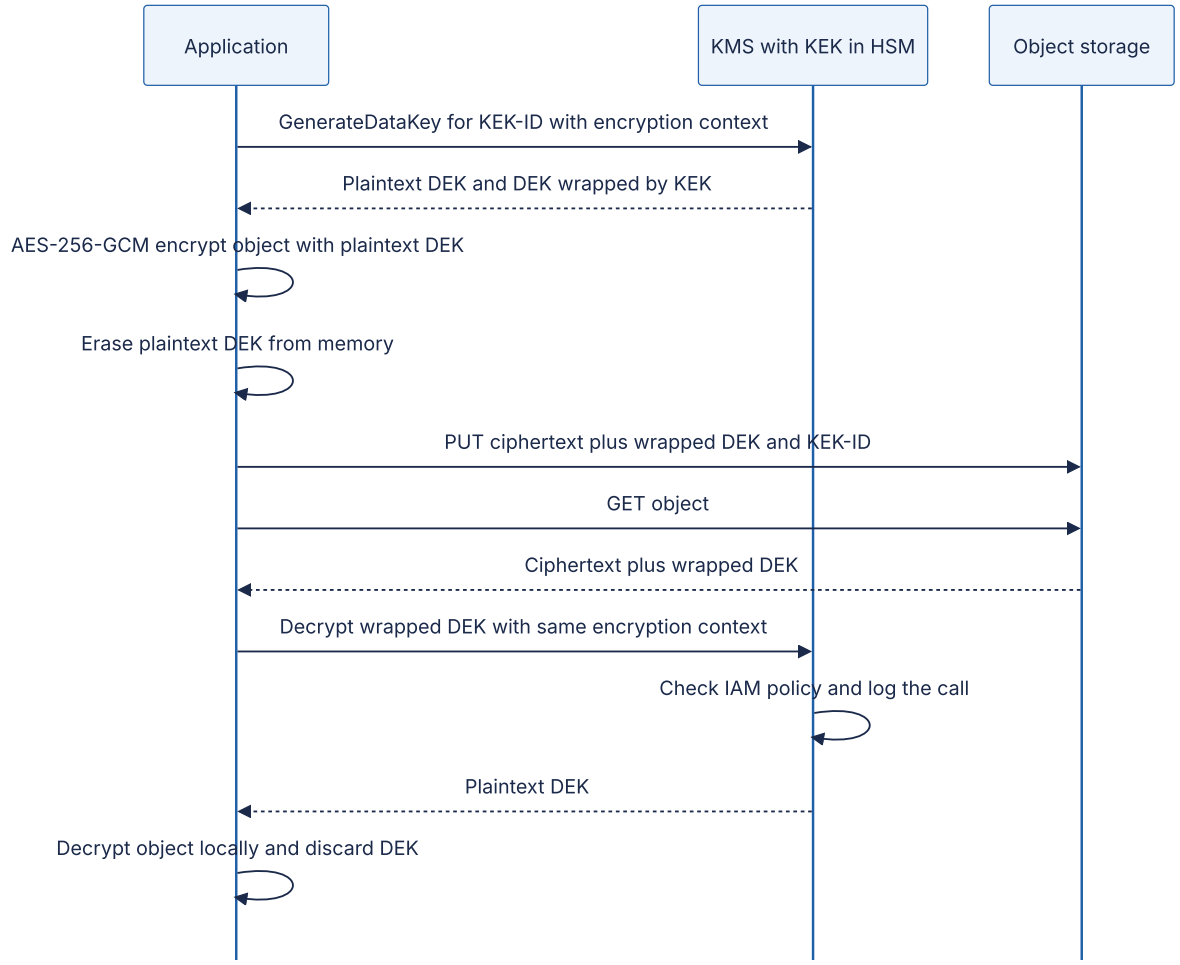


Figure 10.8 — Zero-trust access in the style of BeyondCorp.

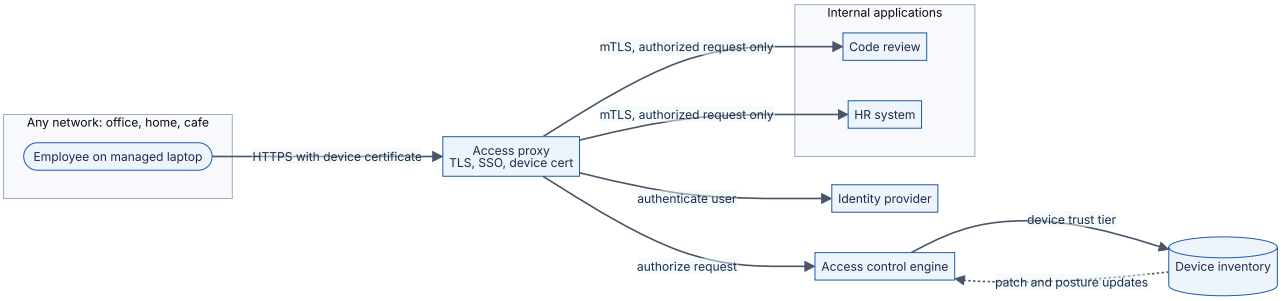


Figure 10.9 — DDoS mitigation in layers.

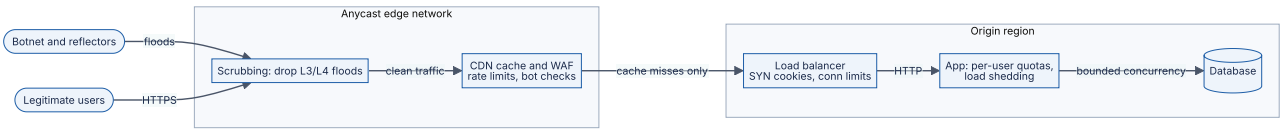
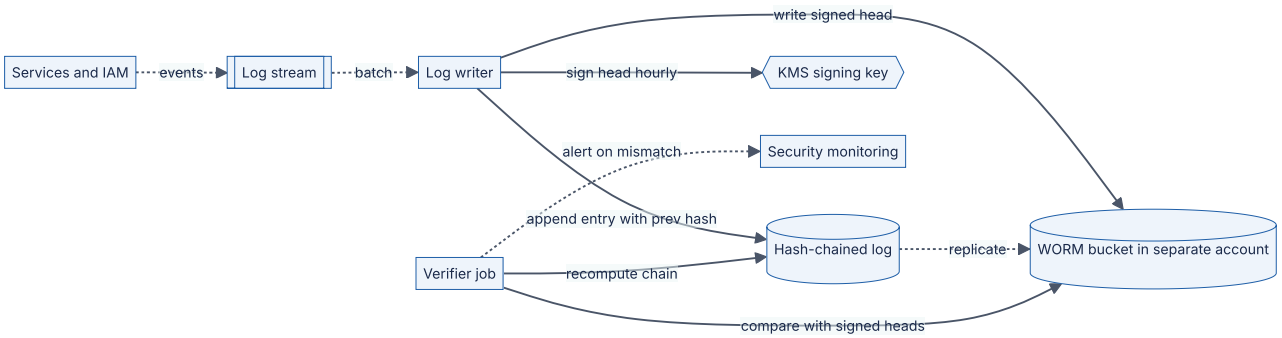


Figure 10.10 — Tamper-evident audit logging pipeline.



Chapter 11 — Relational Databases

Figure 11.1 — Decomposing the flat order export into BCNF tables.

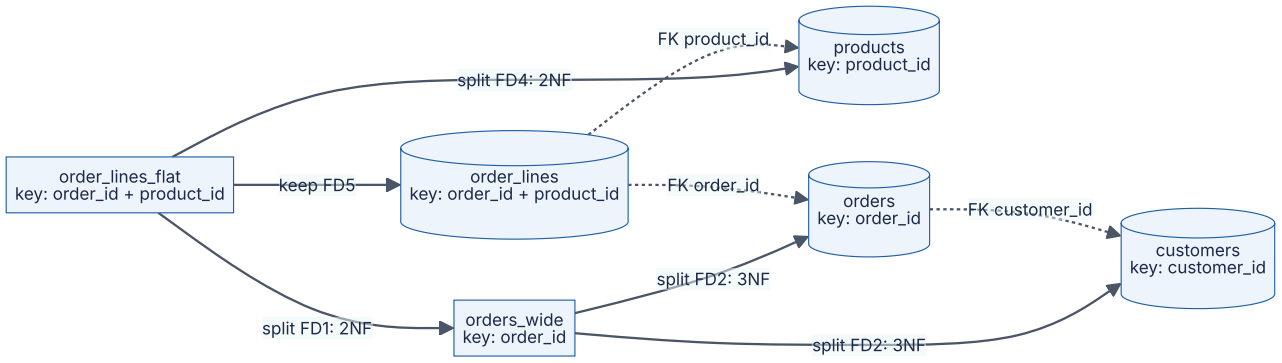


Figure 11.2 — Hash join: build on the smaller input, probe with the larger.

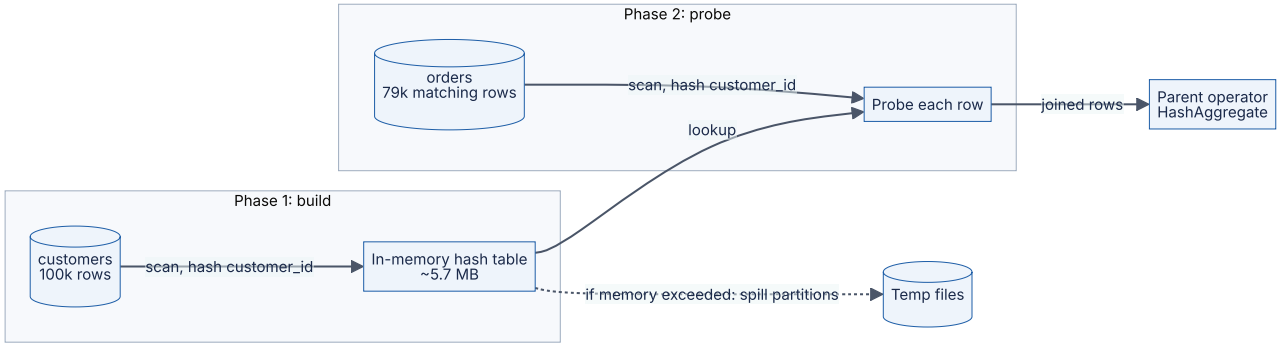


Figure 11.3 — Secondary-index lookup paths in InnoDB and PostgreSQL.

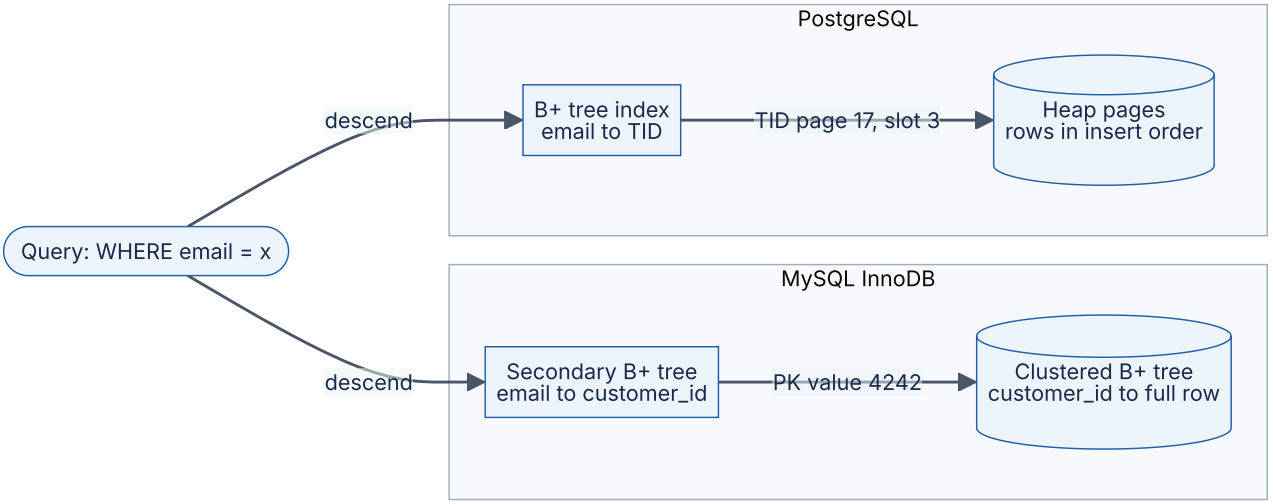


Figure 11.4 — A B+ tree on `order_id` with three levels.

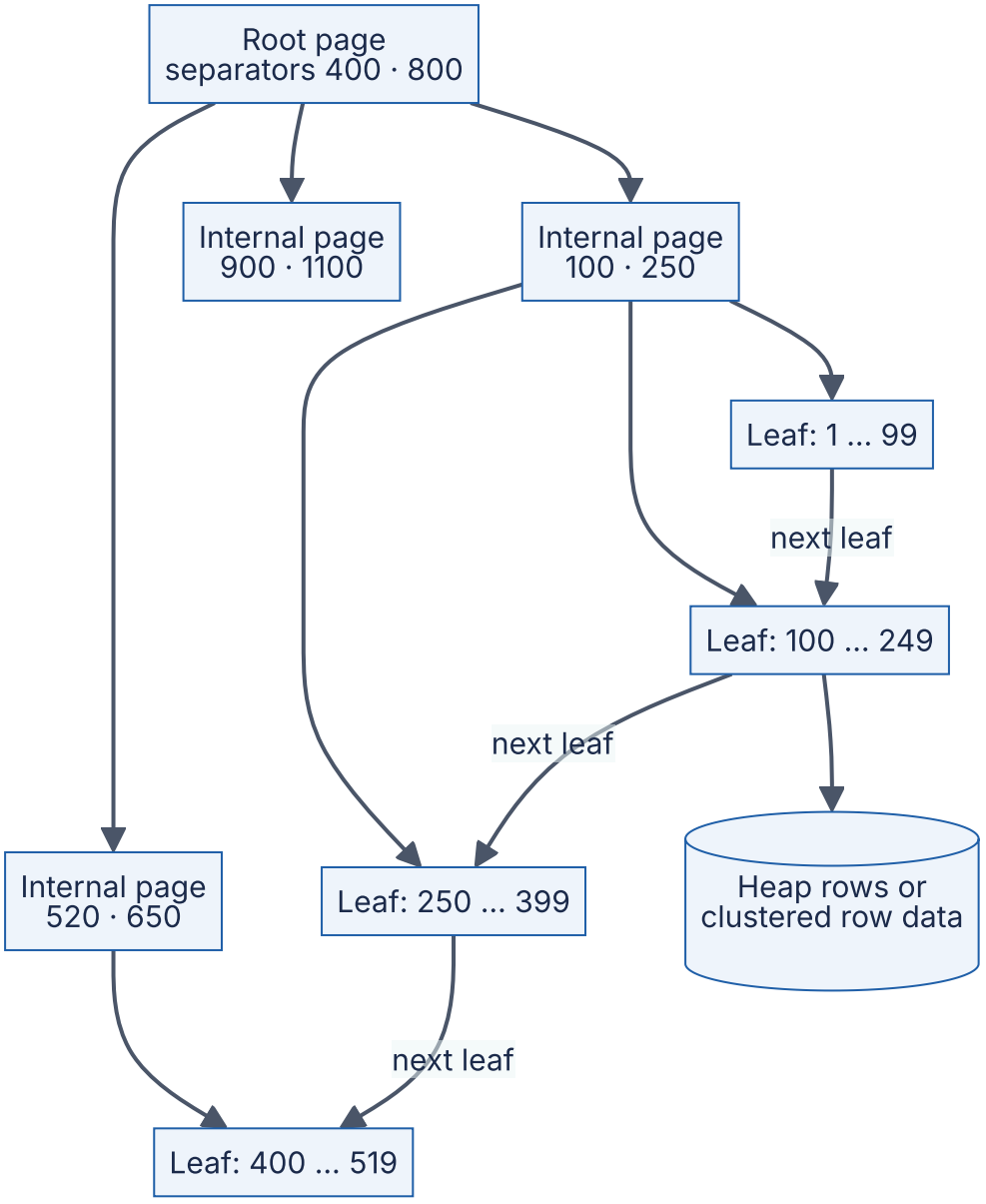


Figure 11.5 — The executed plan tree for “revenue by city over the last day”.

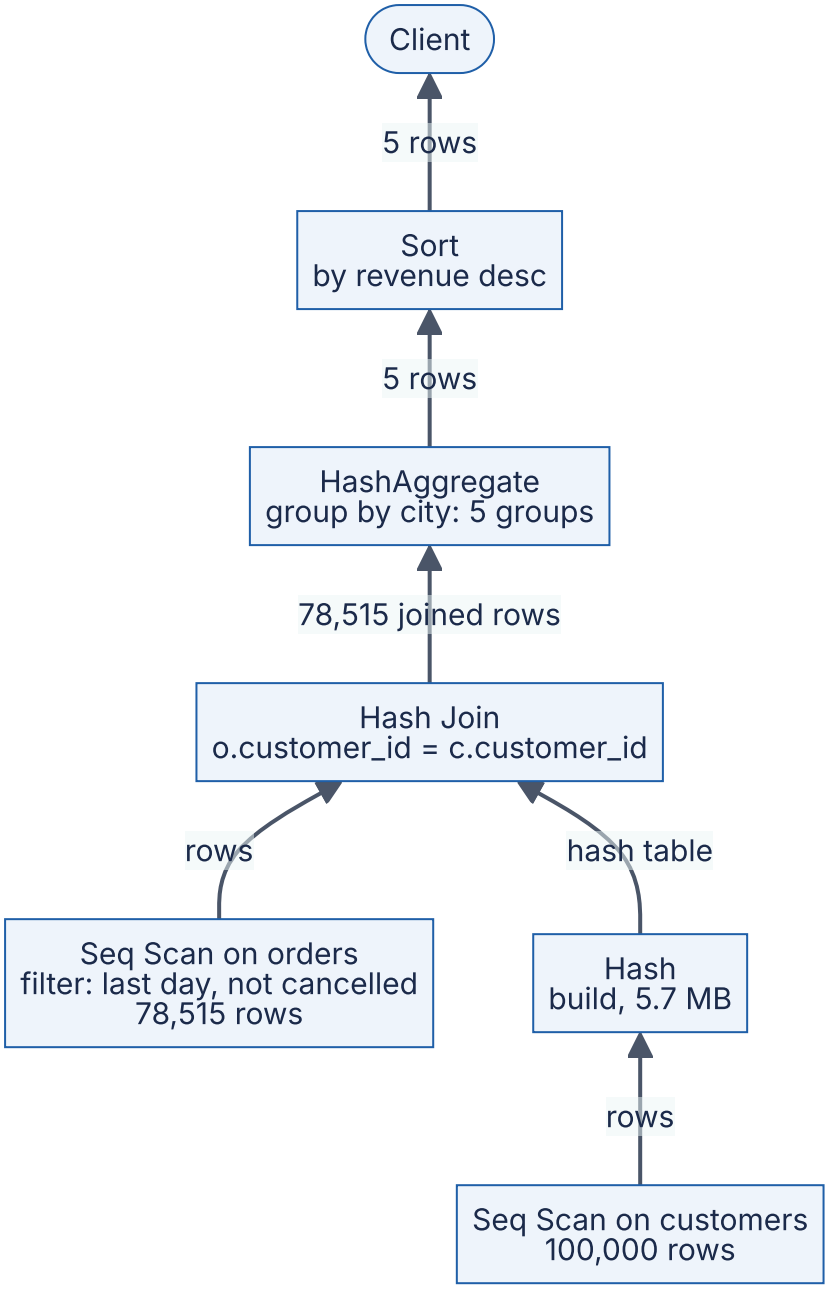


Figure 11.6 — Non-repeatable read under READ COMMITTED.

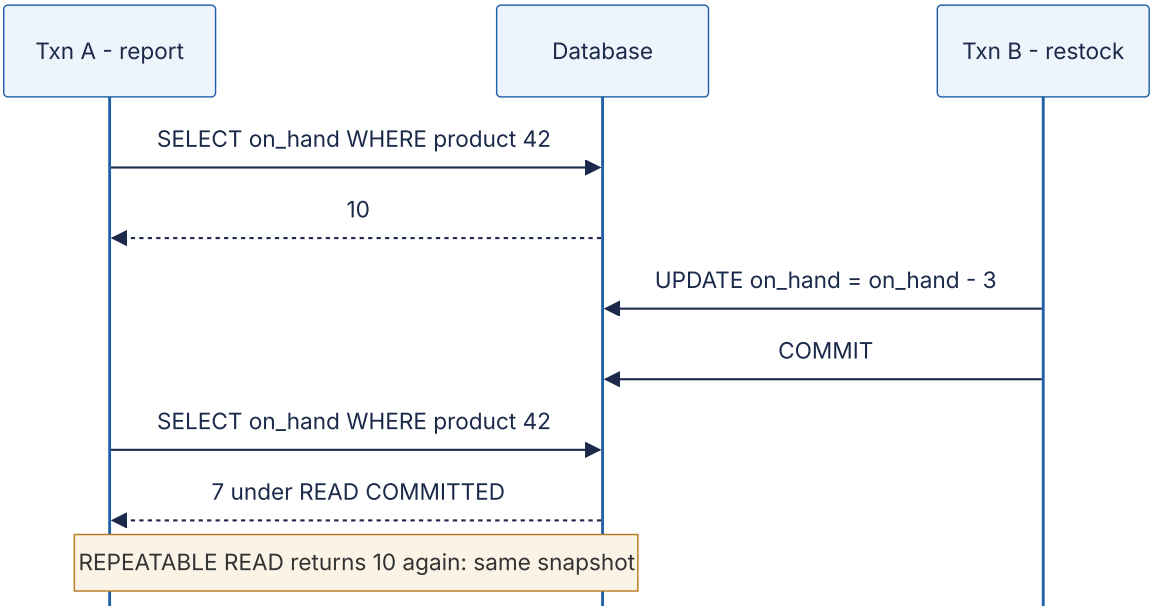


Figure 11.7 — Lost update from an application-side read-modify-write.

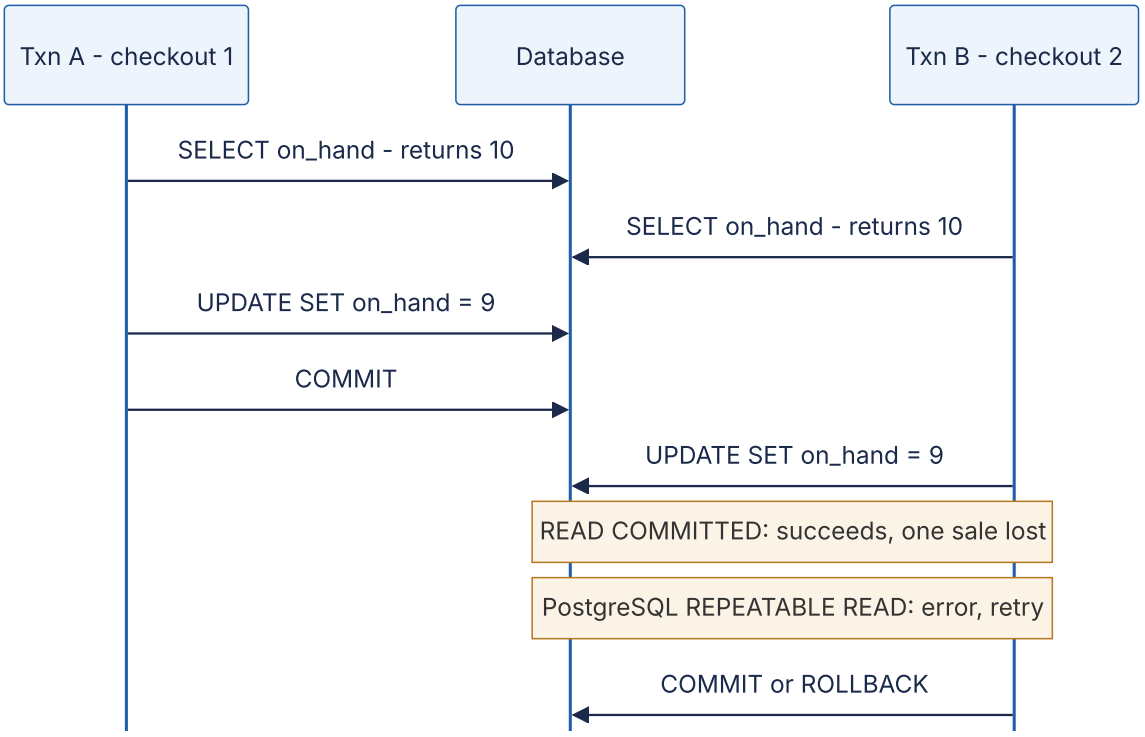


Figure 11.8 — Write skew: both doctors go off call.

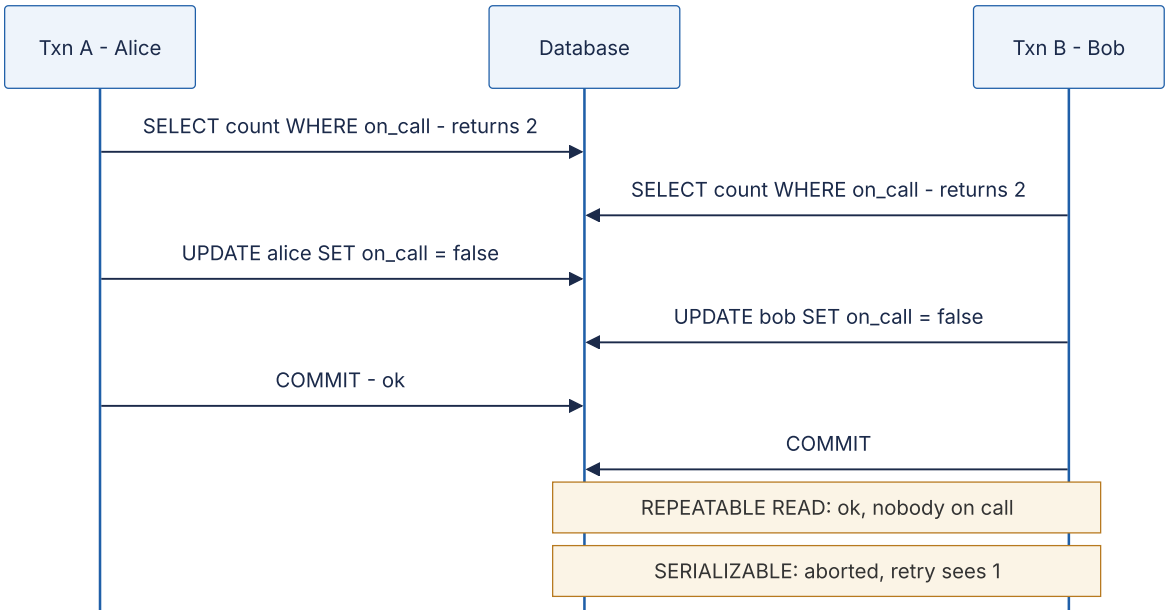


Figure 11.9 — PostgreSQL version chain for product 42.

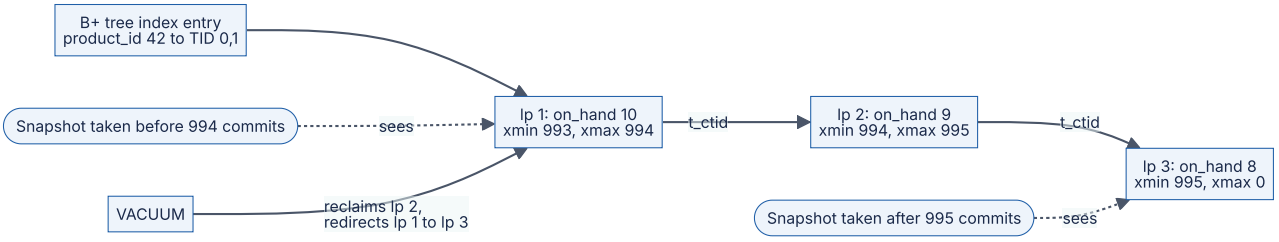


Figure 11.10 — InnoDB current row and undo chain for product 42.

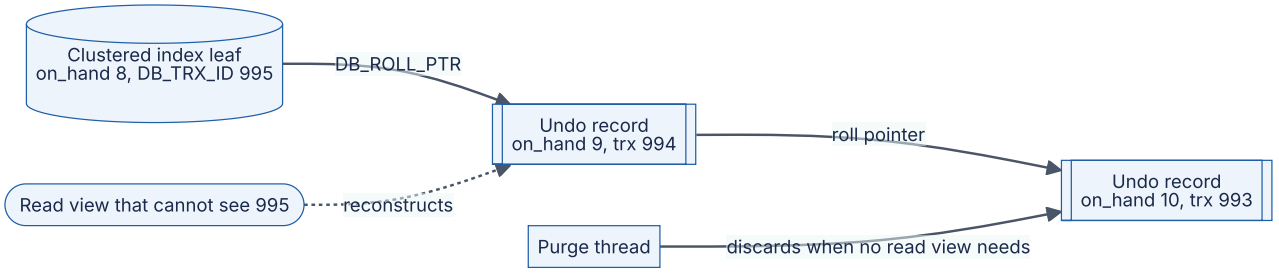


Figure 11.11 — A deadlock between two checkouts, as reproduced on PostgreSQL 16.

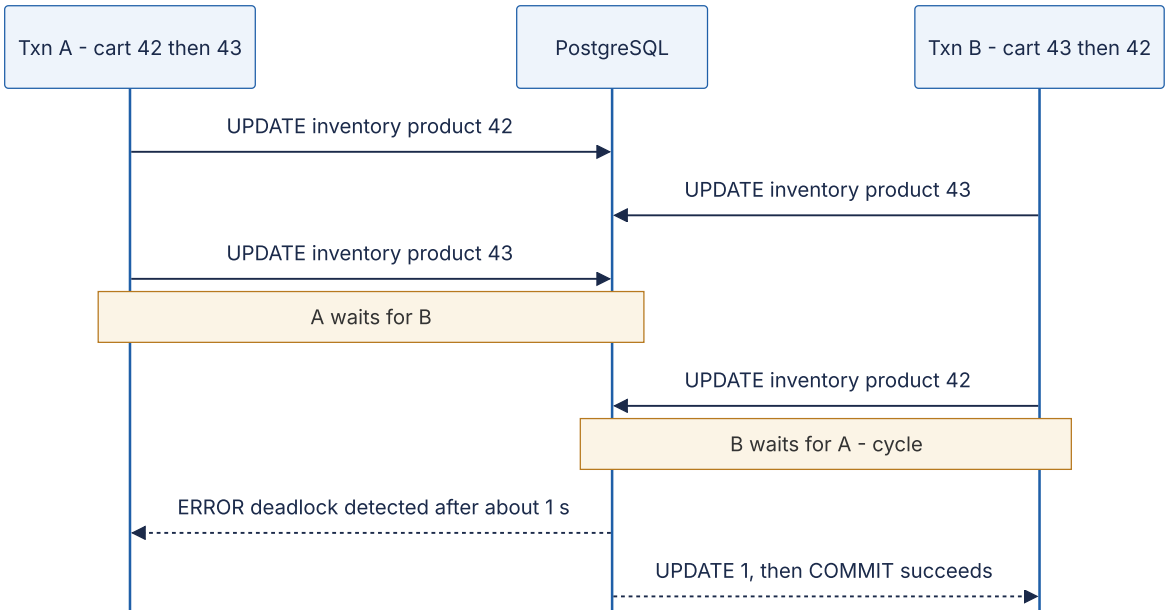
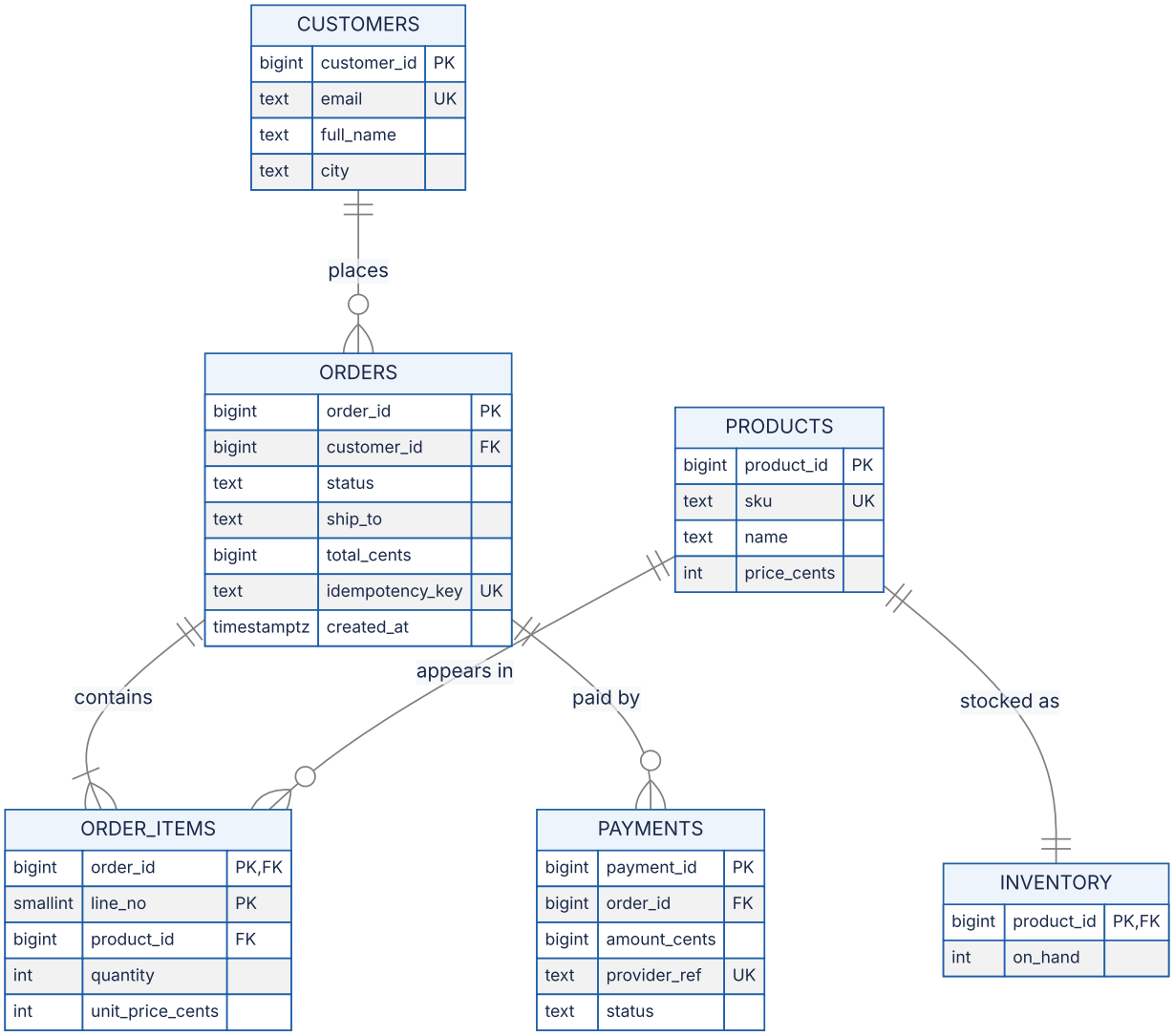
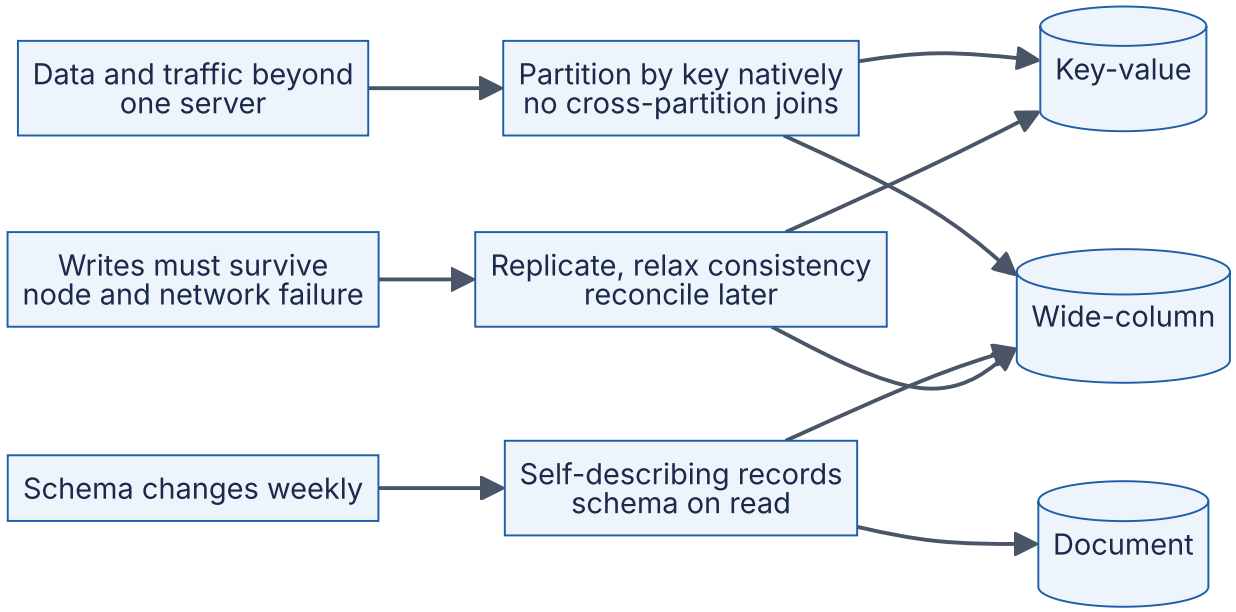


Figure 11.12 — Entity-relationship diagram of the order system.



Chapter 12 — NoSQL Databases

Figure 12.1 — Forces that produced NoSQL and the design responses.



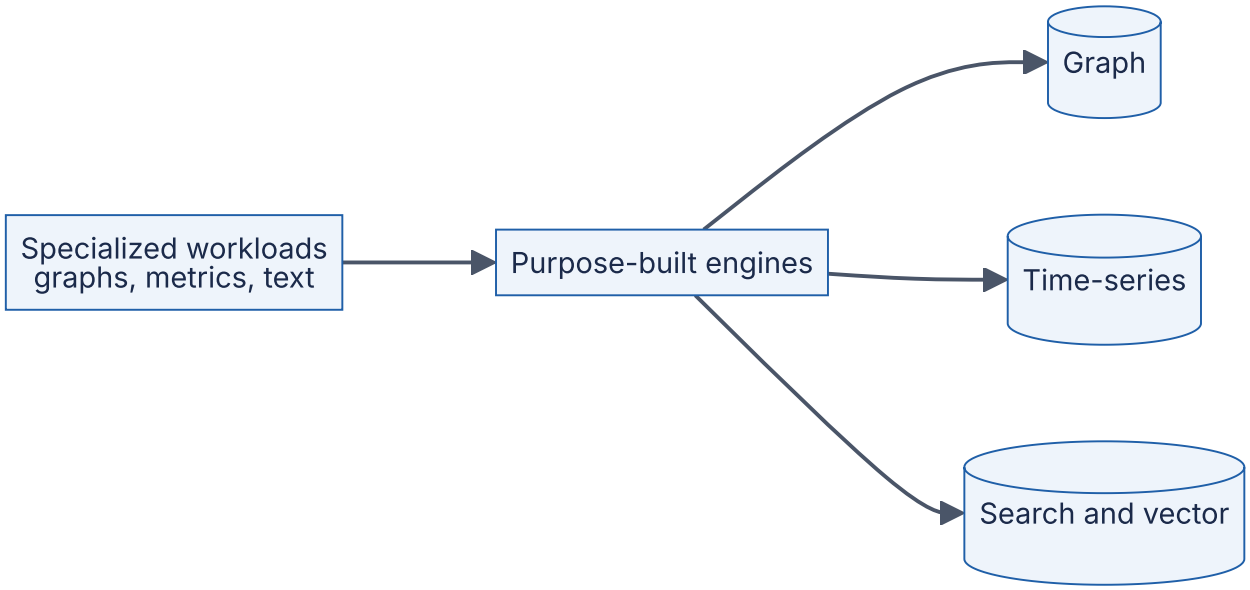


Figure 12.2 — DynamoDB data model and request path.

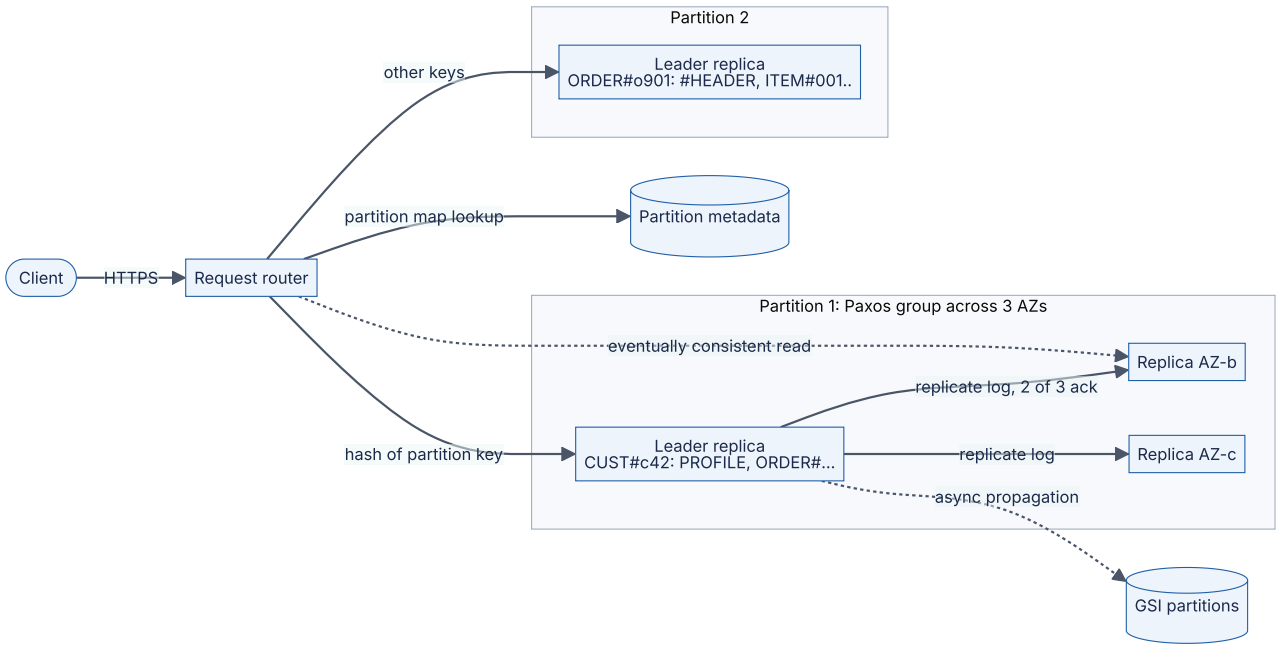


Figure 12.3 — Single-table design with two global secondary indexes.

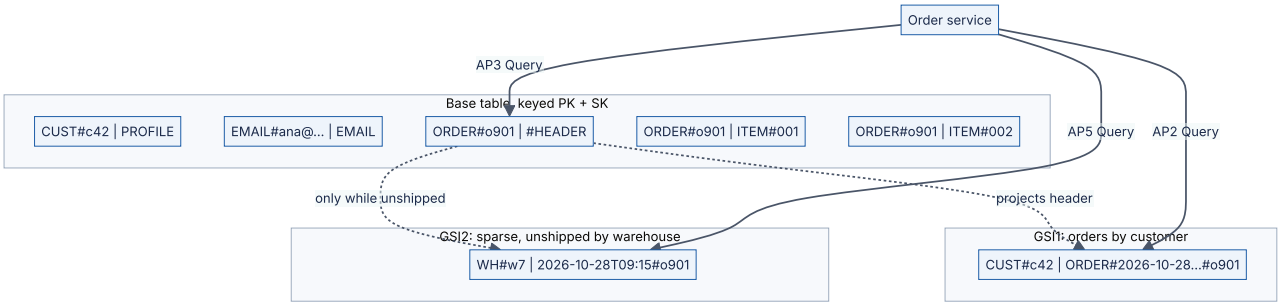


Figure 12.4 — Embedding versus referencing in a document model.

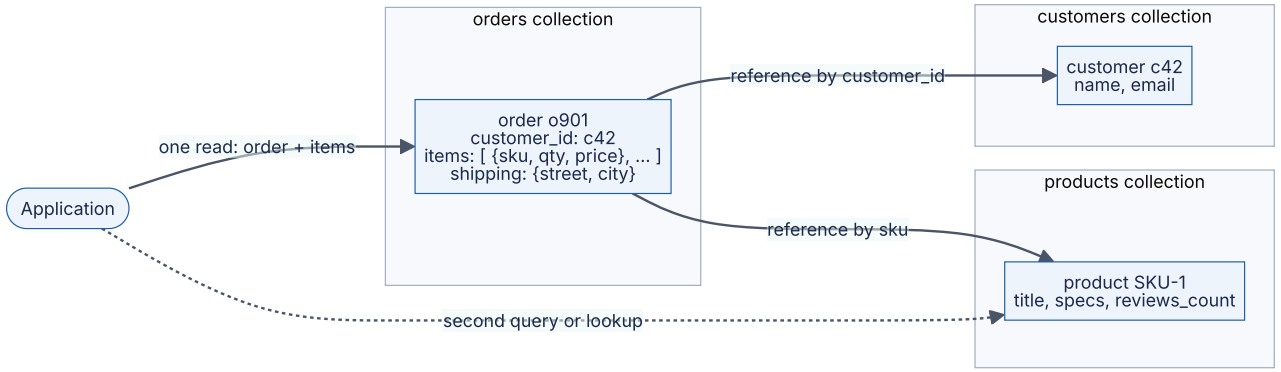


Figure 12.5 — Bigtable data model: the Webtable example.

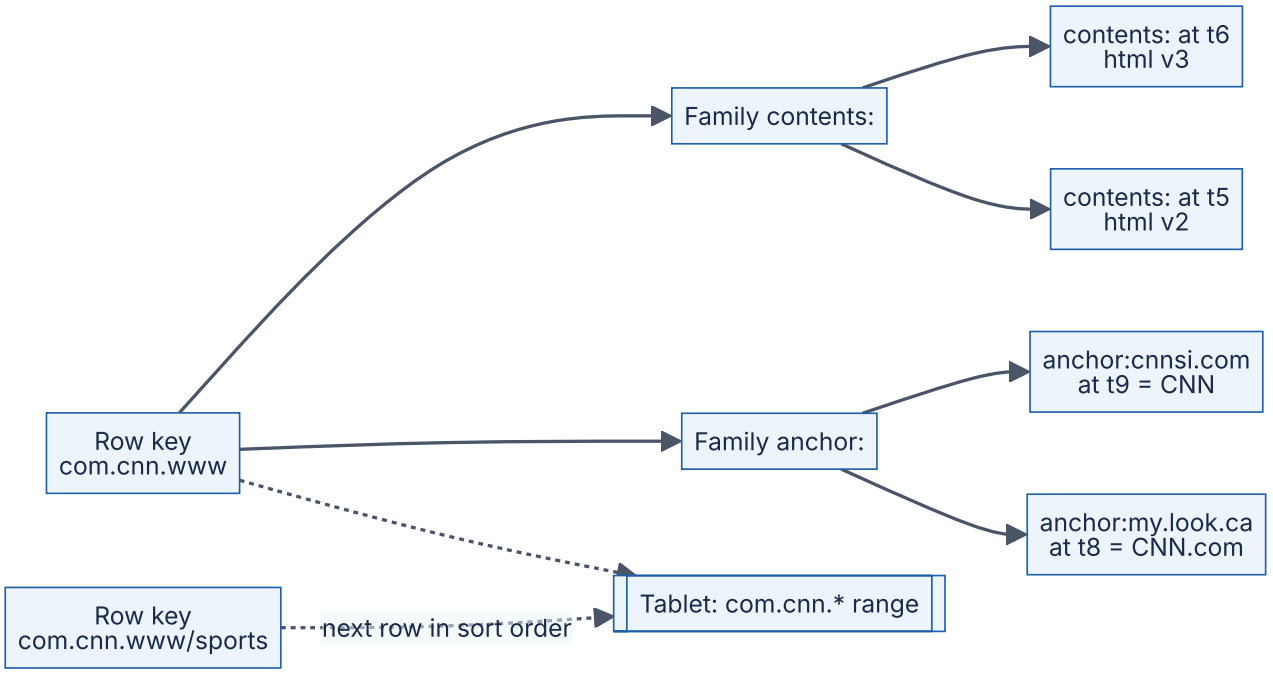


Figure 12.6 — Query-first modeling and partition placement in Cassandra.

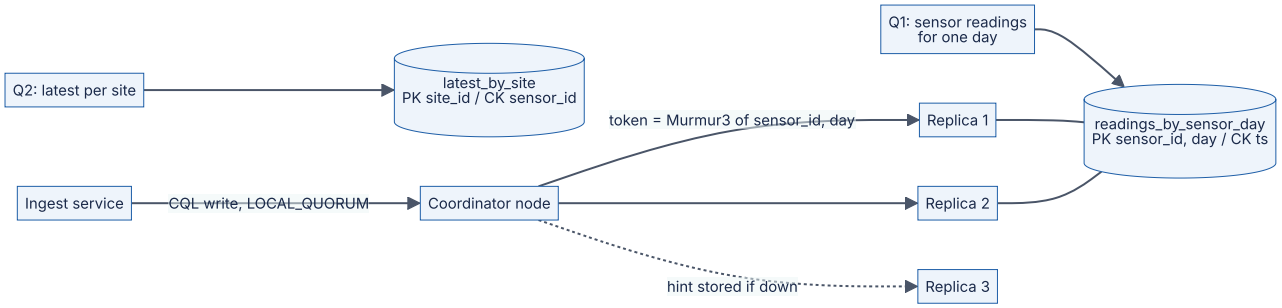


Figure 12.7 — A property graph for fraud and social queries.

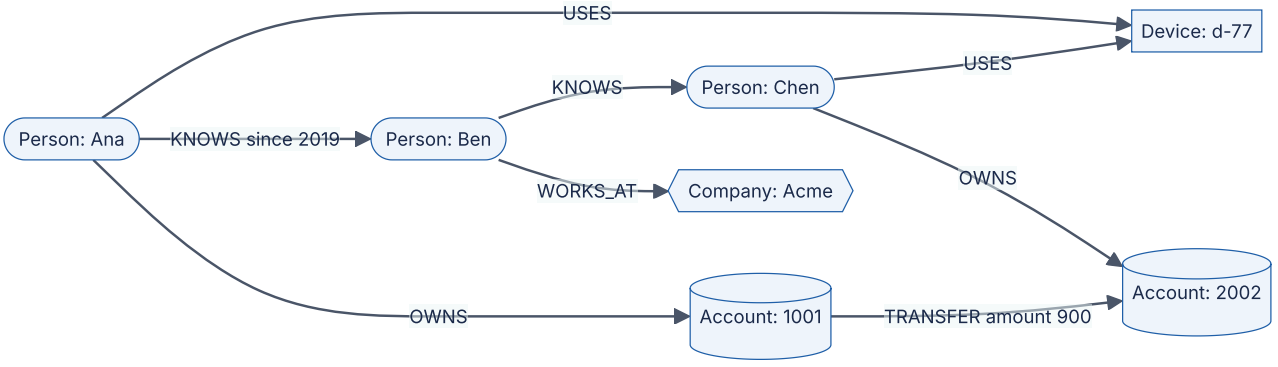


Figure 12.8 — Prometheus TSDB write and storage path.

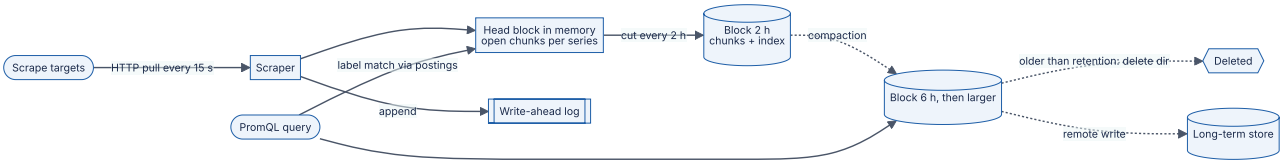


Figure 12.9 — Gorilla’s per-sample encoding decisions.

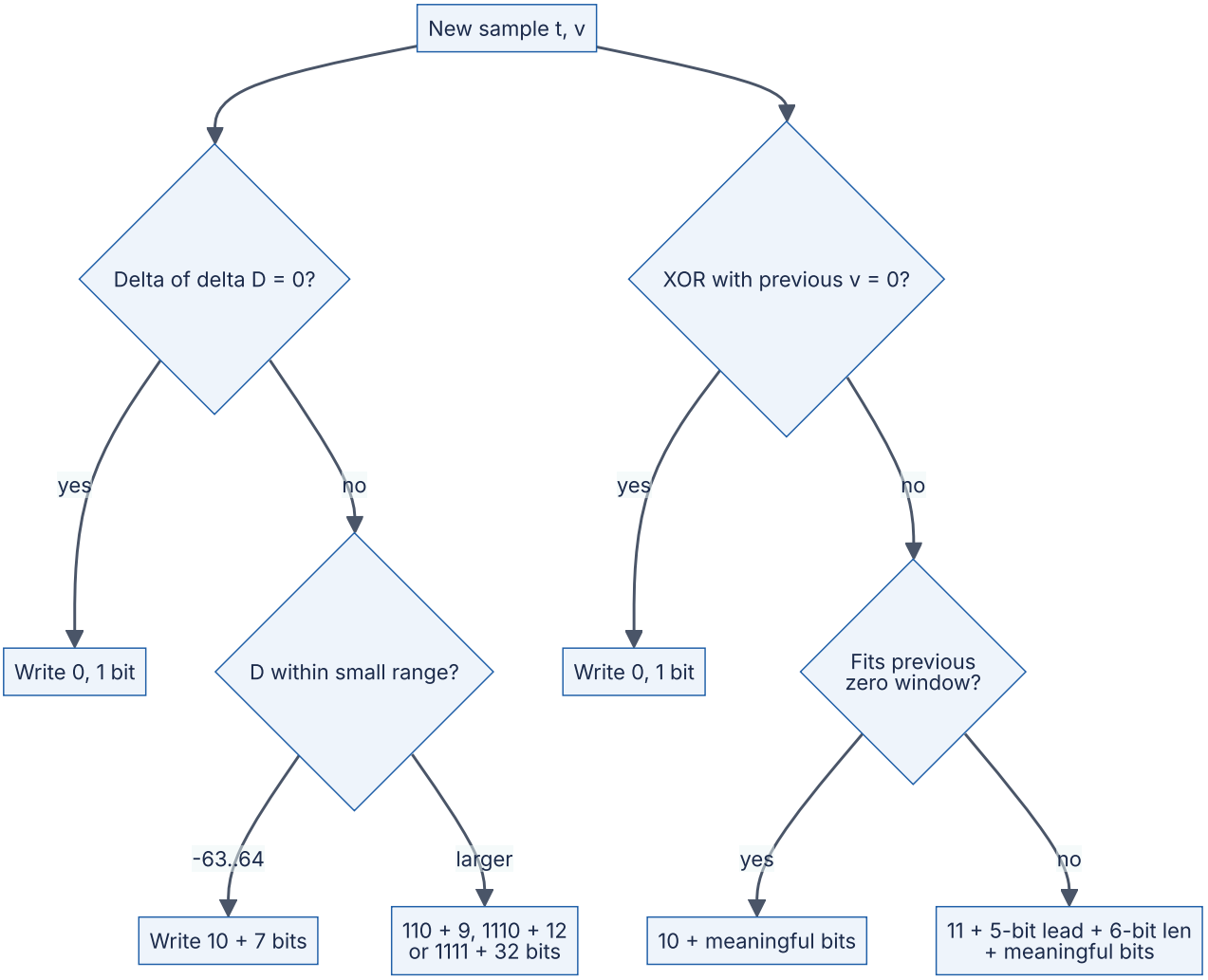


Figure 12.10 — Database selection flowchart.

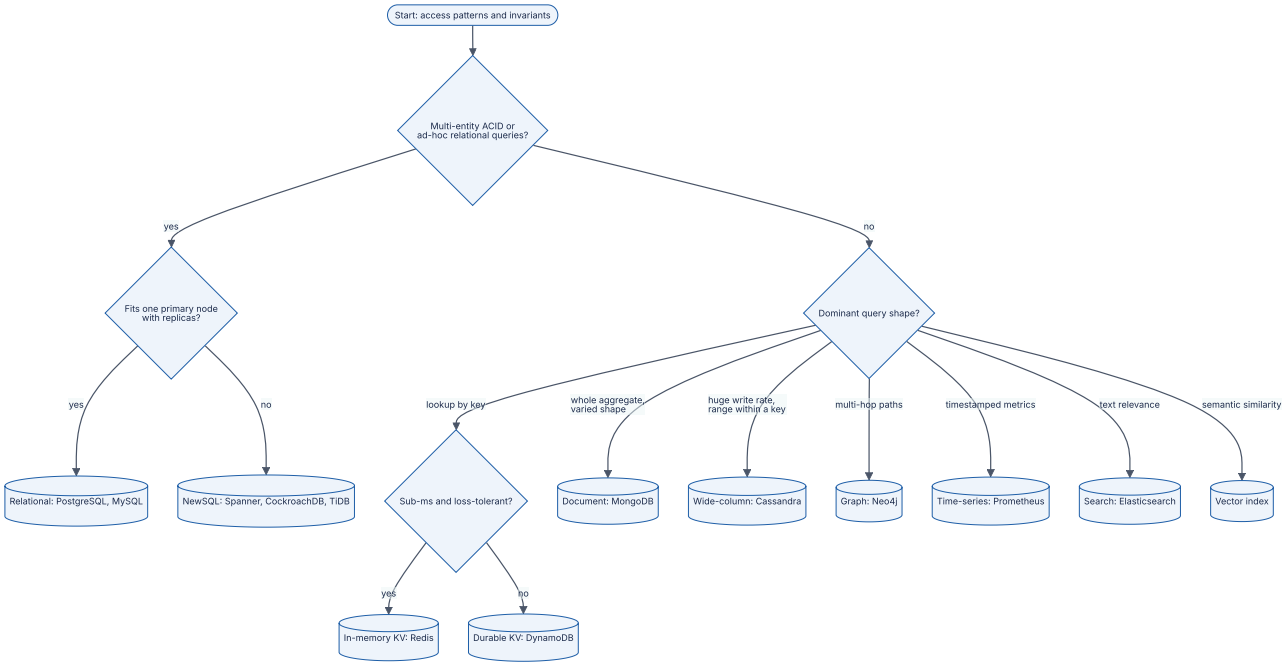
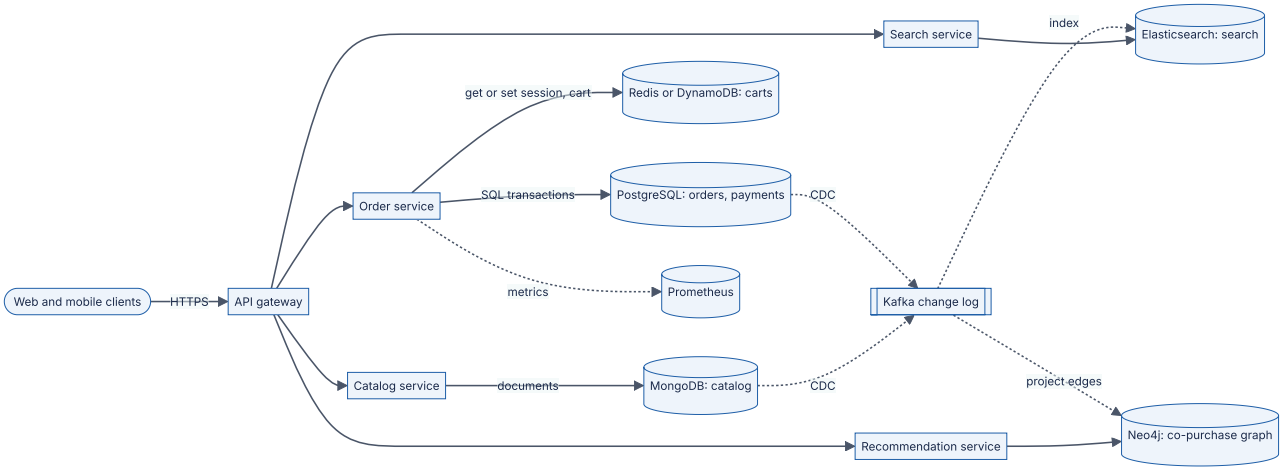


Figure 12.11 — Polyglot persistence in an e-commerce platform.



Chapter 13 — Storage Engine Internals

Figure 13.1 — A slotted page.

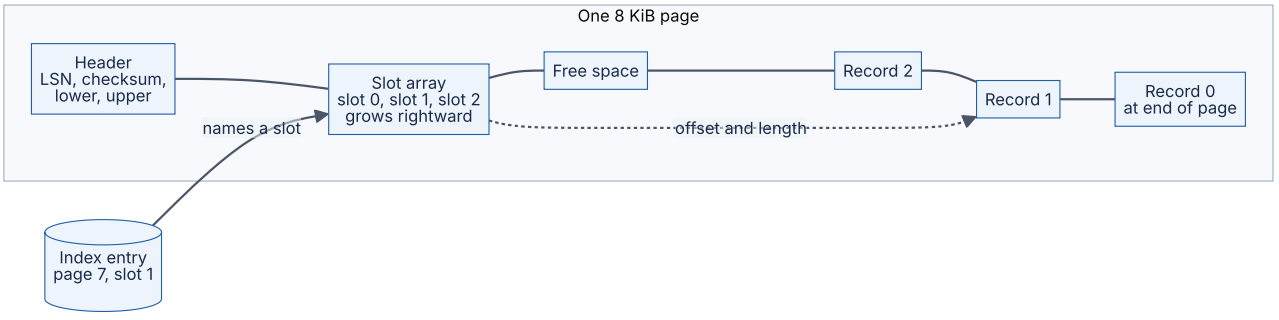


Figure 13.2 — Buffer pool components and the page request path.

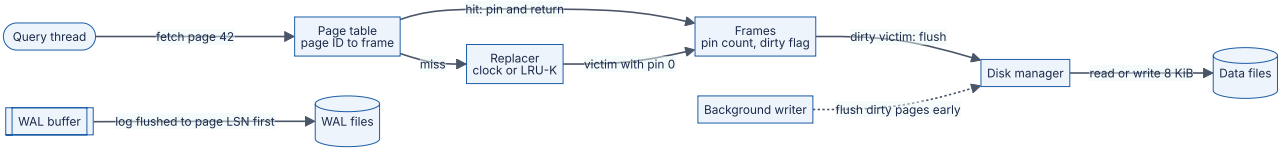


Figure 13.3 — WAL, commit and checkpoint timeline.

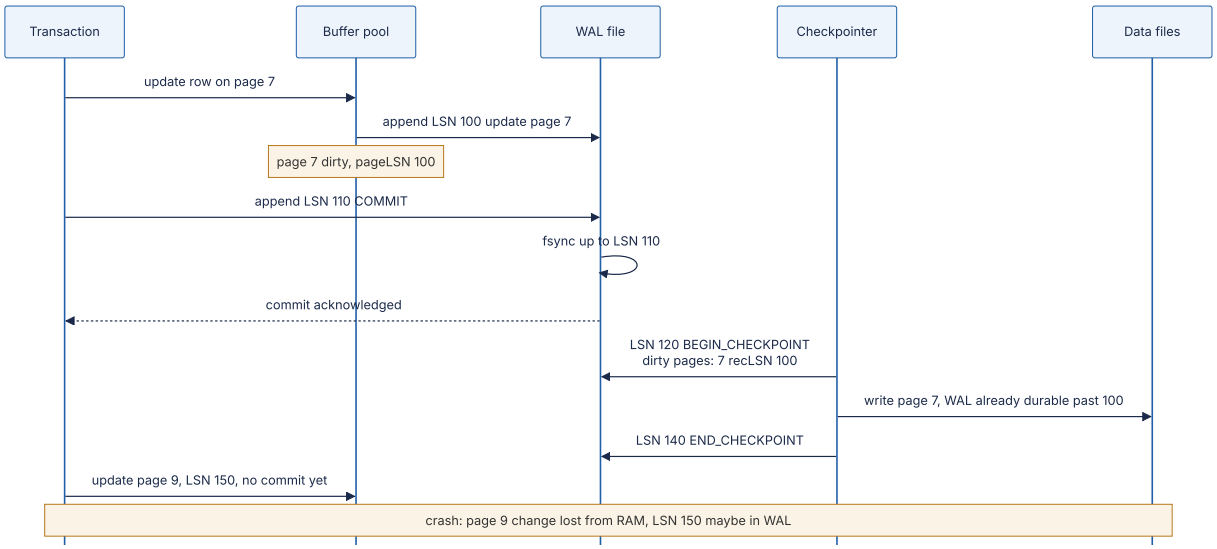


Figure 13.4 — The three ARIES passes over the log.



Figure 13.5 — A leaf page split propagating a separator to the parent.

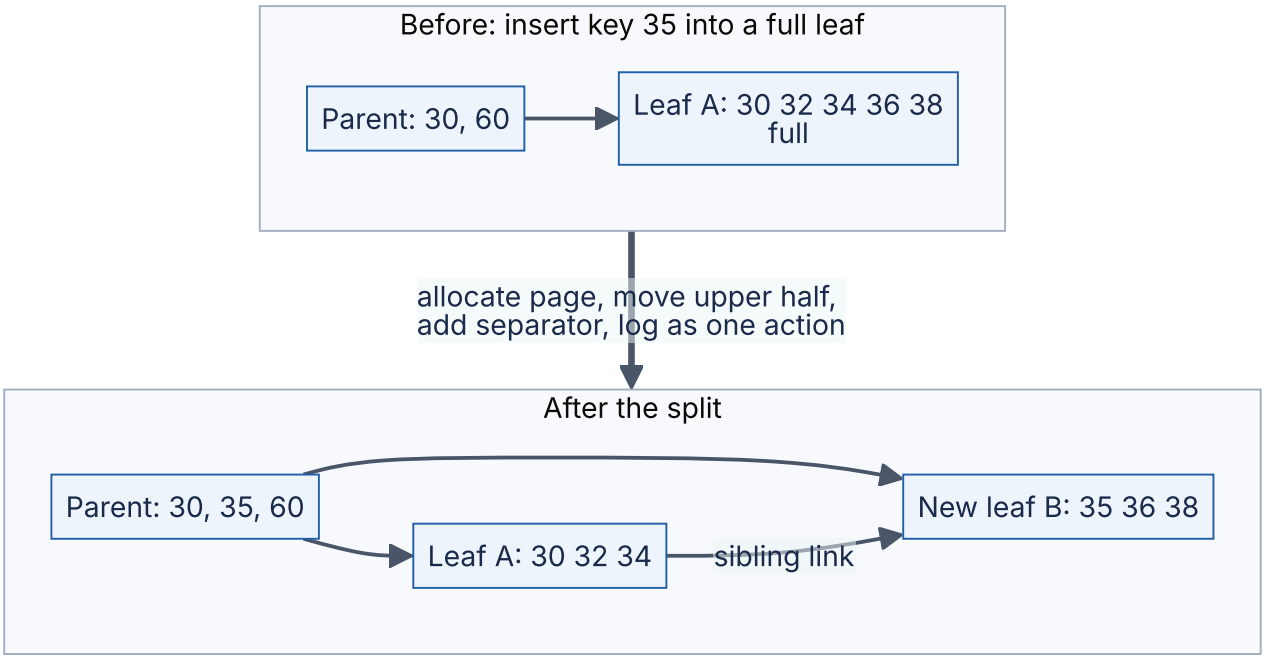


Figure 13.6 — Bitcask: an in-memory hash index over append-only files.

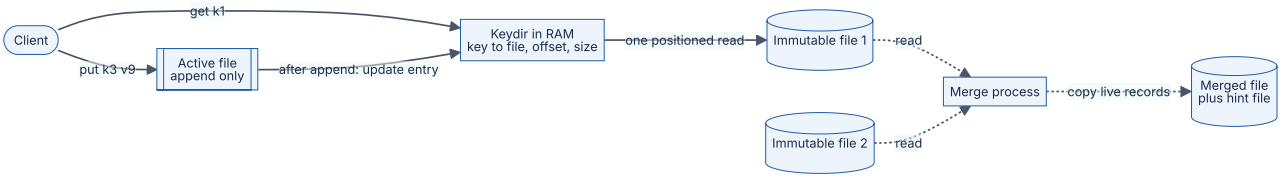


Figure 13.7 — LSM tree write path and levels.

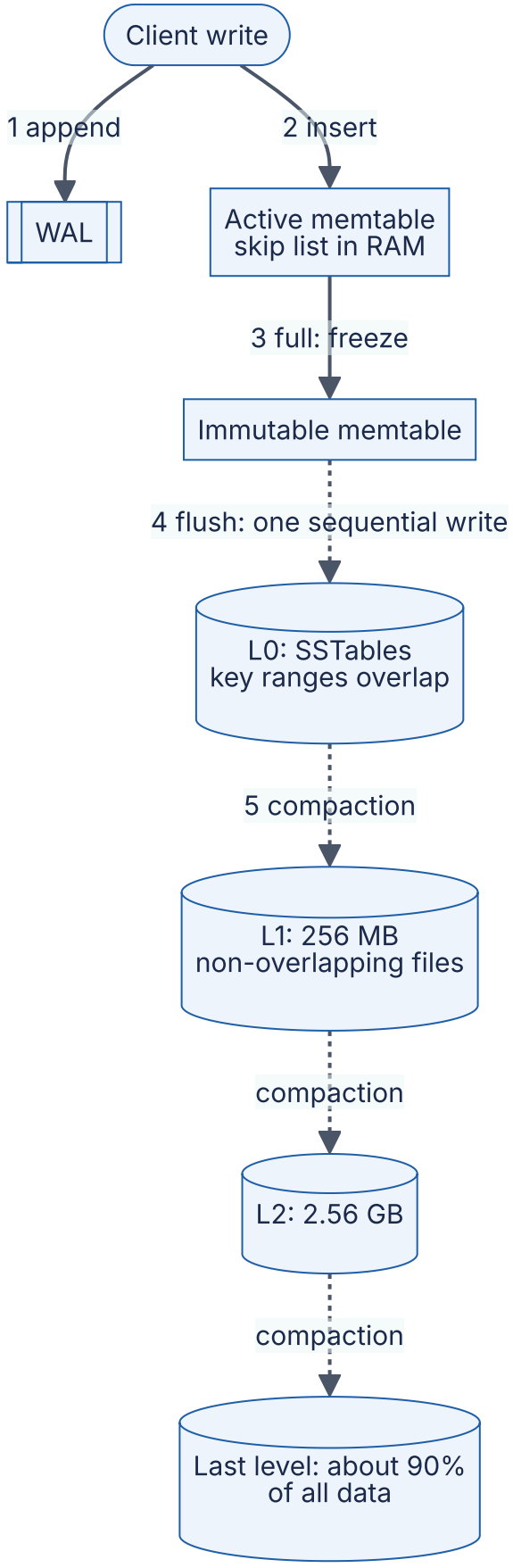


Figure 13.8 — LSM point-lookup read path.

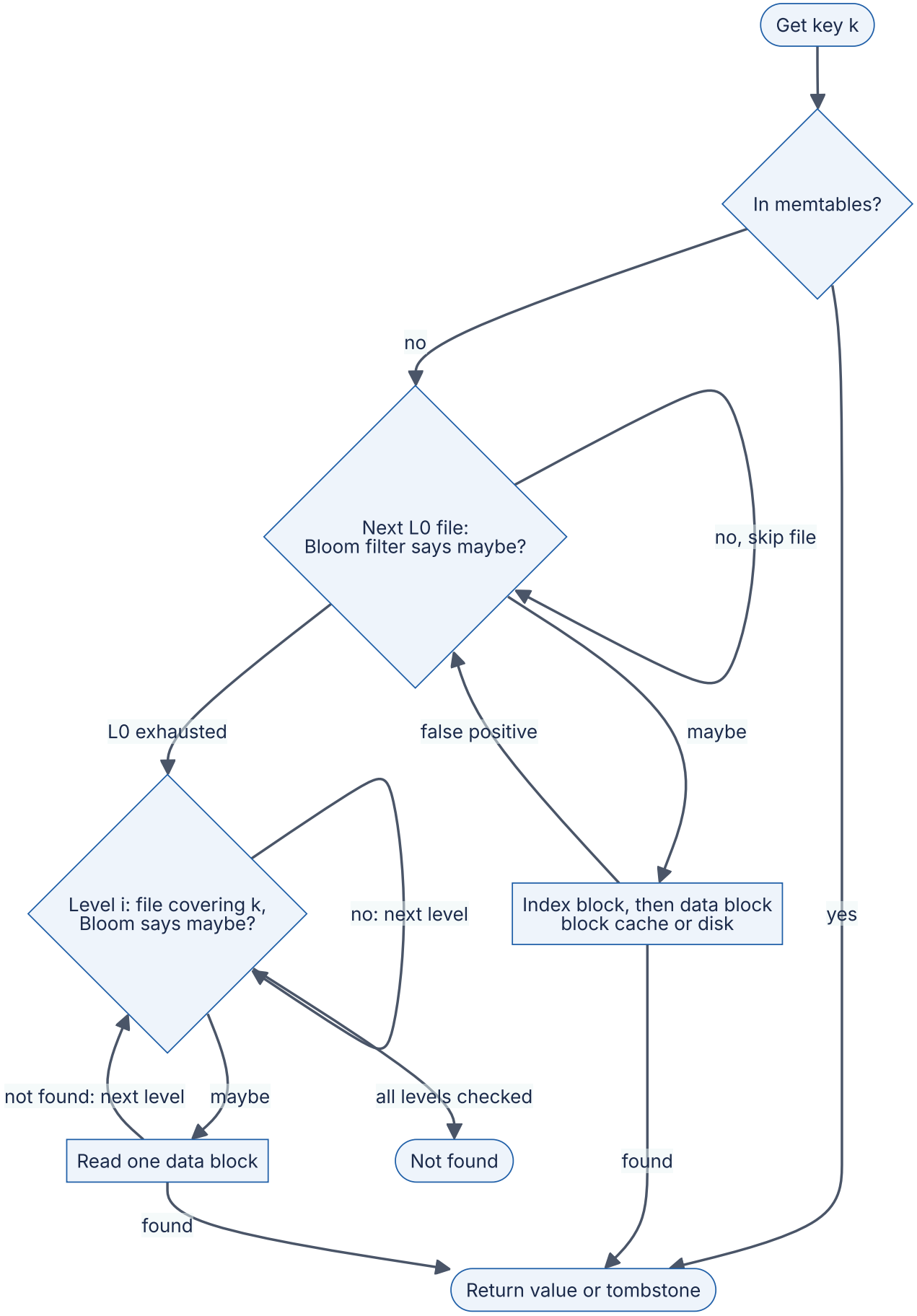
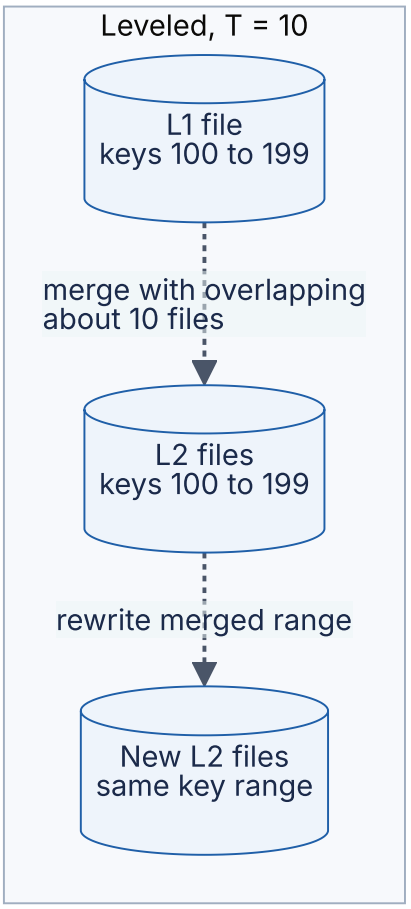

Get key k
In memtables?
yes
no
Next L0 file:
Bloom filter says maybe?
no, skip file
maybe
false positive
L0 exhausted
Level i: file covering k,
Bloom says maybe?
no: next level
Index block, then data block
block cache or disk
found
all levels checked
Not found
not found: next level
maybe
Read one data block
found
Return value or tombstone

Figure 13.9 — Size-tiered versus leveled compaction.



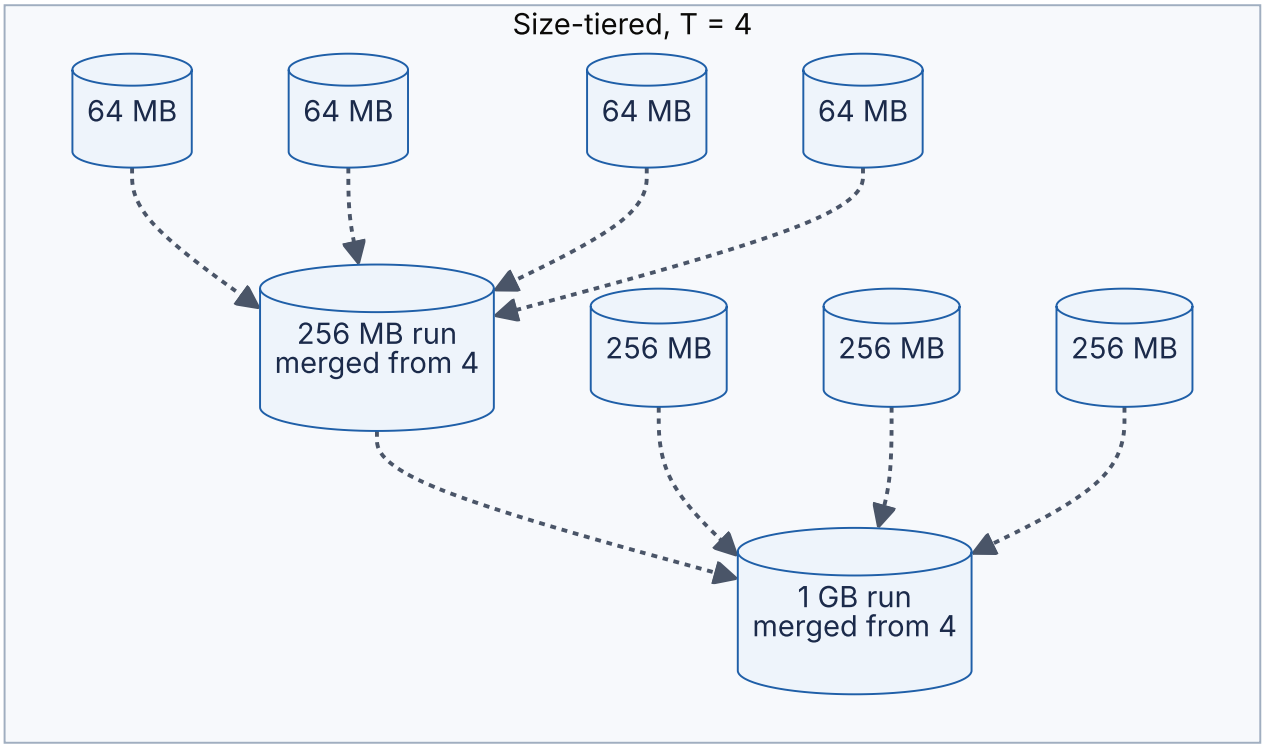


Figure 13.10 — InnoDB doublewrite protecting against a torn page.

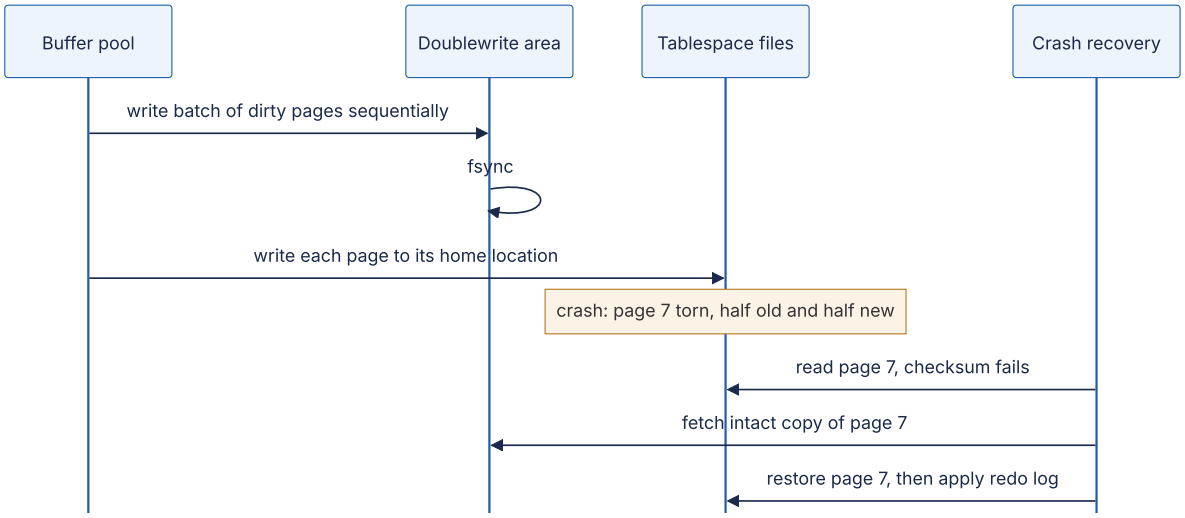


Figure 13.11 — Row-oriented versus column-oriented layout of the same table.



Chapter 14 — Database Scaling: Replication, Partitioning and Sharding

Figure 14.1 — Primary-replica architecture with read/write splitting.

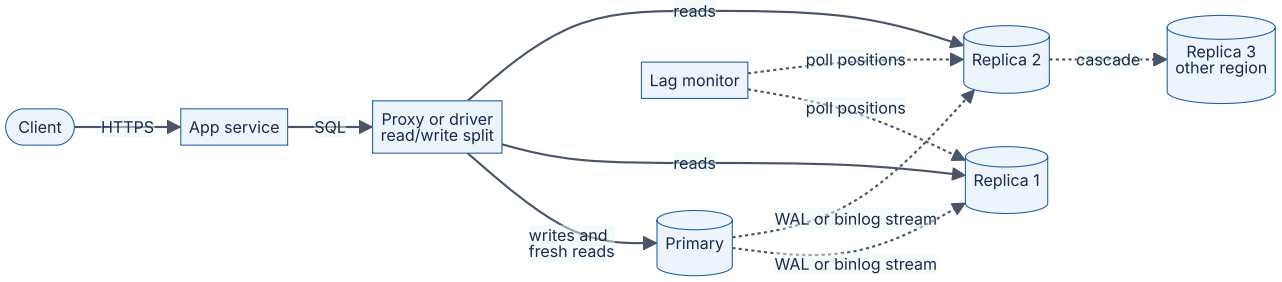
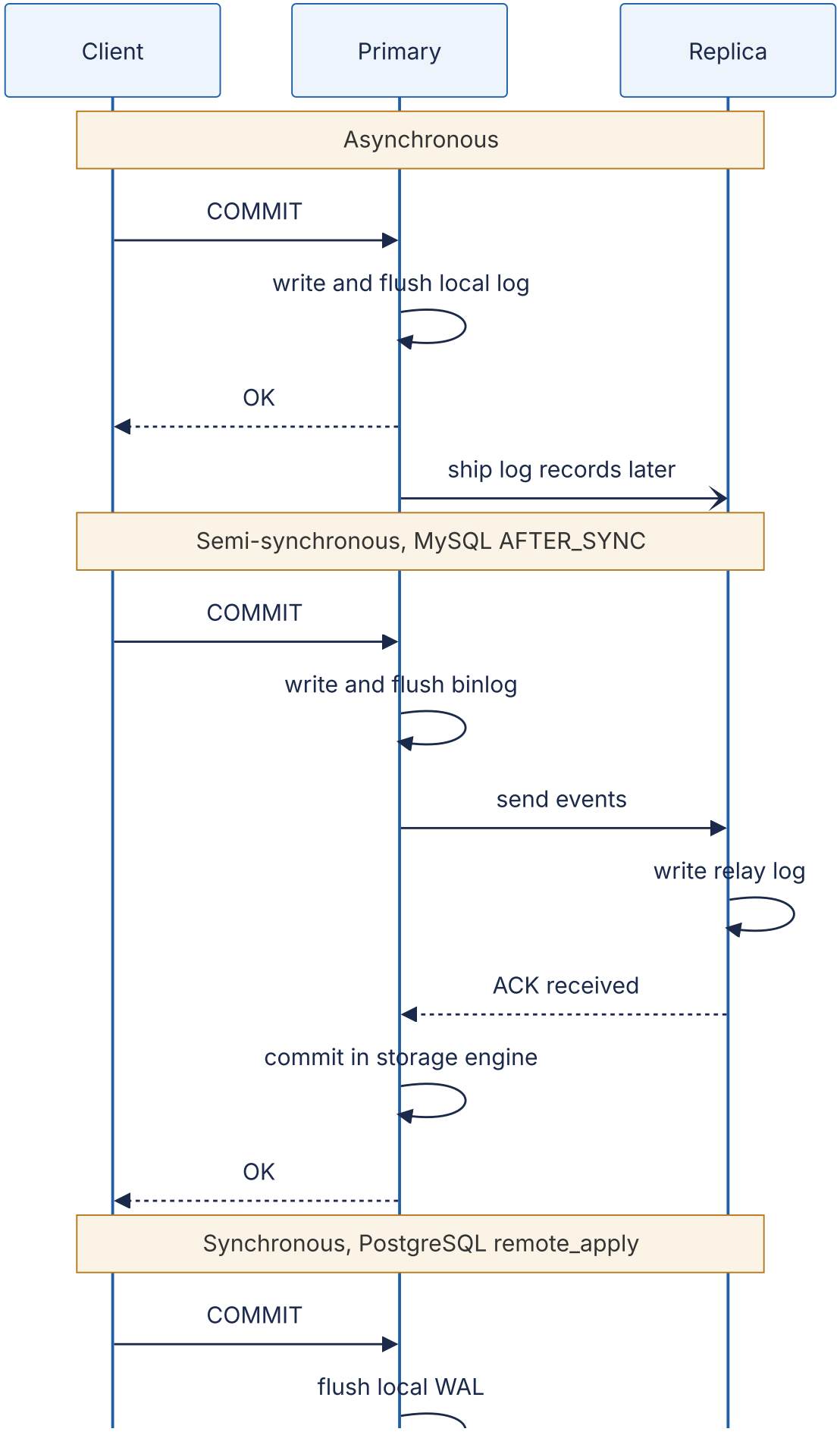


Figure 14.2 — Commit paths under asynchronous, semi-synchronous and synchronous replication.


Client
Primary
Replica
Asynchronous
COMMIT
write and flush local log
OK
ship log records later
Semi-synchronous, MySQL AFTER_SYNC
COMMIT
write and flush binlog
send events
write relay log
ACK received
commit in storage engine
OK
Synchronous, PostgreSQL remote_apply
COMMIT
flush local WAL

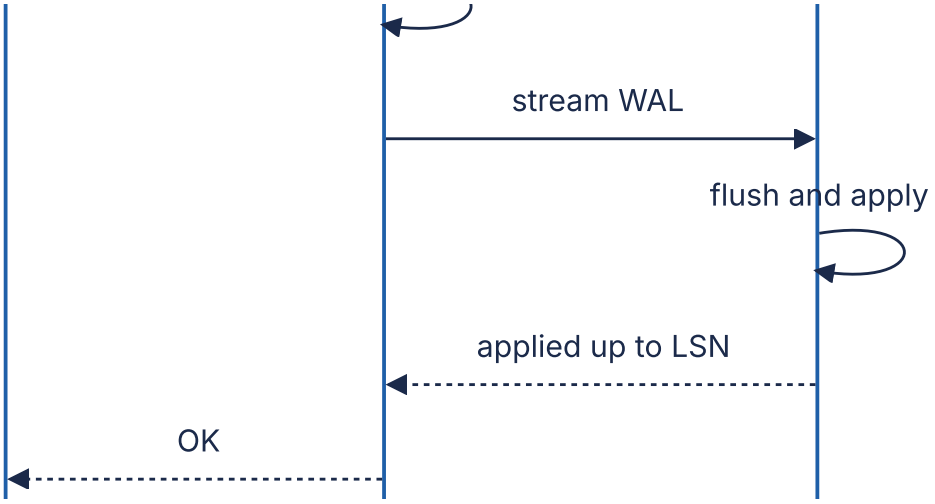


Figure 14.3 — Two lag anomalies: lost own write and time going backwards.

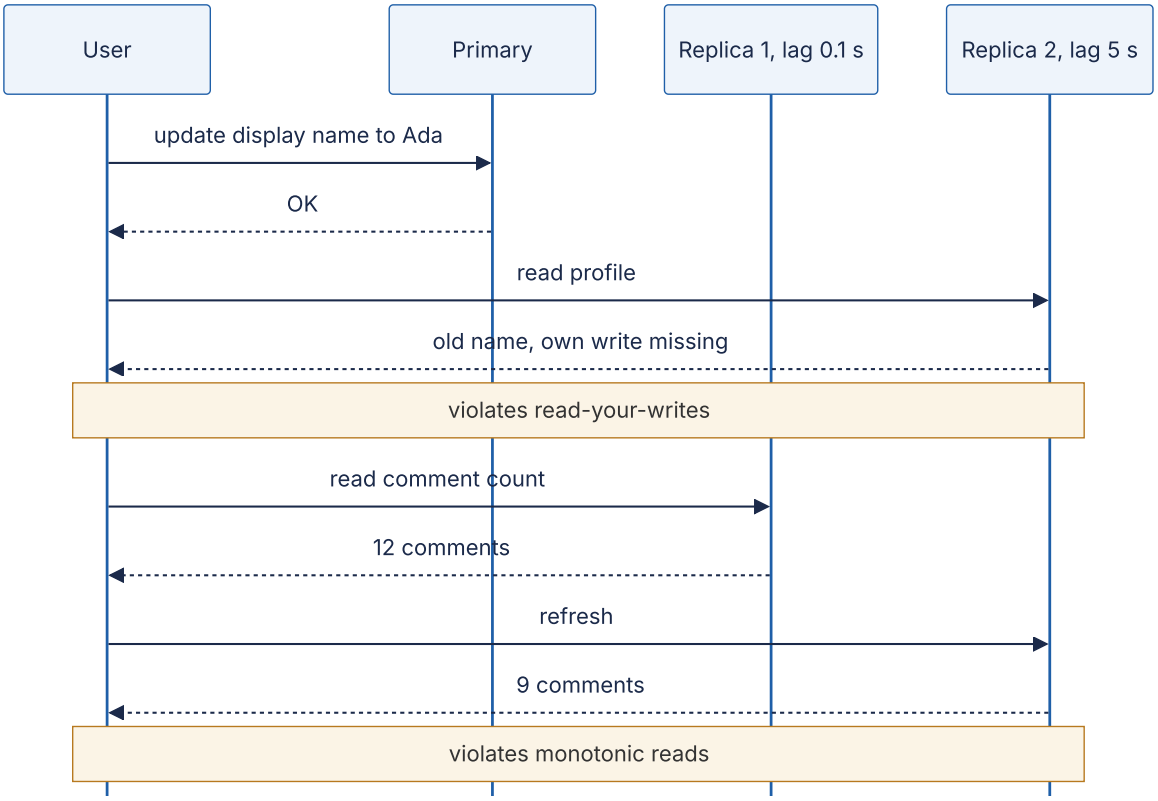


Figure 14.4 — Two-region multi-primary replication and a write conflict.

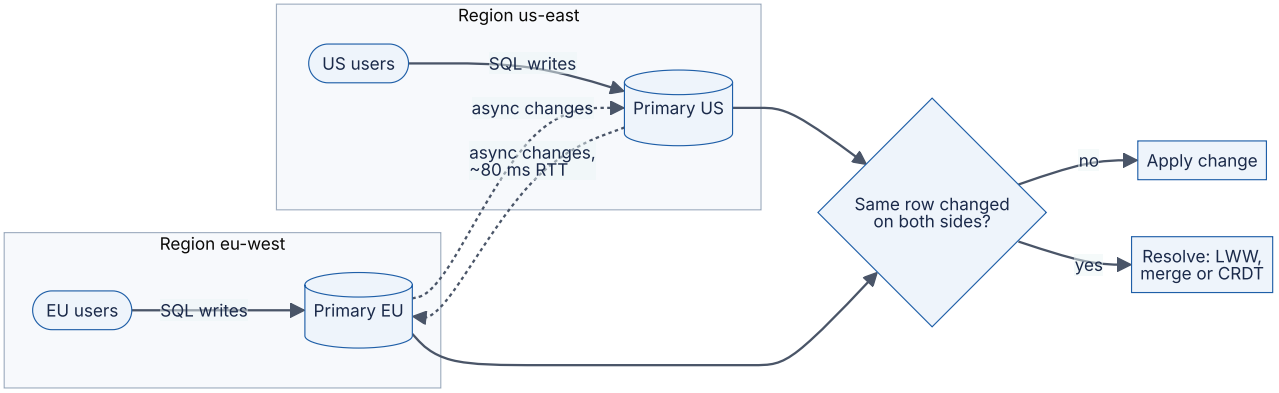


Figure 14.5 — Four ways to answer “which partition holds this key?”.

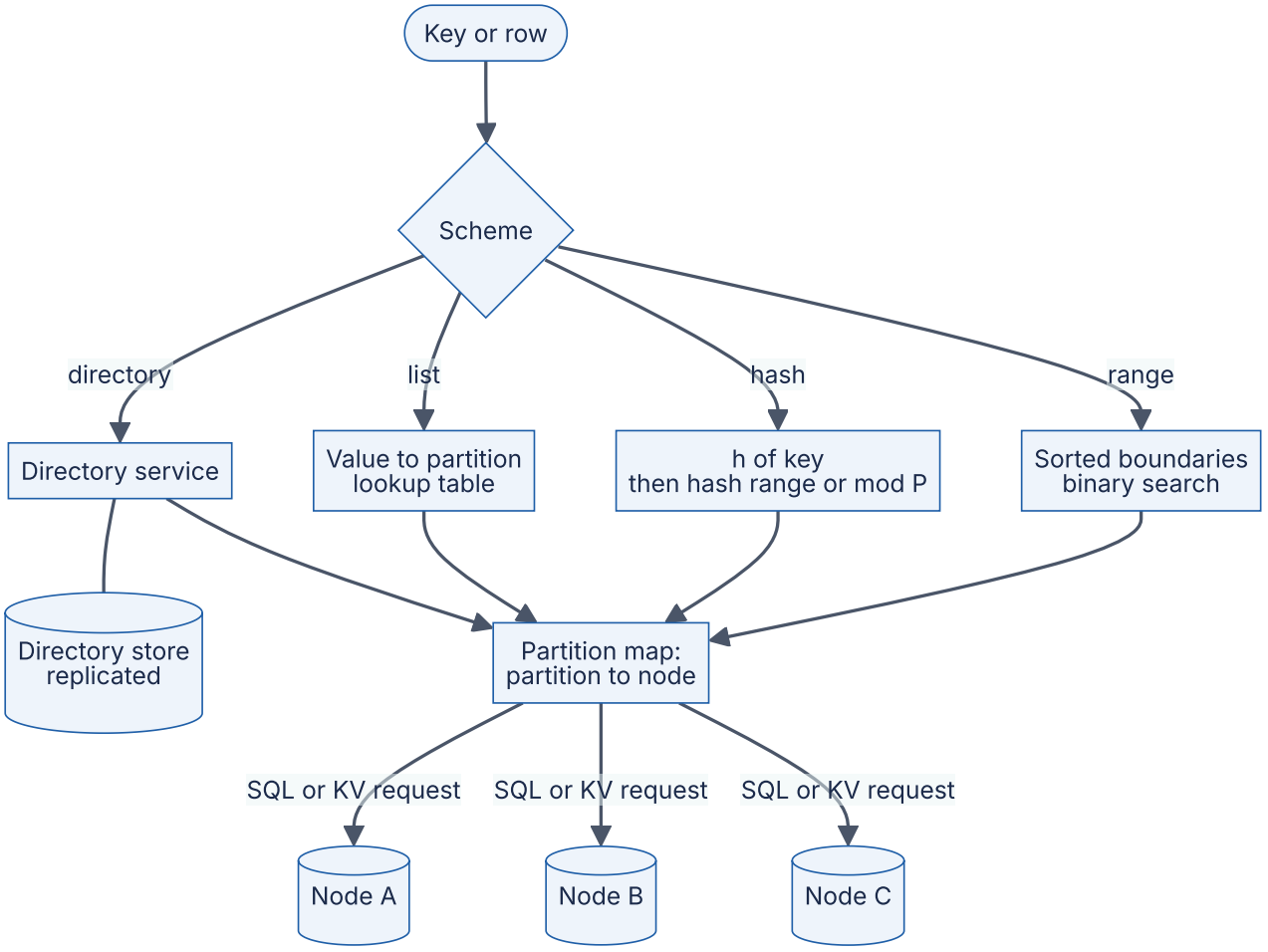


Figure 14.6 — Multi-tenant sharding with logical shards and a tenant directory.

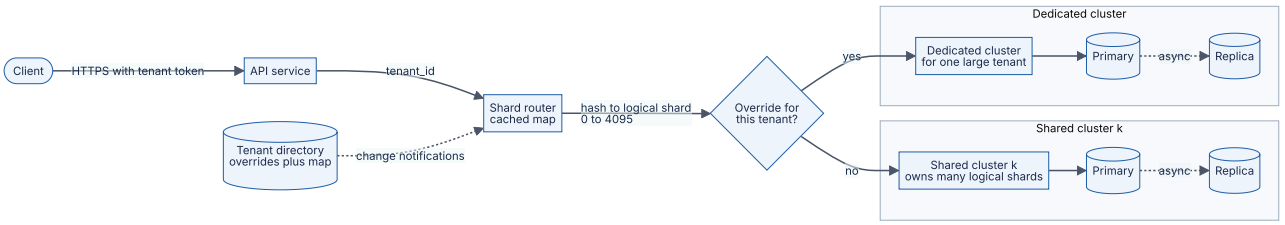


Figure 14.7 — A ring with three nodes and two vnodes each.

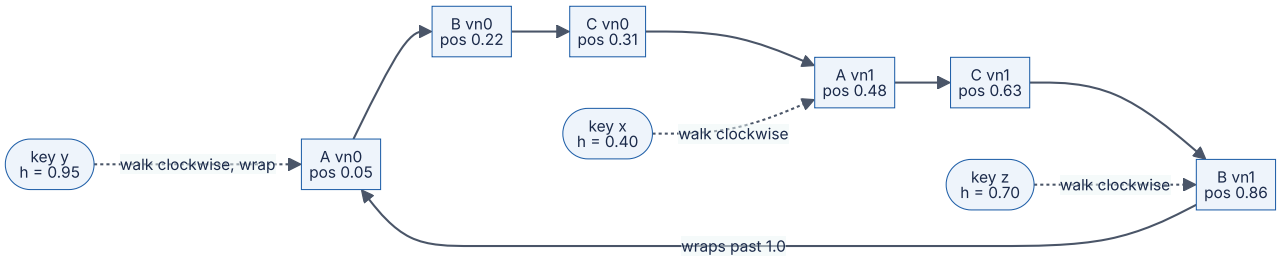
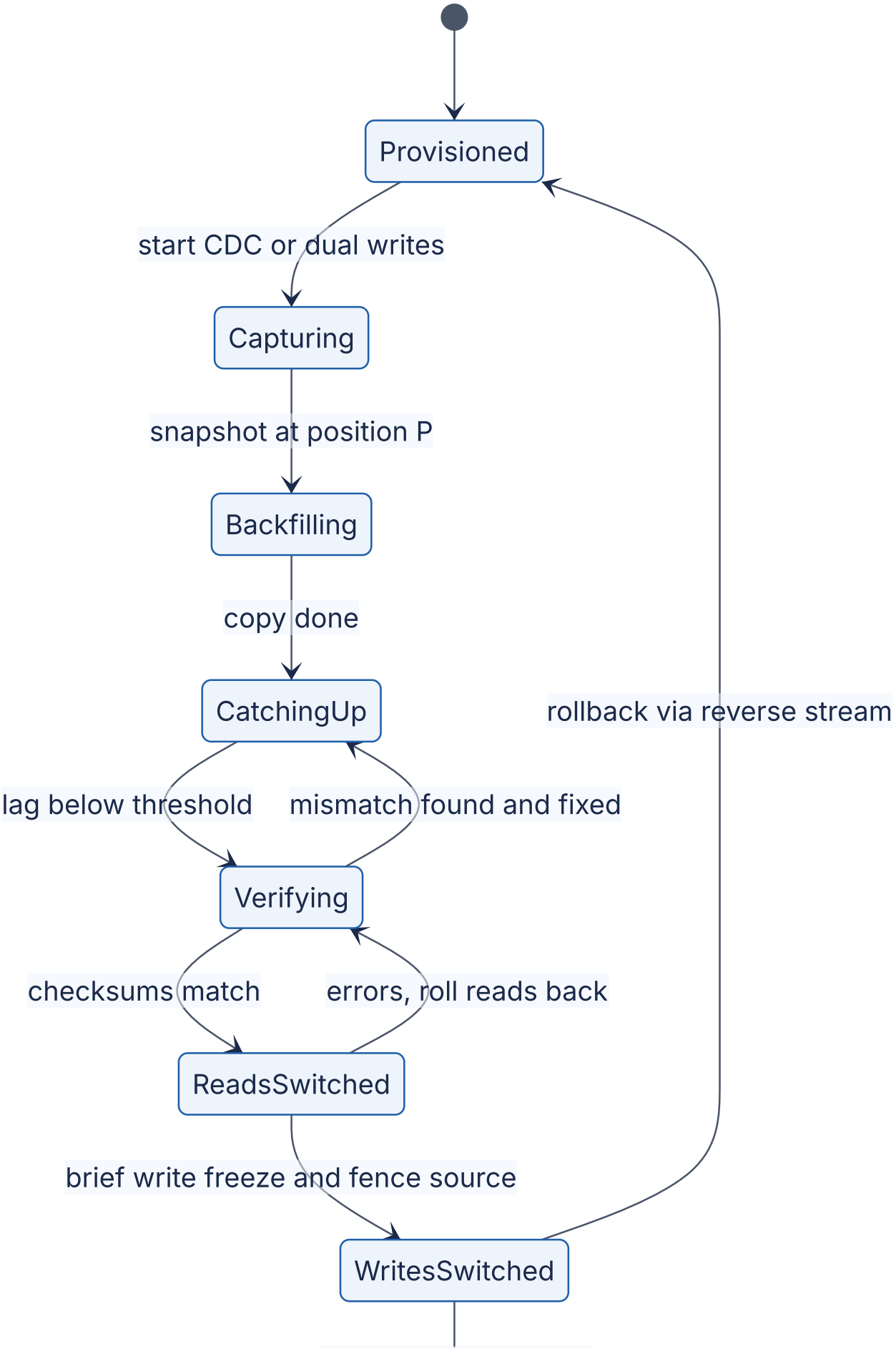


Figure 14.8 — Online resharding as a state machine.


Provisioned
start CDC or dual writes
Capturing
snapshot at position P
Backfilling
copy done
CatchingUp
rollback via reverse stream
lag below threshold
mismatch found and fixed
Verifying
checksums match
errors, roll reads back
ReadsSwitched
brief write freeze and fence source
WritesSwitched

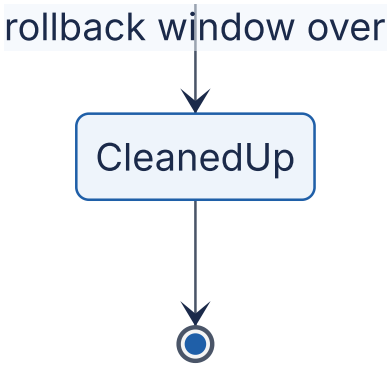


Figure 14.9 — Salting a hot counter across partitions.

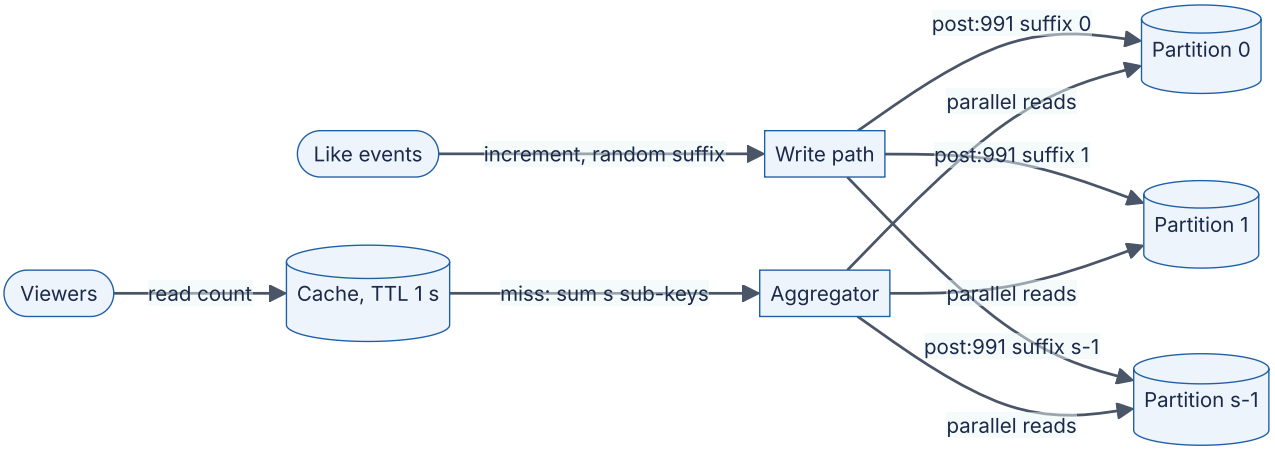
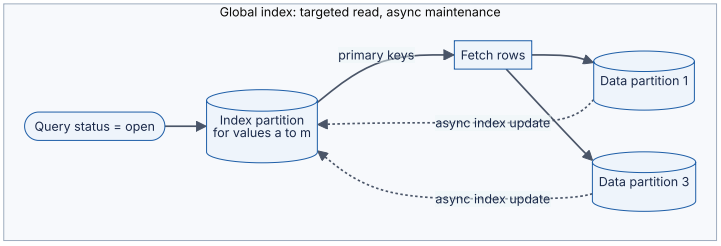


Figure 14.10 — Local versus global secondary index for a query on `status`.



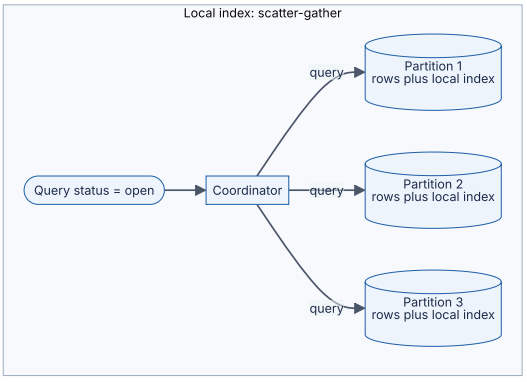


Figure 14.11 — Two-phase commit across two shards (preview of Chapter 28).

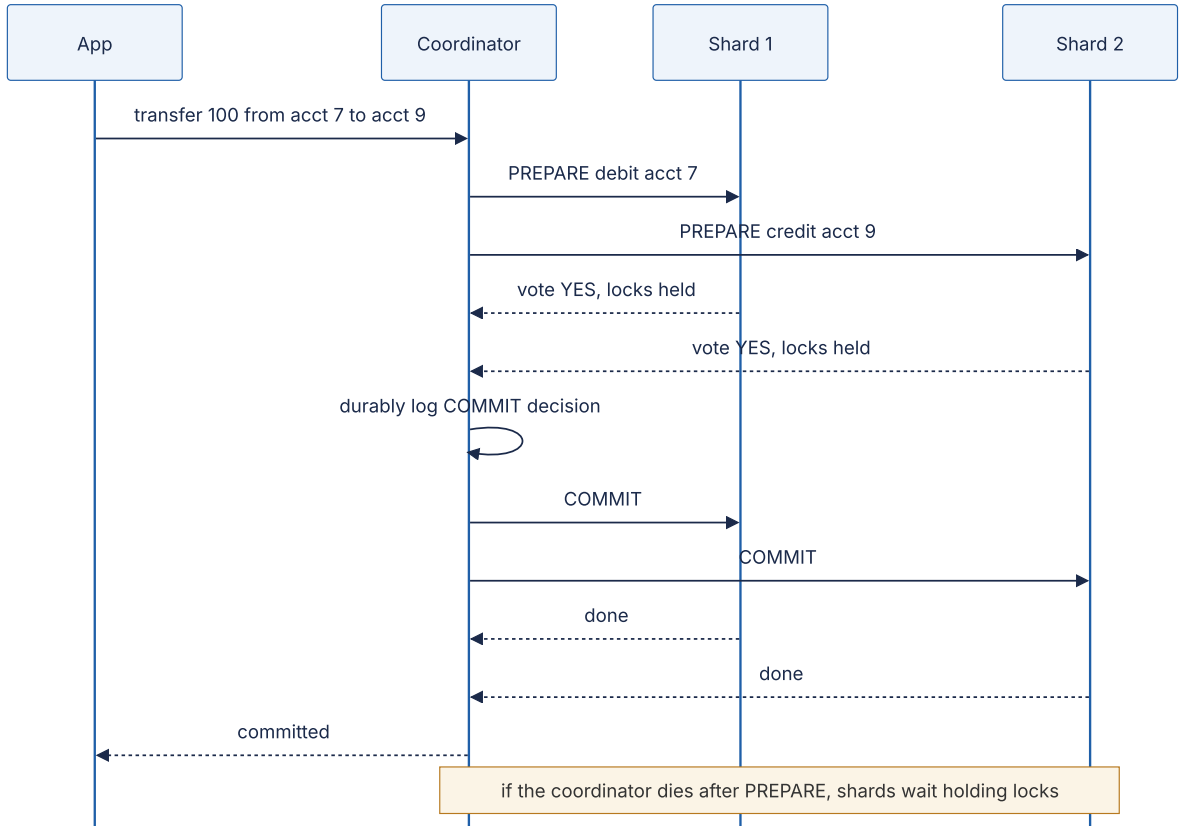


Figure 14.12 — PgBouncer in transaction-pooling mode.

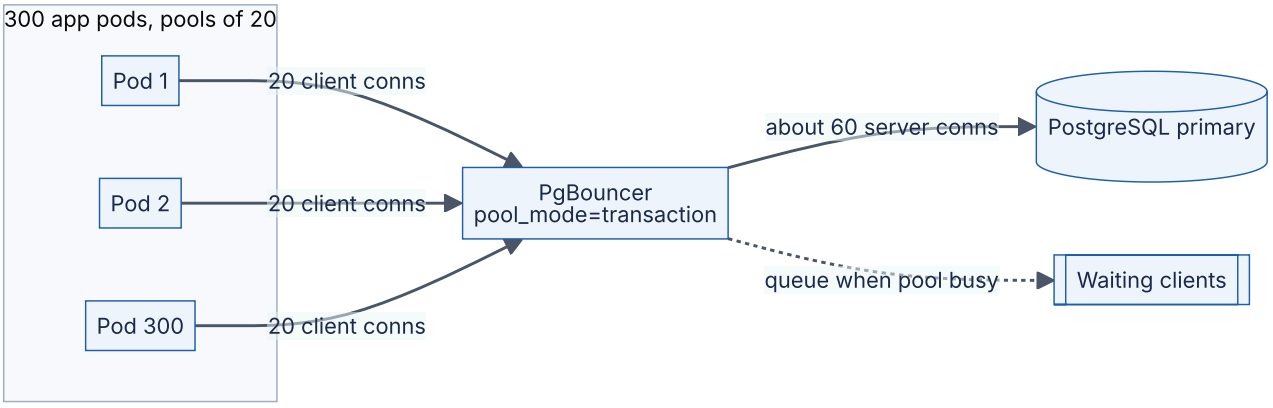
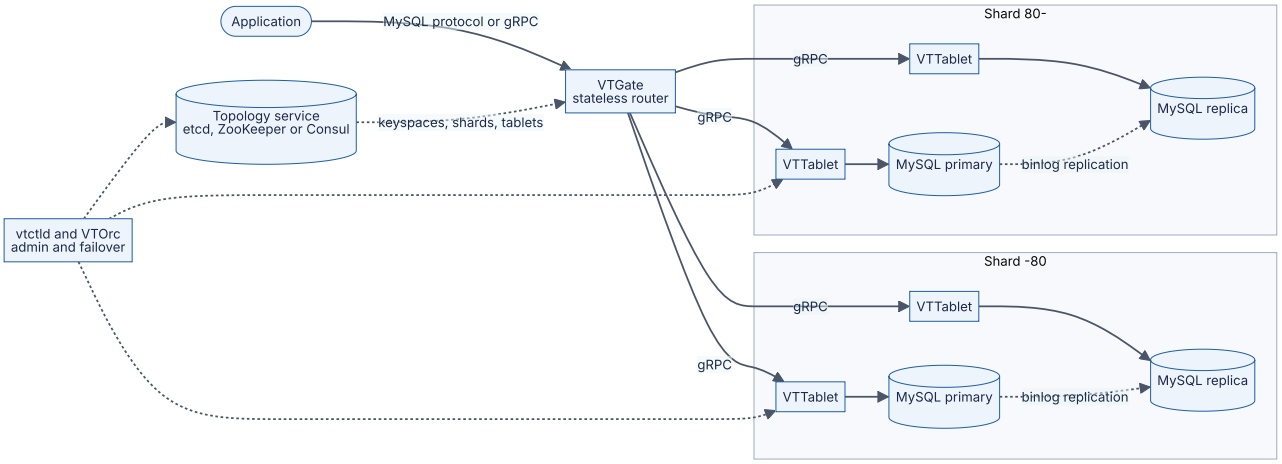


Figure 14.13 — Vitess architecture.



Chapter 15 — Object, Block and File Storage

Figure 15.1 — Three storage interfaces over shared disks.

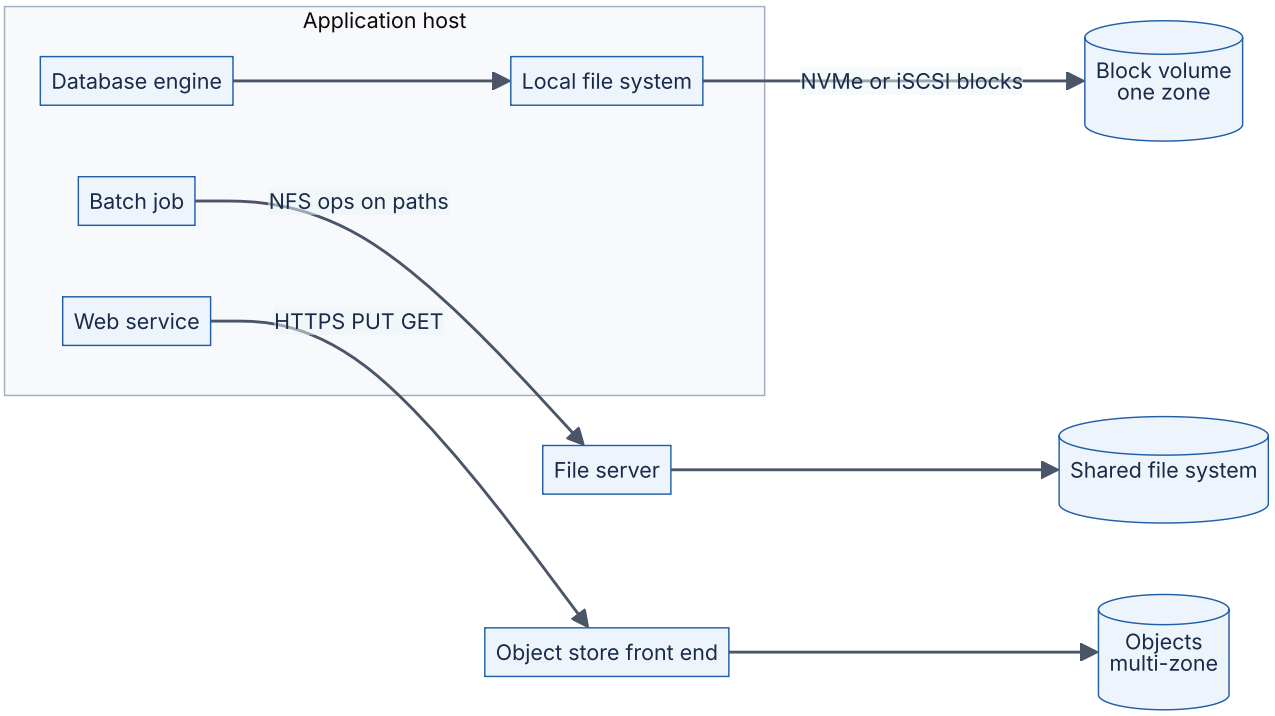


Figure 15.2 — Generic S3-style object store (illustrative).

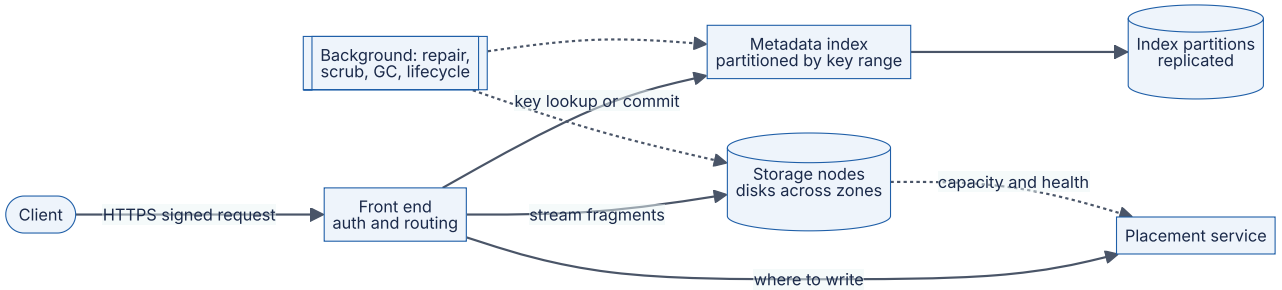


Figure 15.3 — PUT then GET in an S3-style store (illustrative).

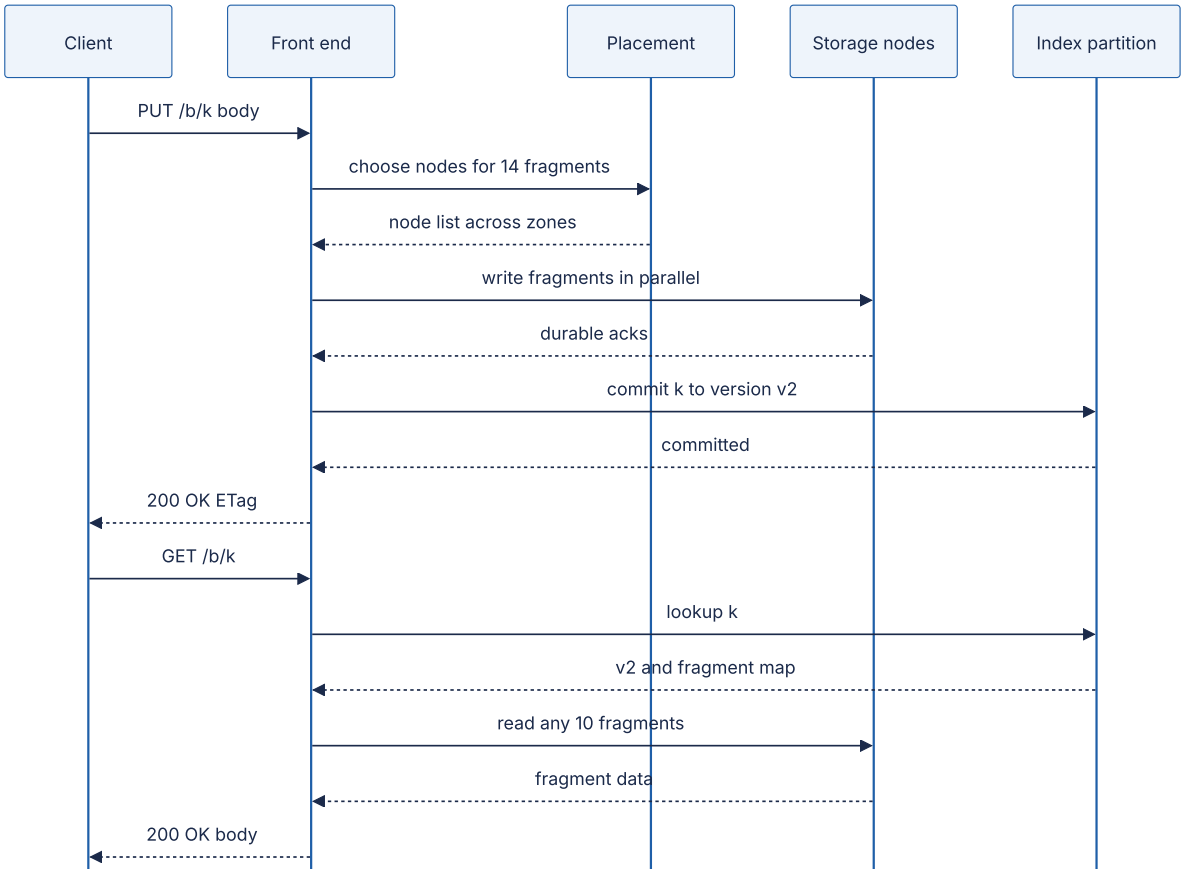


Figure 15.4 — GFS write with a lease-holding primary.

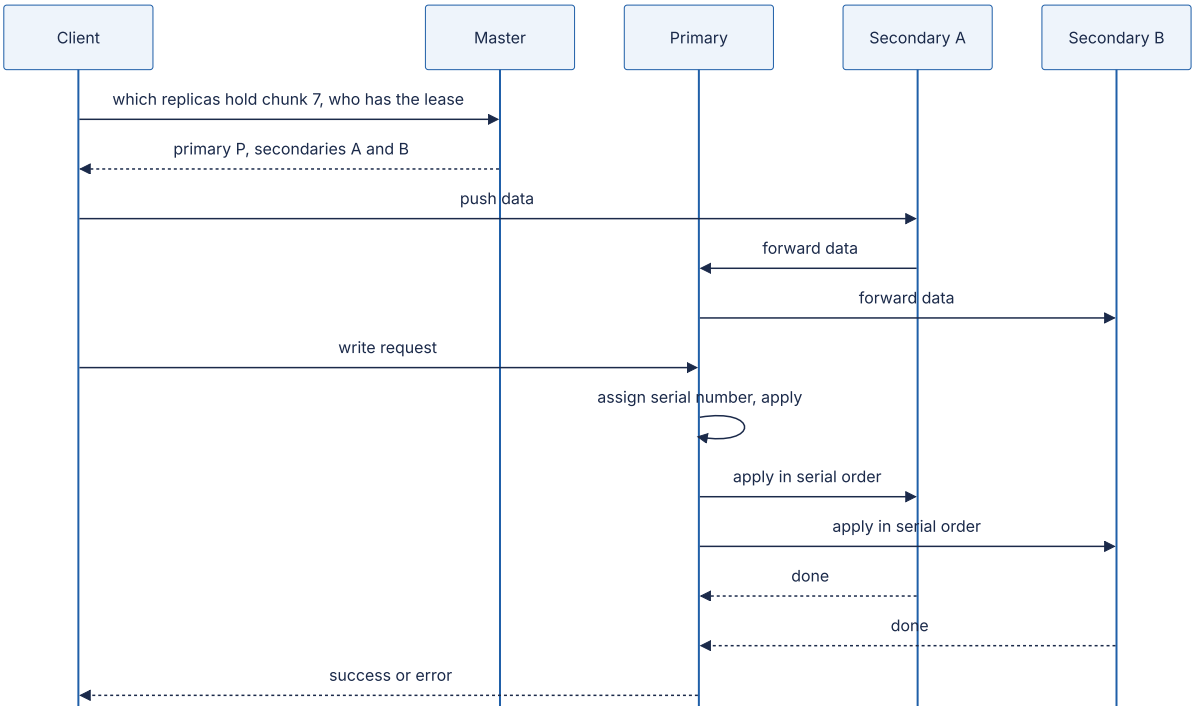


Figure 15.5 — HDFS with NameNode high availability.

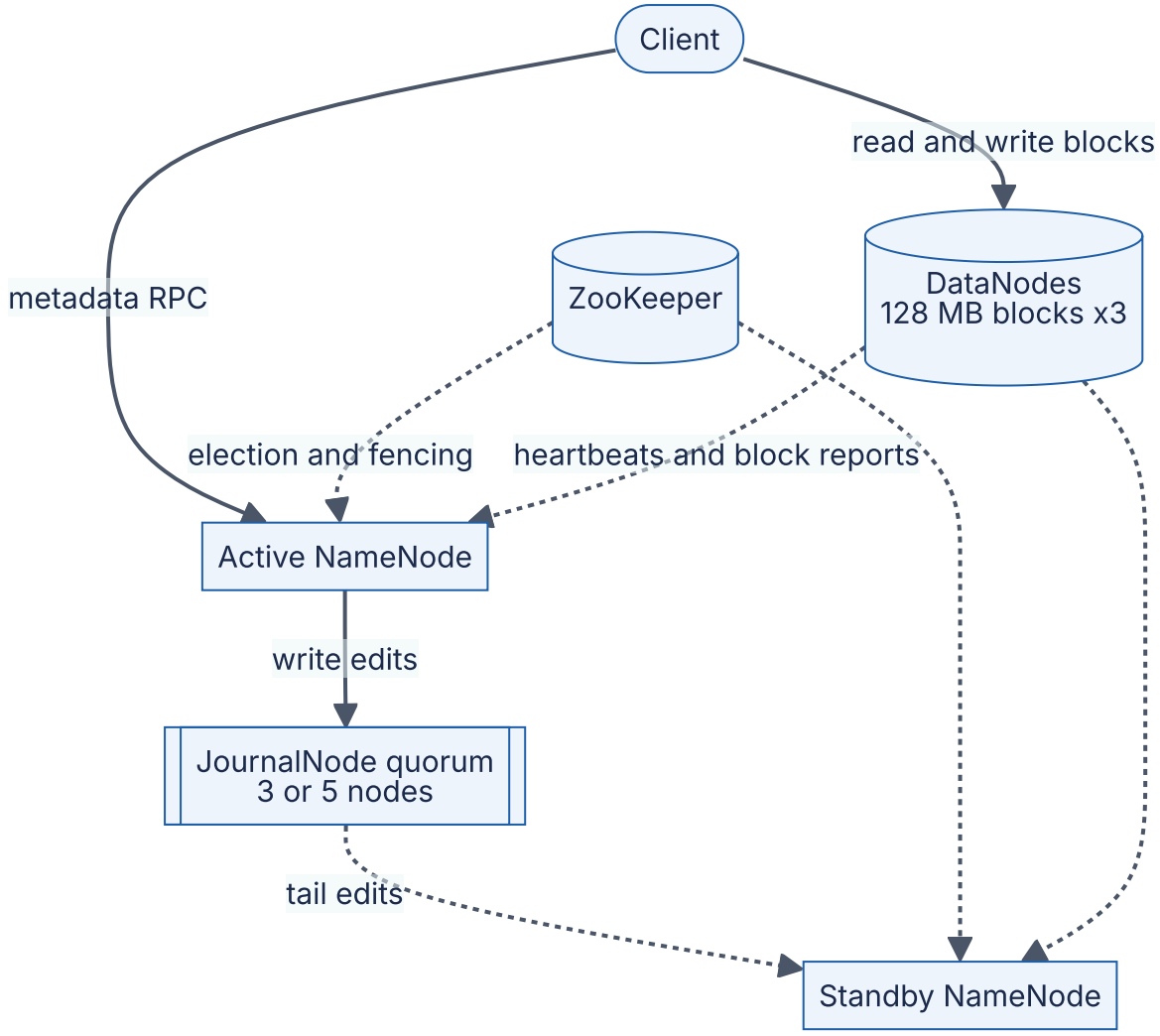


Figure 15.6 — An RS(10,4) stripe across 14 failure domains.

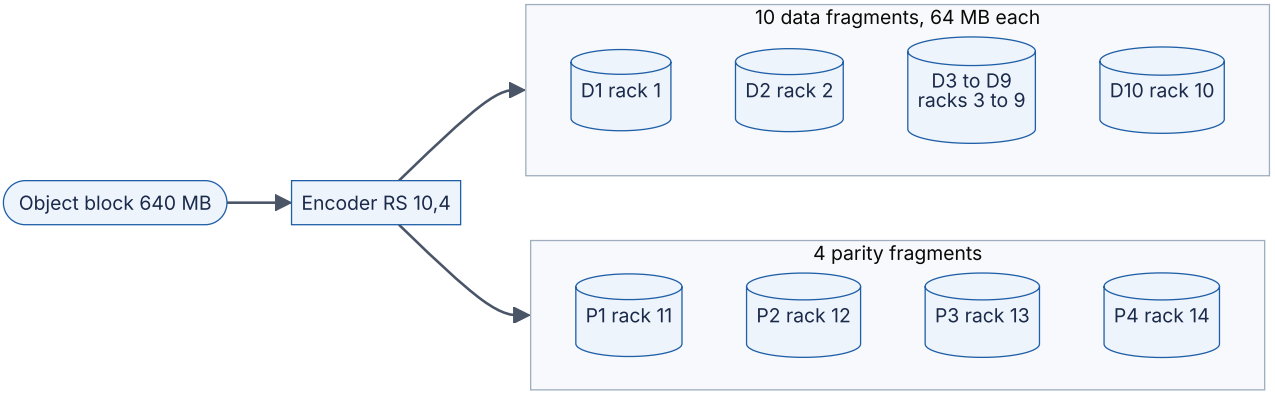


Figure 15.7 — Azure-style LRC(12,2,2) stripe.

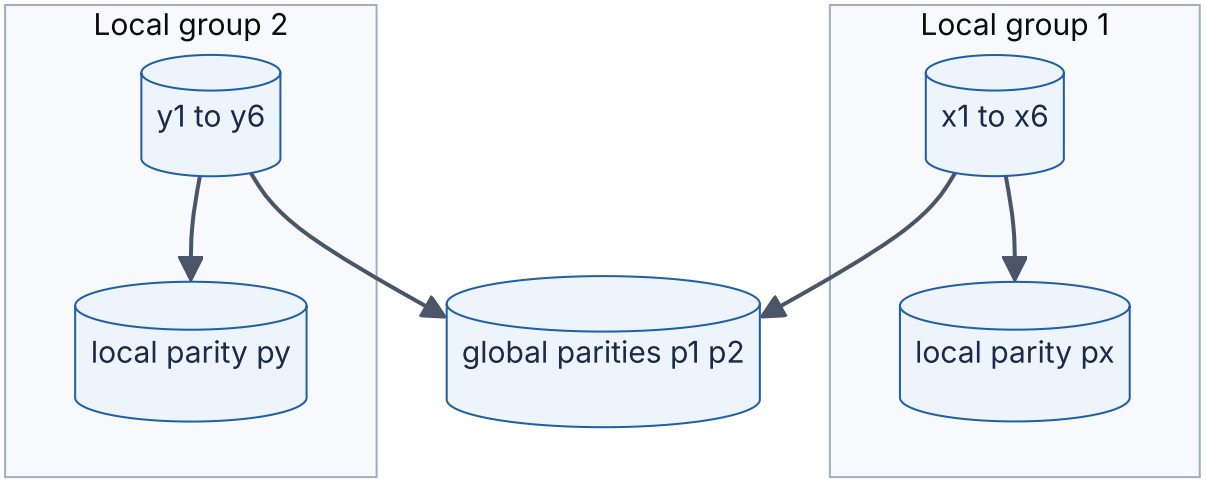


Figure 15.8 — Layered, partitioned file-system metadata (after Tectonic).

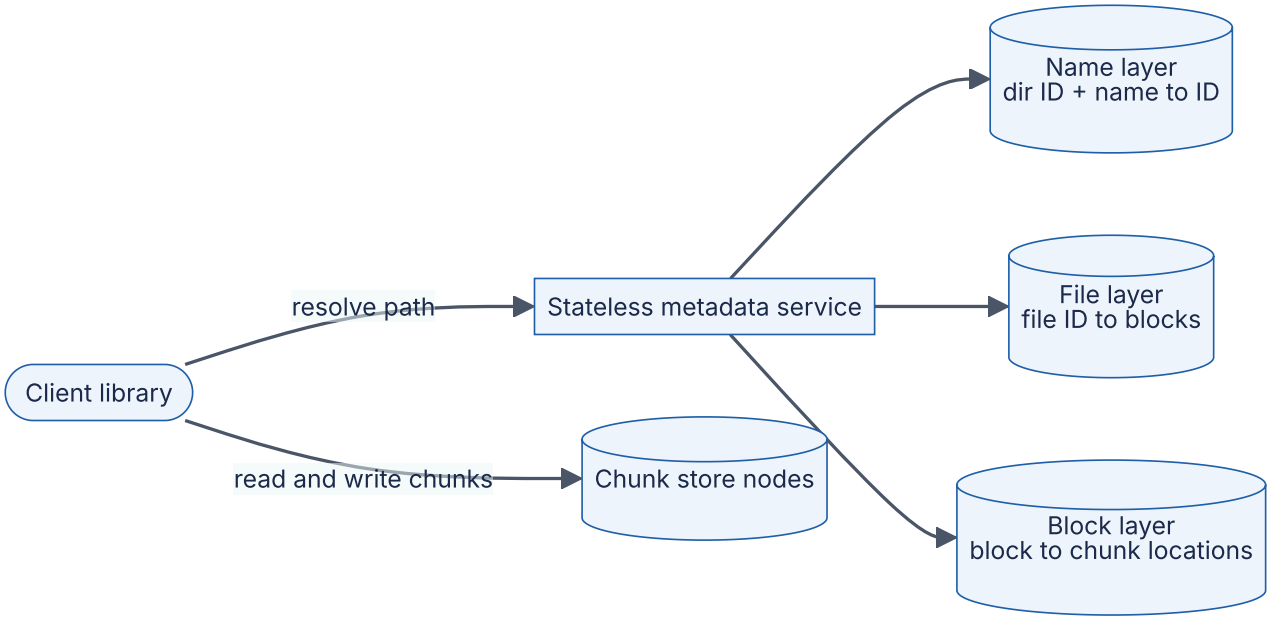


Figure 15.9 — Multipart upload sequence.

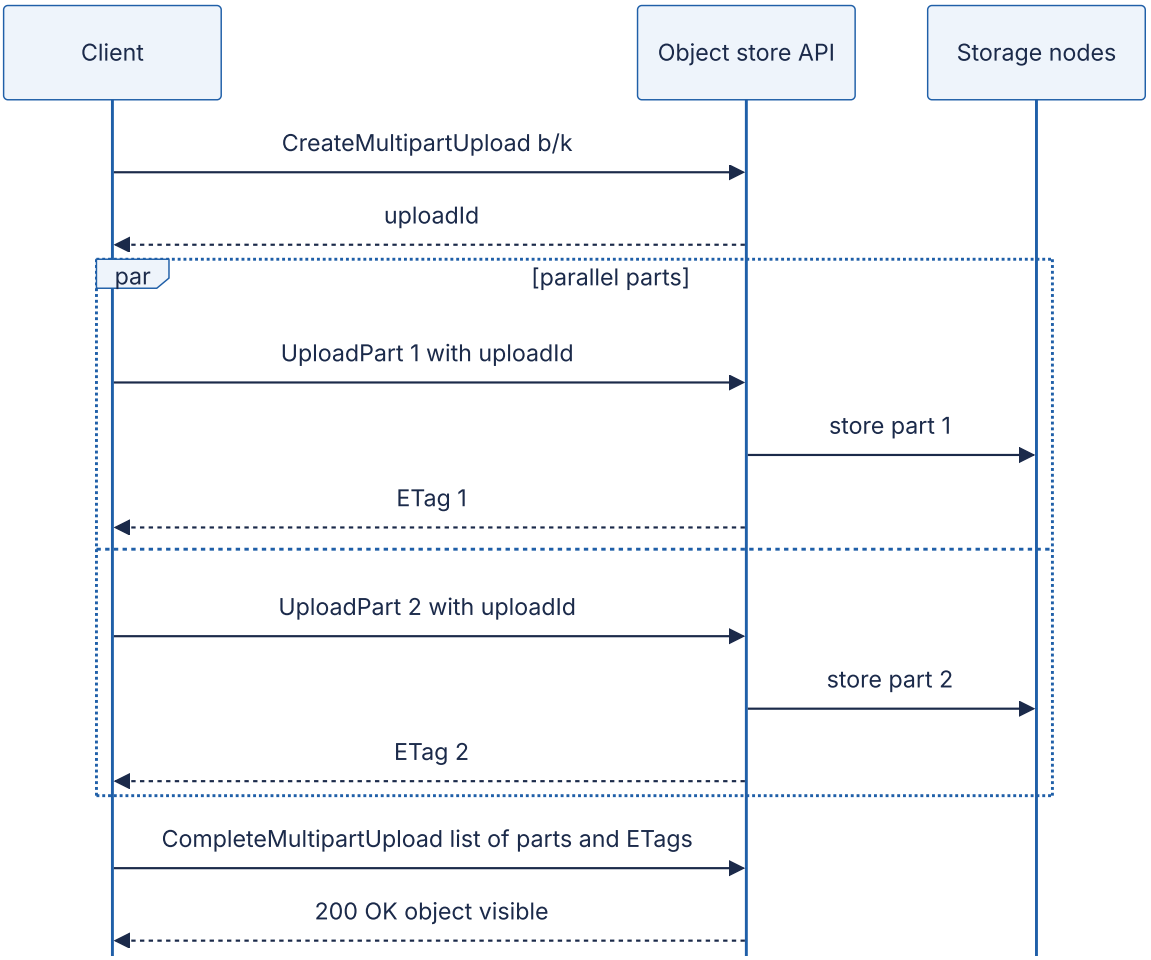
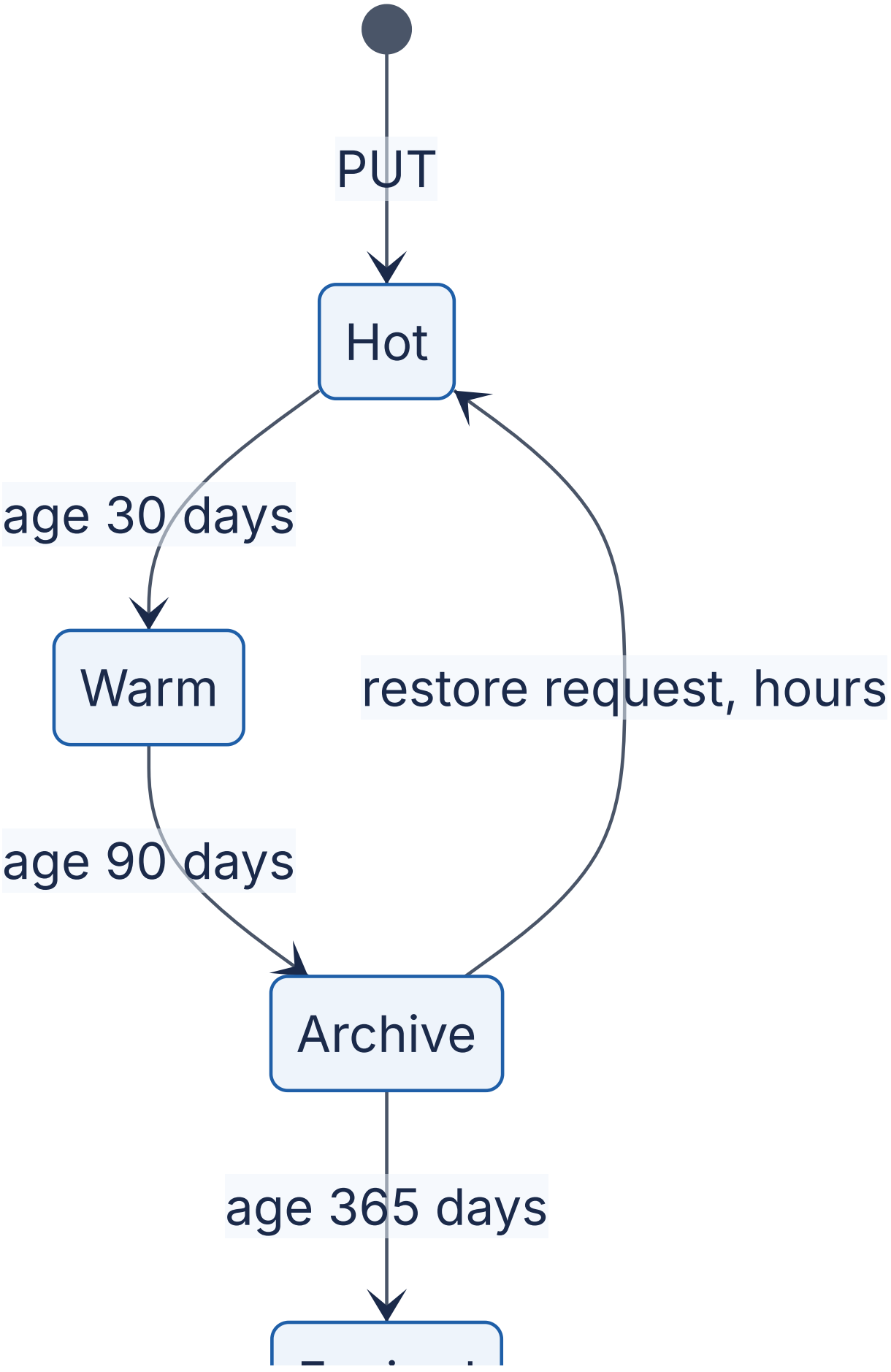
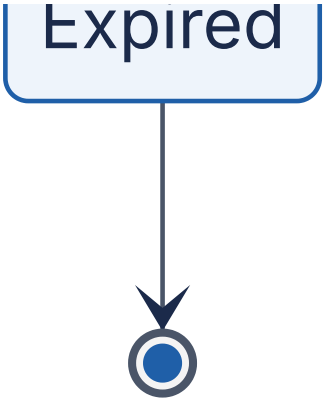


Figure 15.10 — A lifecycle policy for log objects.


PUT
Hot
age 30 days
Warm
restore request, hours
age 90 days
Archive
age 365 days



Chapter 16 — Search Systems

Figure 16.1 — The two paths of a search system.

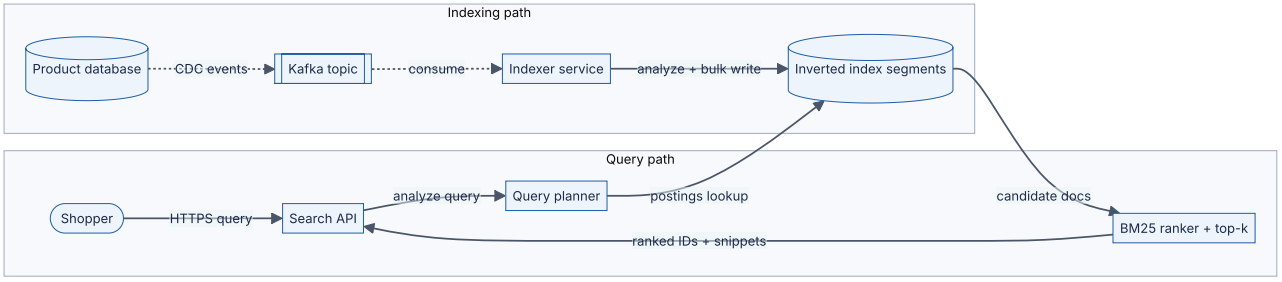


Figure 16.2 — Dictionary, postings lists and positions.

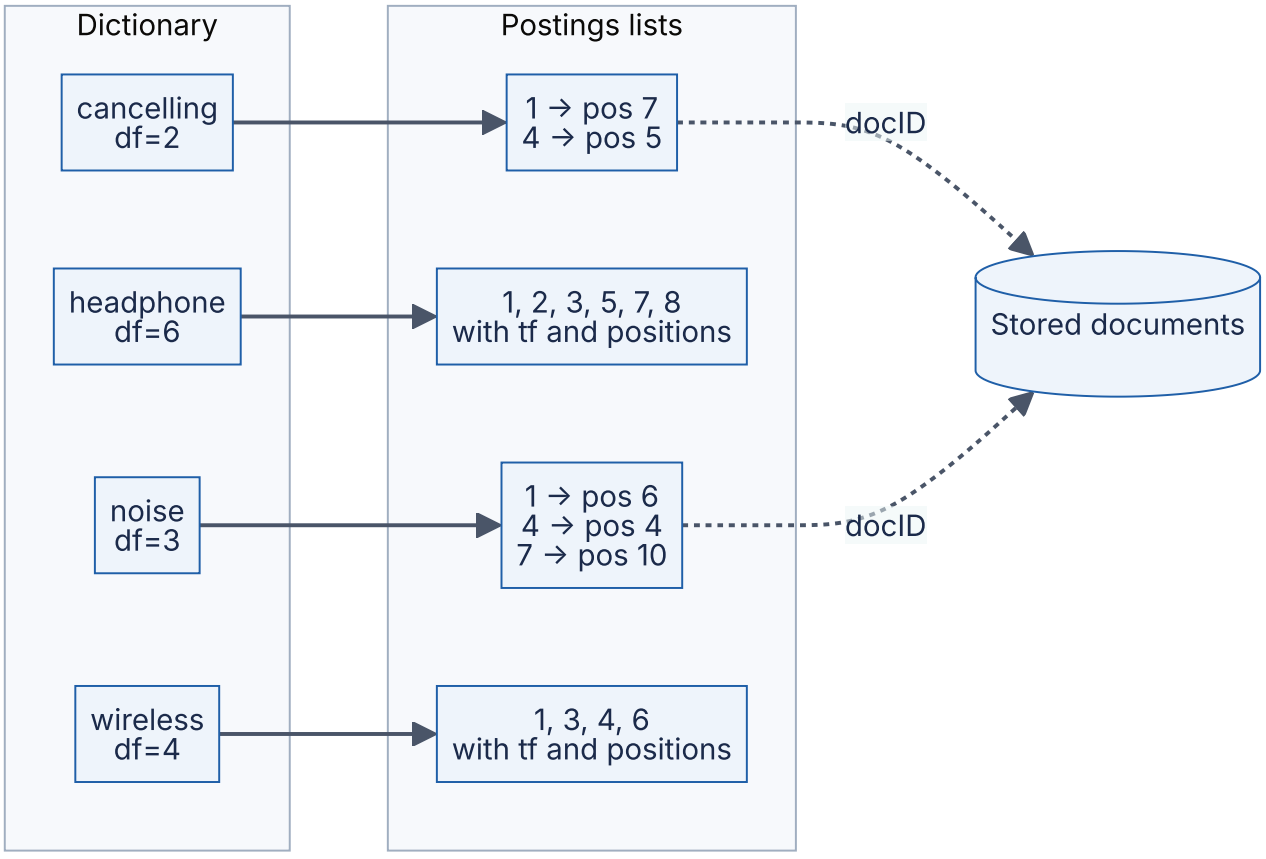


Figure 16.3 — An analysis chain applied at index time and at query time.

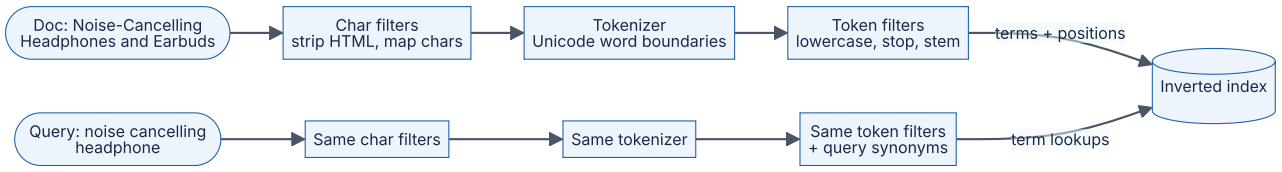


Figure 16.4 — Ranked query pipeline inside one shard.

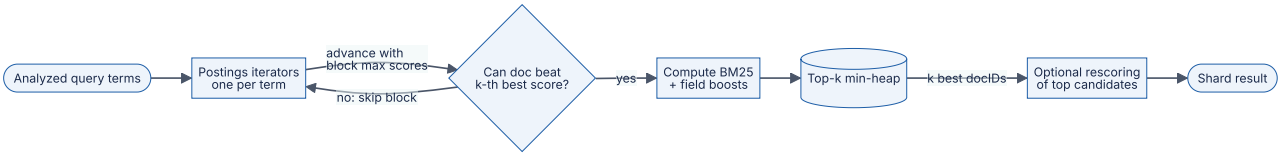
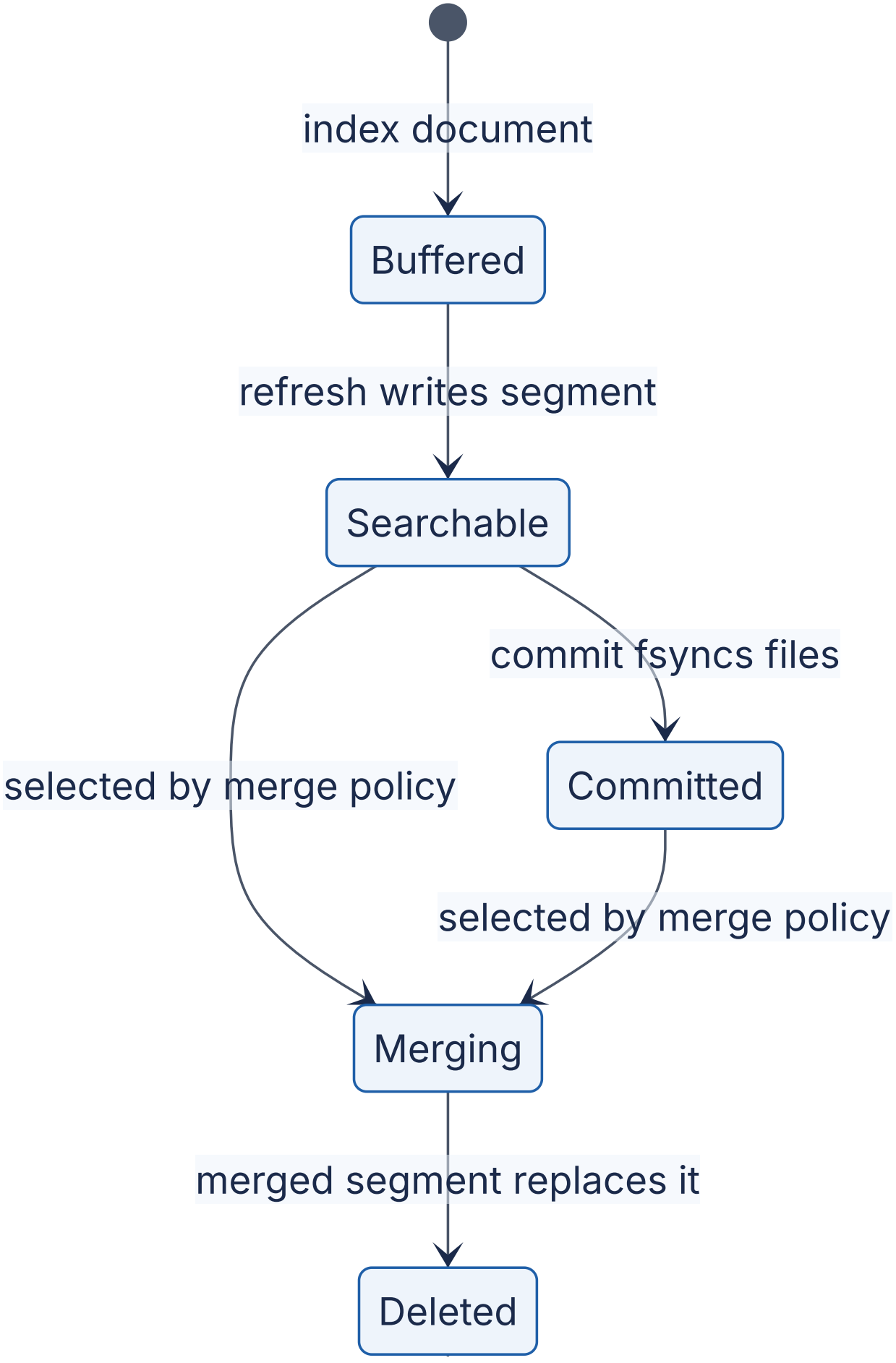


Figure 16.5 — Lifecycle of a Lucene segment.


index document
Buffered
refresh writes segment
Searchable
commit fsyncs files
selected by merge policy
Committed
selected by merge policy
Merging
merged segment replaces it
Deleted

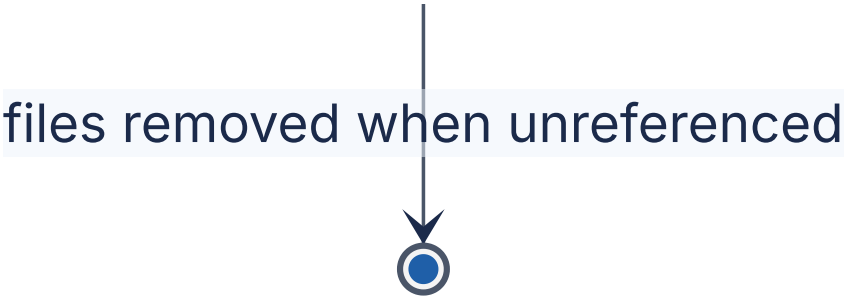


Figure 16.6 — Document partitioning versus term partitioning.



Figure 16.7 — Query-then-fetch across three shards.

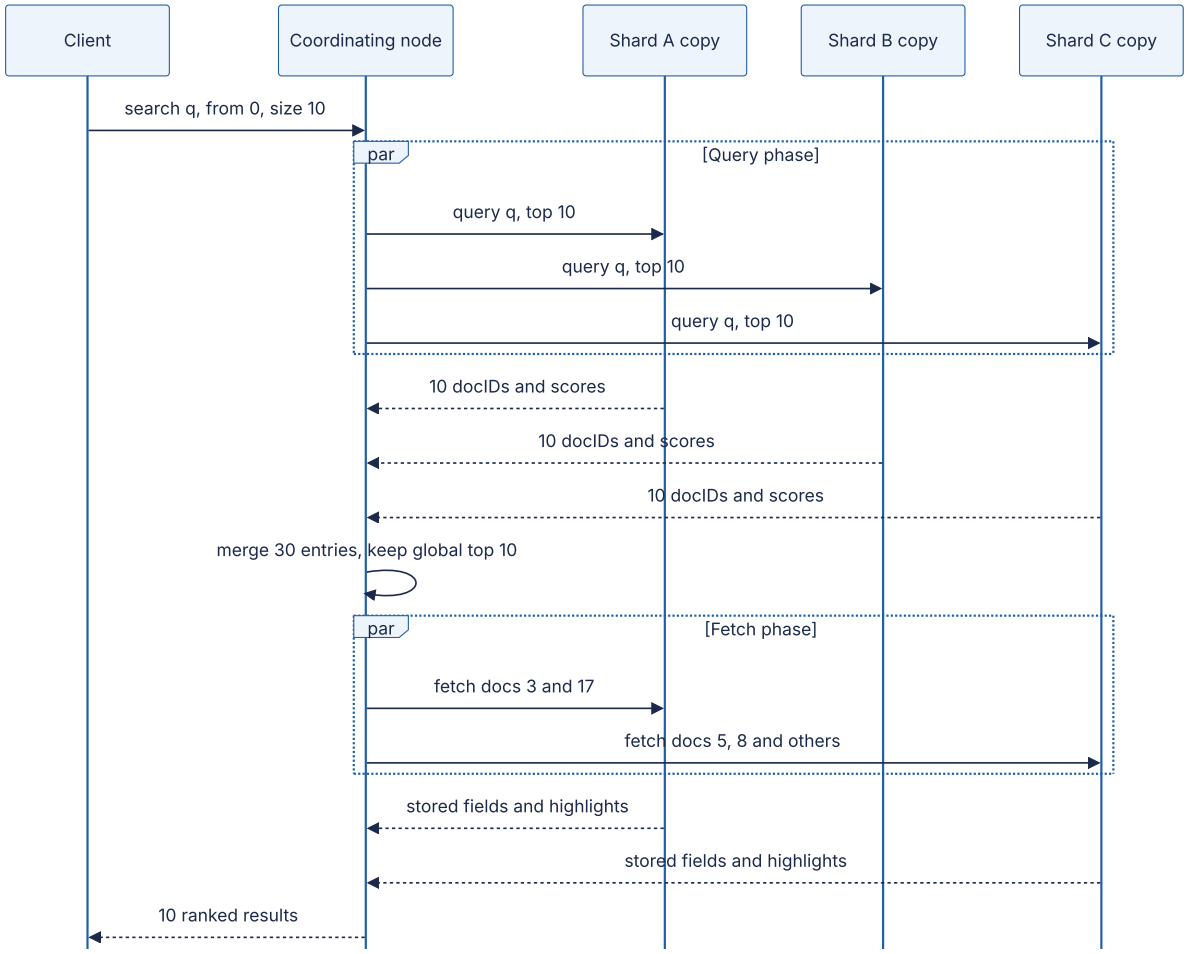


Figure 16.8 — A three-zone Elasticsearch cluster with two primaries and one replica each.

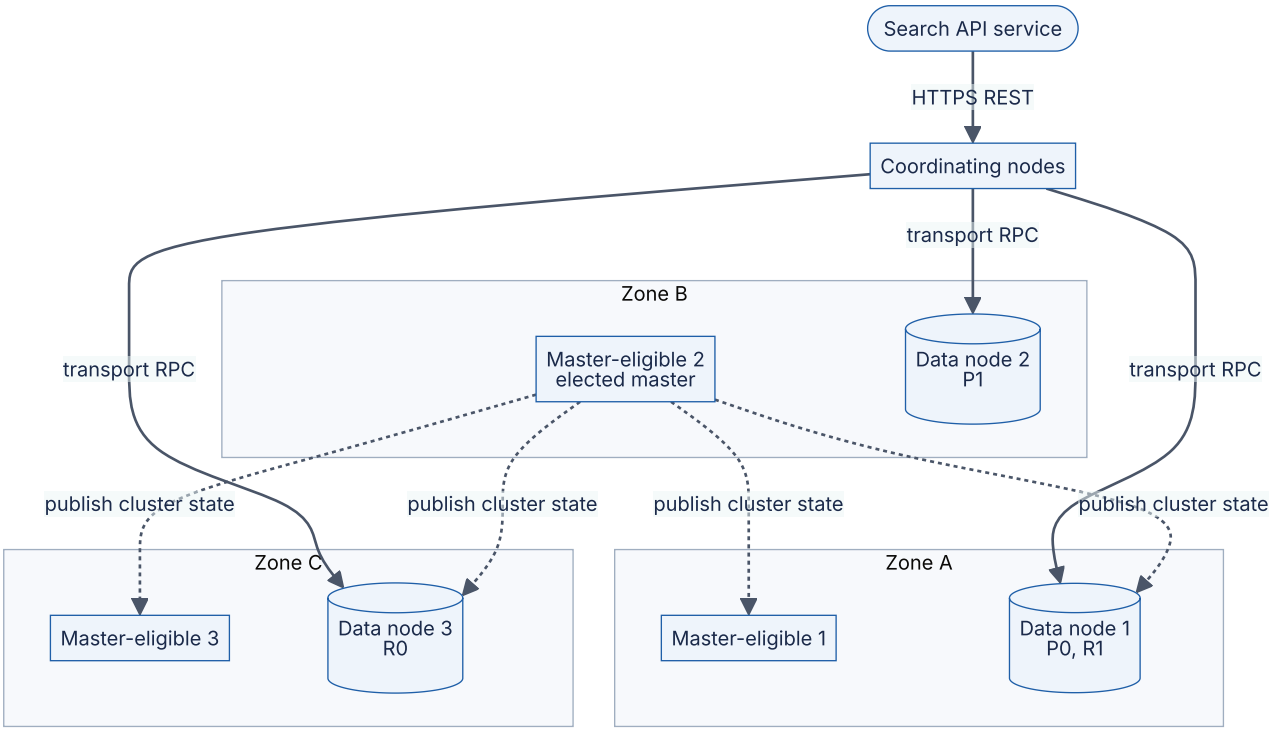


Figure 16.9 — Indexing one document with one replica.

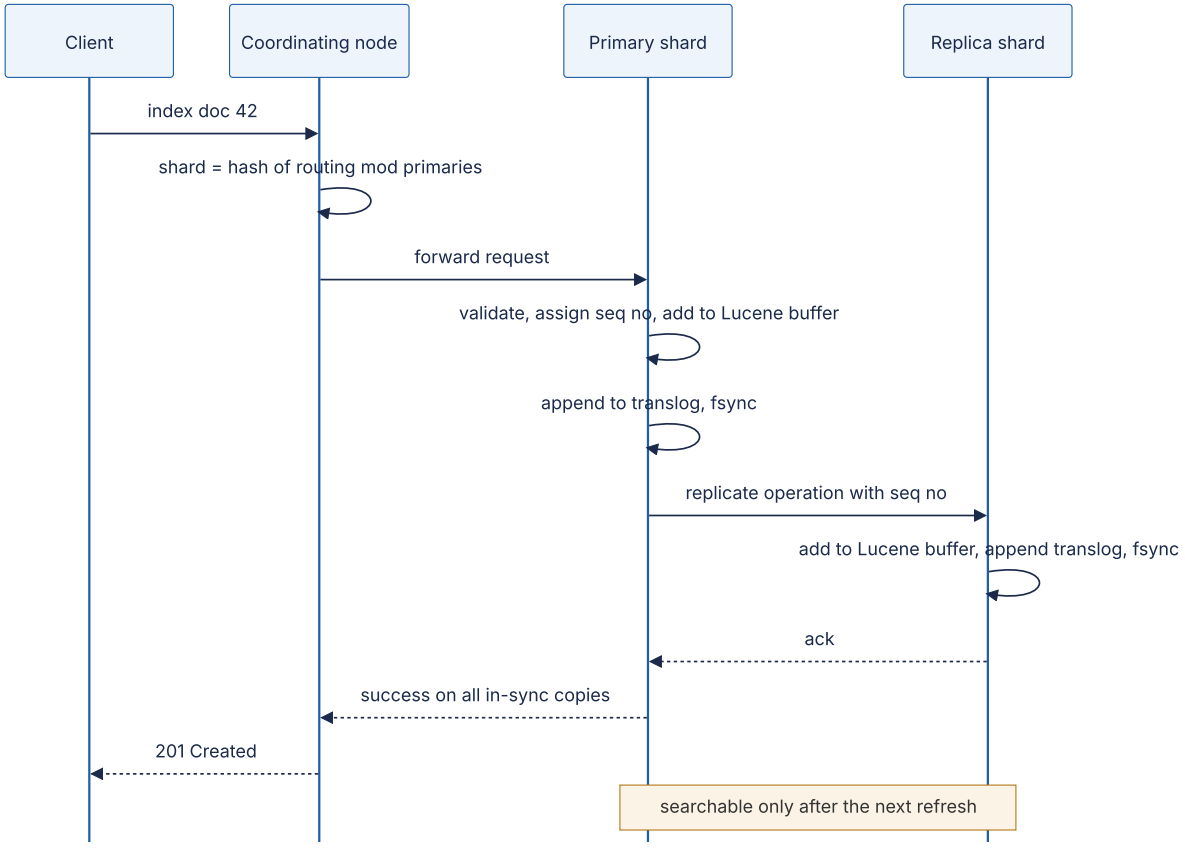


Figure 16.10 — A trie with top-2 completions stored at each node.

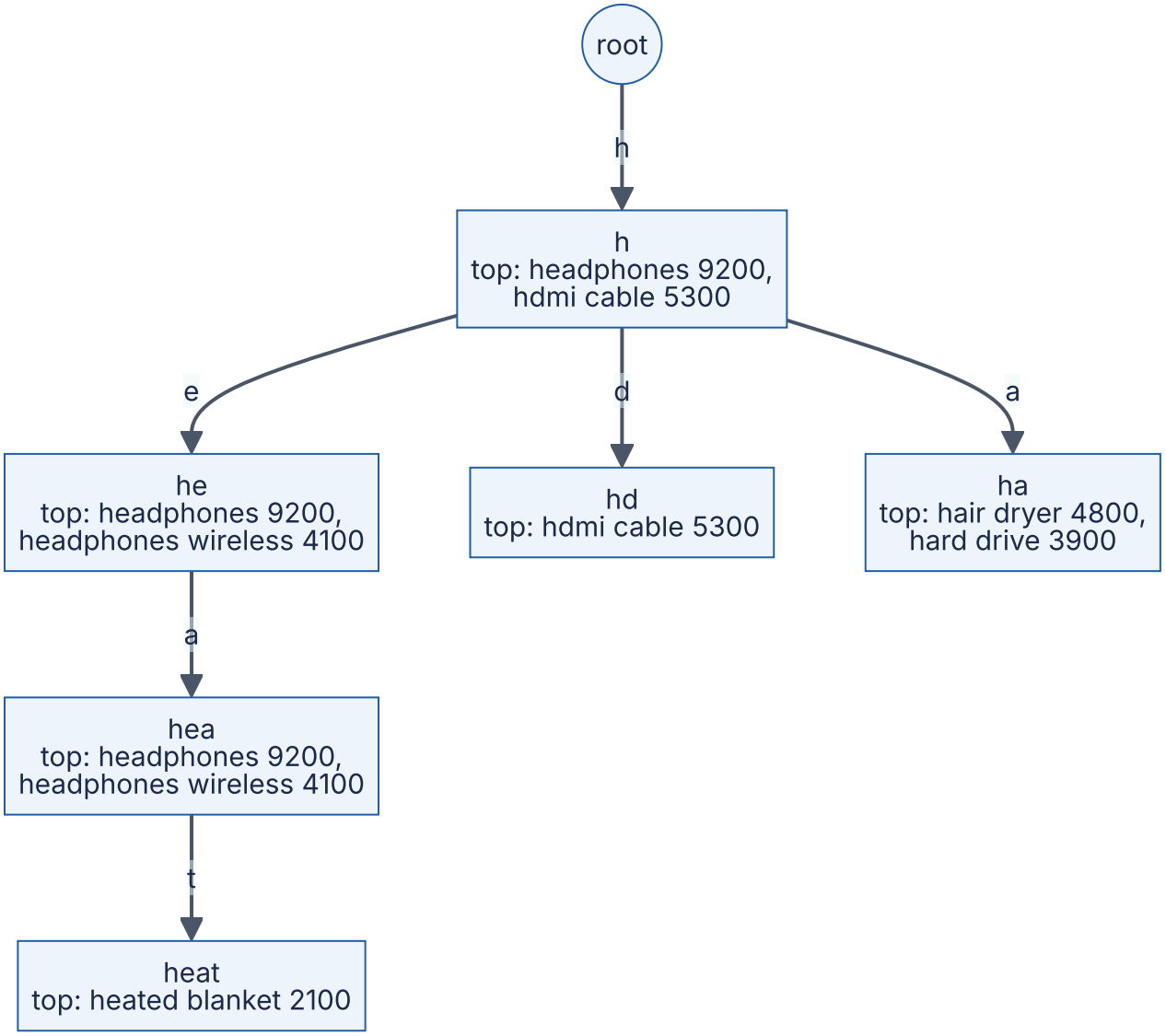


Figure 16.11 — Autocomplete: offline build and online serving.

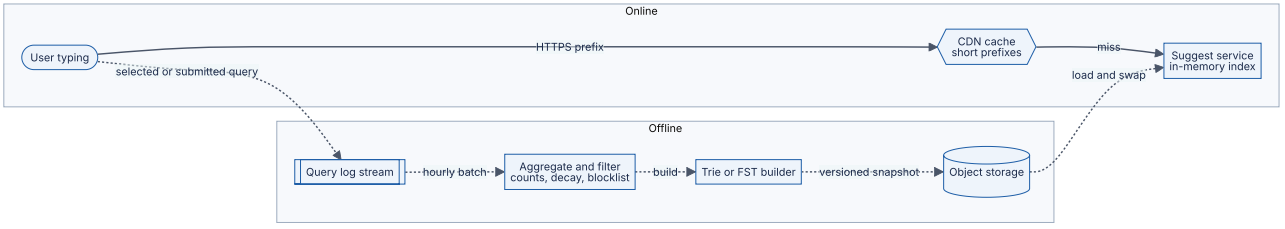


Figure 16.12 — Near-real-time indexing pipeline with CDC and Kafka.

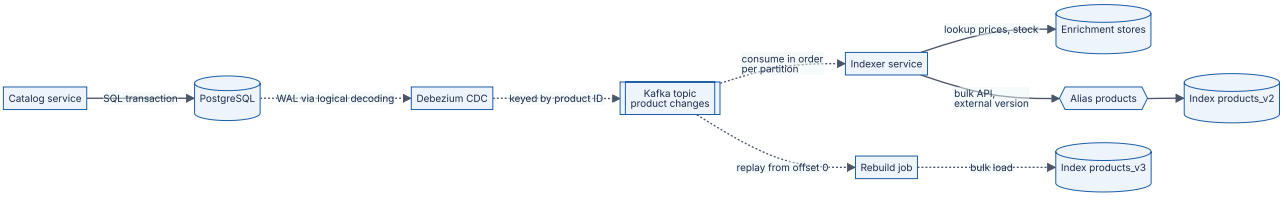
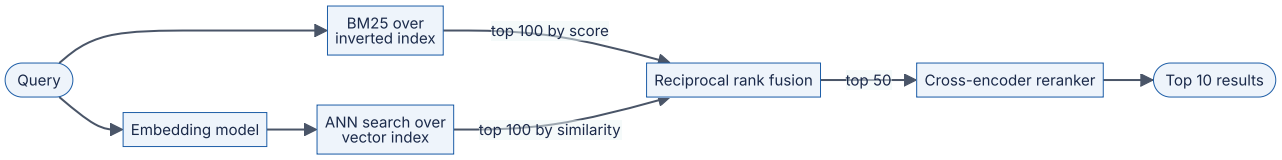


Figure 16.13 — Hybrid retrieval (preview of Chapters 74–75).



Chapter 17 — Data Structures and Algorithms for Scale

Figure 17.1 — Uniform hashing versus hash flooding.

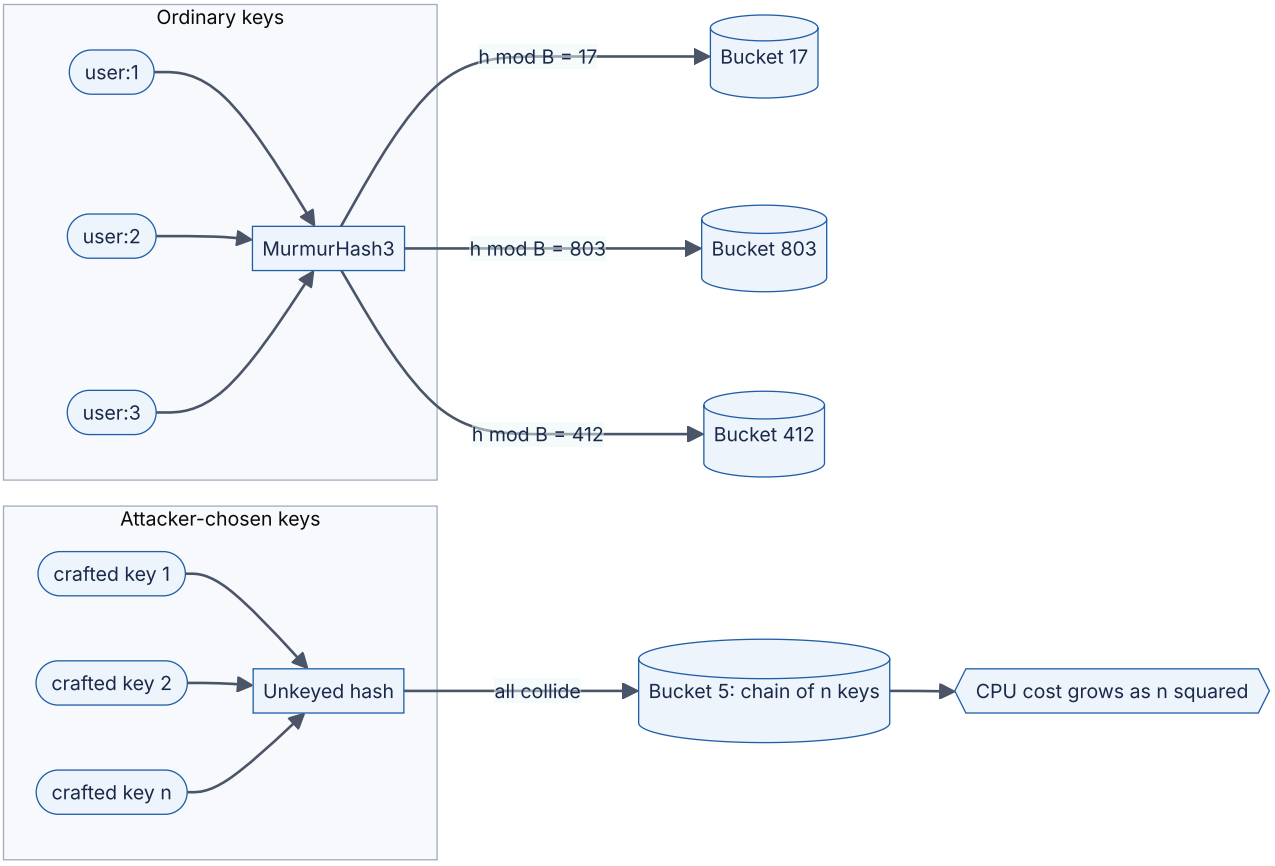


Figure 17.2 — Inserting and querying a Bloom filter with k = 3.

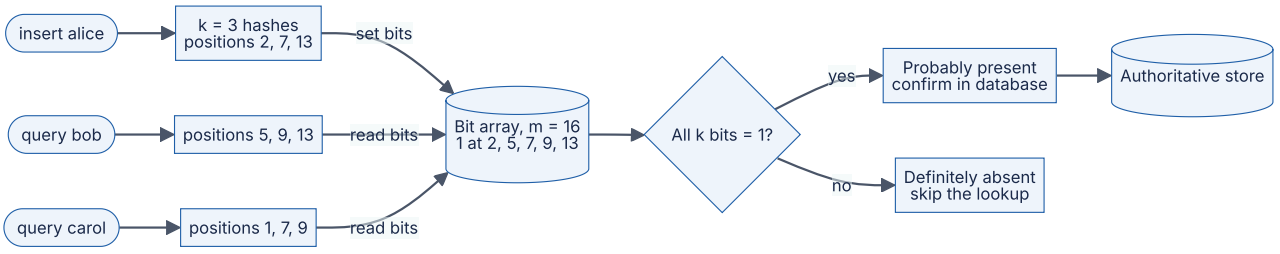


Figure 17.3 — Bloom filters on the read path of an LSM-tree storage engine.

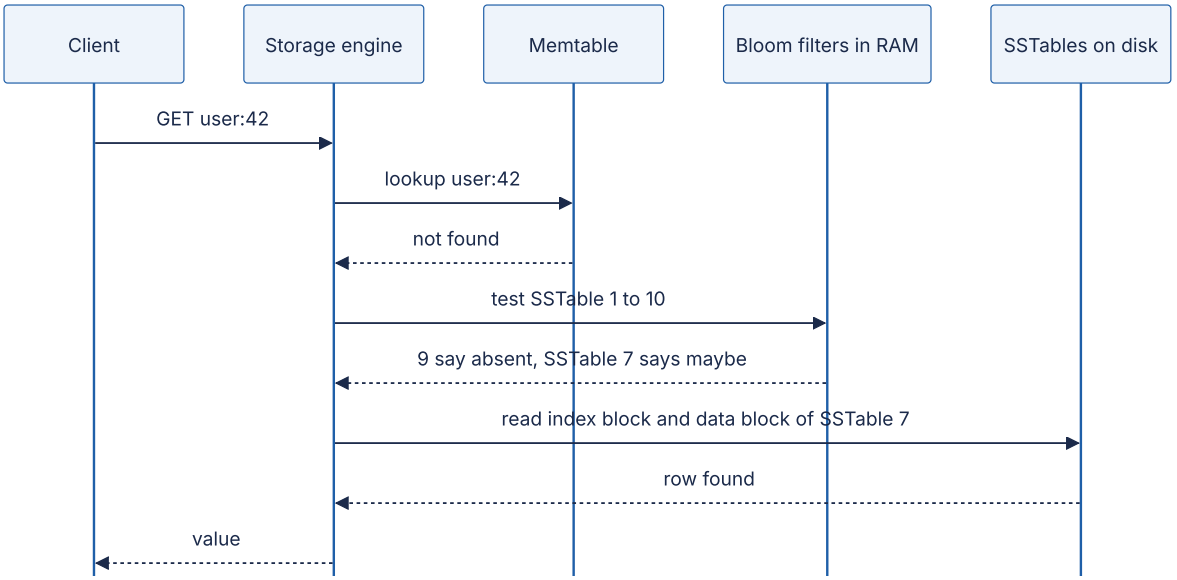


Figure 17.4 — Inserting into a full cuckoo filter by relocation.

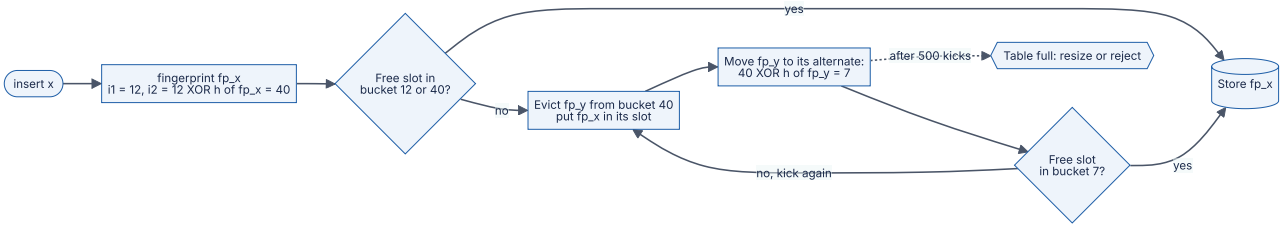


Figure 17.5 — The HyperLogLog update and estimate path.

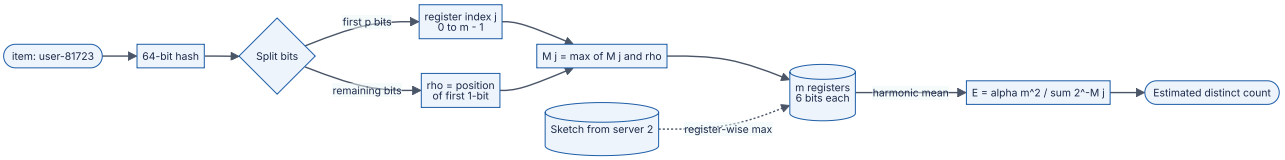


Figure 17.6 — A Count-Min sketch with d = 3 rows.

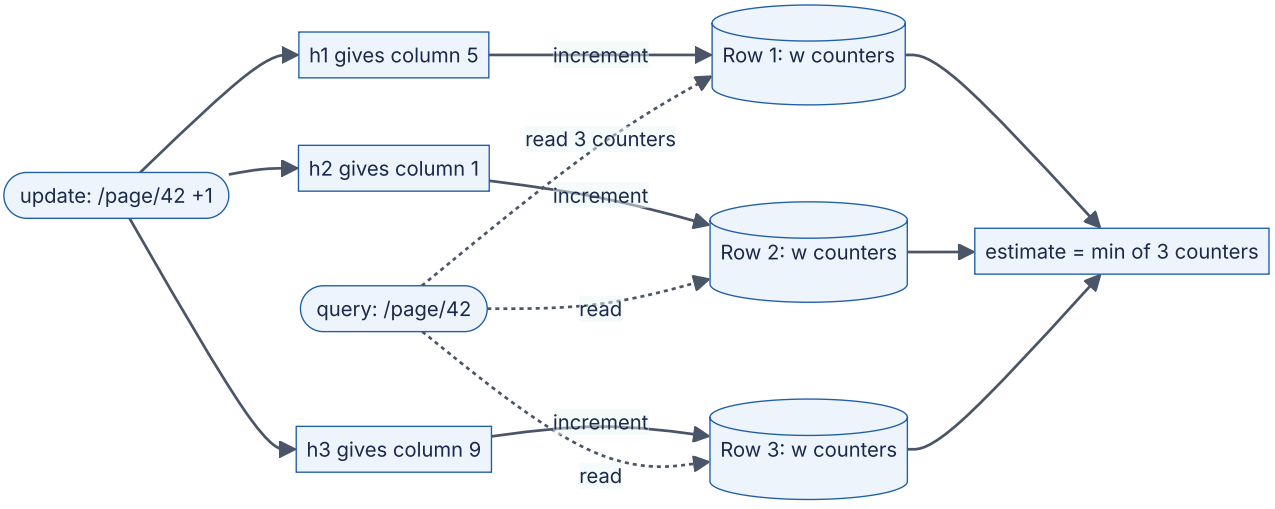


Figure 17.7 — A distributed top-k pipeline.

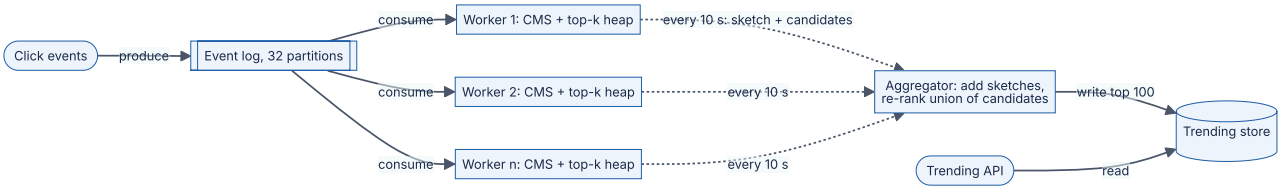


Figure 17.8 — A skip list with three levels; the bold path finds 40.

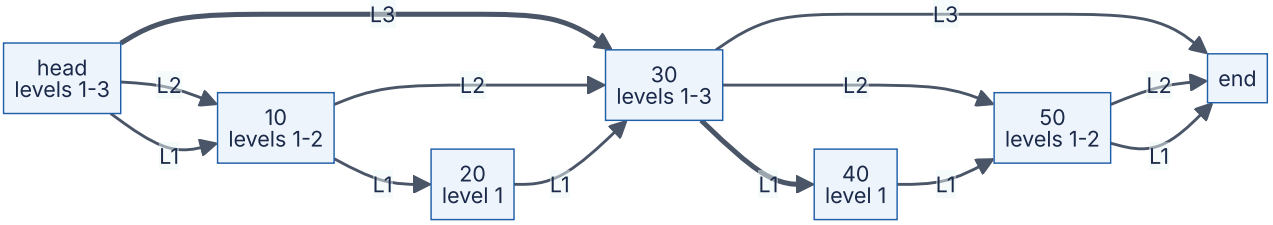


Figure 17.9 — A Merkle tree over four records and the inclusion proof for record C.

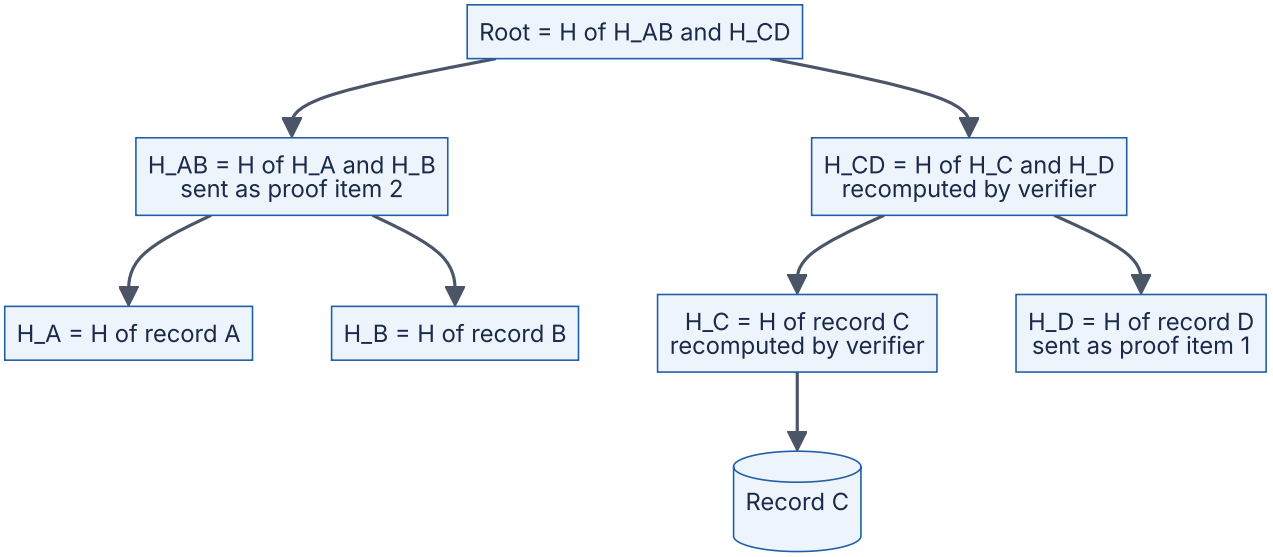


Figure 17.10 — Merkle-tree anti-entropy between two replicas.

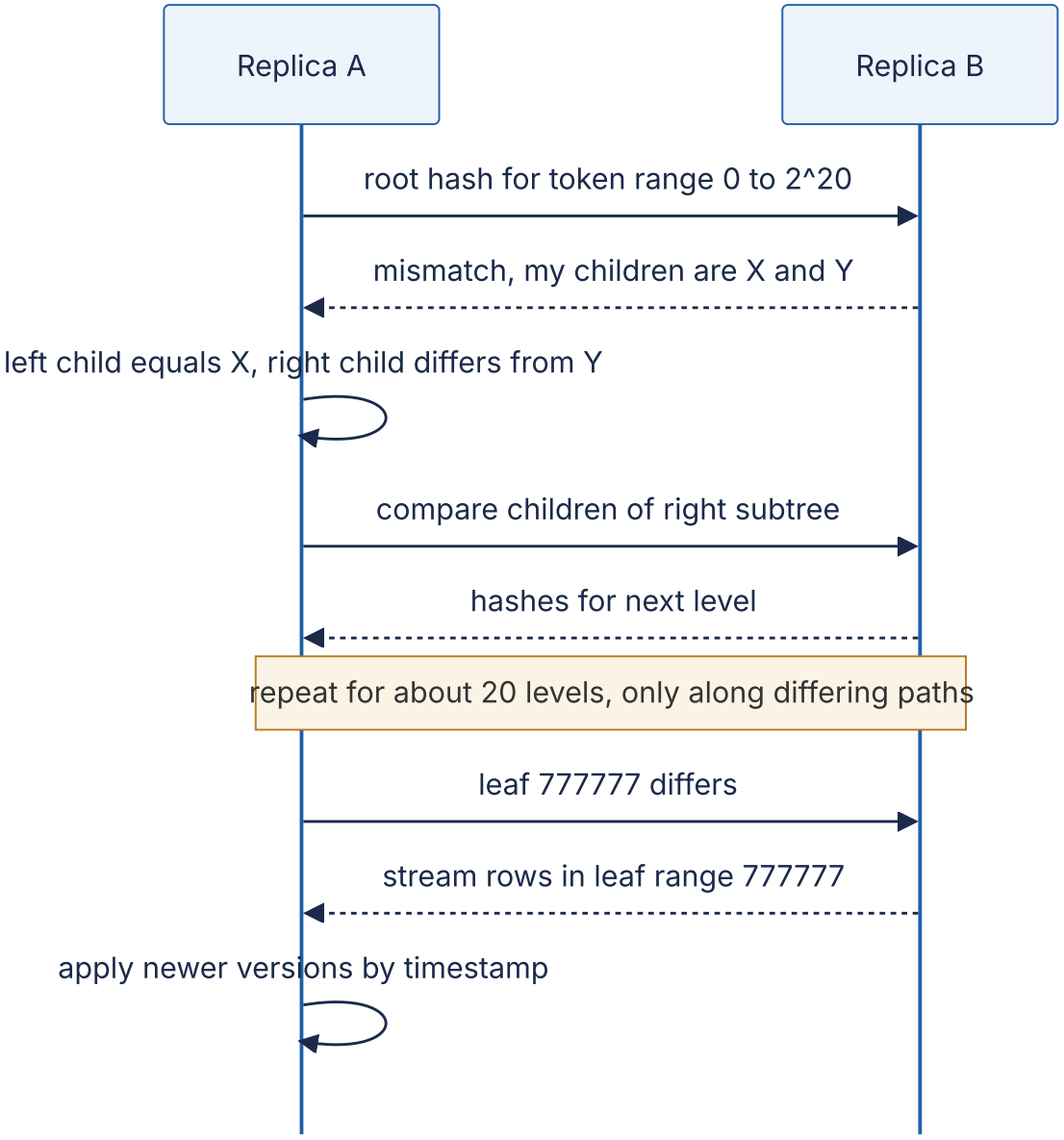


Figure 17.11 — Geohash refinement: each character narrows the cell.

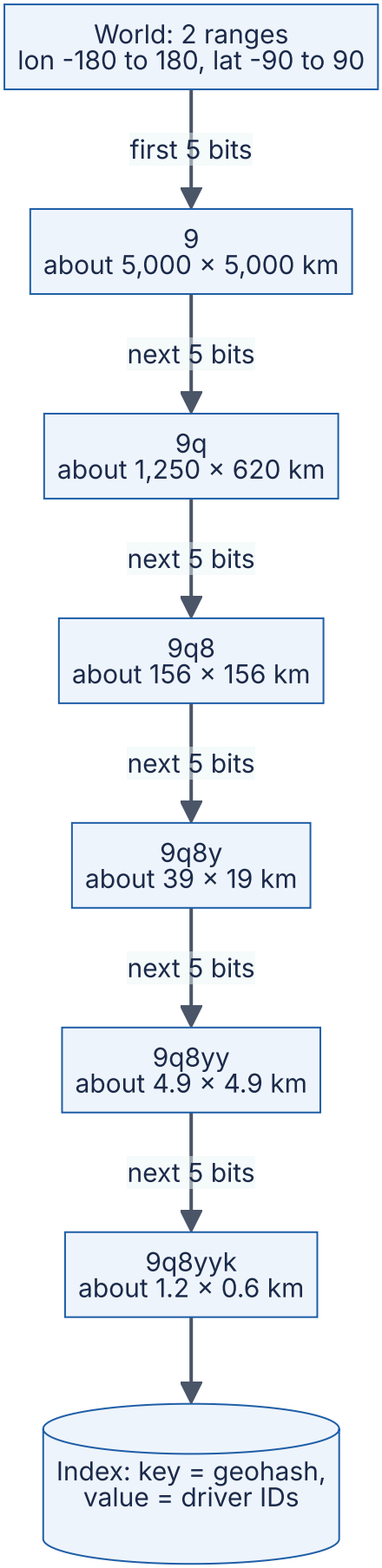

World: 2 ranges
lon -180 to 180, lat -90 to 90
first 5 bits
9
about 5,000 × 5,000 km
next 5 bits
9q
about 1,250 × 620 km
next 5 bits
9q8
about 156 × 156 km
next 5 bits
9q8y
about 39 × 19 km
next 5 bits
9q8yy
about 4.9 × 4.9 km
next 5 bits
9q8yyk
about 1.2 × 0.6 km
Index: key = geohash,
value = driver IDs

Figure 17.12 — An adaptive quadtree: dense areas split further.

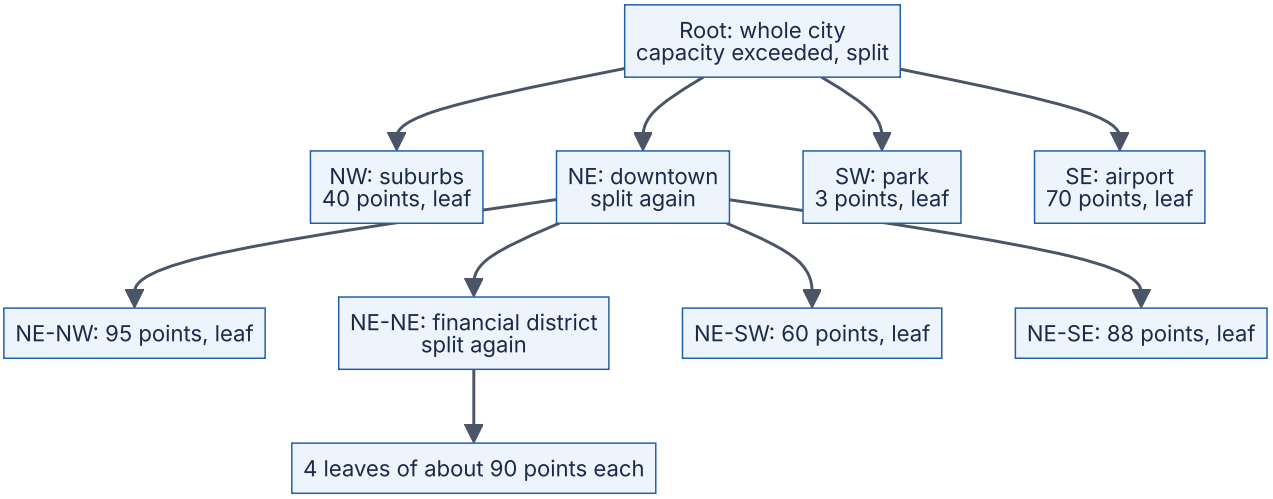


Figure 17.13 — A nearby-drivers service built on a cell index.



Figure 17.14 — The Snowflake 64-bit ID layout.



Figure 17.15 — Flickr-style ticket servers.

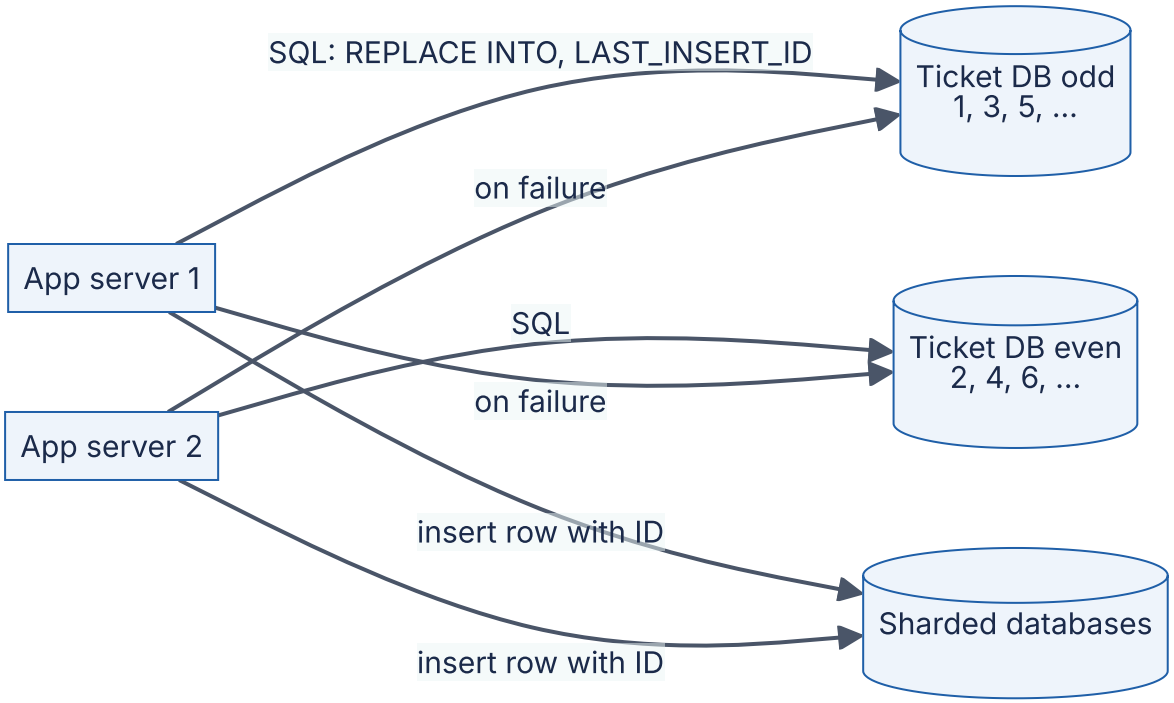
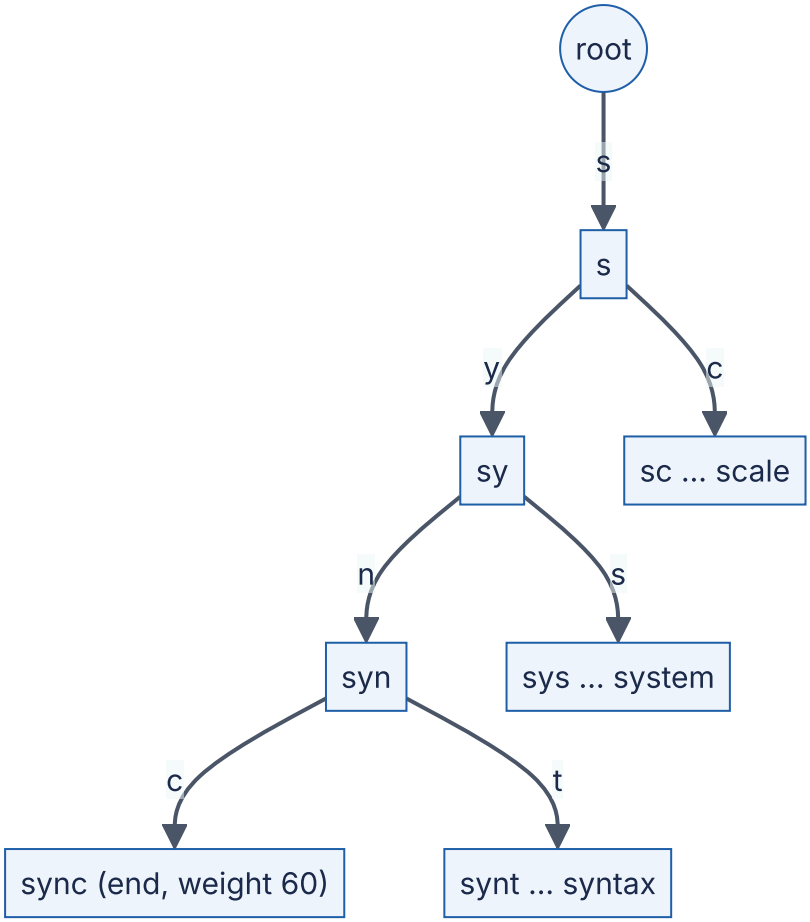
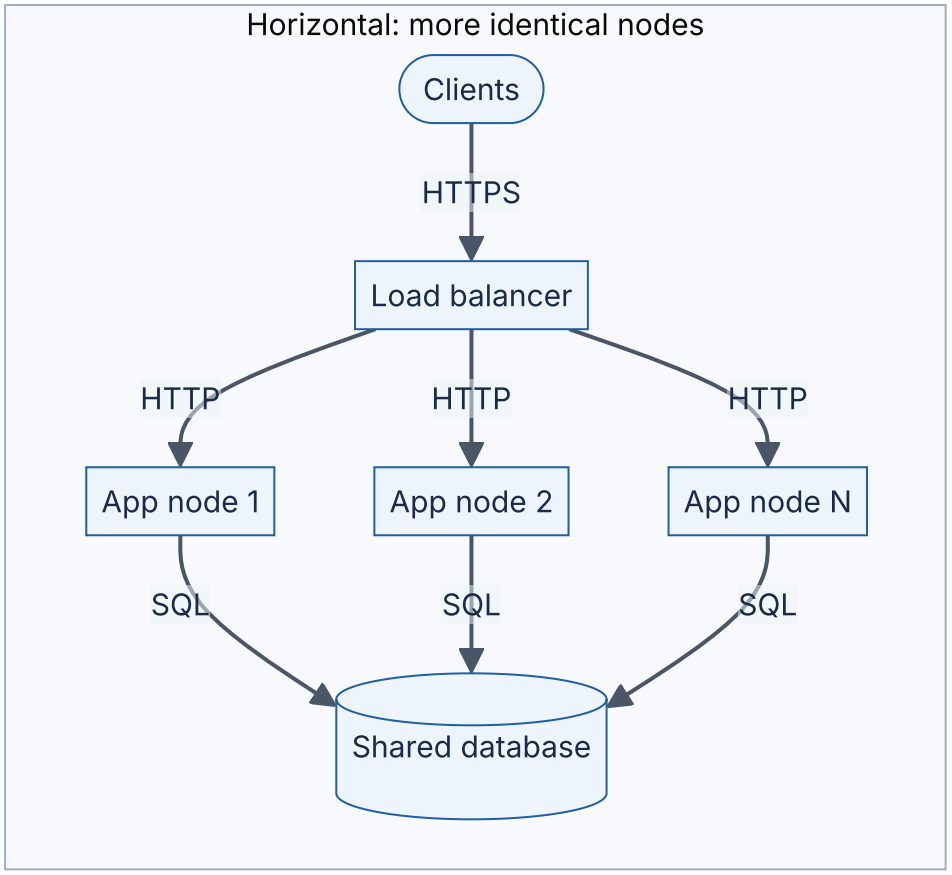


Figure 17.16 — A trie holding sync, syntax, system and scale.



Chapter 18 — Scalability

Figure 18.1 — Vertical versus horizontal scaling.



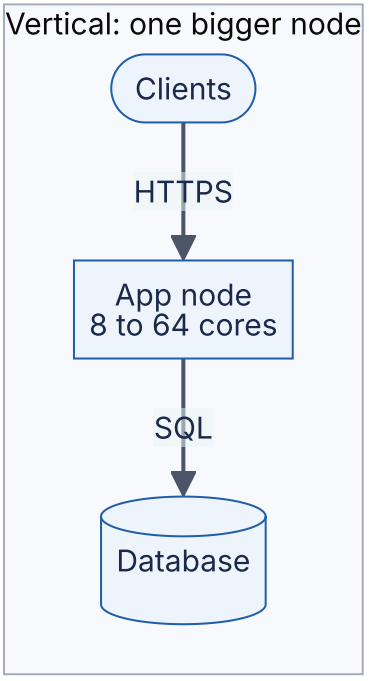
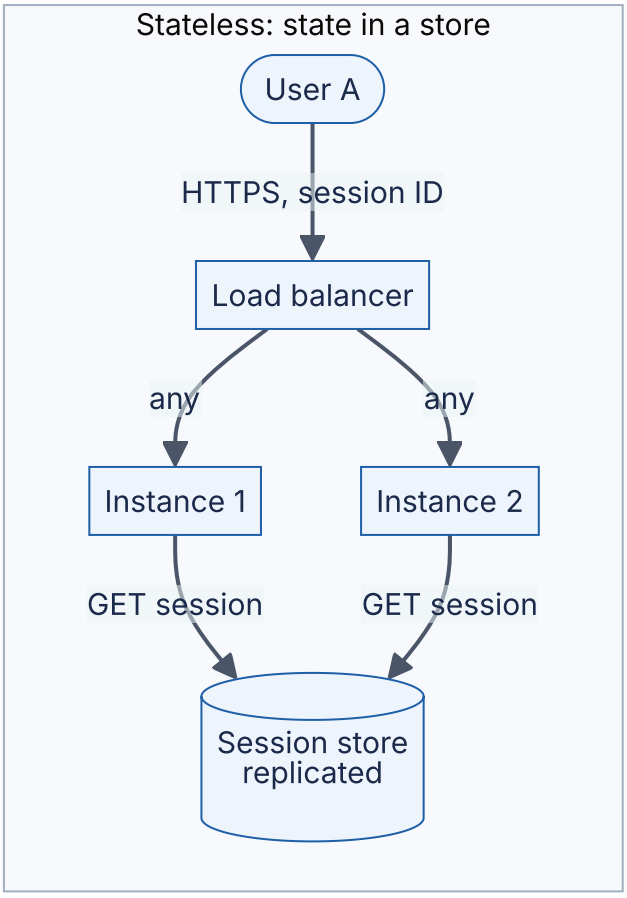


Figure 18.2 — Sticky sessions versus an externalized session store.



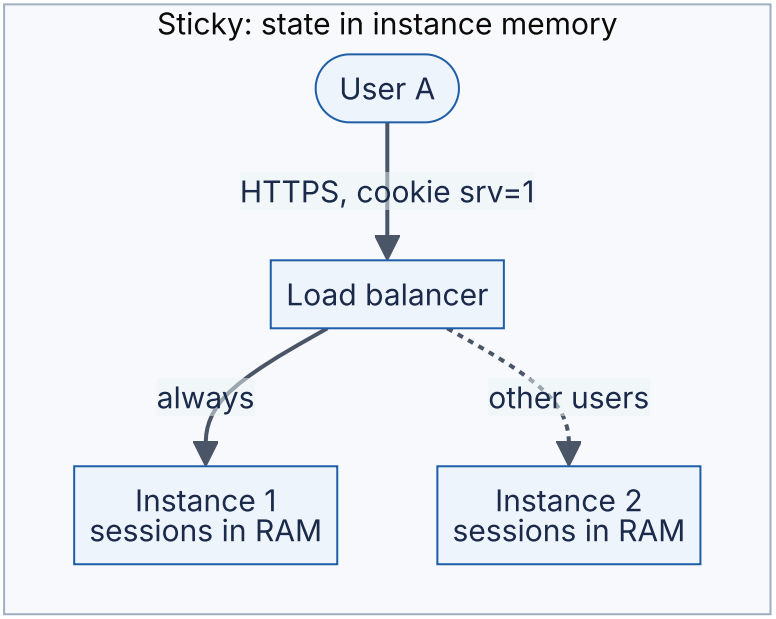


Figure 18.3 — Layers of load distribution.

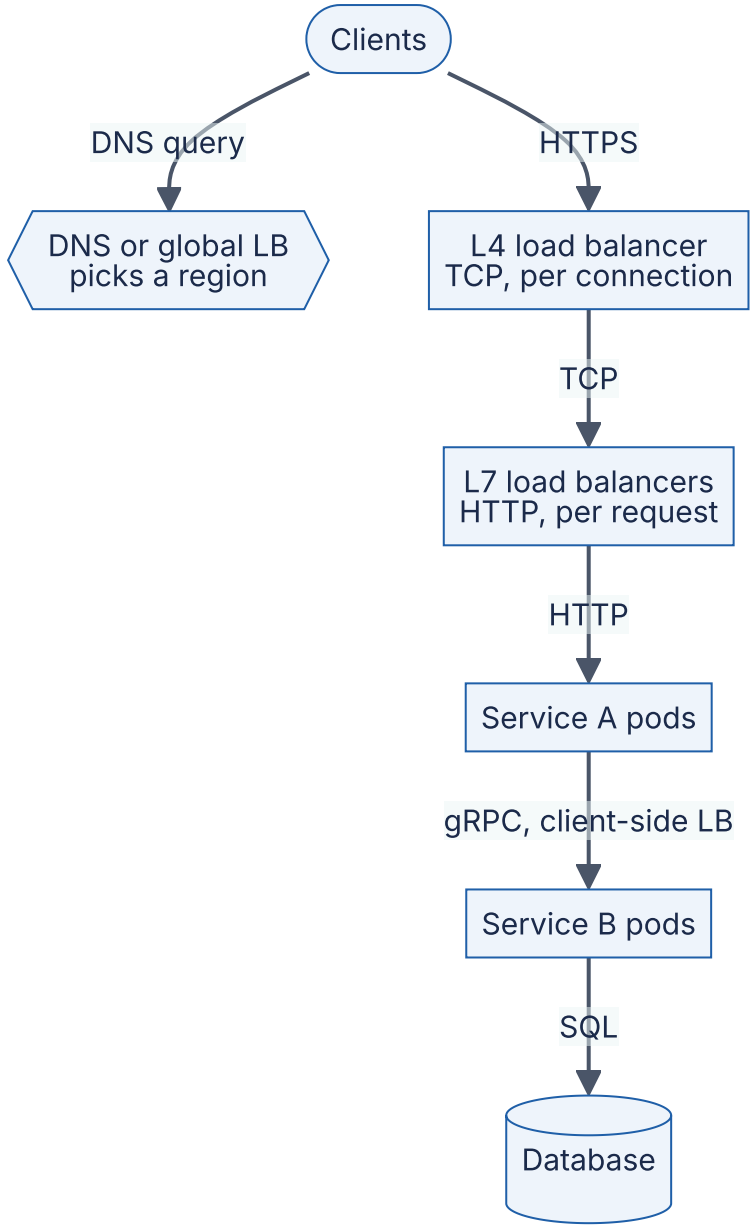


Figure 18.4 — The shared-resource funnel.

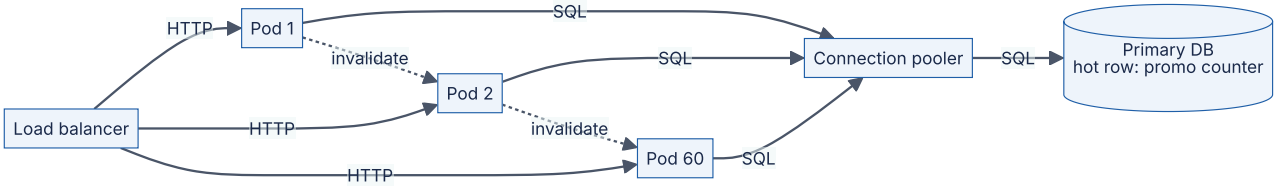


Figure 18.5 — The autoscaling control loop.

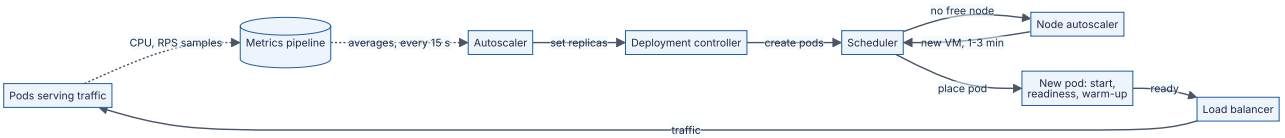


Figure 18.6 — Two hundred pods against one connection limit.

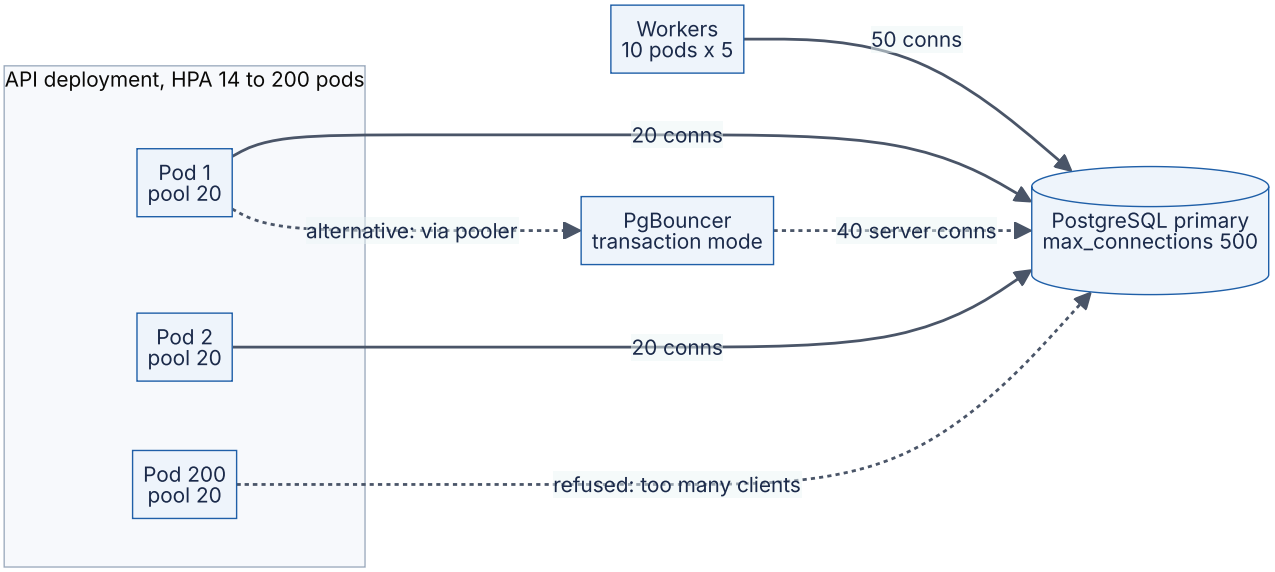


Figure 18.7 — The scale cube applied to QuickBite.

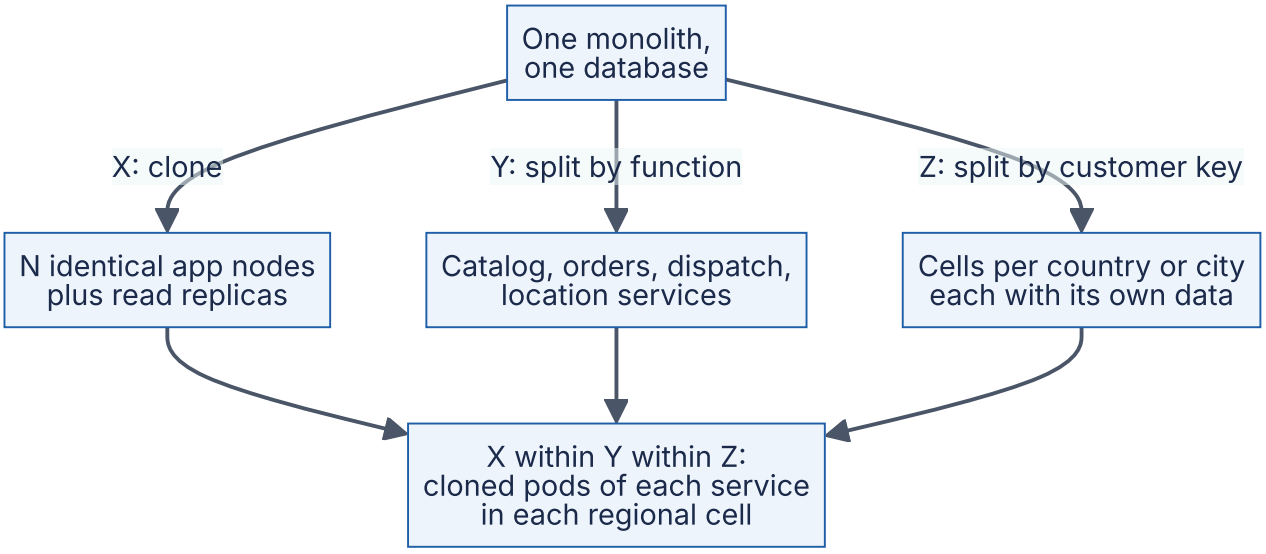


Figure 18.8 — The capacity planning loop.

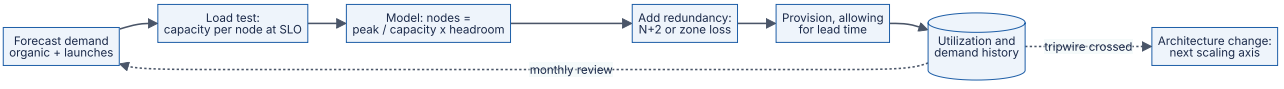


Figure 18.9 — Stage S1: 1,000 DAU.

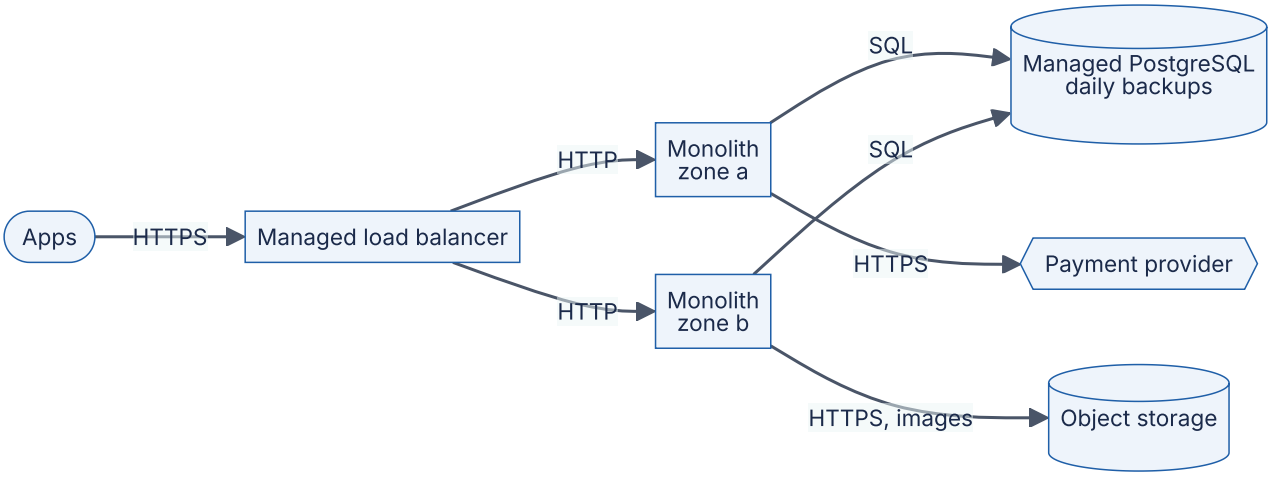


Figure 18.10 — Stage S2: 20,000 DAU (one city).

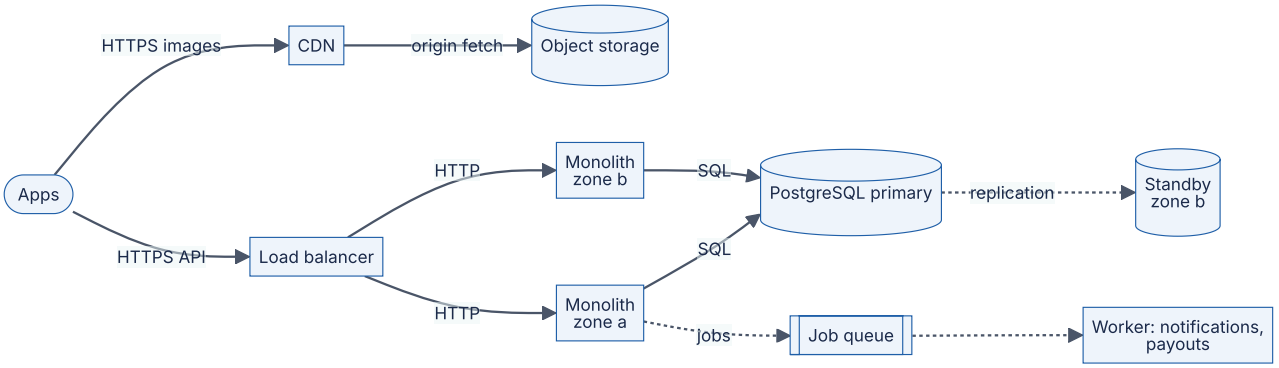


Figure 18.11 — Stage S3: 200,000 DAU (one region).

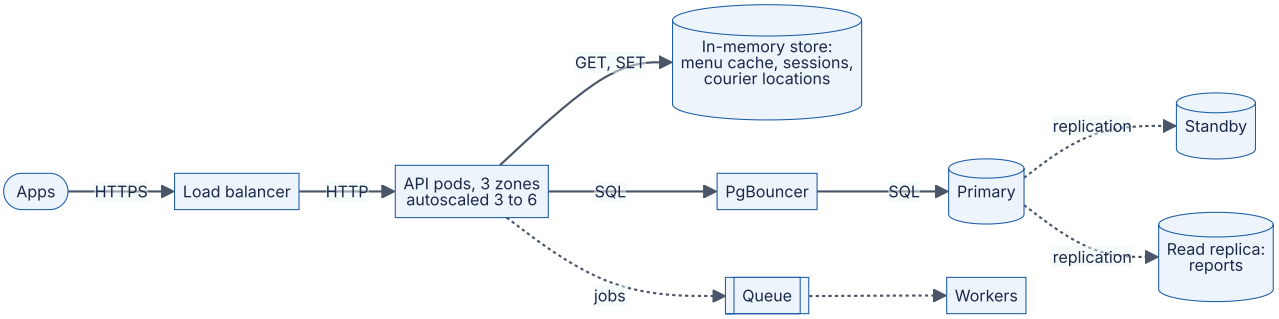


Figure 18.12 — Stage S4: 2,000,000 DAU (one country).

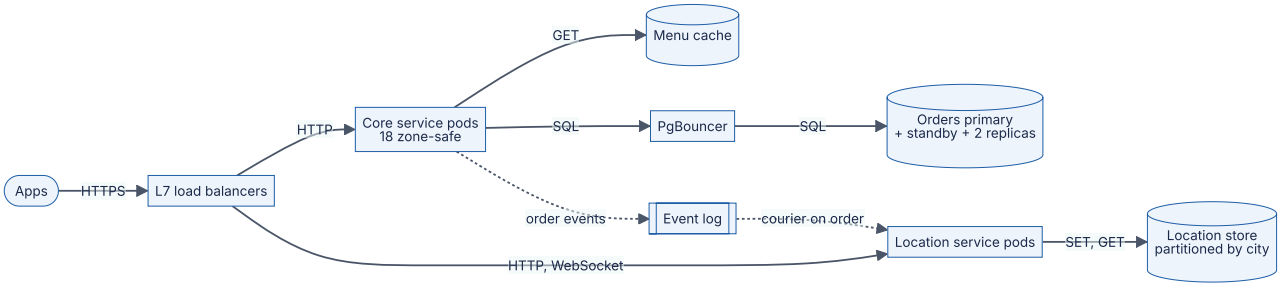
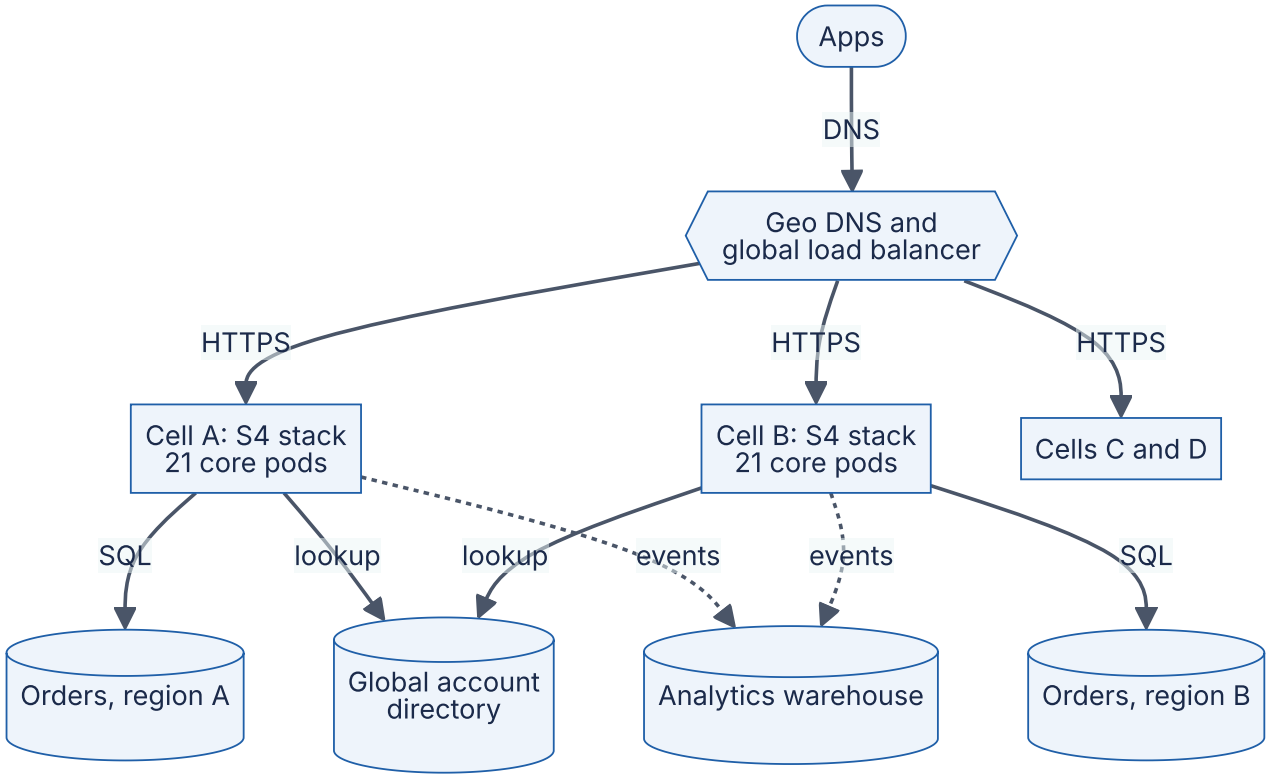


Figure 18.13 — Stage S5: 10,000,000 DAU (several countries).



Chapter 19 — Load Balancing

Figure 19.1 — An L4 balancer forwards flows; an L7 balancer terminates connections and routes requests.

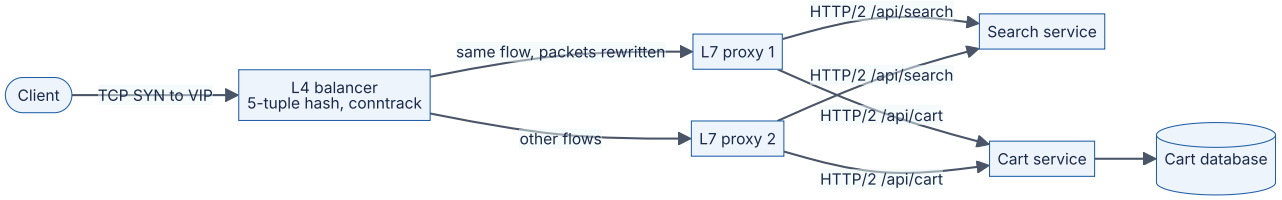


Figure 19.2 — Full-proxy (NAT) forwarding versus direct server return.

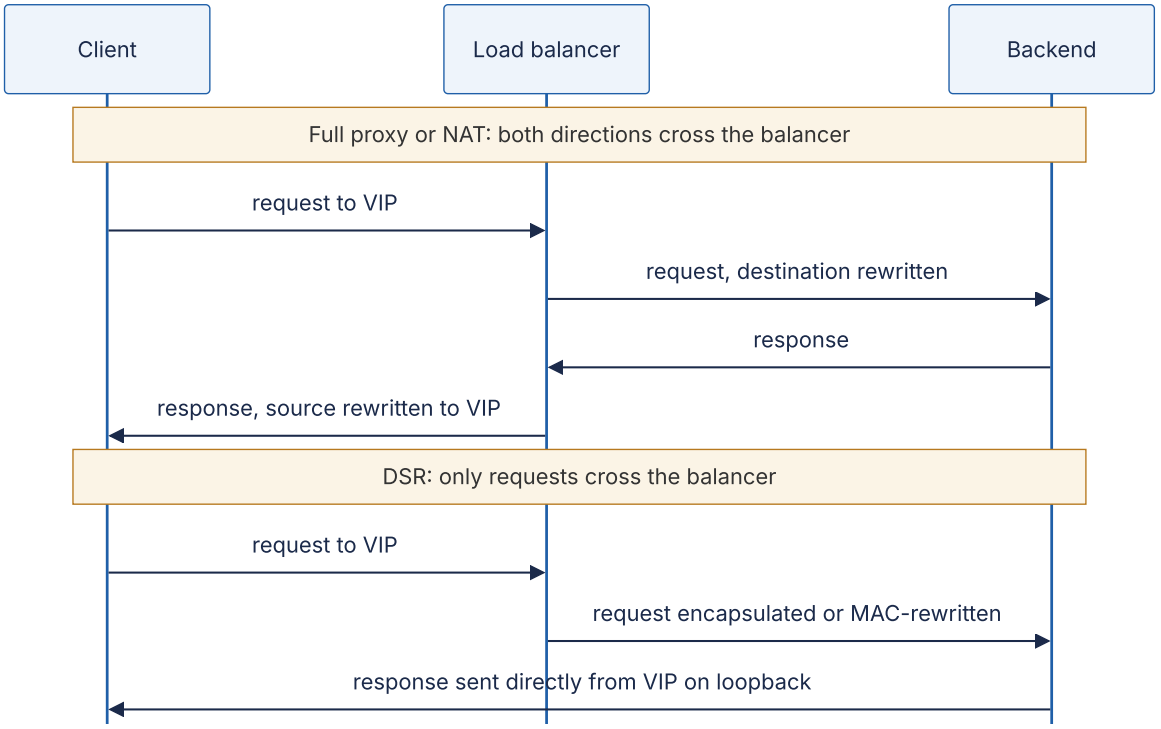
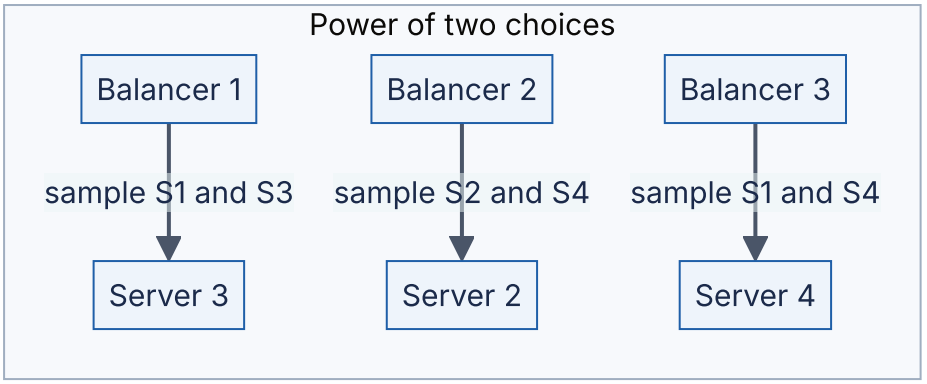


Figure 19.3 — Herd behavior with stale load information, and how two random choices avoid it.



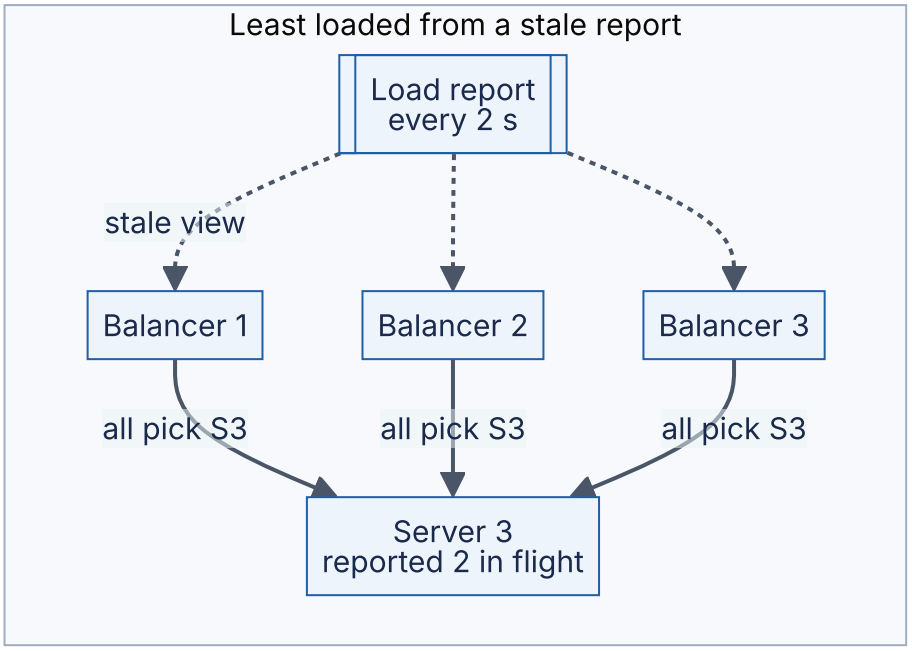


Figure 19.4 — Maglev: ECMP spreads packets over balancer machines; each machine maps flows to backends with the same lookup table.

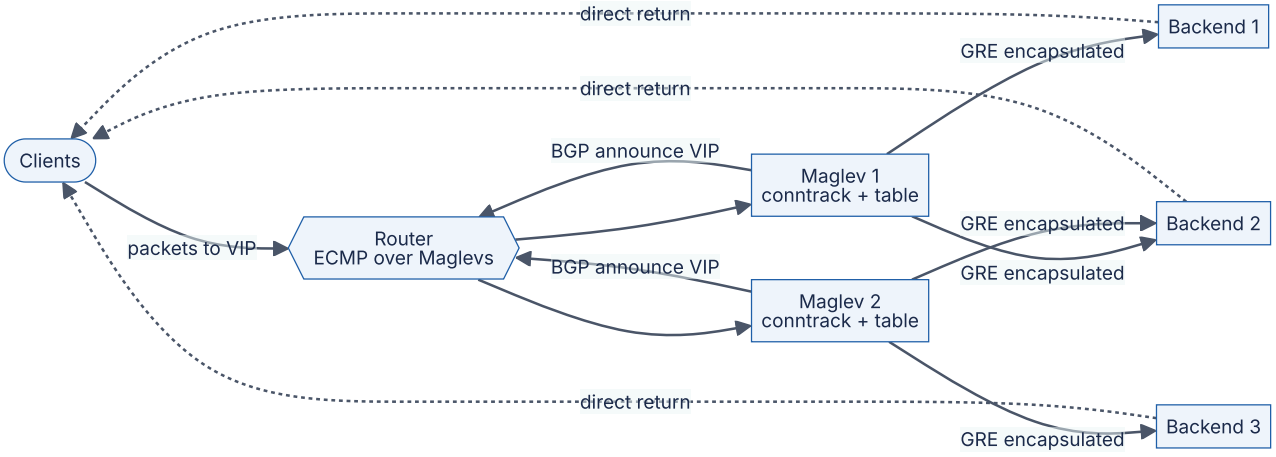


Figure 19.5 — Health state of one backend as seen by the balancer.

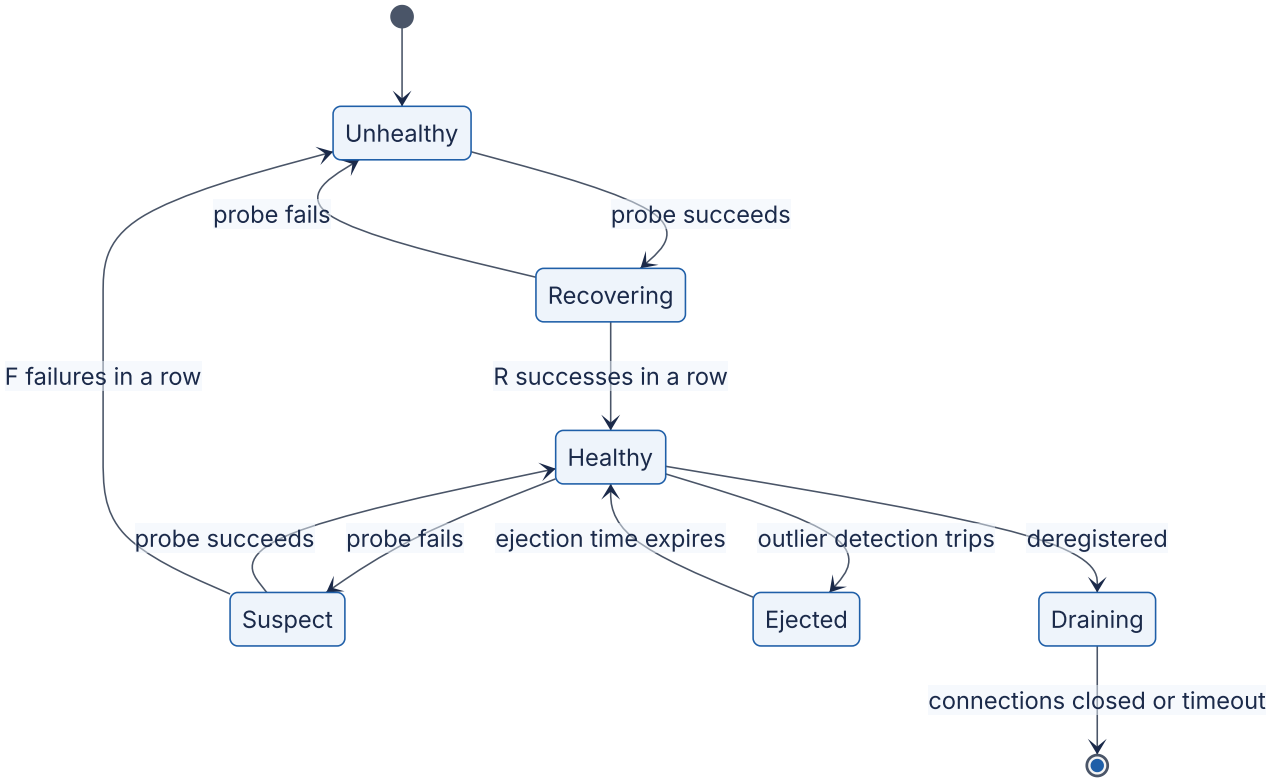


Figure 19.6 — Balancer-inserted affinity cookie.

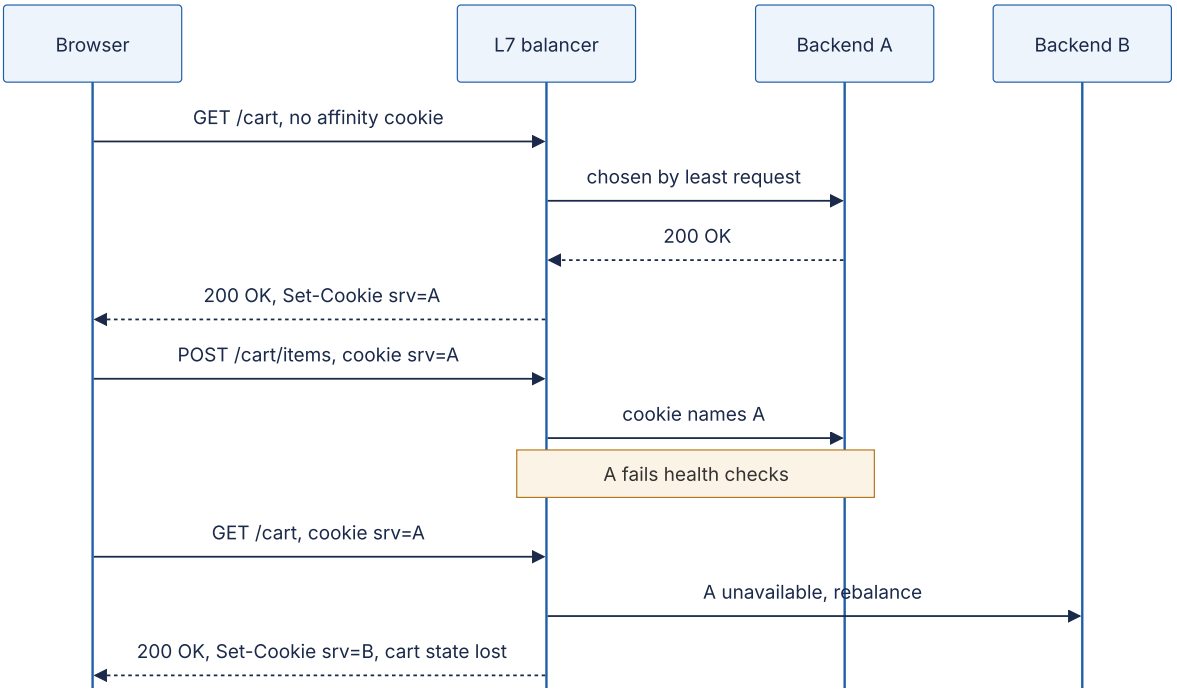
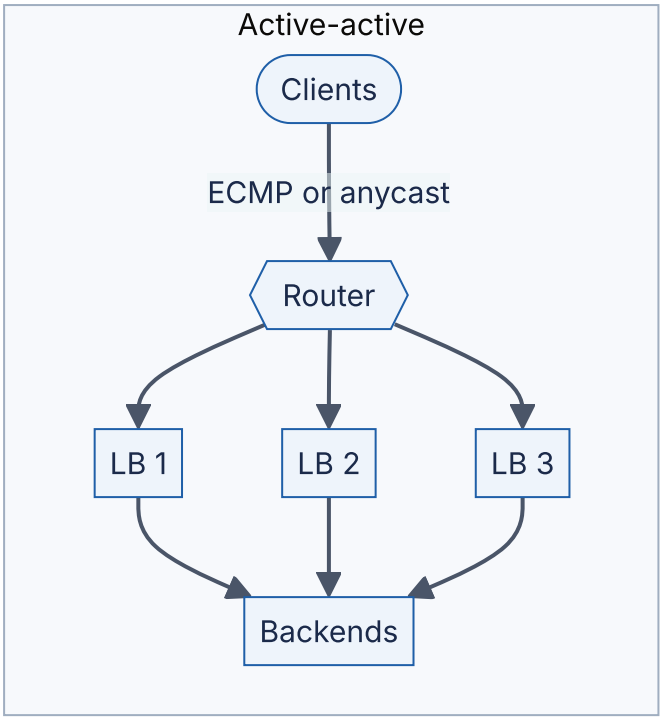


Figure 19.7 — Active-passive versus active-active balancer tiers.



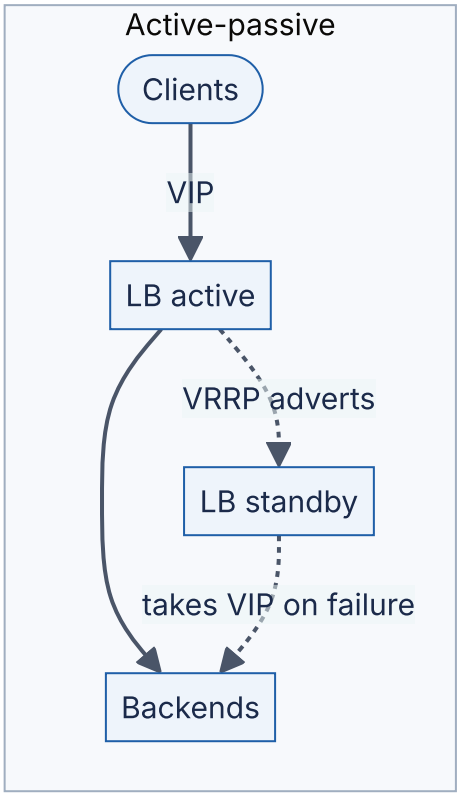


Figure 19.8 — VRRP failover between two balancers.

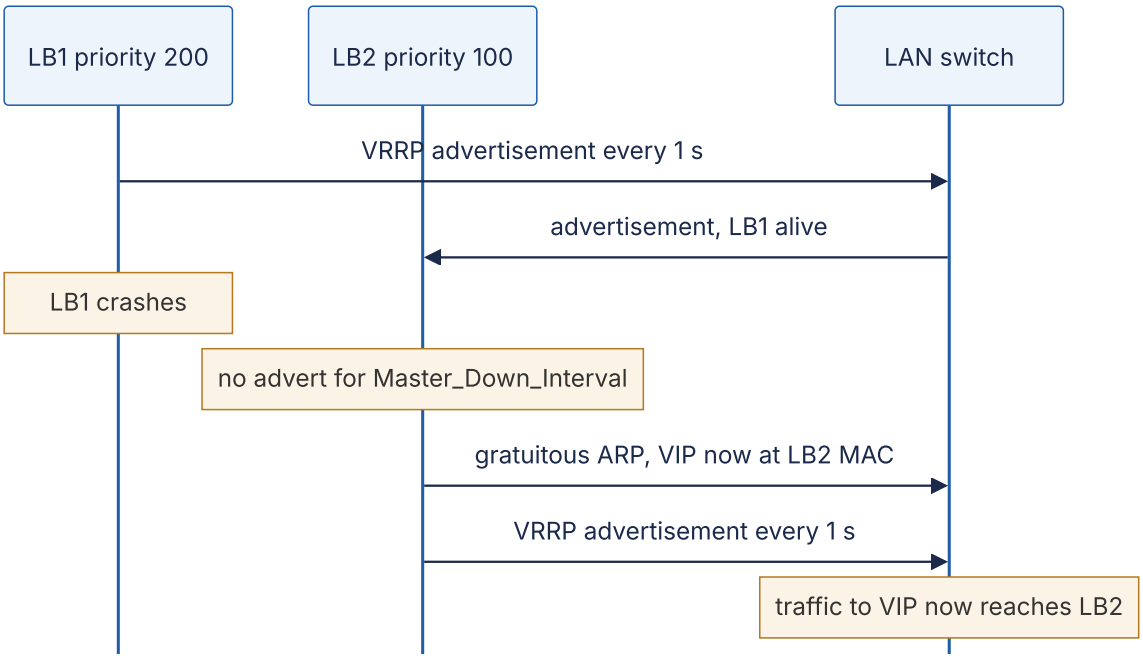


Figure 19.9 — A layered, redundant front end.

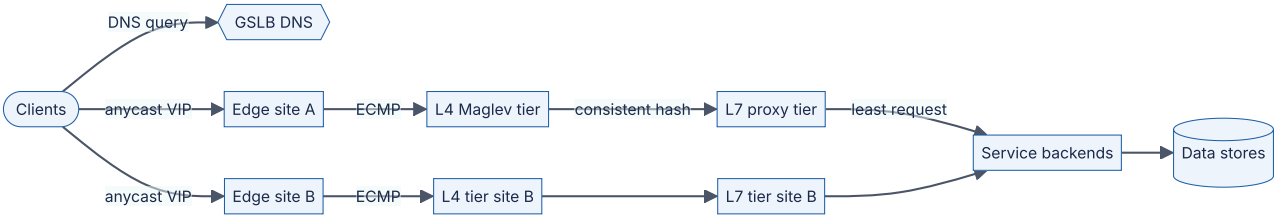


Figure 19.10 — DNS-based global server load balancing.

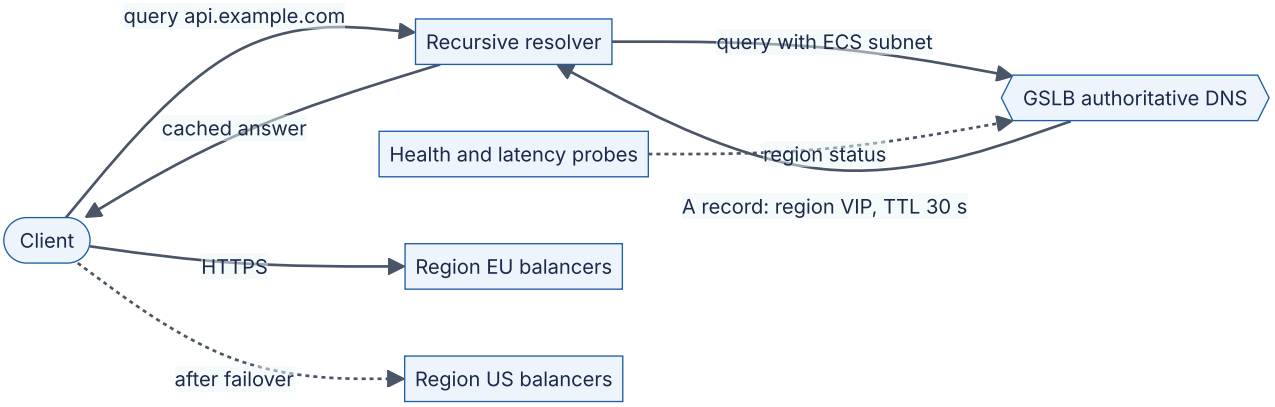
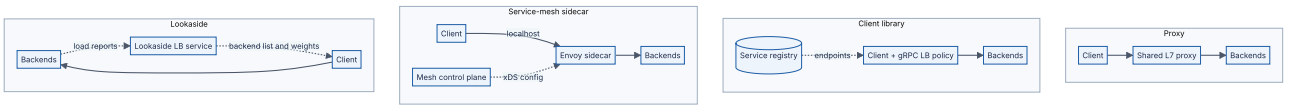


Figure 19.11 — Four places to put the balancing decision.



Chapter 20 — Caching

Figure 20.1 — The hit and miss paths of a cache in front of a database.

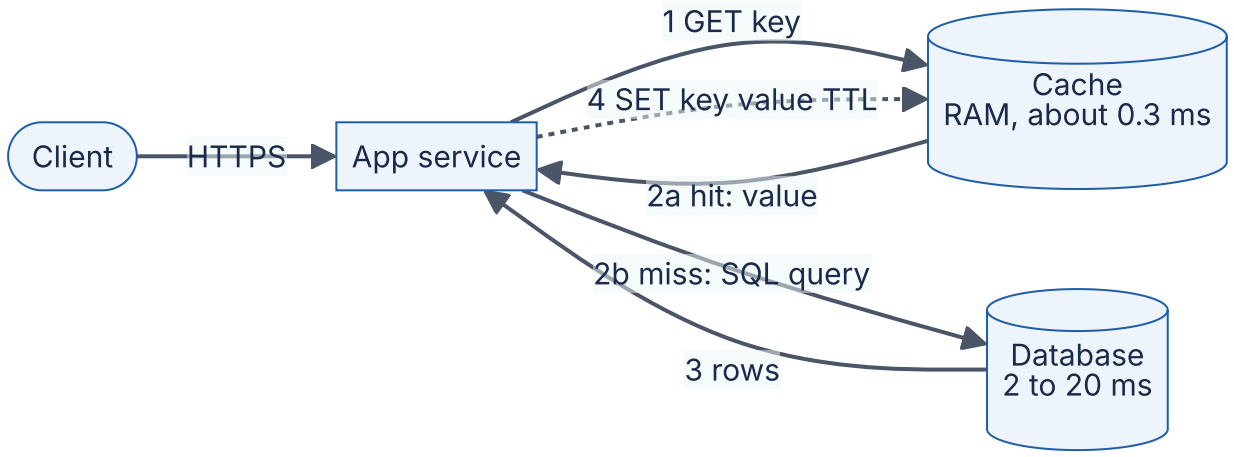


Figure 20.2 — Cache layers from the client to the database.

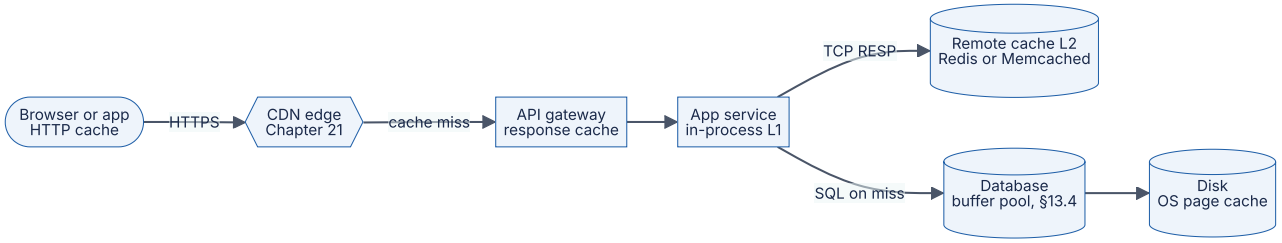


Figure 20.3 — Redis's event loop with I/O threads.

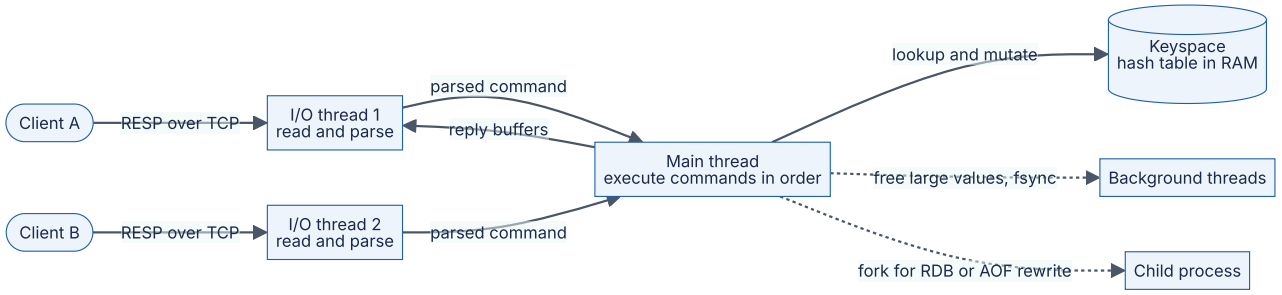


Figure 20.4 — Redis Cluster: hash slots, redirects and replicas.

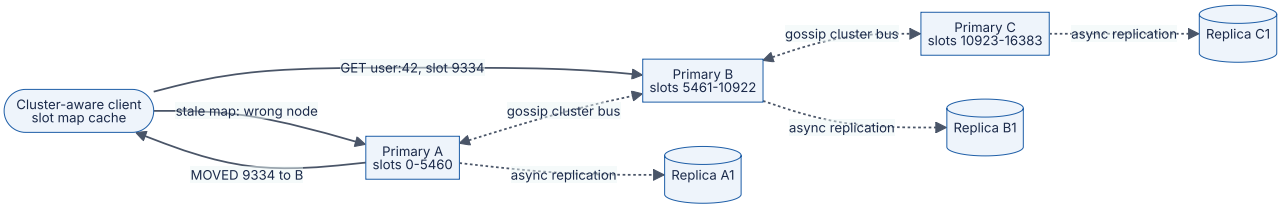


Figure 20.5 — Memcached's slab allocator and per-class LRUs.

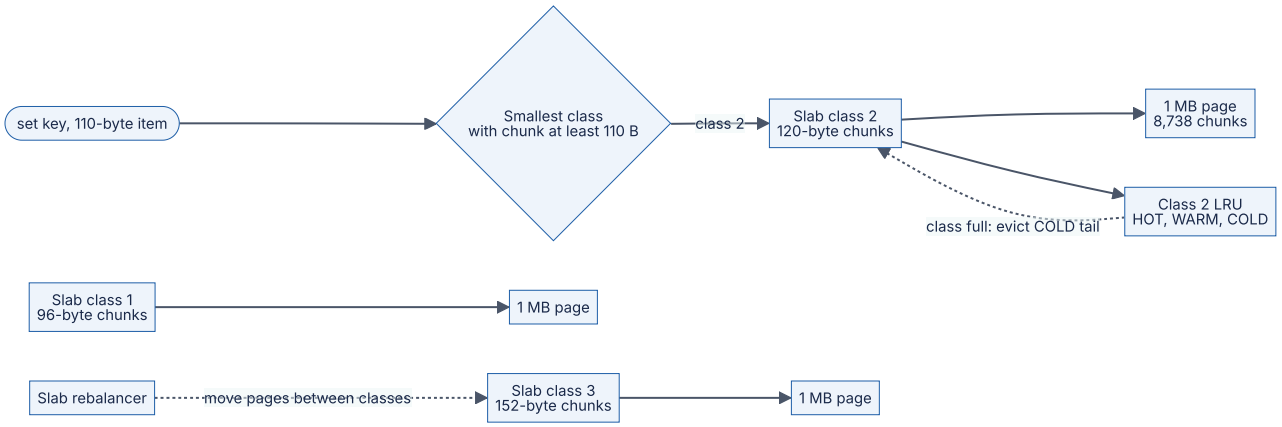


Figure 20.6 — Cache-aside read and write paths.

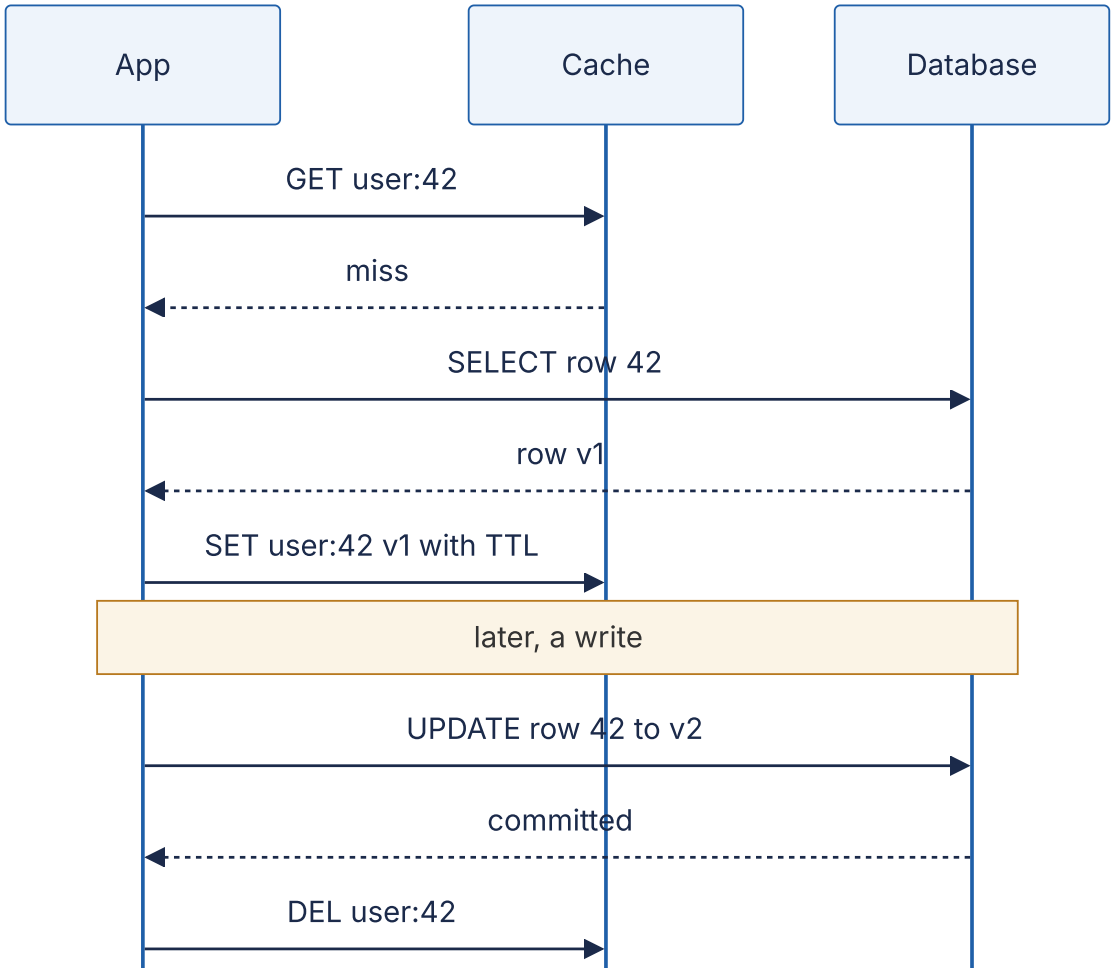


Figure 20.7 — The stale-set race in cache-aside.

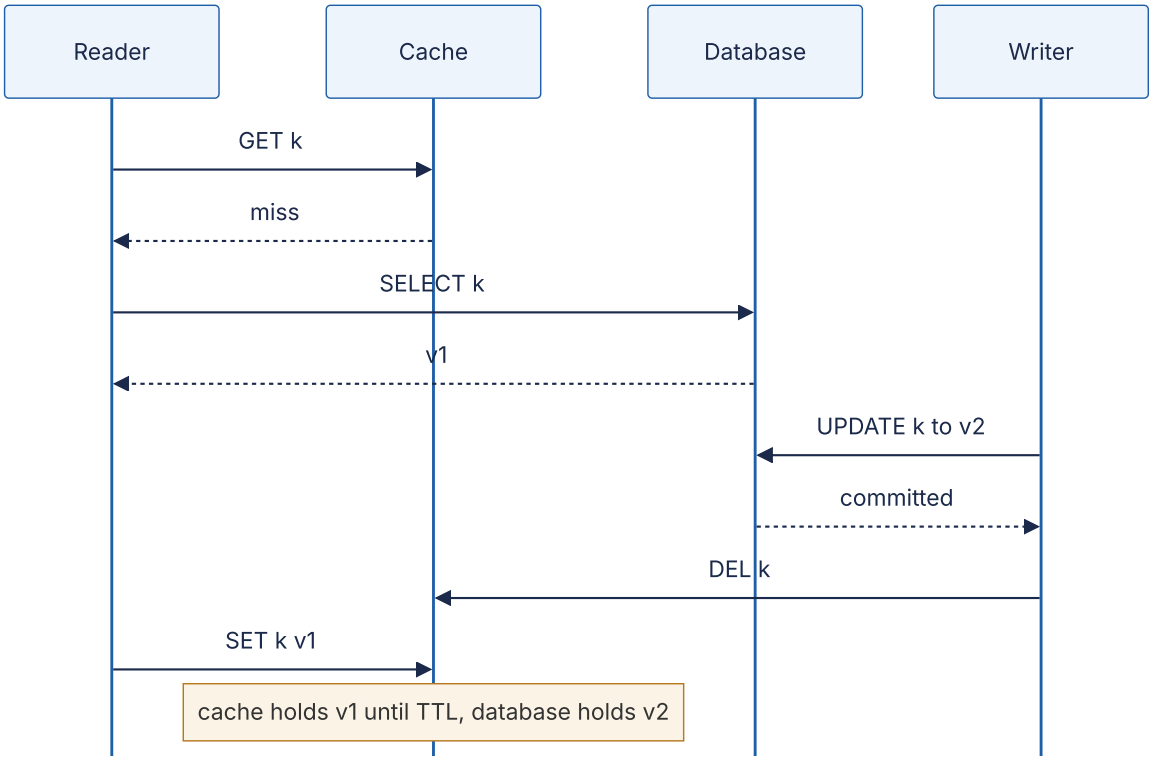


Figure 20.8 — Read-through.

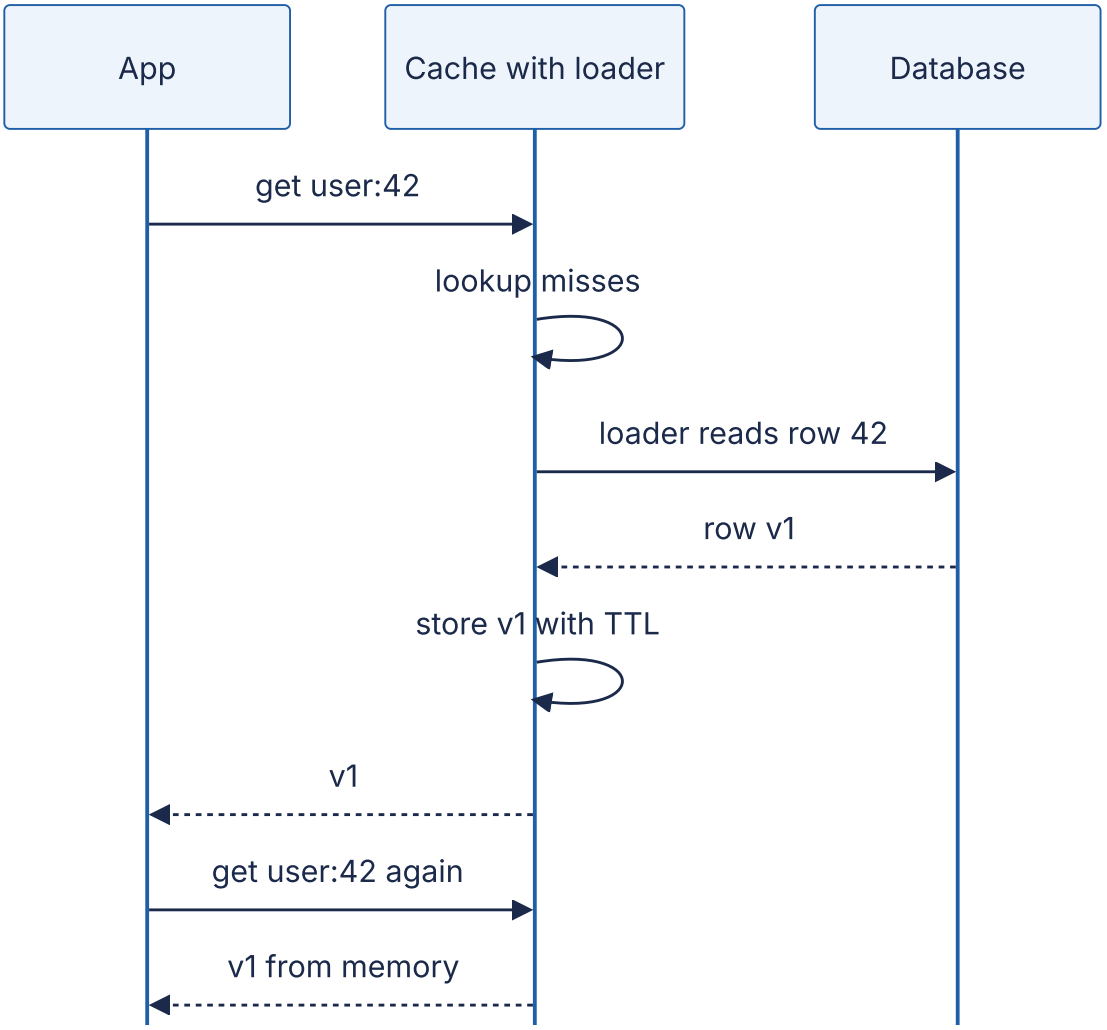


Figure 20.9 — Write-through.

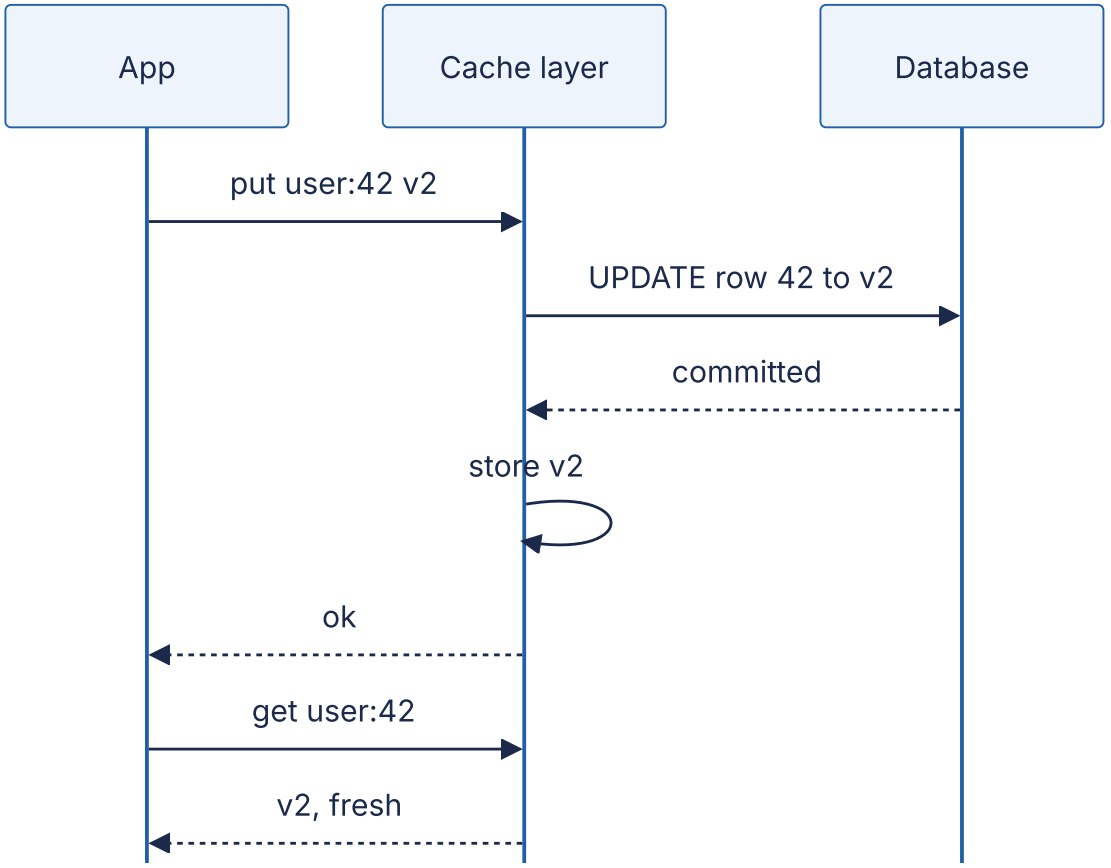


Figure 20.10 — Write-back.

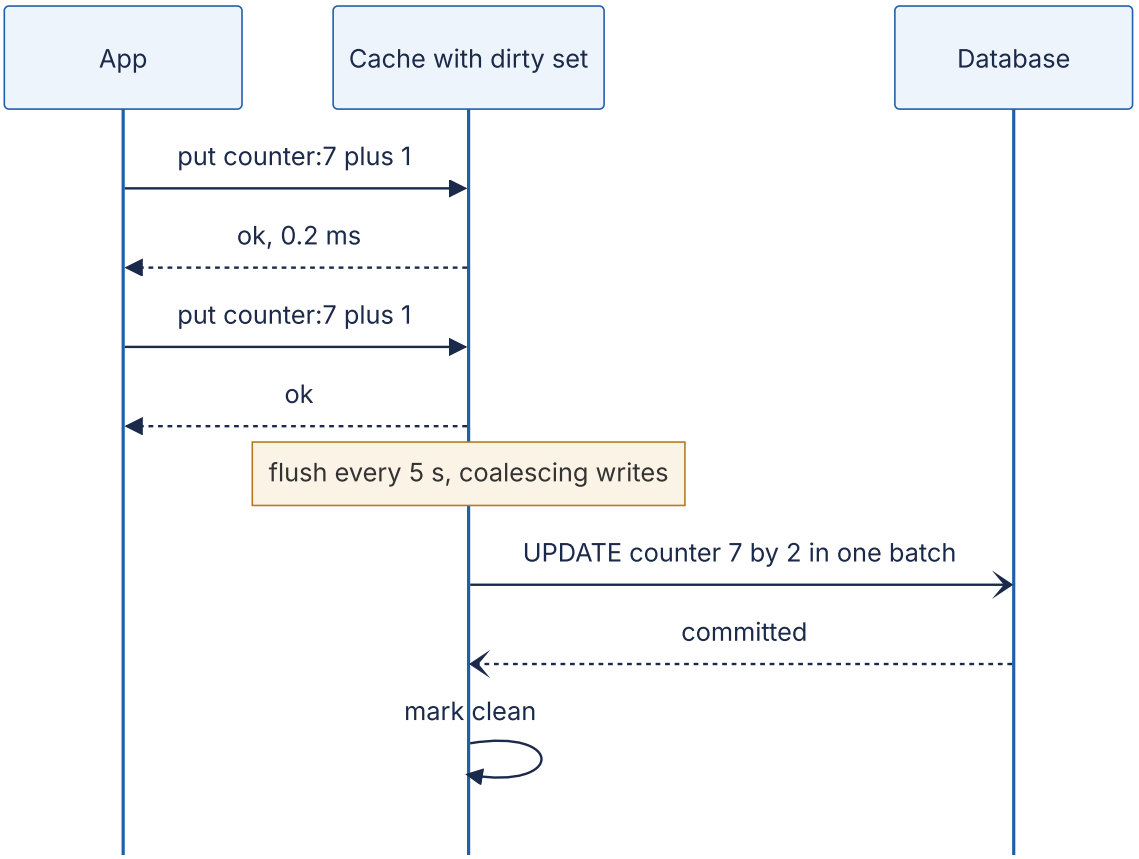


Figure 20.11 — Write-around.

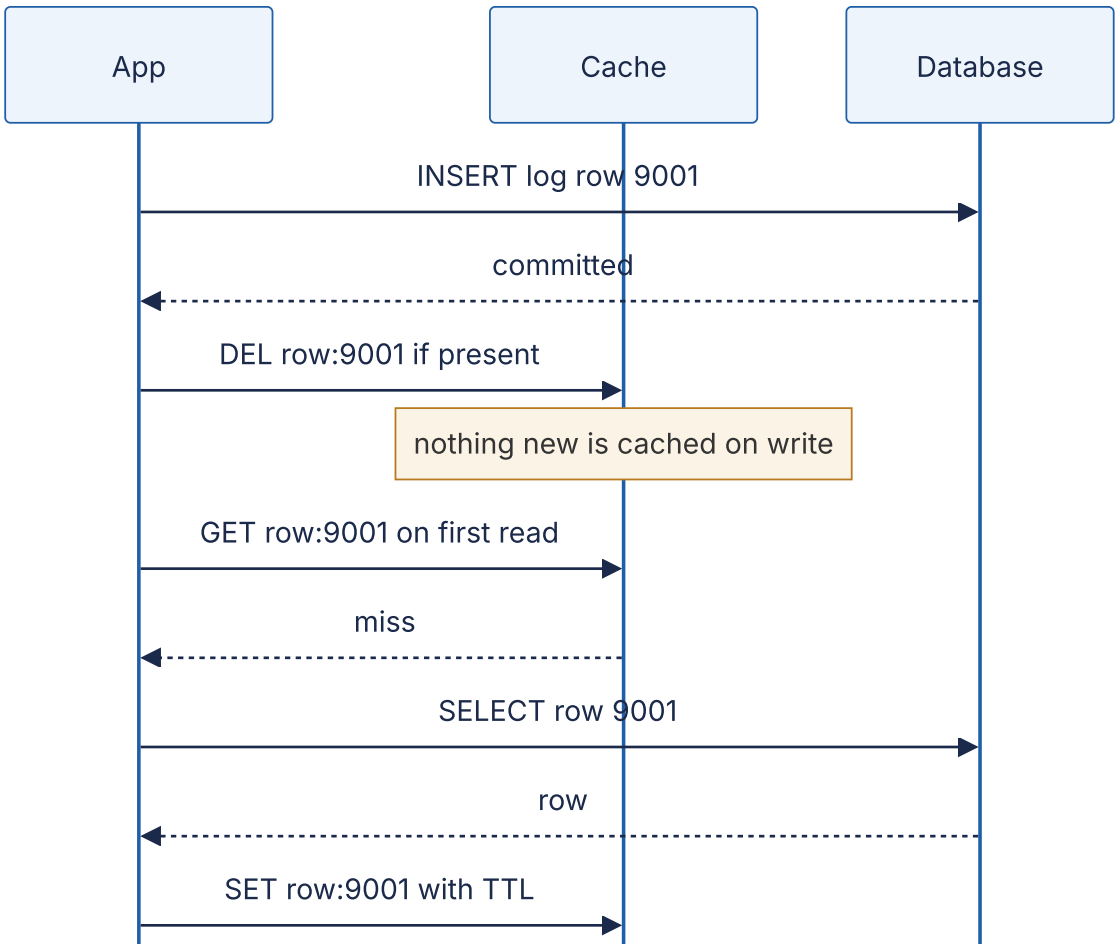


Figure 20.12 — W-TinyLFU: window, admission filter and segmented main cache.

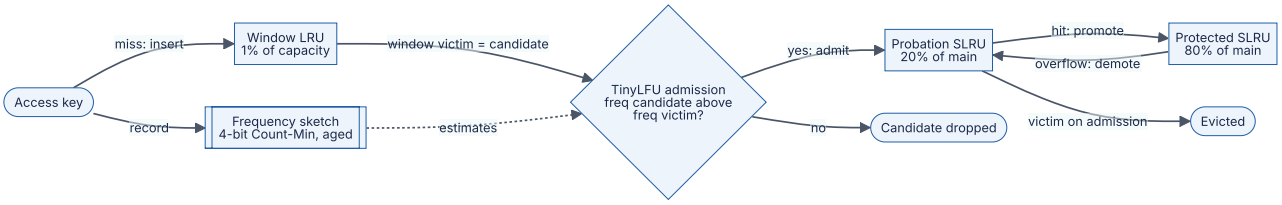


Figure 20.13 — CDC-driven invalidation across regions (after Facebook's mcsqueal).

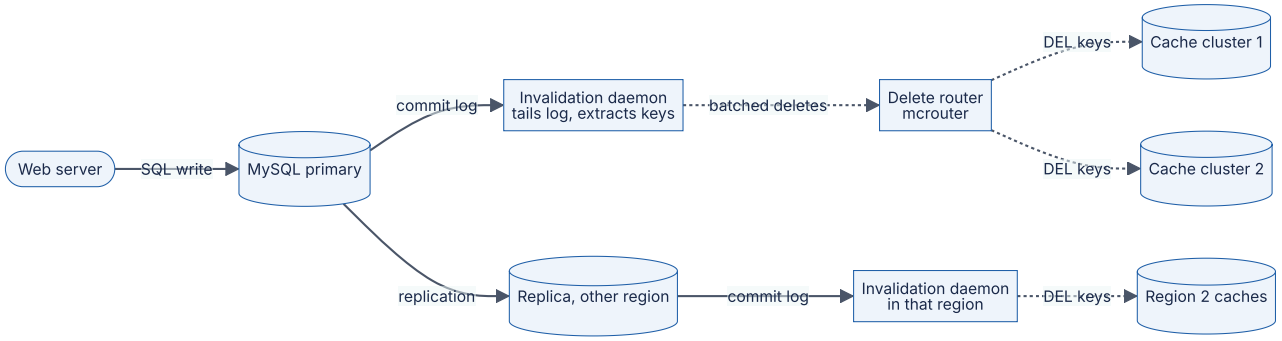


Figure 20.14 — Stampede protection with a lease: one recompute, others wait or serve stale.

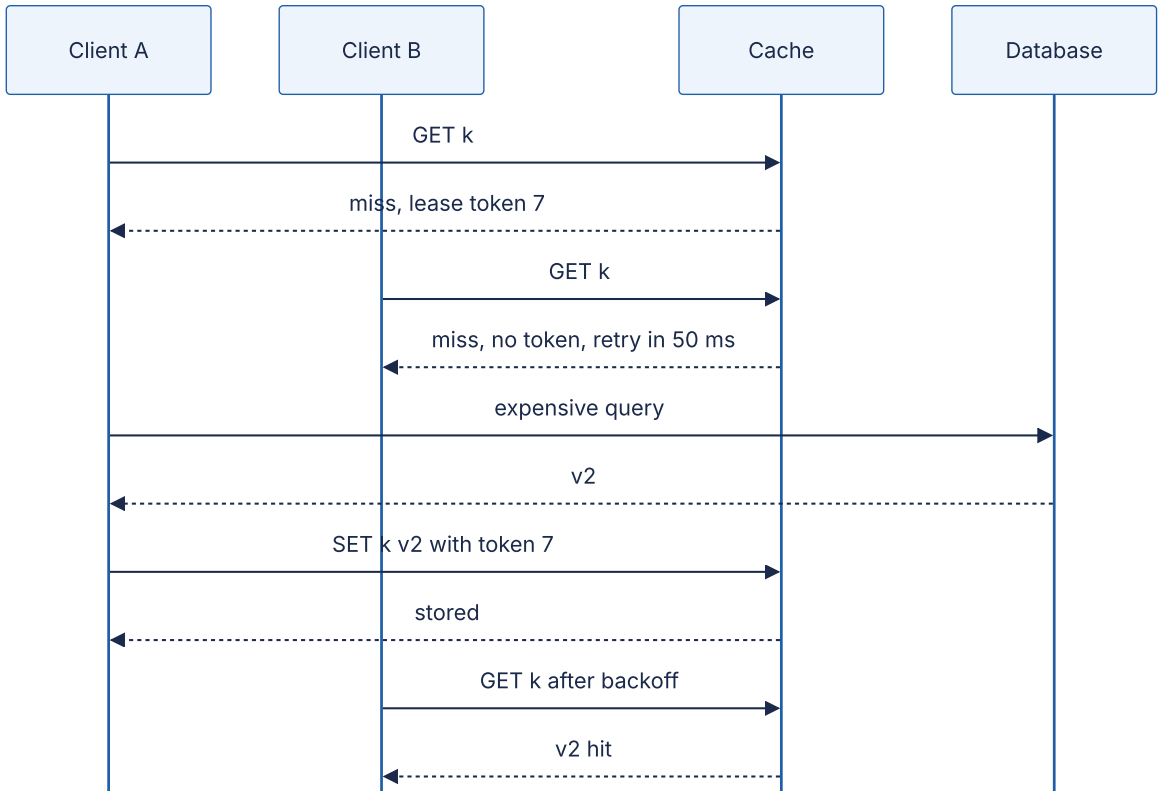


Figure 20.15 — Defenses against cache penetration.

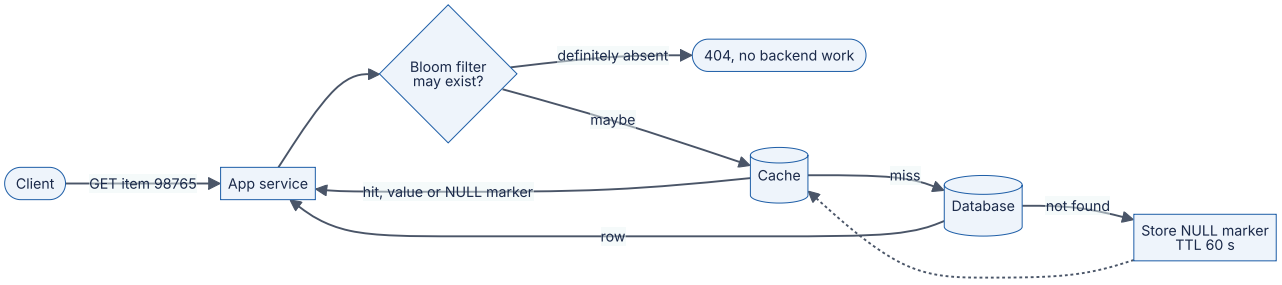
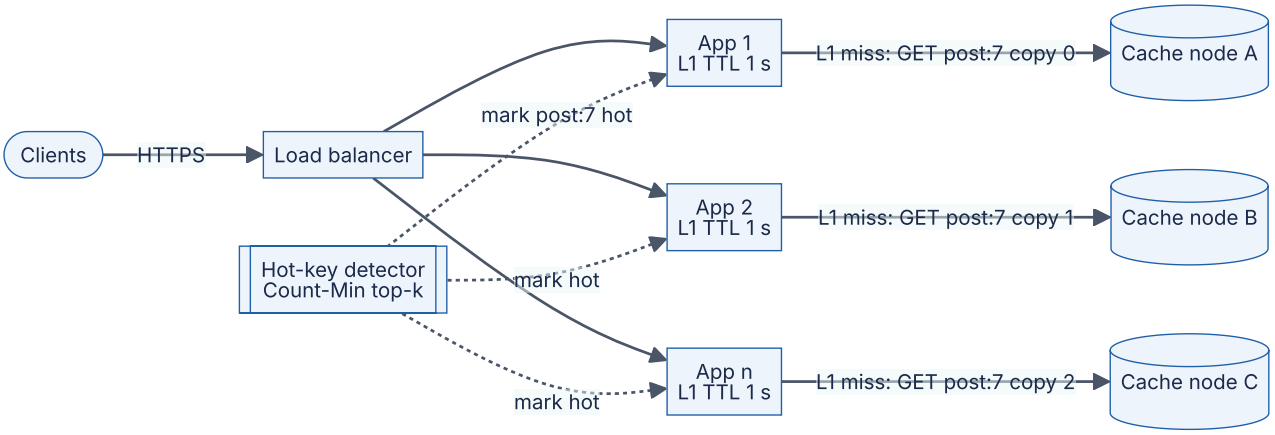


Figure 20.16 — Hot-key mitigation with L1 caches and replicated keys.



Chapter 21 — Content Delivery Networks

Figure 21.1 — A CDN: users, PoPs, mid-tier and origin.

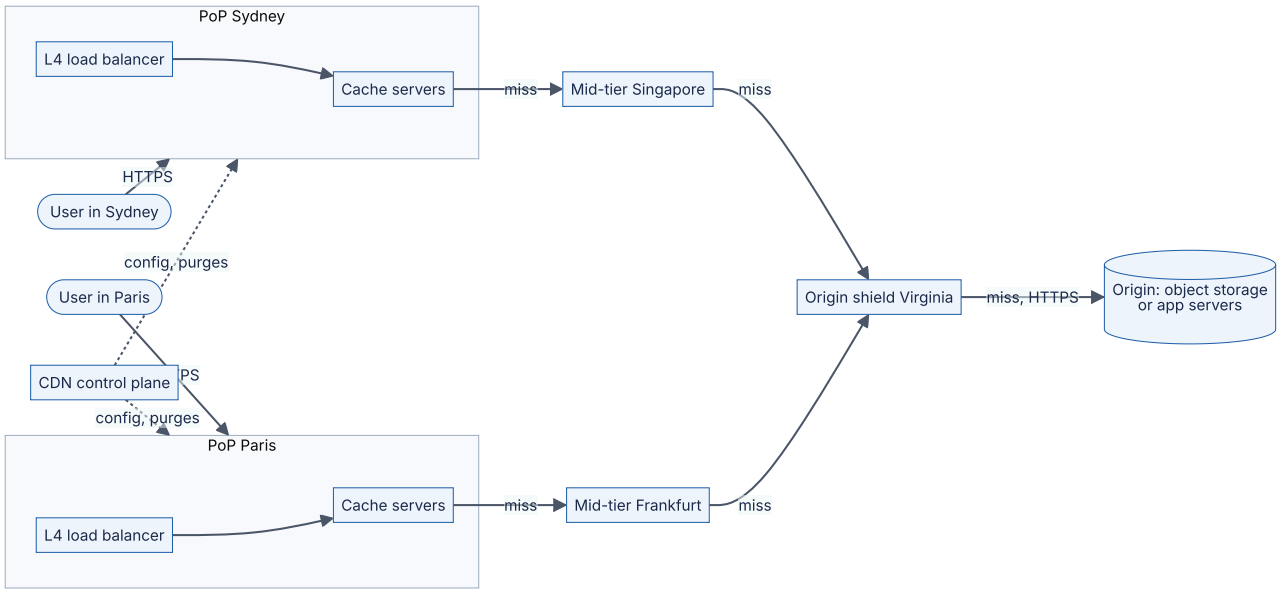


Figure 21.2 — Request routing with DNS mapping versus anycast.

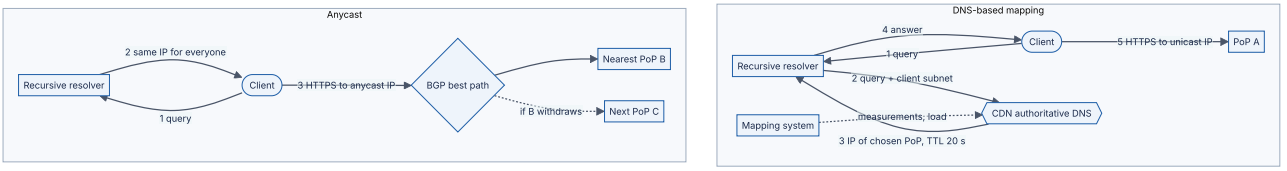


Figure 21.3 — A cache hit and a cache miss at an edge PoP.

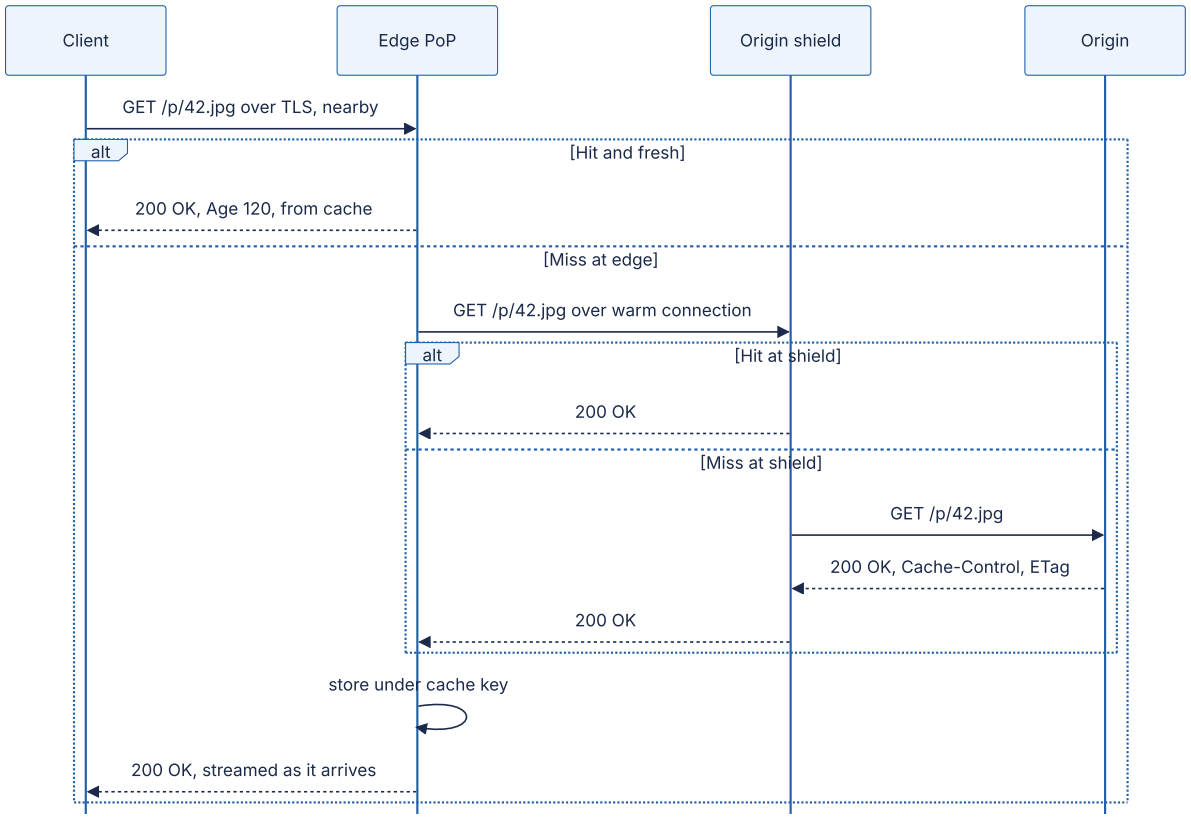


Figure 21.4 — Origin traffic without and with a shield.

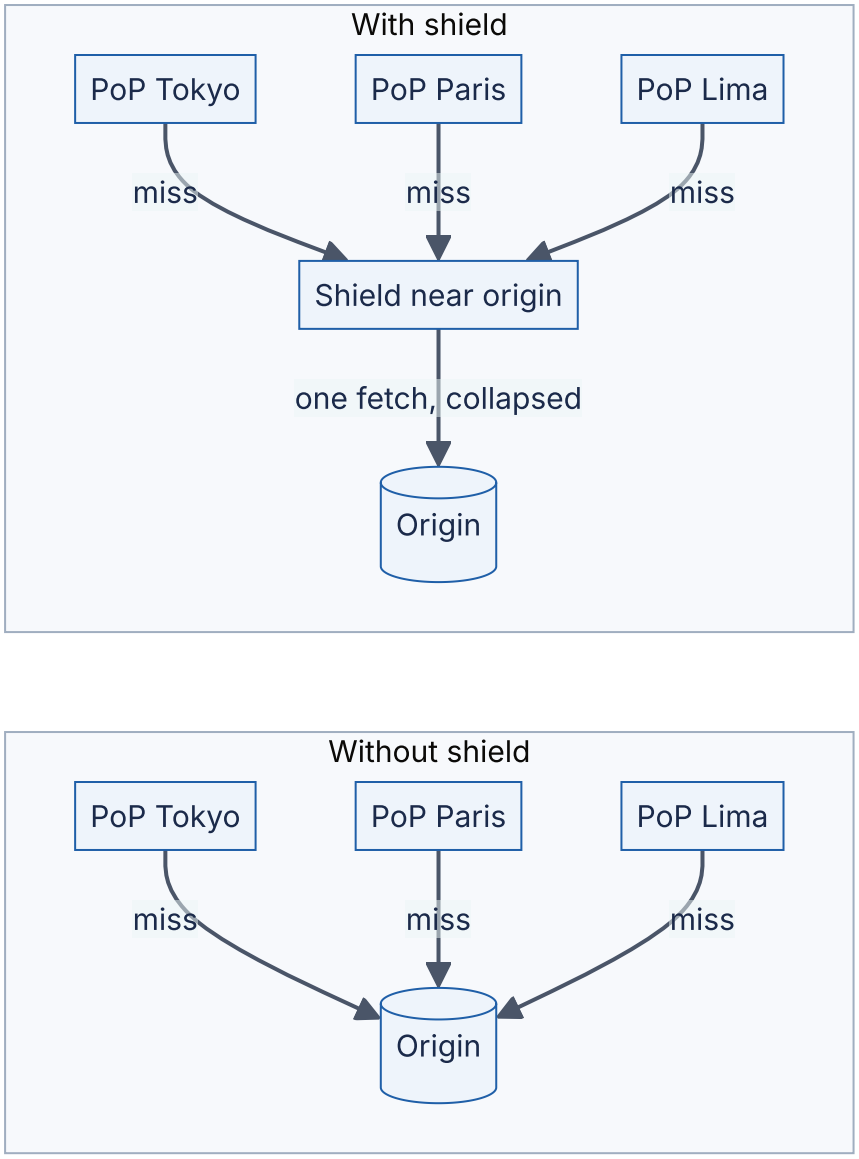


Figure 21.5 — Life of a cached response at the edge.

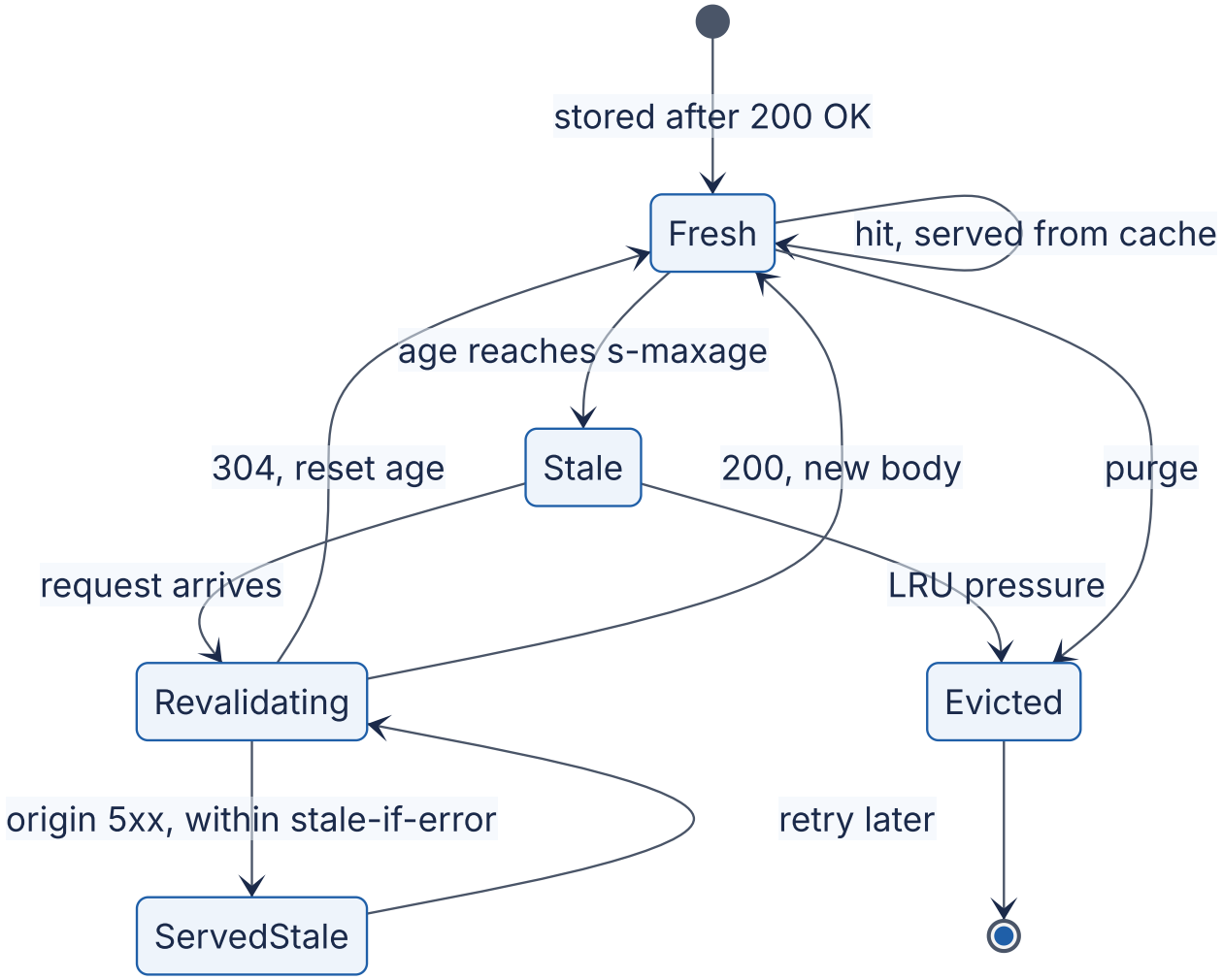


Figure 21.6 — Tag-based purge propagation.

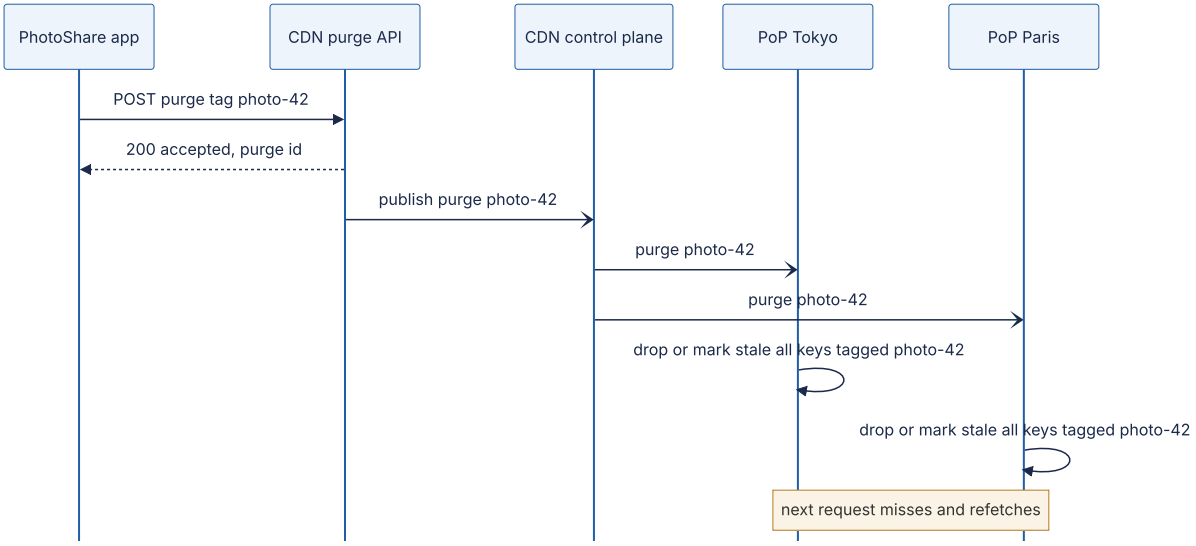


Figure 21.7 — Dynamic site acceleration.

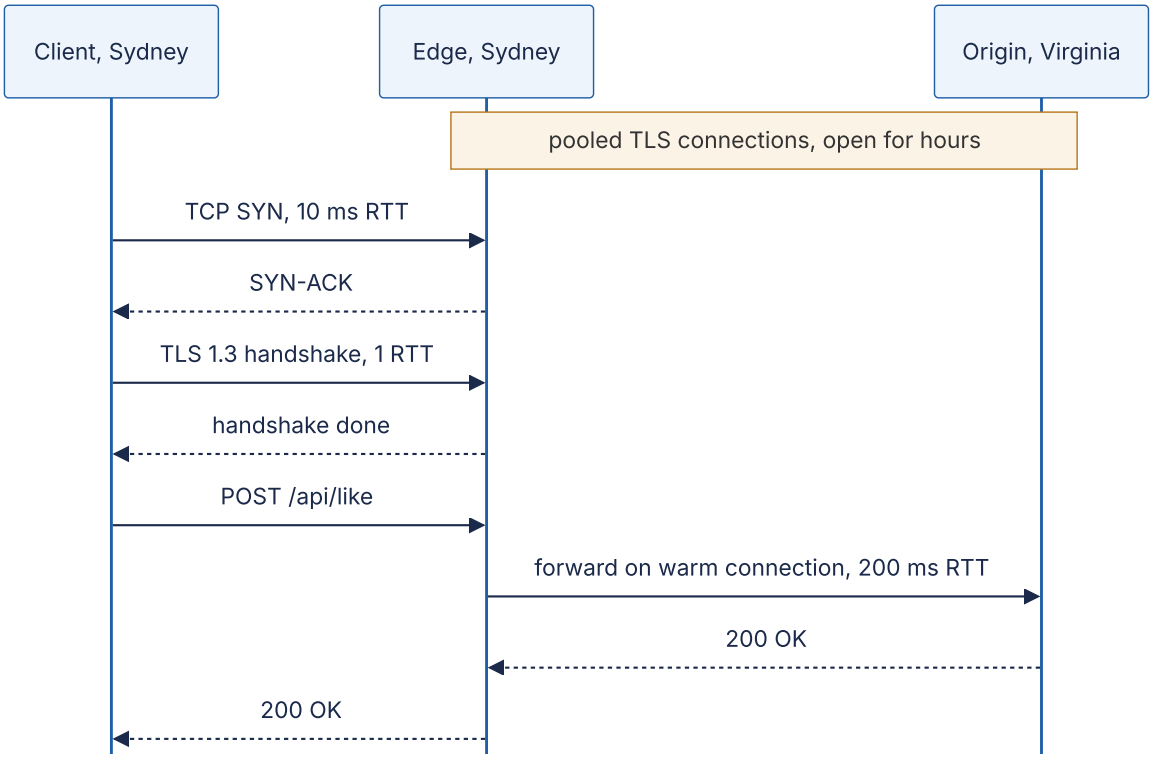


Figure 21.8 — HLS and DASH structure on a CDN.

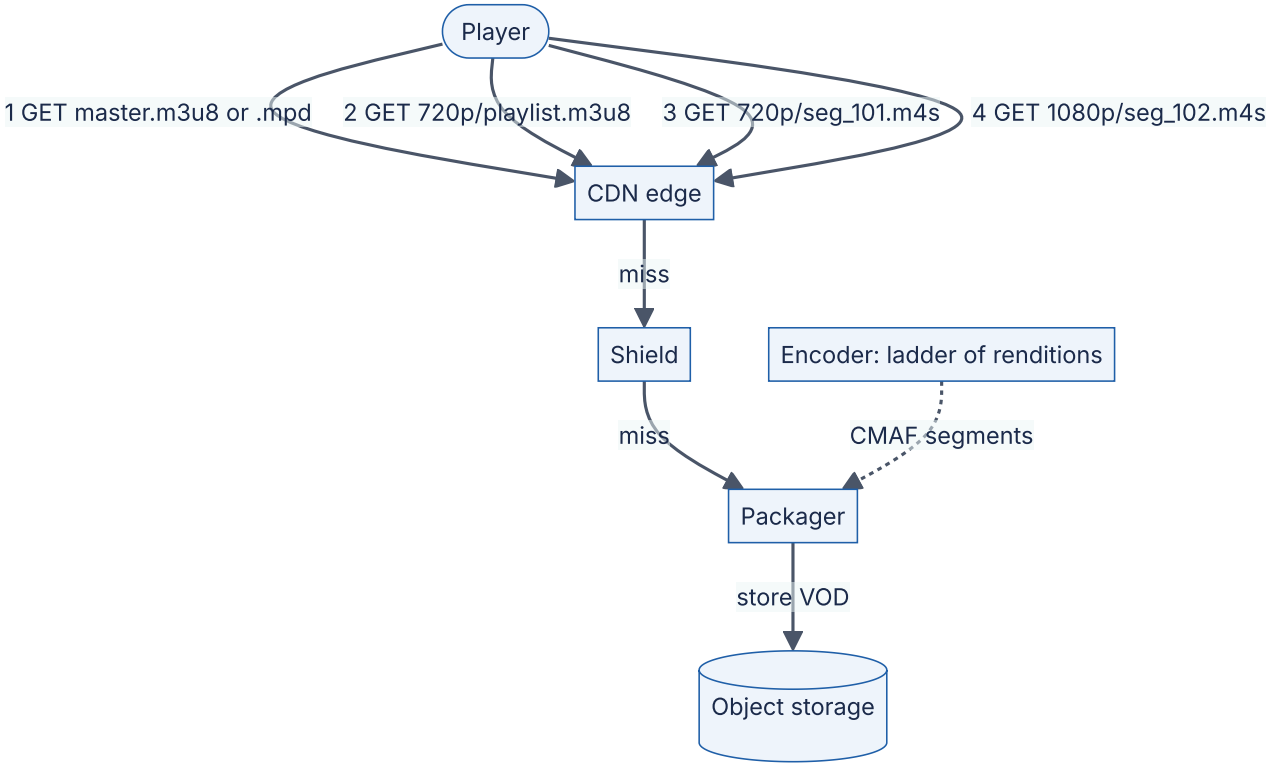


Figure 21.9 — The ABR control loop.

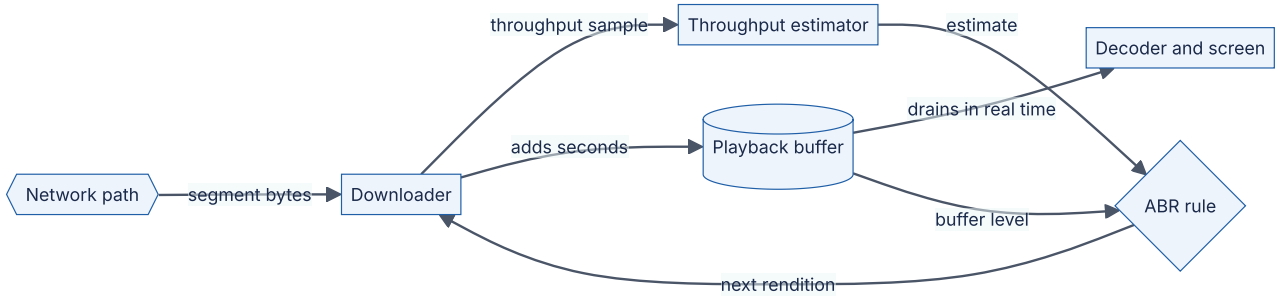
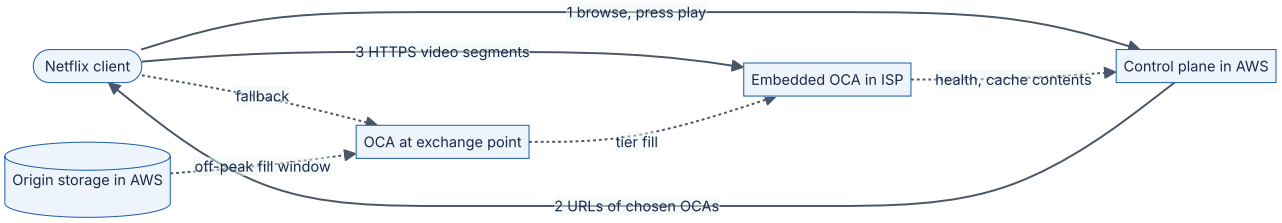


Figure 21.10 — Open Connect: control plane in the cloud, bytes from the ISP.



Chapter 22 — API and Traffic Management

Figure 22.1 — Layered rate limiting for the PhotoShare API.

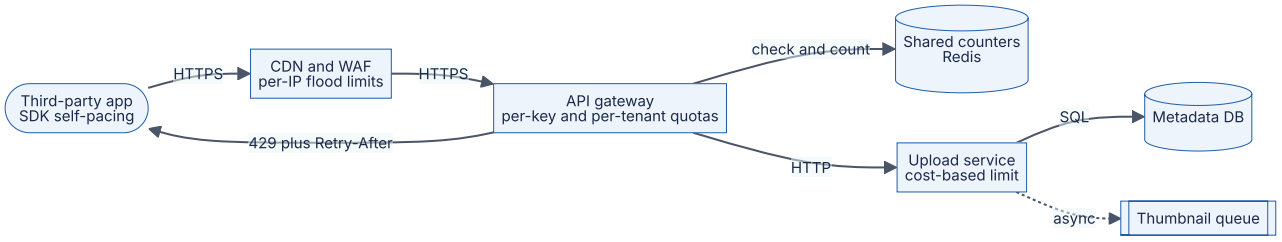


Figure 22.2 — Token bucket admission decision.

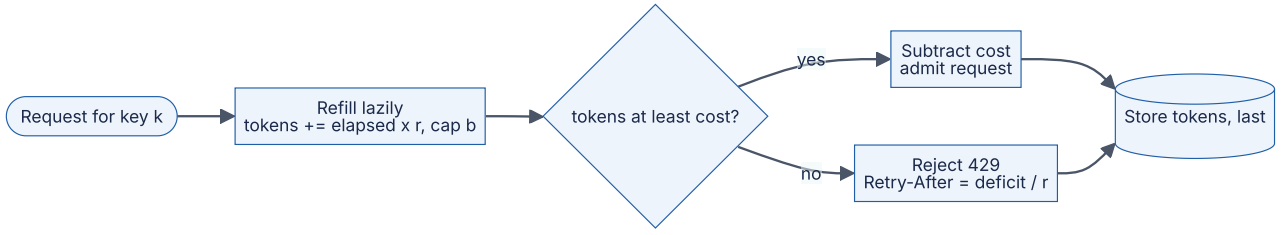
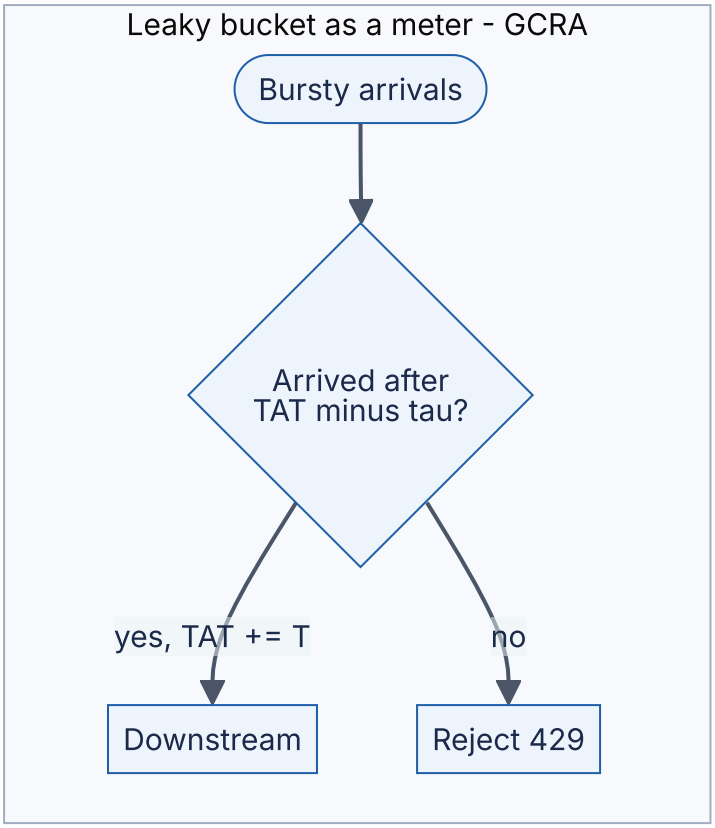


Figure 22.3 — Leaky bucket as a queue versus as a meter.



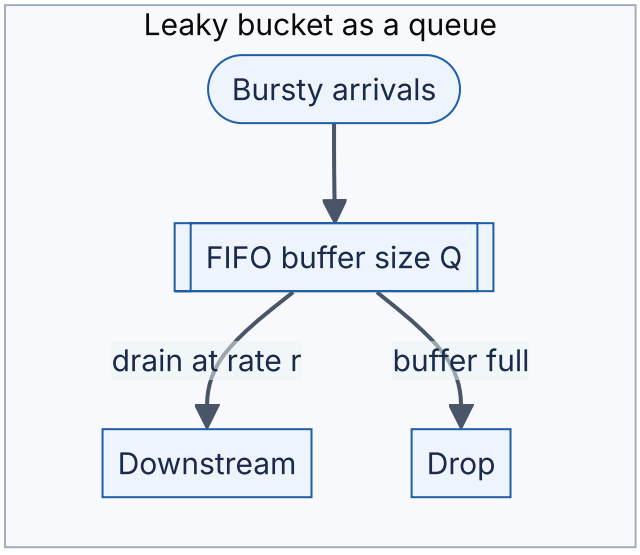


Figure 22.4 — The fixed-window boundary burst.

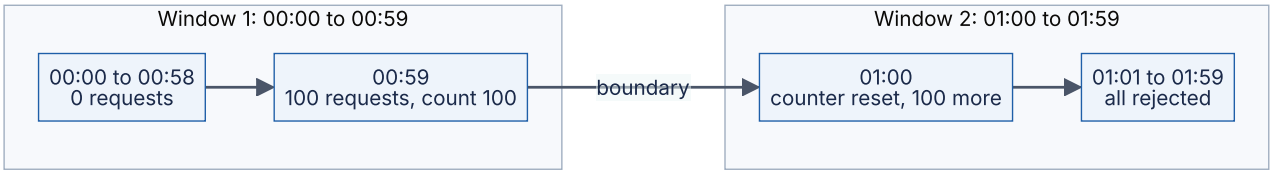


Figure 22.5 — The read-modify-write race in a shared counter.

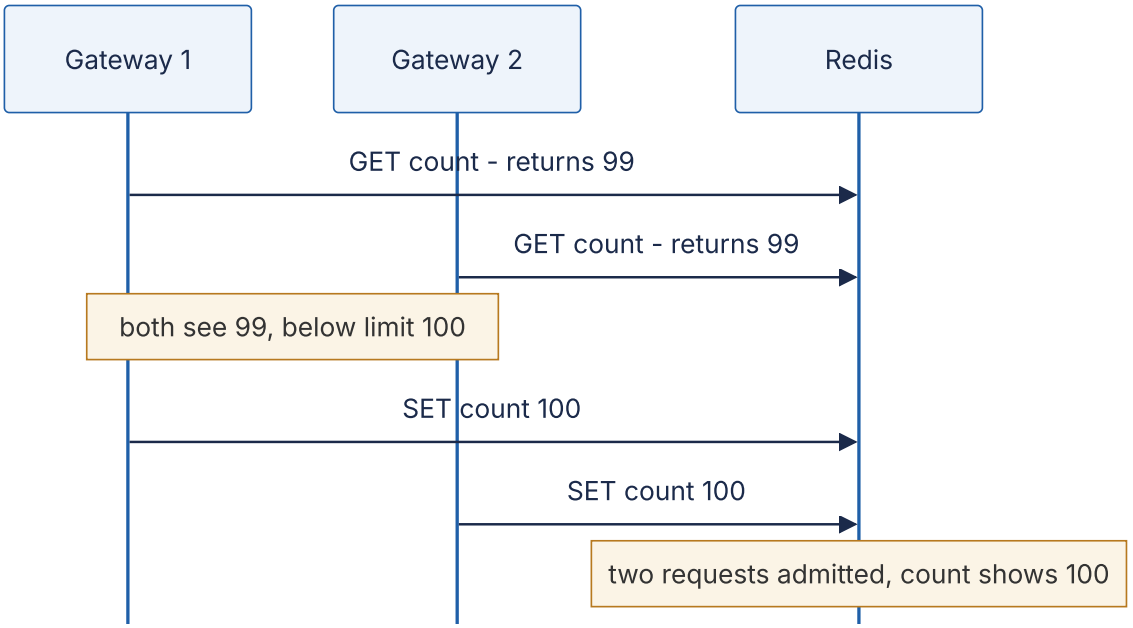


Figure 22.6 — Centralized rate limiting with an atomic script.

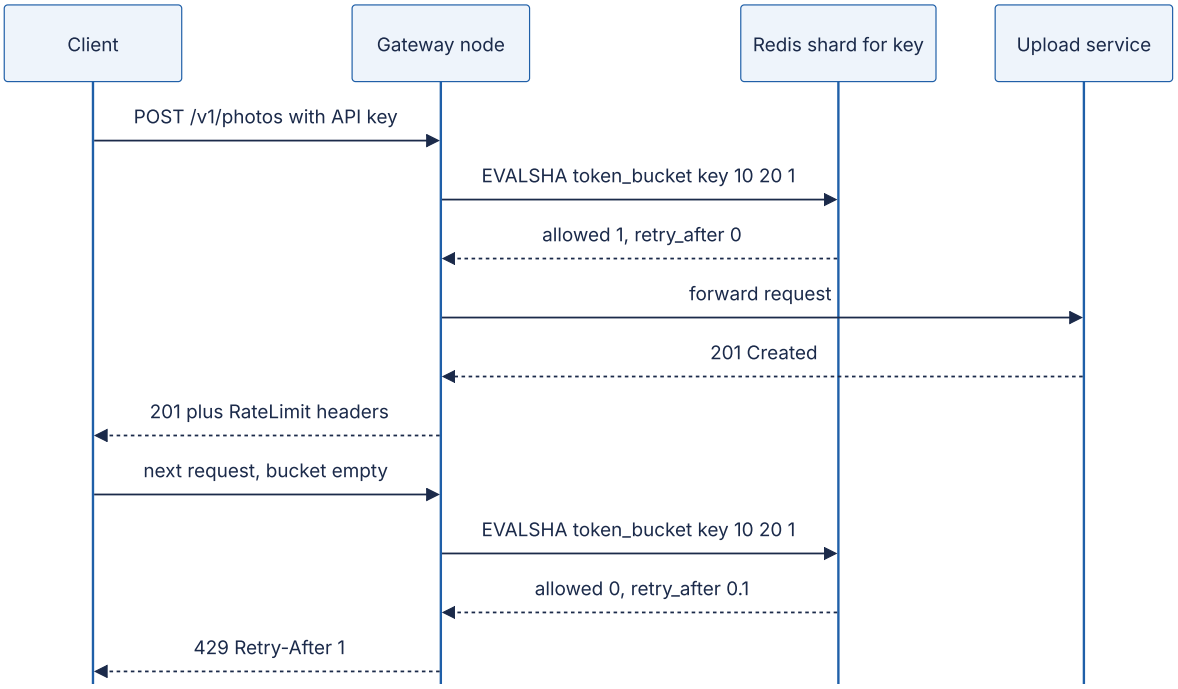


Figure 22.7 — Local enforcement with synchronization or quota leases.

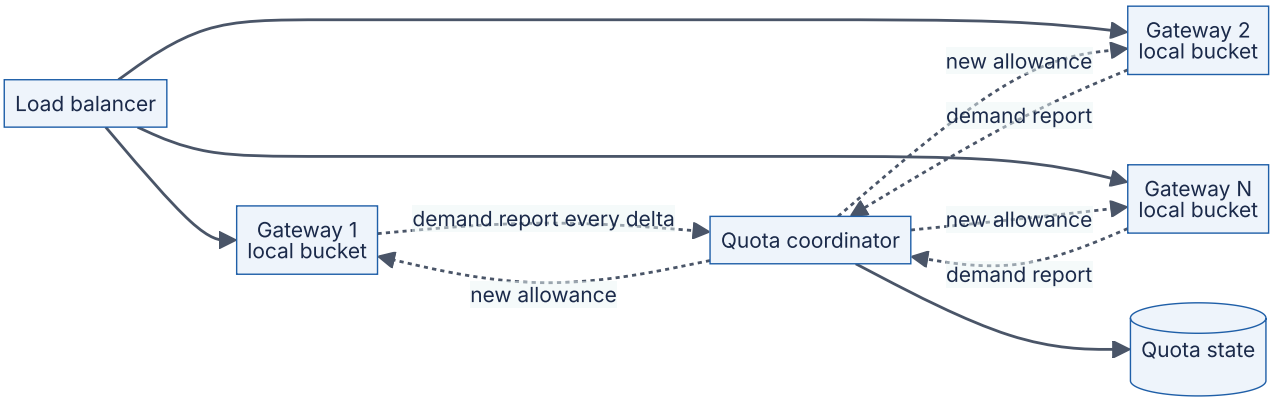


Figure 22.8 — Overload control at a service: priority shedding and an adaptive queue.

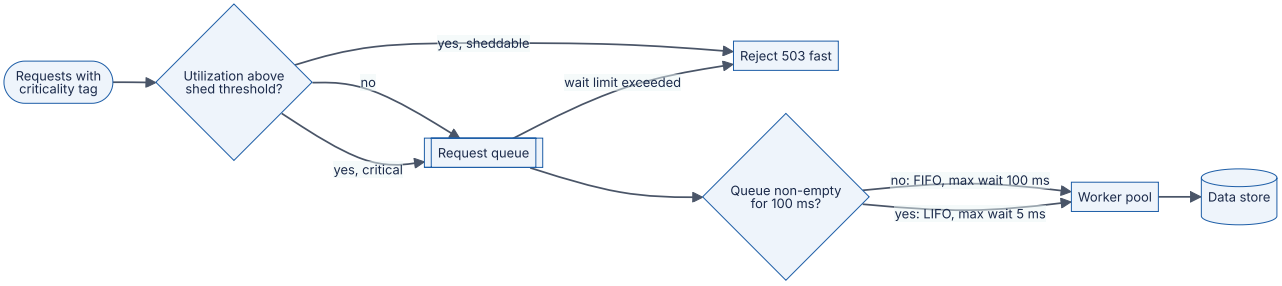


Figure 22.9 — Backpressure through a processing pipeline.



Figure 22.10 — Deadline propagation through the PhotoShare Pro purchase path.

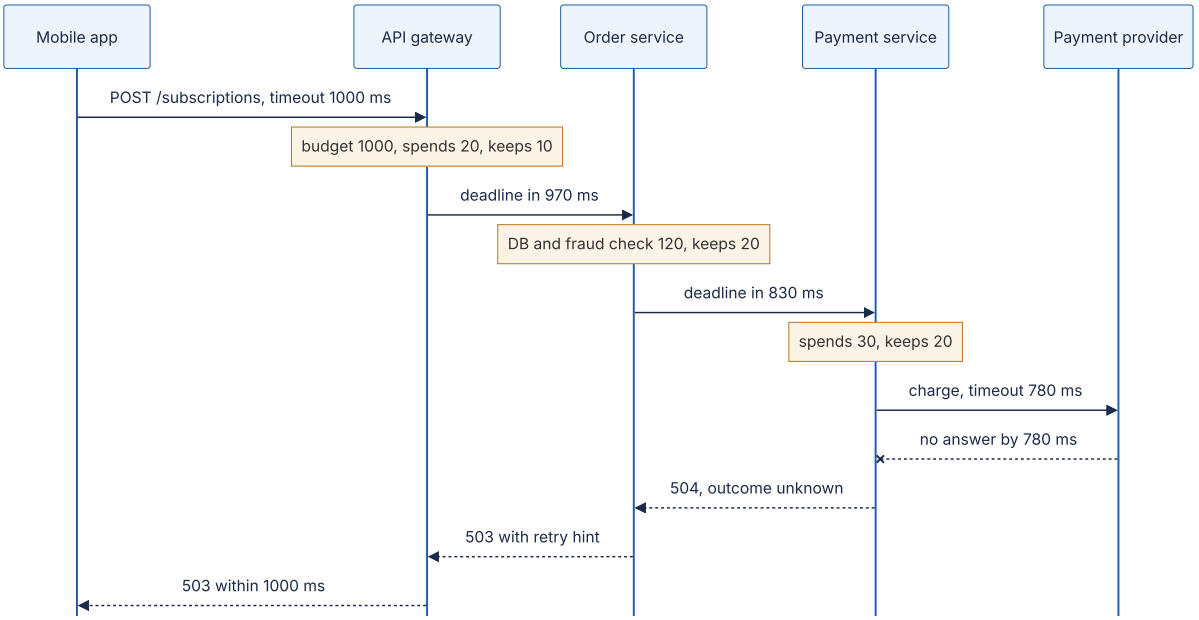


Figure 22.11 — Retry amplification in a four-layer call graph.



Figure 22.12 — Circuit breaker state machine.

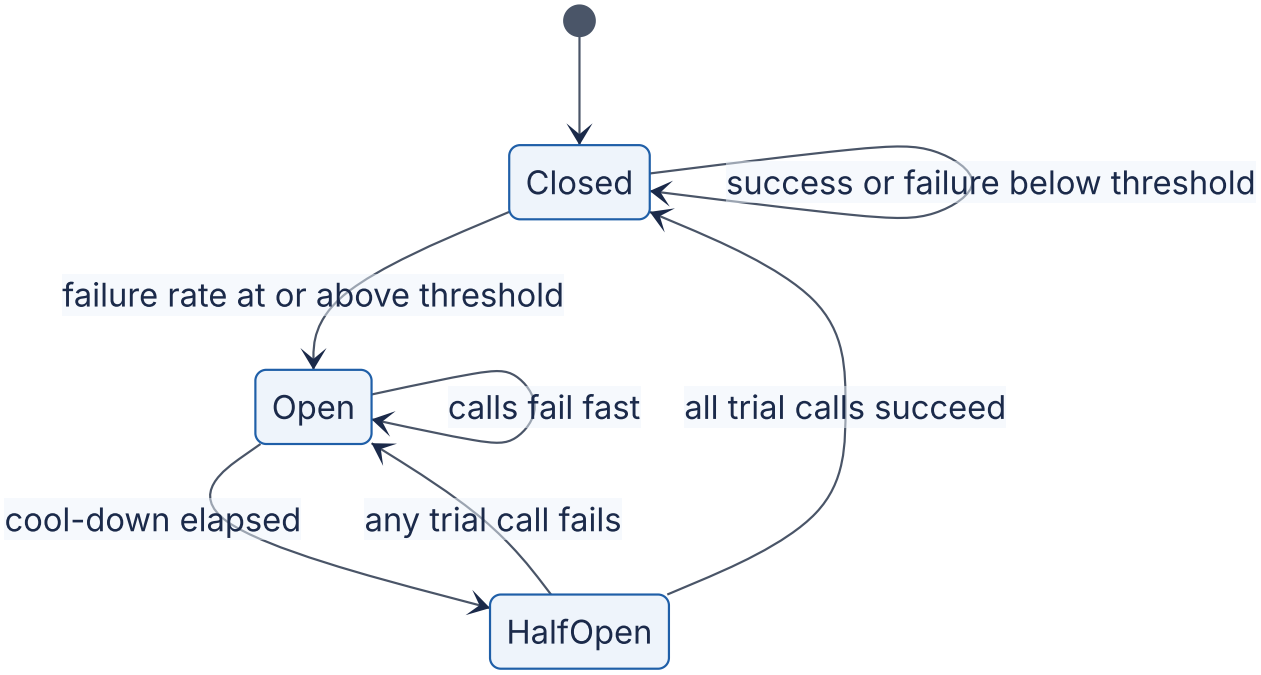


Figure 22.13 — Bulkheads isolating dependencies in the feed service.

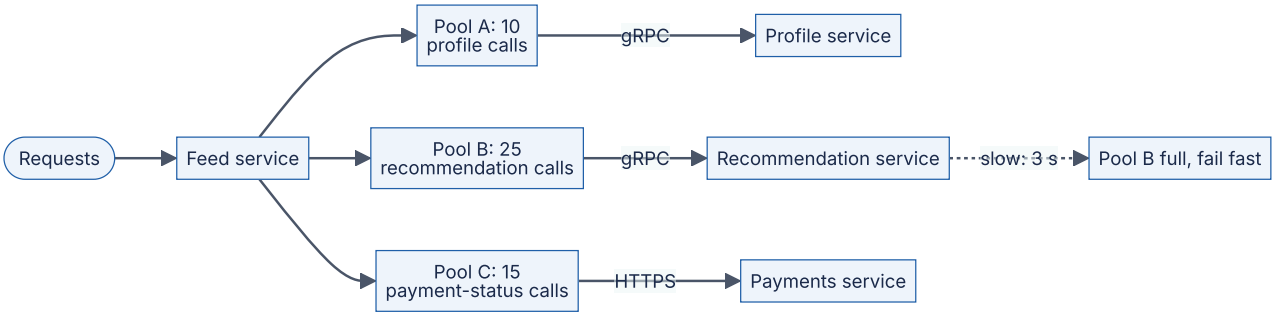


Figure 22.14 — An idempotent purchase retried after a lost response.

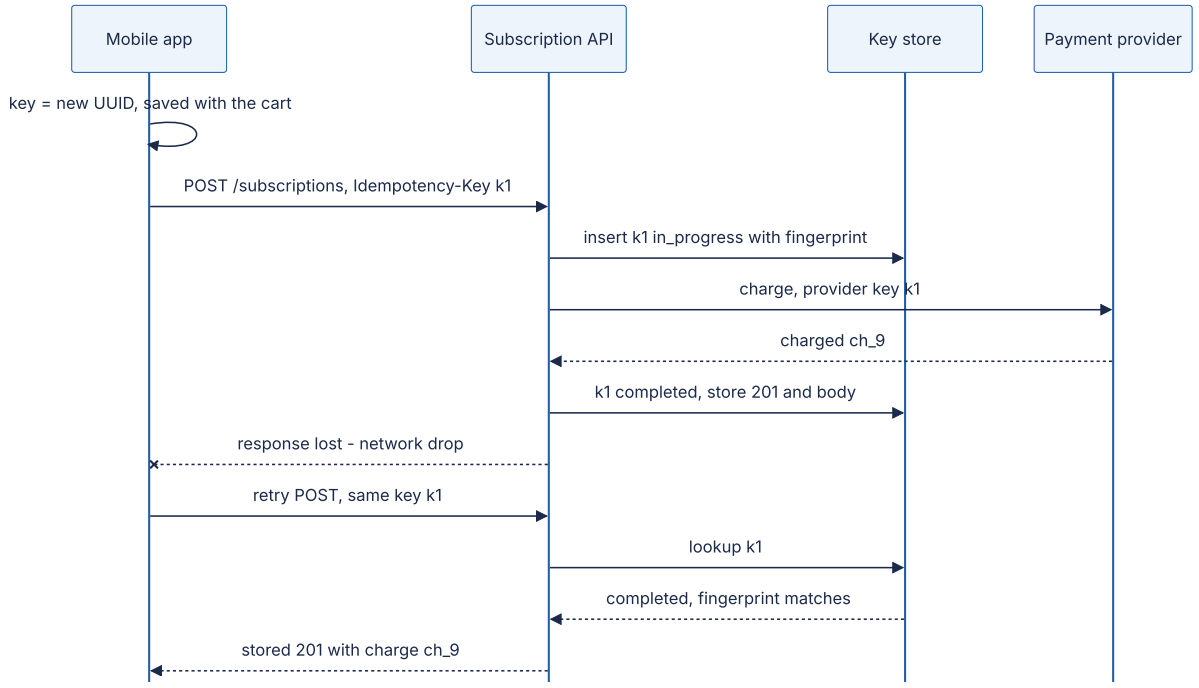
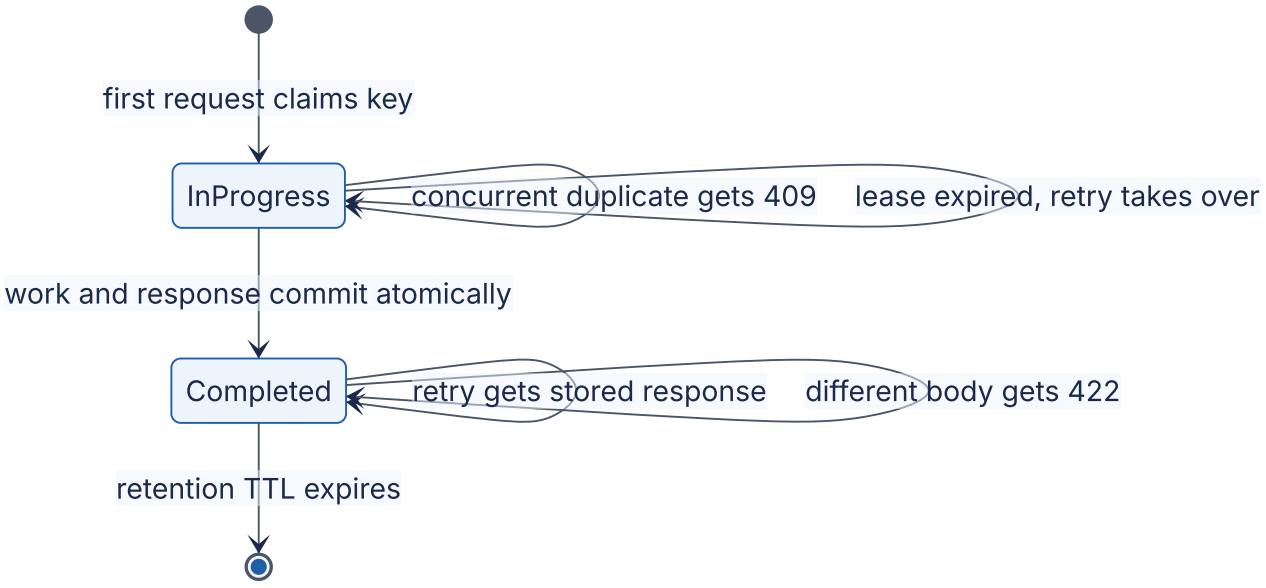
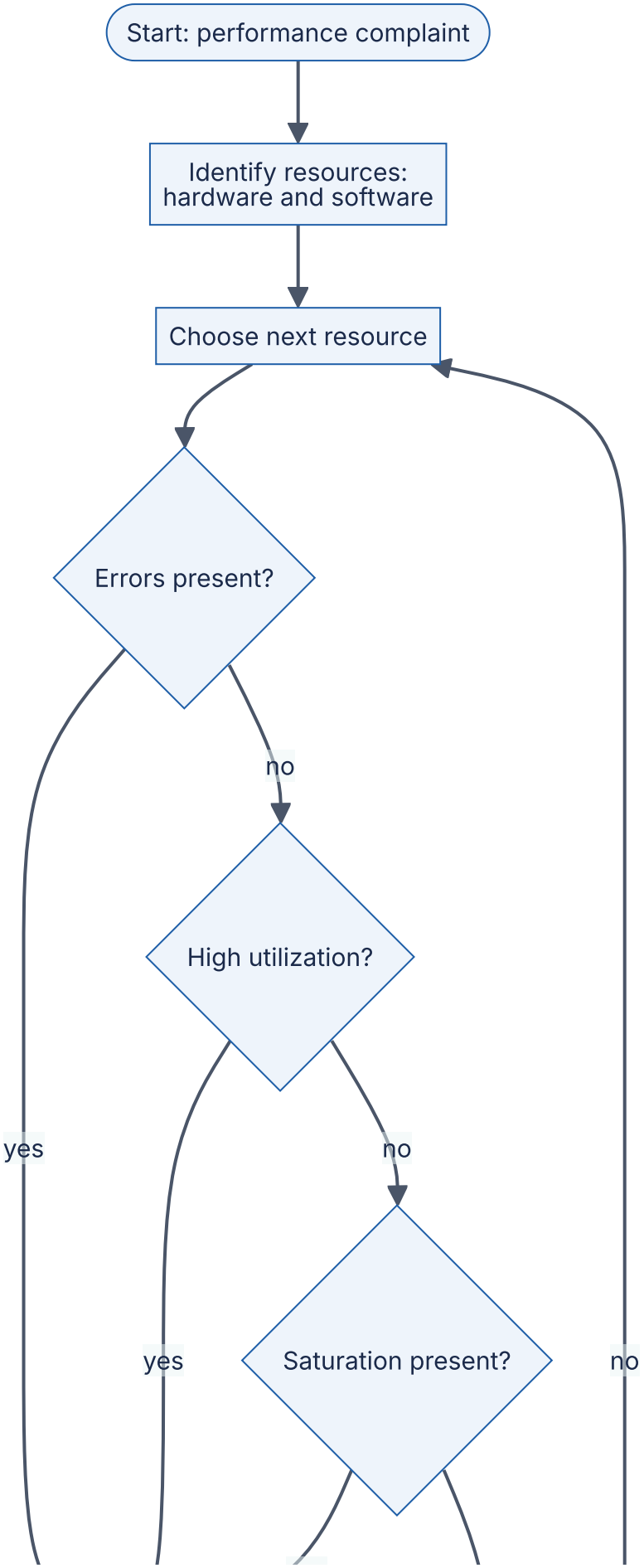


Figure 22.15 — Lifecycle of an idempotency-key record.



Chapter 23 — Performance Engineering

Figure 23.1 — The USE method procedure.


Start: performance complaint
Identify resources: hardware and software
Choose next resource
Errors present?
yes
no
High utilization?
yes
no
Saturation present?
no

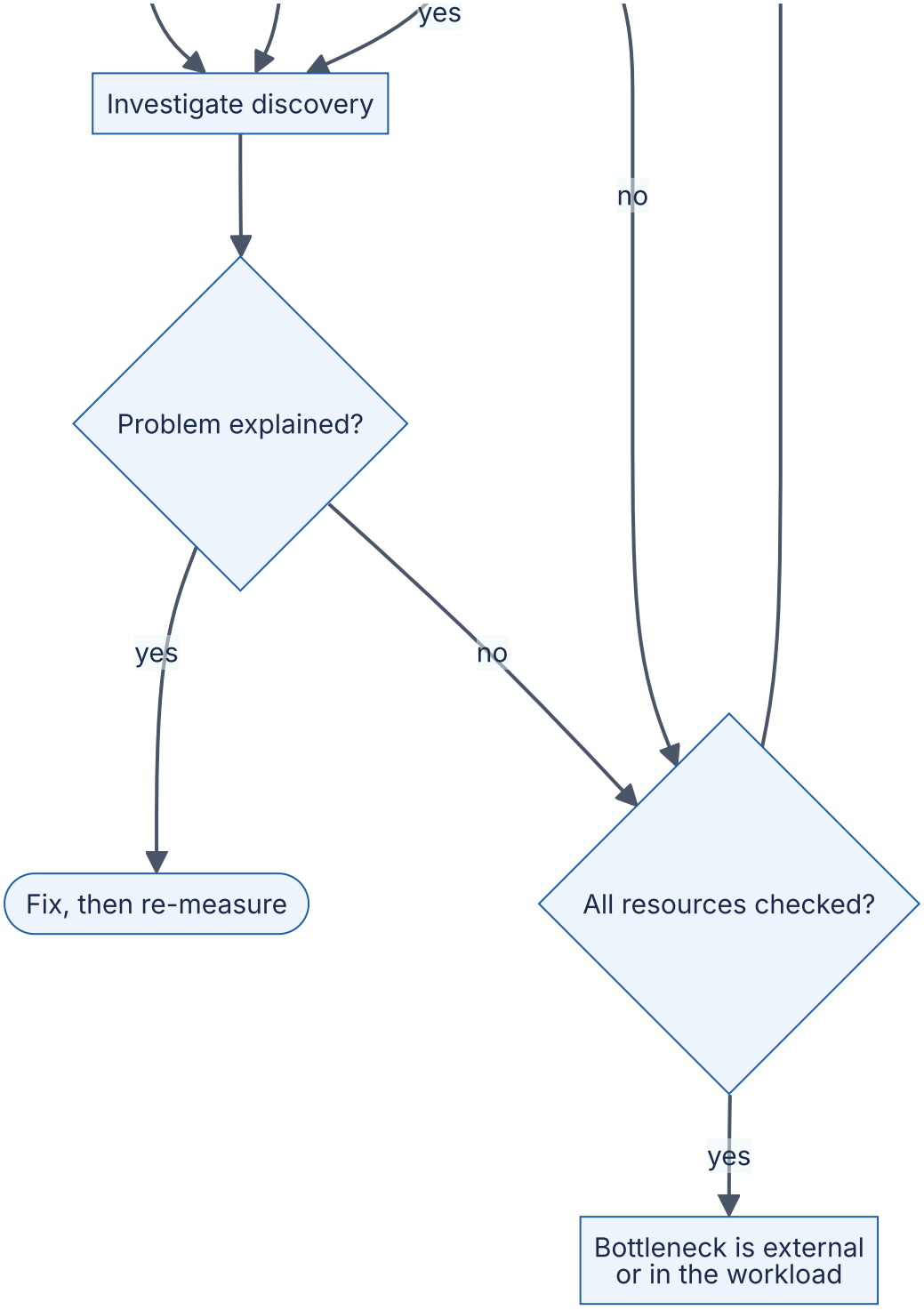


Figure 23.2 — Workflow from symptom to cause.

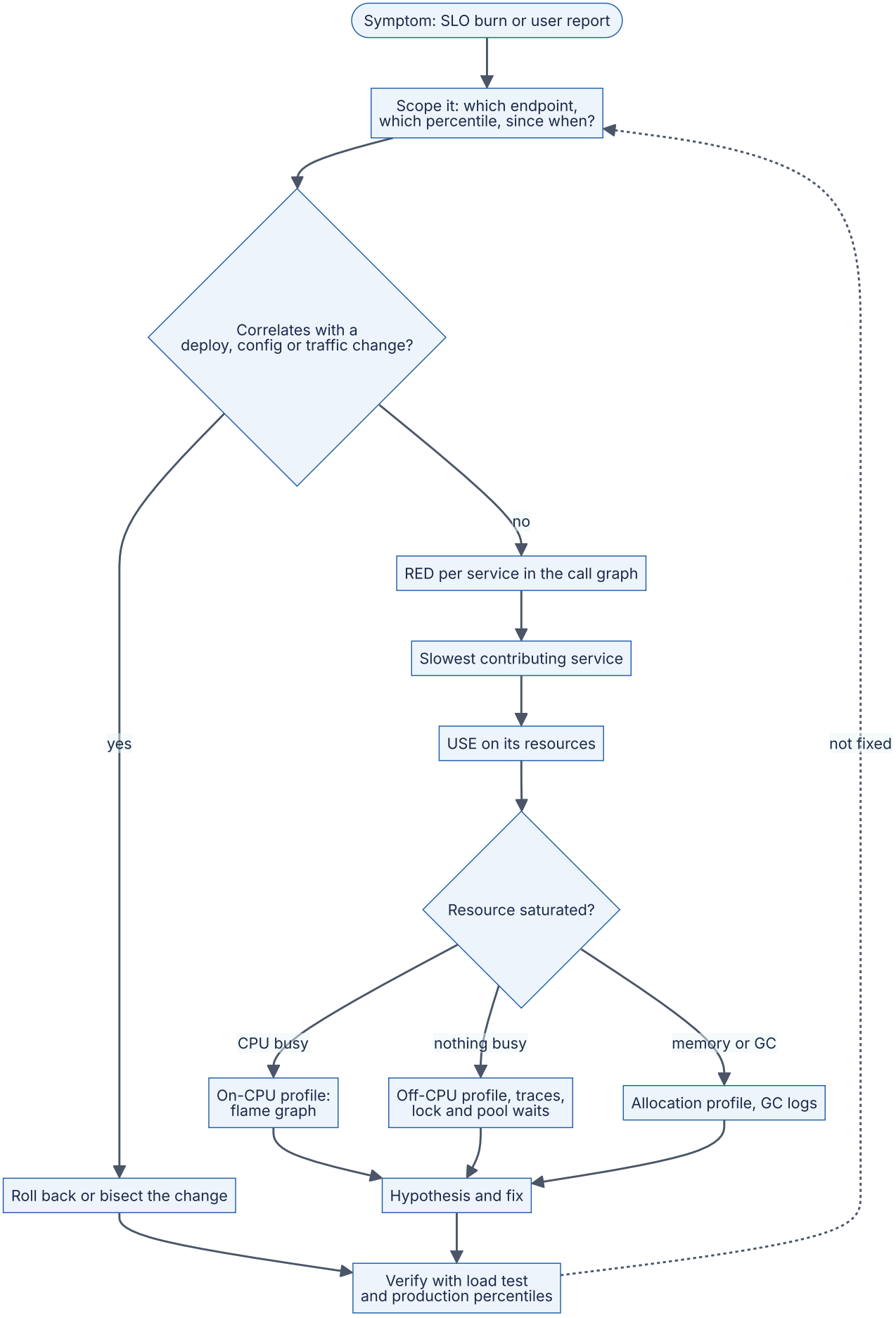

Symptom: SLO burn or user report
Scope it: which endpoint, which percentile, since when?
Correlates with a deploy, config or traffic change?
yes
no
not fixed
RED per service in the call graph
Slowest contributing service
USE on its resources
Resource saturated?
CPU busy
nothing busy
memory or GC
On-CPU profile: flame graph
Off-CPU profile, traces, lock and pool waits
Allocation profile, GC logs
Roll back or bisect the change
Hypothesis and fix
Verify with load test and production percentiles

Figure 23.3 — From samples to a flame graph.



Figure 23.4 — On-CPU and off-CPU thread states.

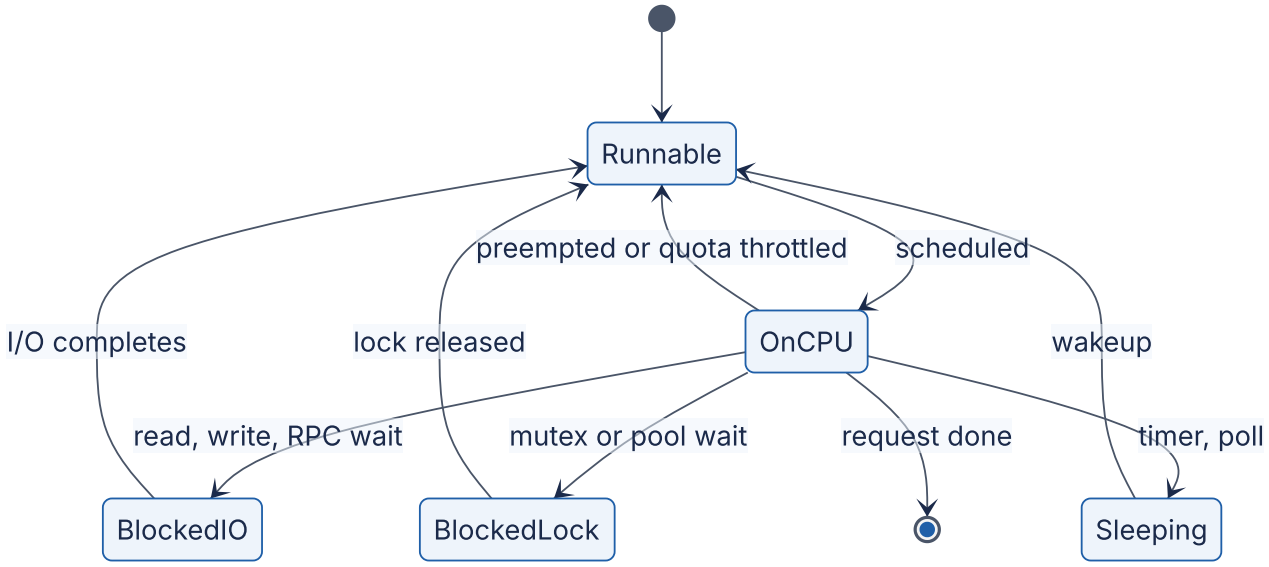


Figure 23.5 — Open and closed load generators.

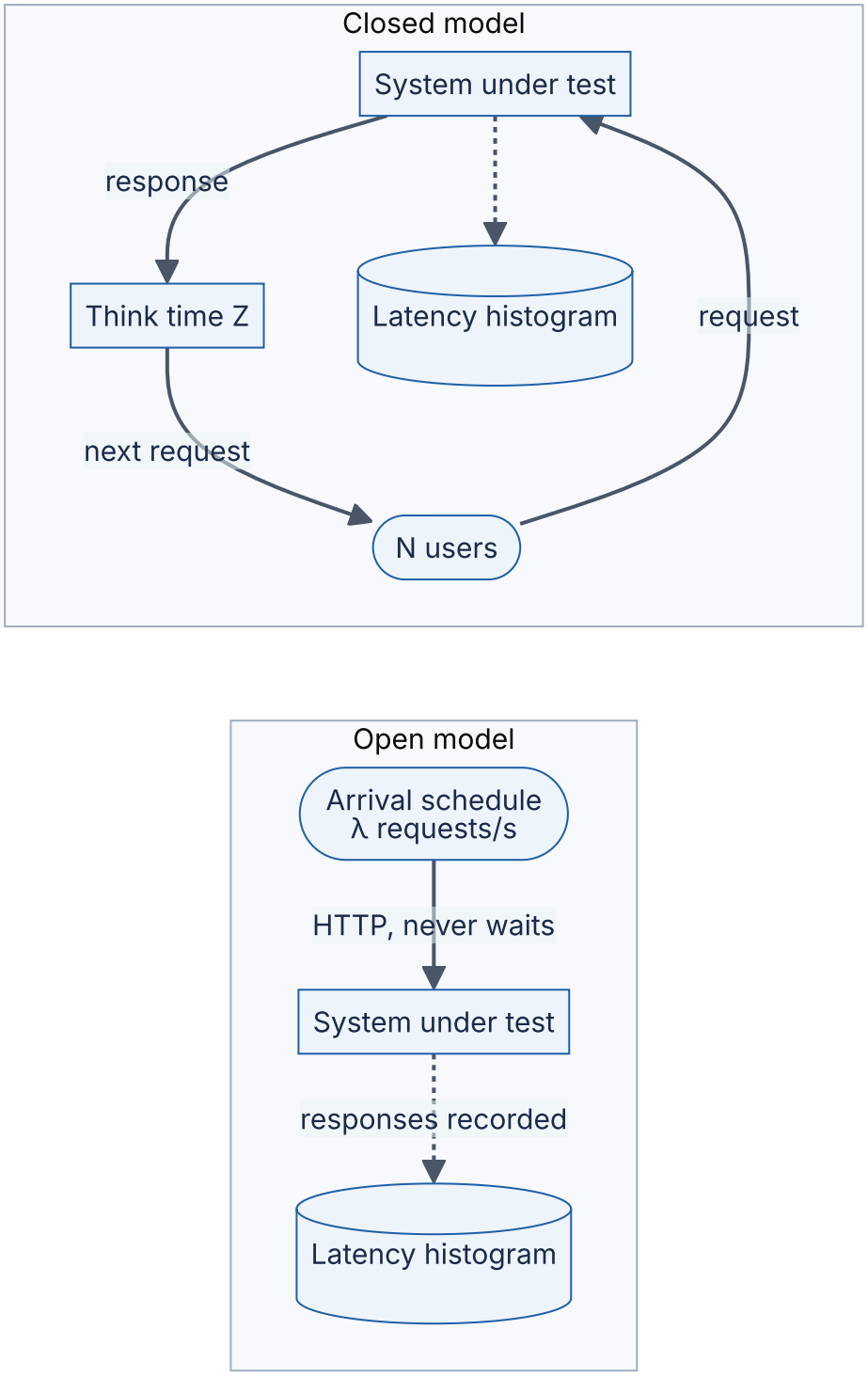


Figure 23.6 — How a closed-loop tester omits the stall.

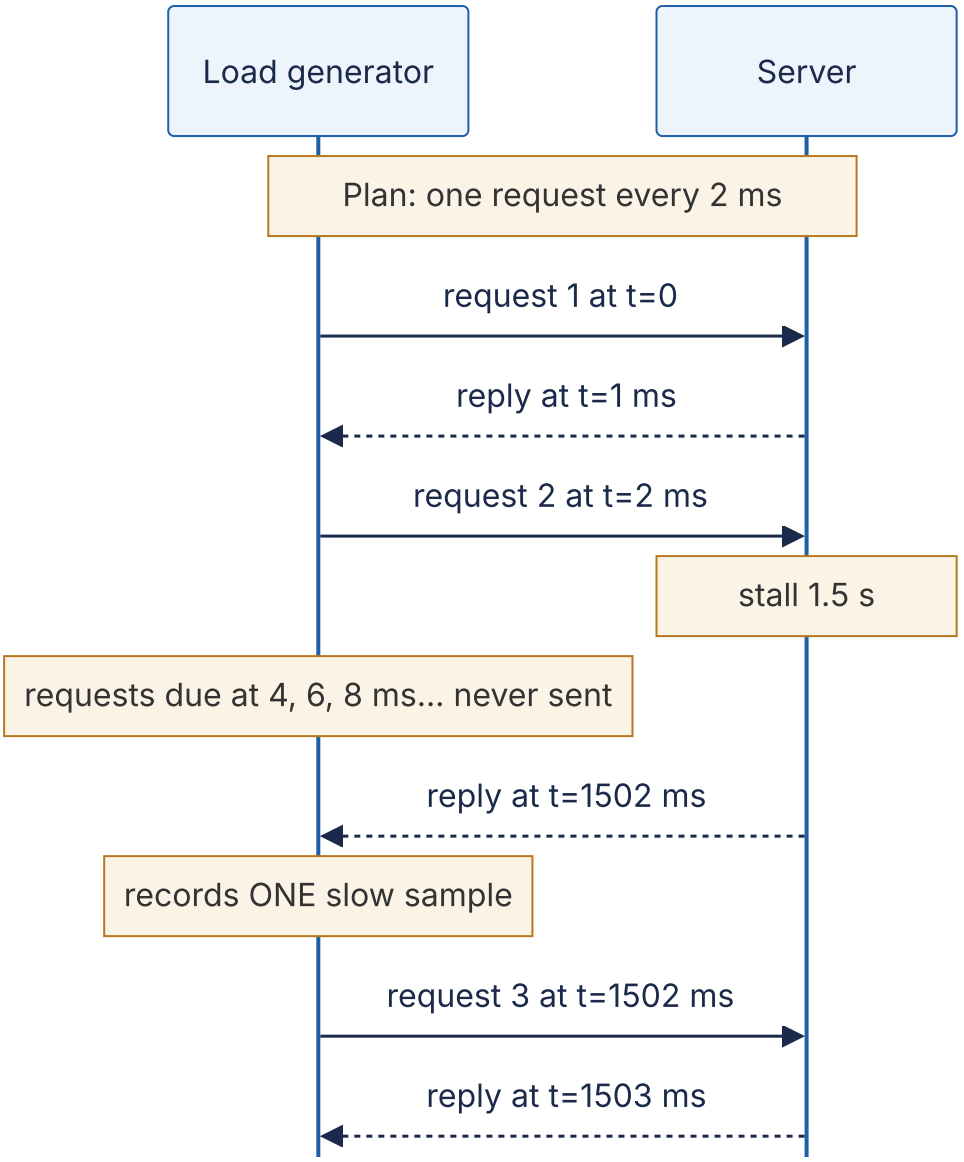


Figure 23.7 — Where variability enters a request.

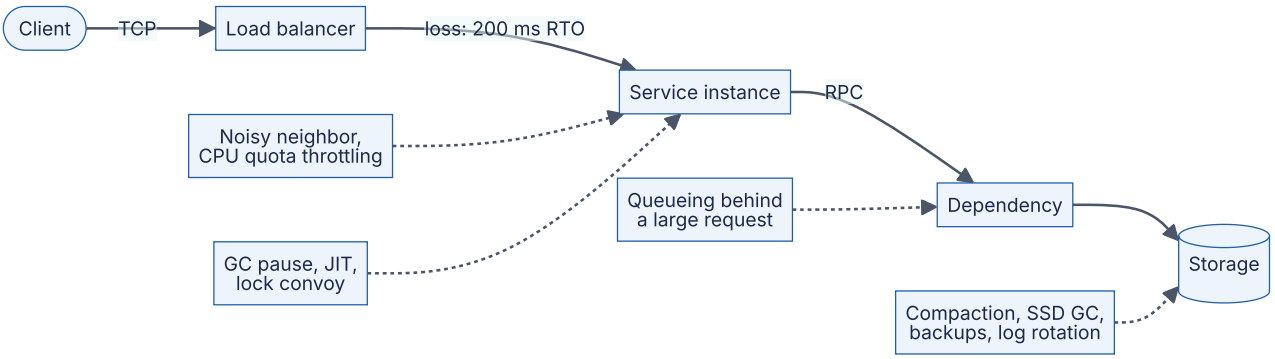


Figure 23.8 — A hedged request.

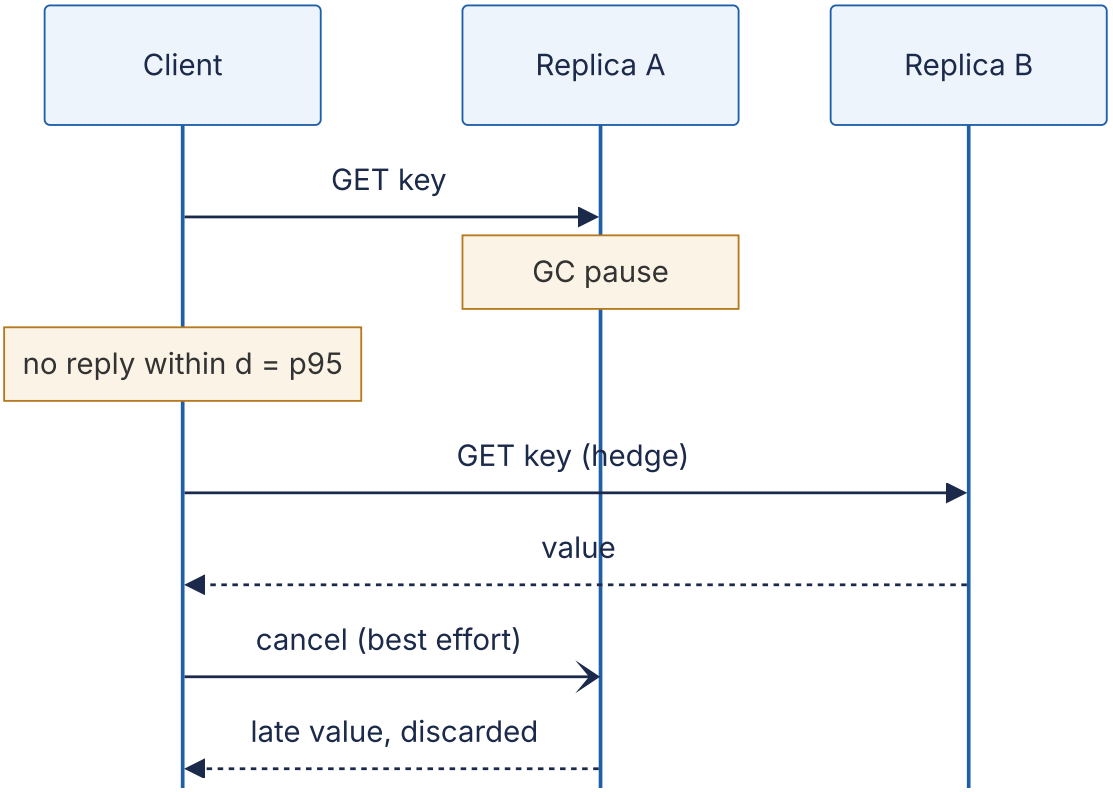


Figure 23.9 — Tied requests with cross-server cancellation.

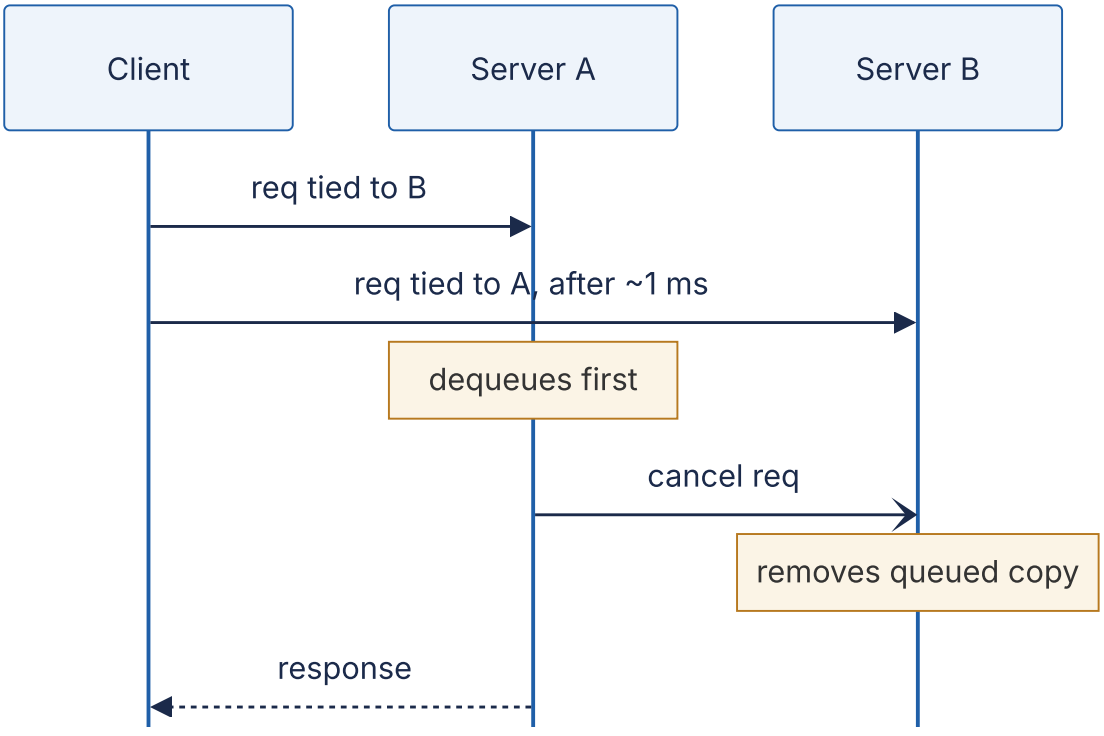


Figure 23.10 — A batcher with size and time triggers.

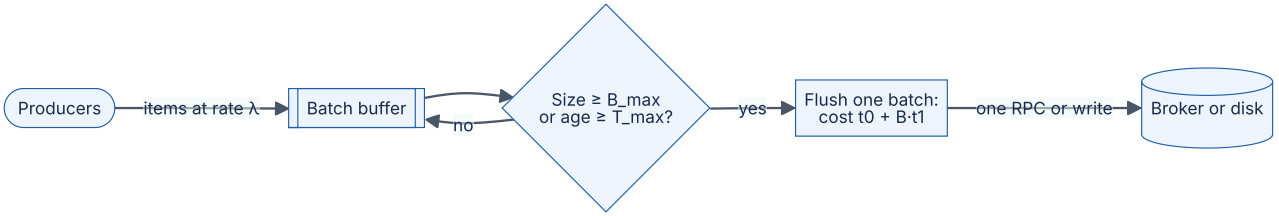


Figure 23.11 — Generational heap.

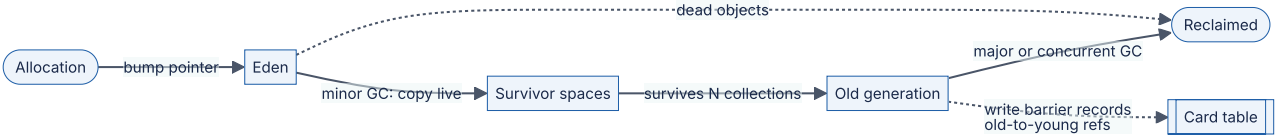
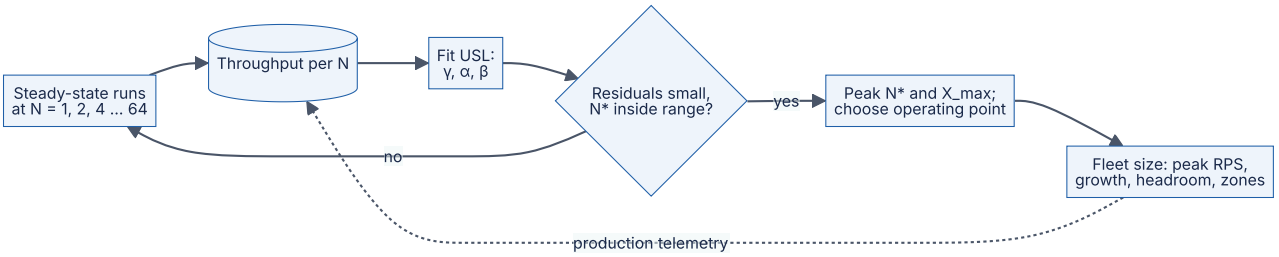


Figure 23.12 — Capacity modeling workflow.



Chapter 24 — Distributed Systems Fundamentals

Figure 24.1 — One computer versus a distributed system.

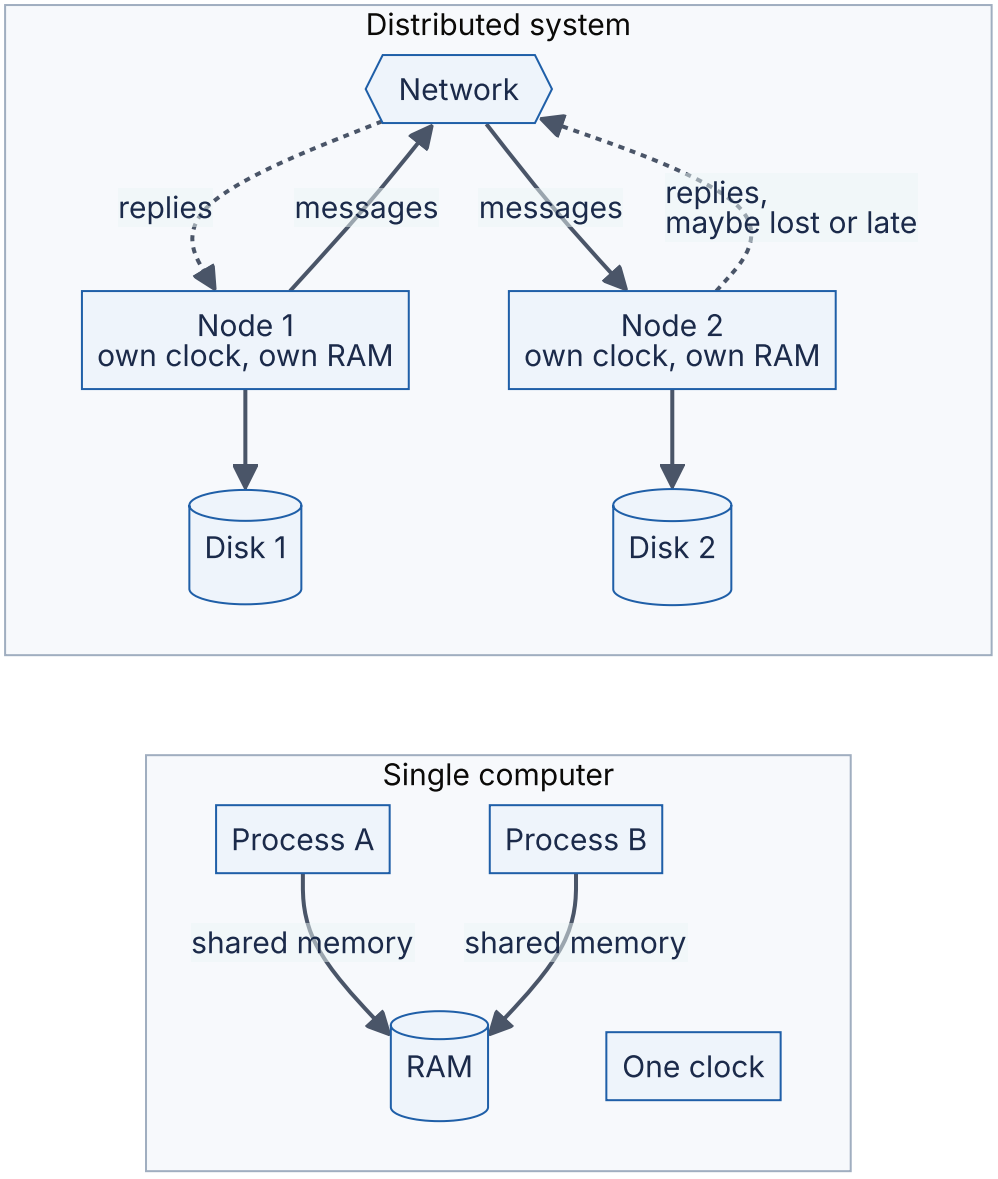


Figure 24.2 — The two generals problem: every acknowledgment needs another acknowledgment.

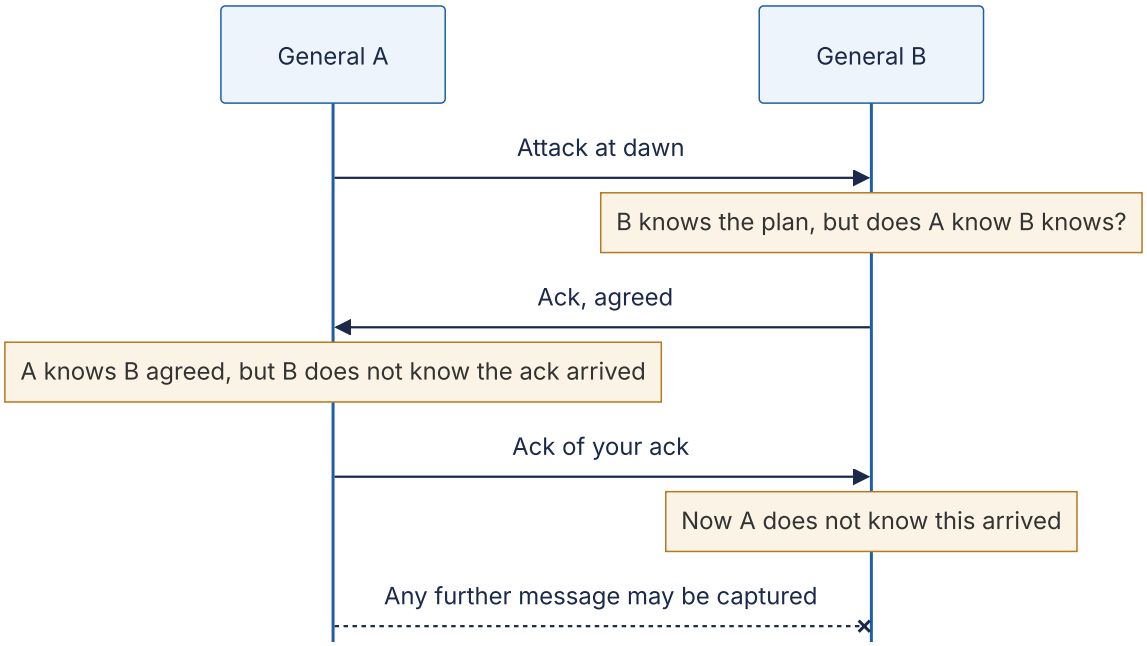


Figure 24.3 — Failure models nest: each one includes the behaviors of those inside it.

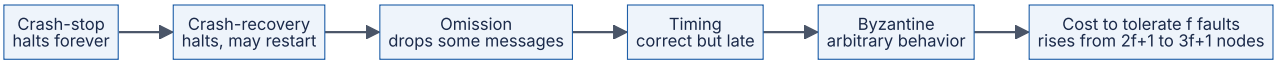
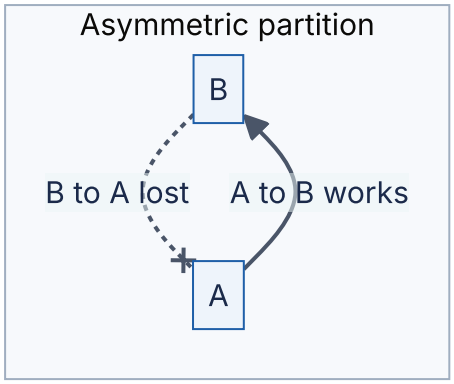
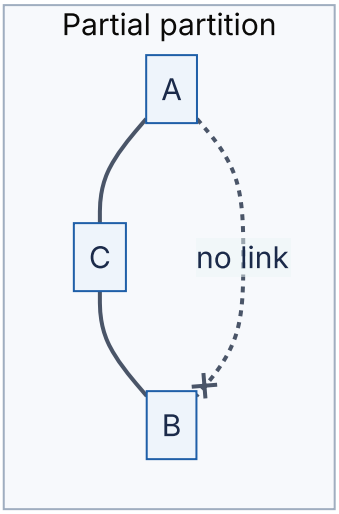


Figure 24.4 — Three kinds of network partition.





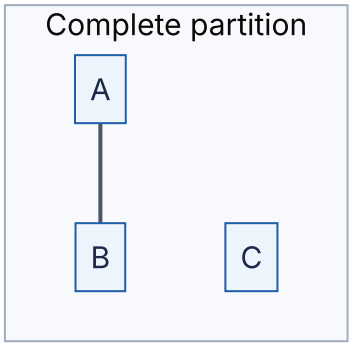


Figure 24.5 — Six things that can happen to one request, all of which look the same to the client.

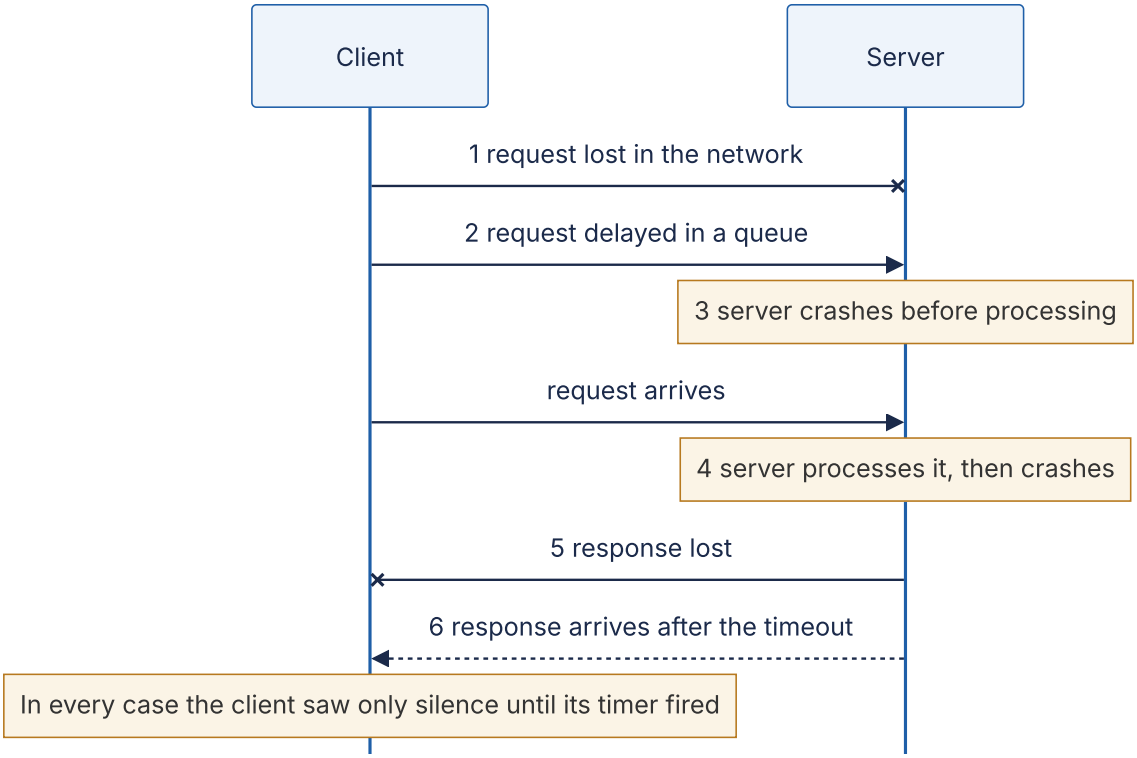


Figure 24.6 — Gray failure is a disagreement between observers.

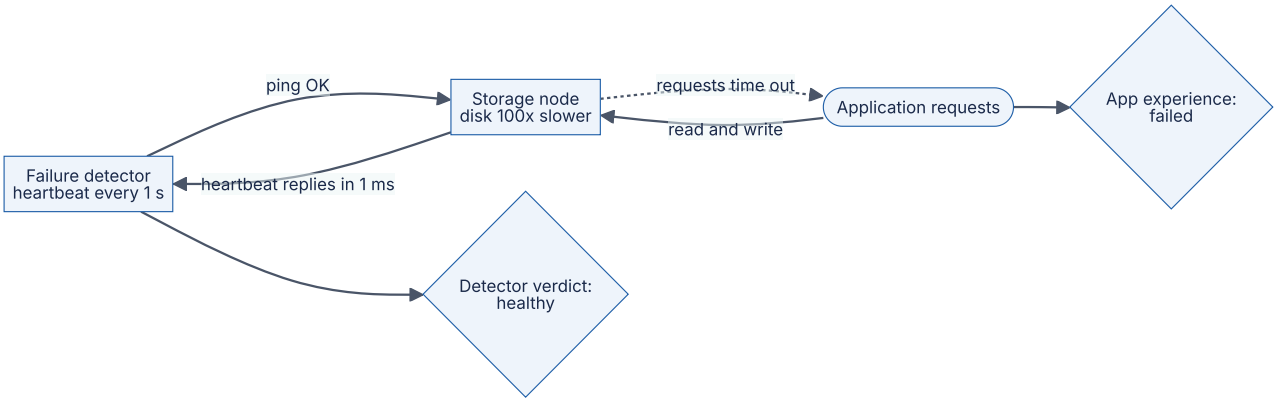


Figure 24.7 — The NTP on-wire exchange.

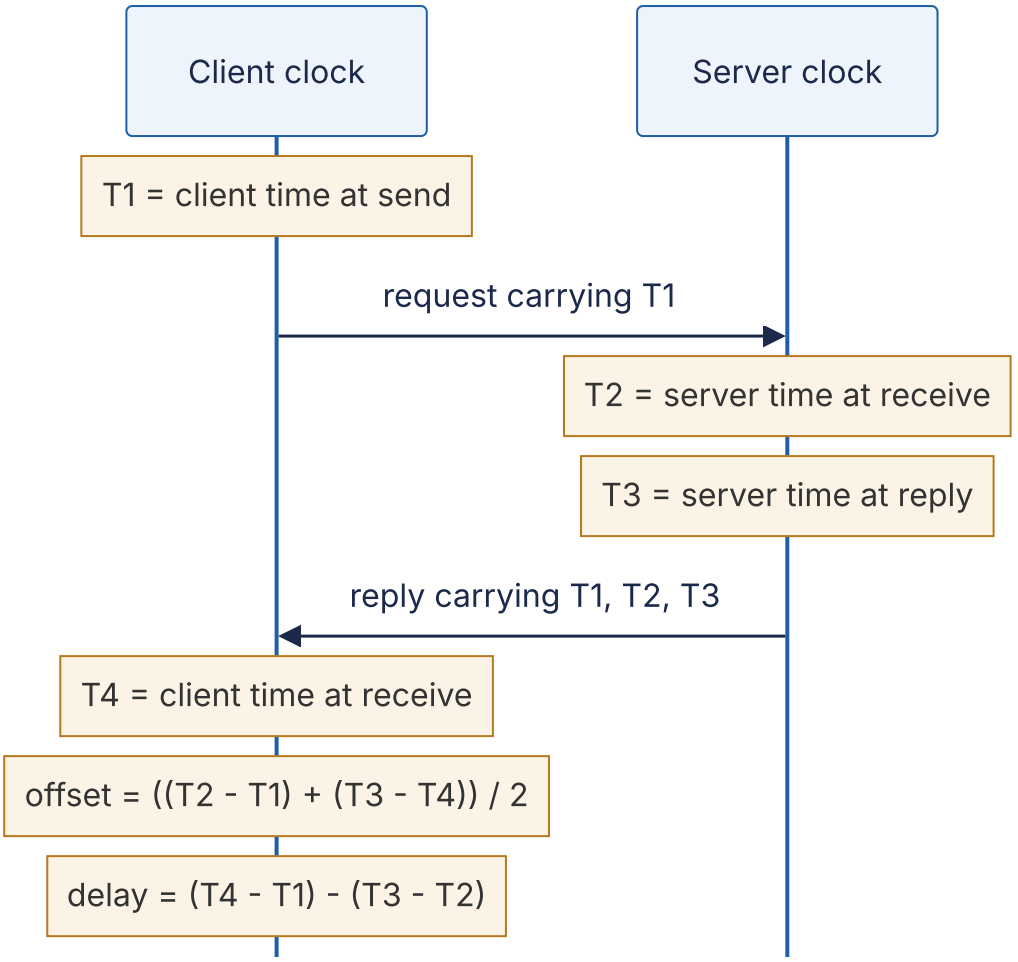


Figure 24.8 — The time-distribution hierarchy: NTP strata and a PTP domain.

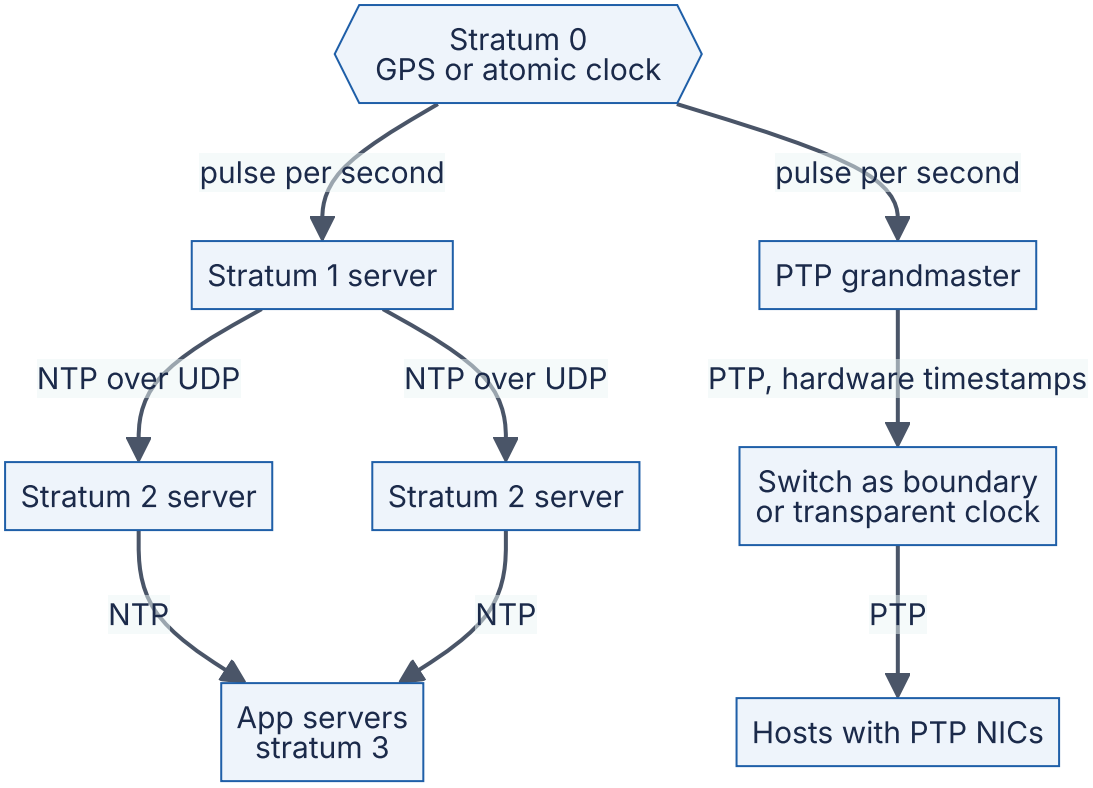


Figure 24.9 — Last-writer-wins discards the later write when one clock runs fast.

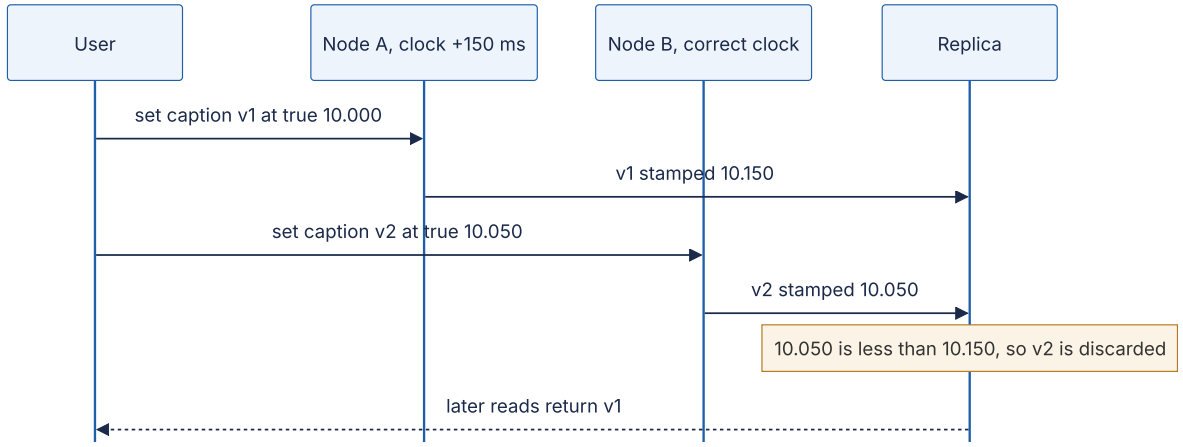


Figure 24.10 — Lamport timestamps on three processes.

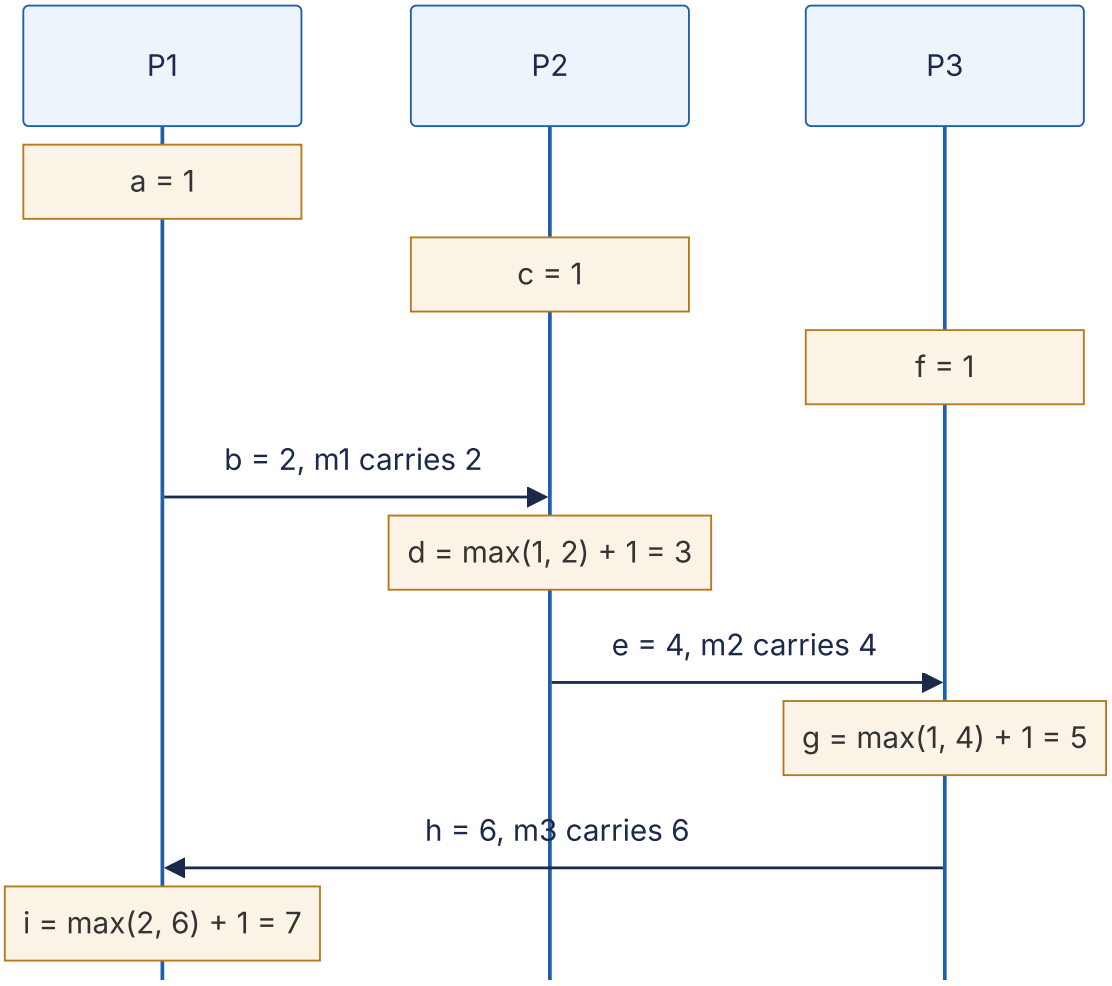


Figure 24.11 — Vector clocks on the same trace.

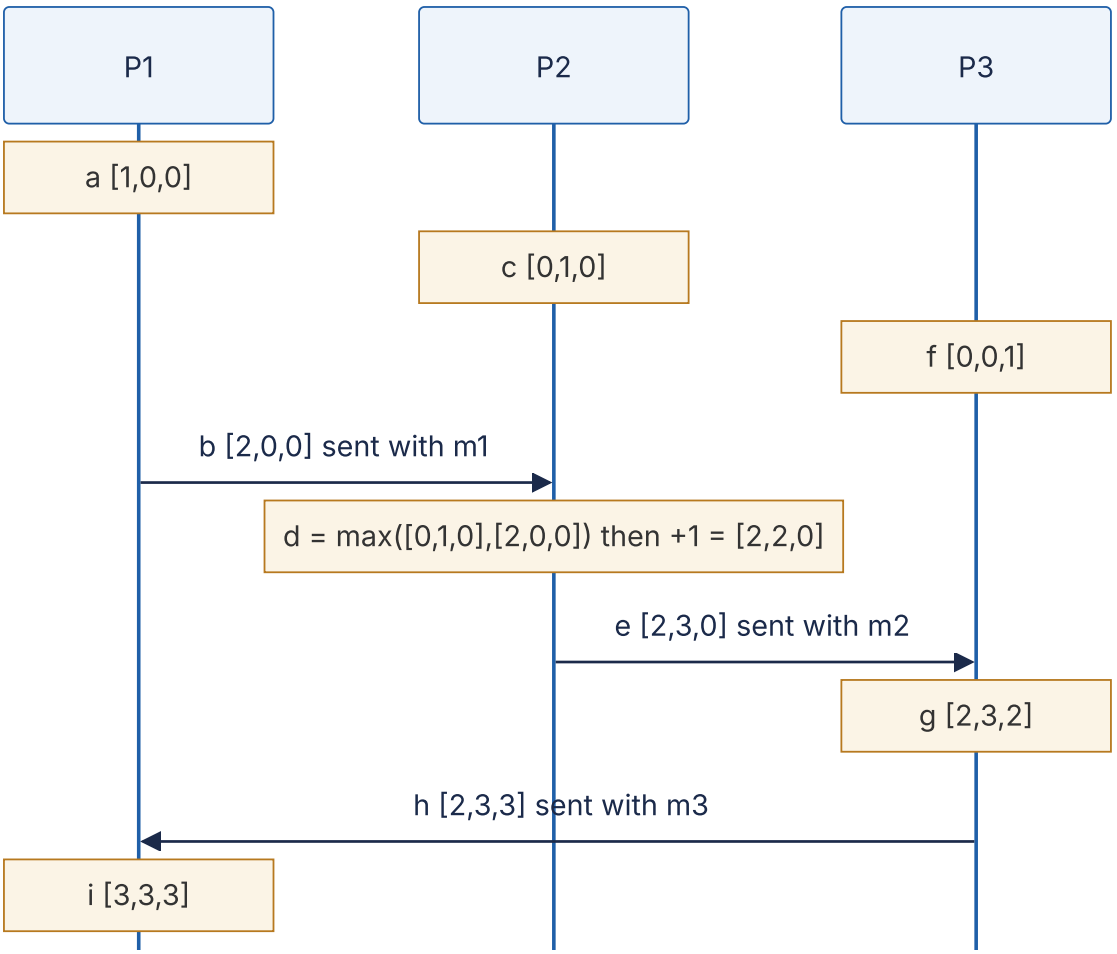


Figure 24.12 — Version vectors detect a concurrent shopping-cart update.

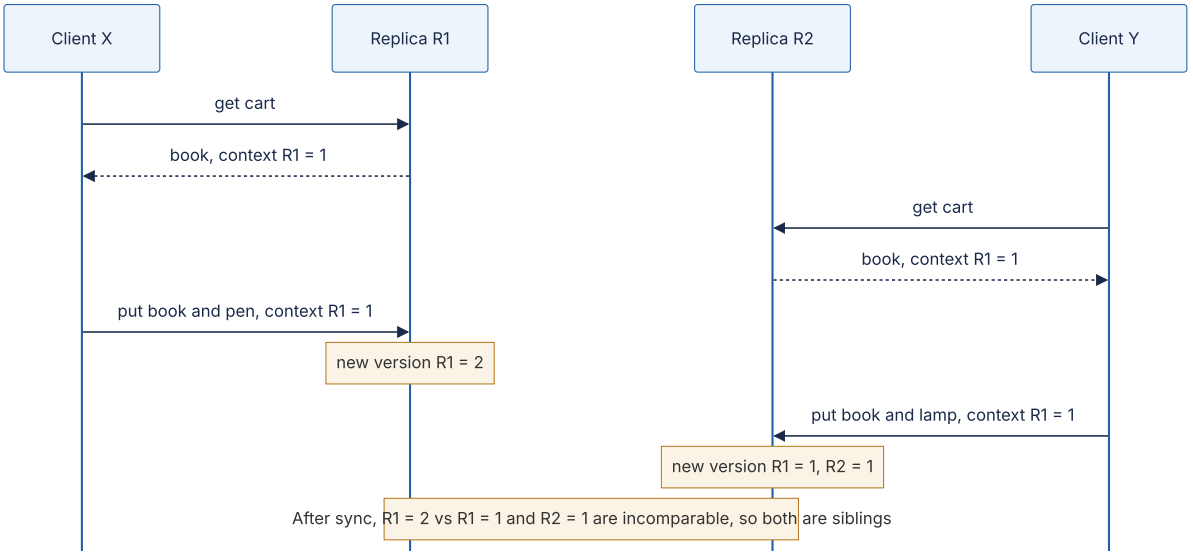


Figure 24.13 — HLC keeps causal order when node A’s clock runs 100 ms fast.

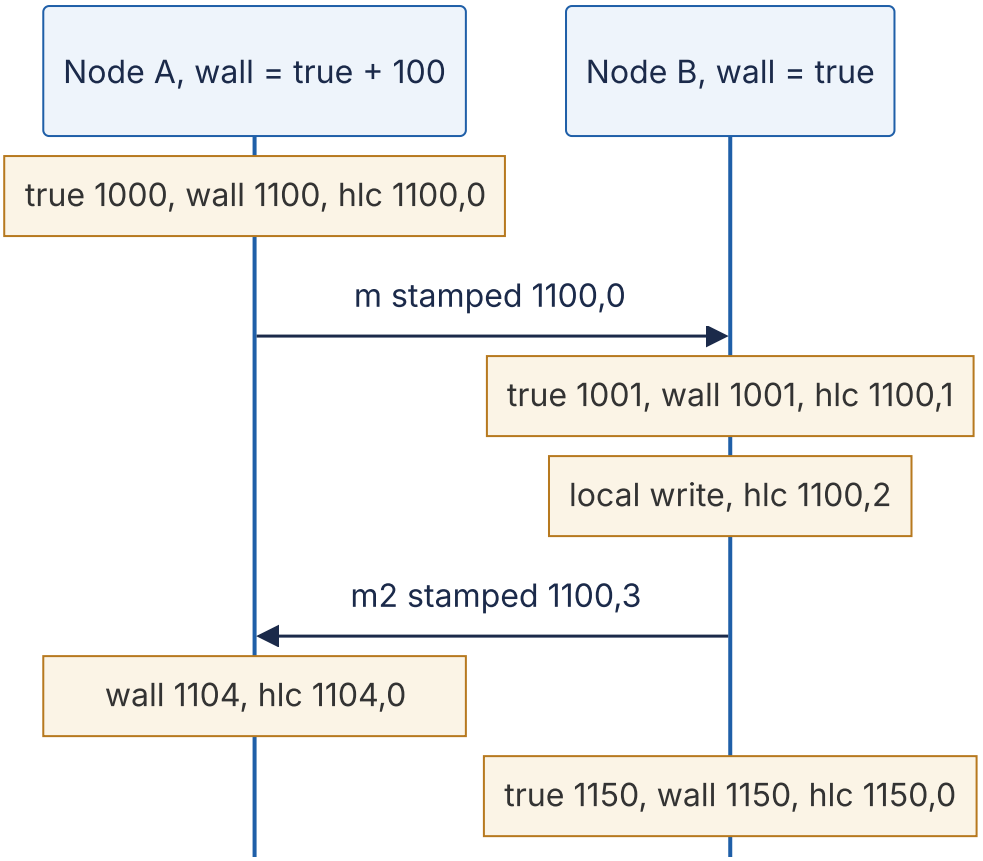


Figure 24.14 — The adversary keeps the protocol undecided.

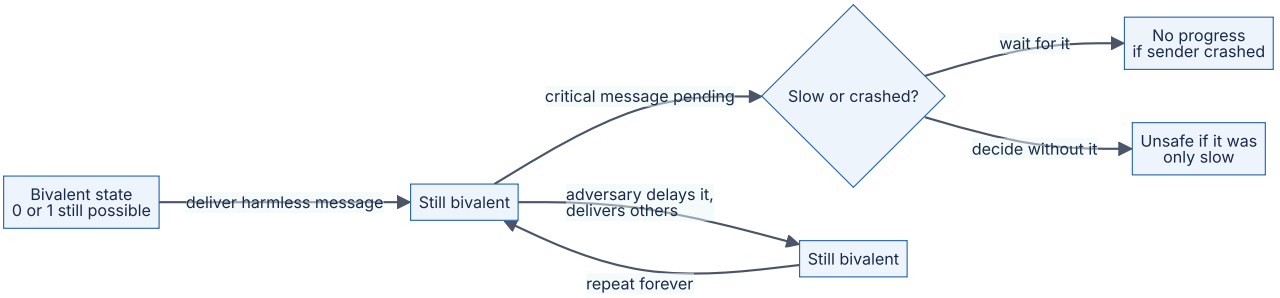
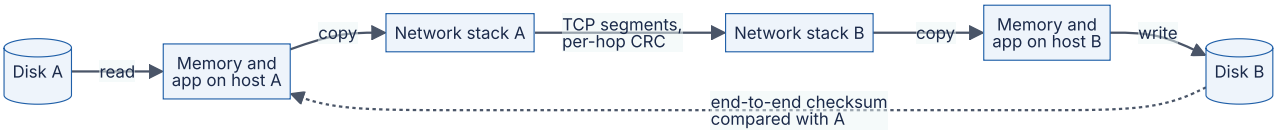


Figure 24.15 — Where errors can enter a file transfer, and which check covers them.



Chapter 25 — Consistency and Availability

Figure 25.1 — A linearizable history and its linearization points.

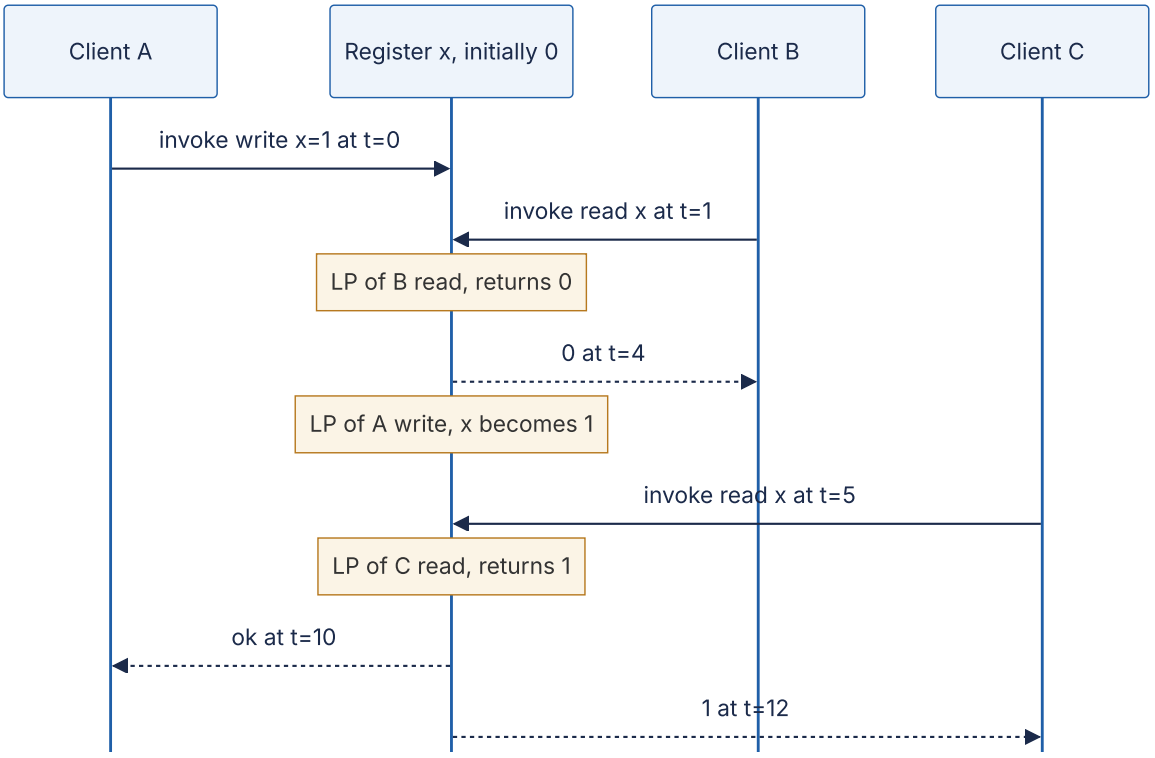


Figure 25.2 — Two non-linearizable histories: a stale read, and a new-then-old inversion.

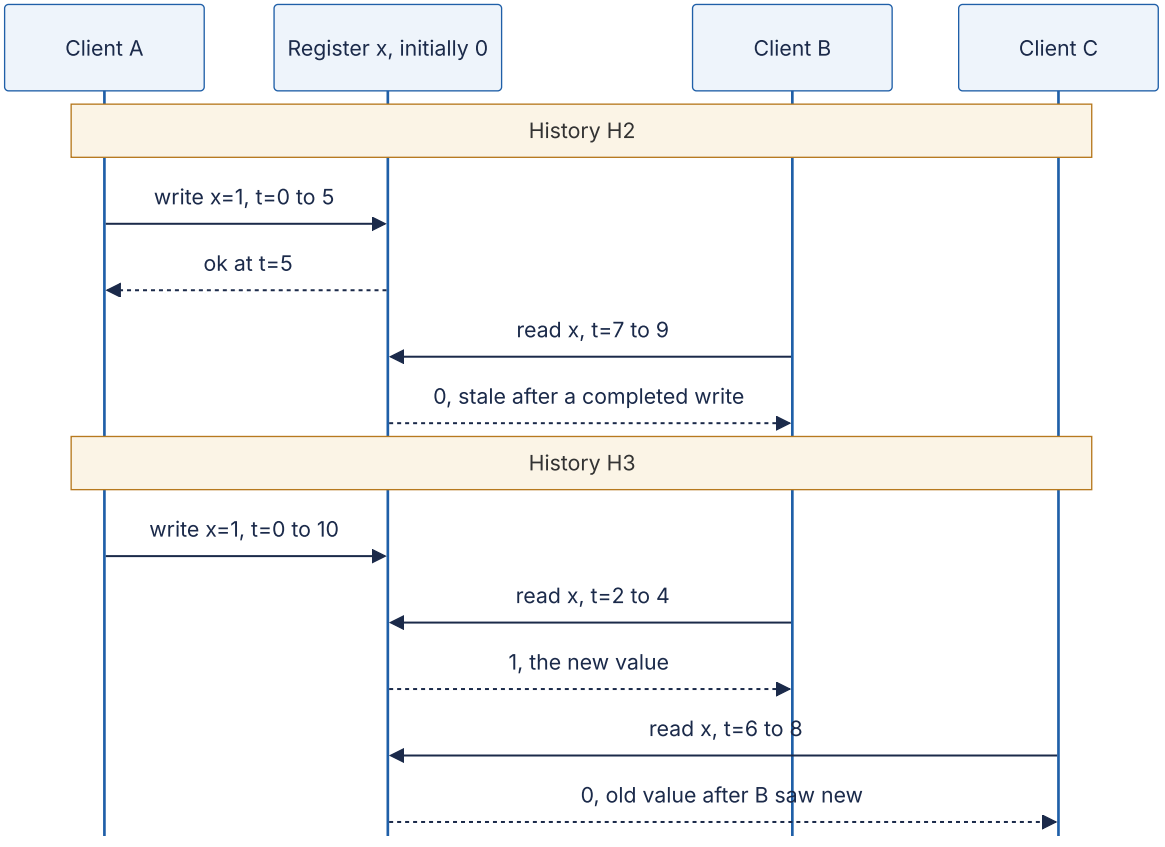


Figure 25.3 — ABD quorum read with write-back on three replicas.

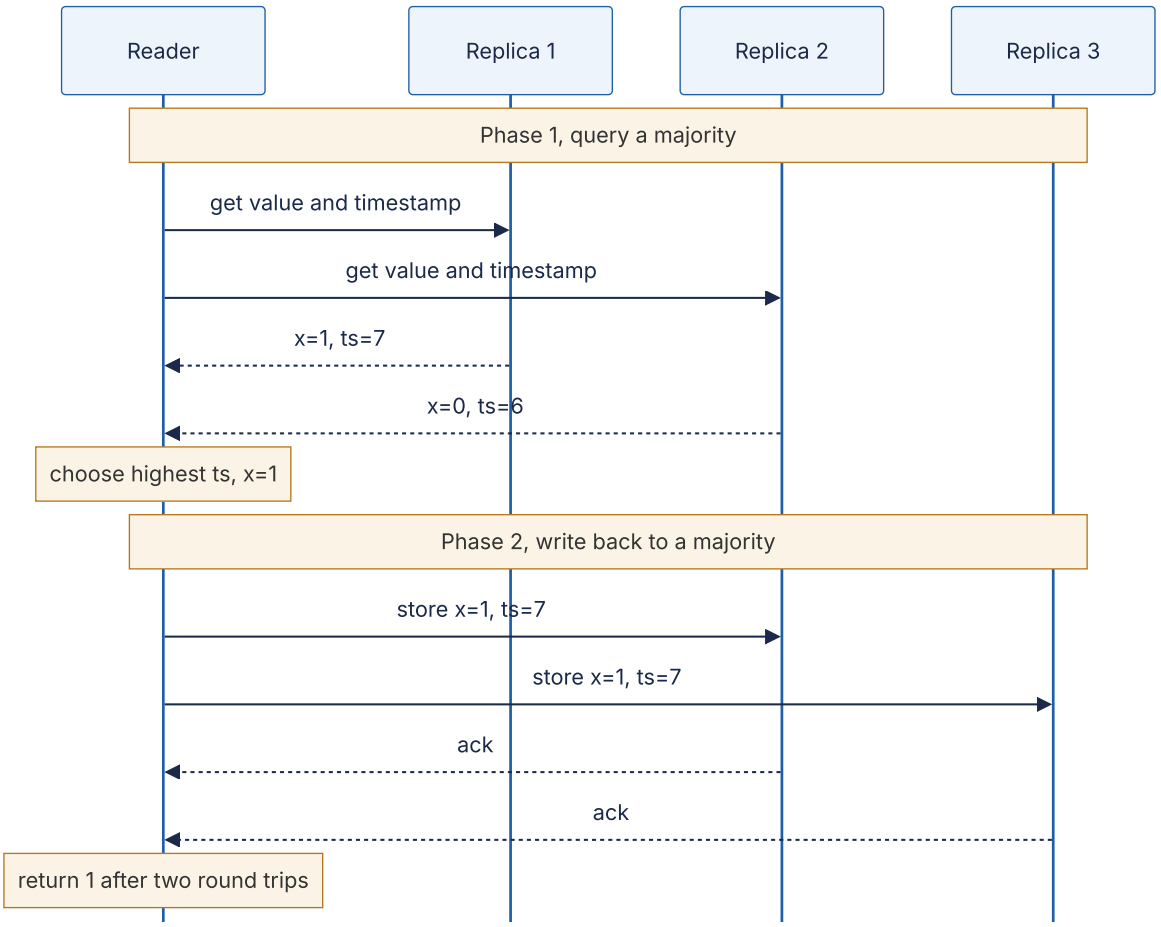


Figure 25.4 — A sequentially consistent but not linearizable history (follower read).

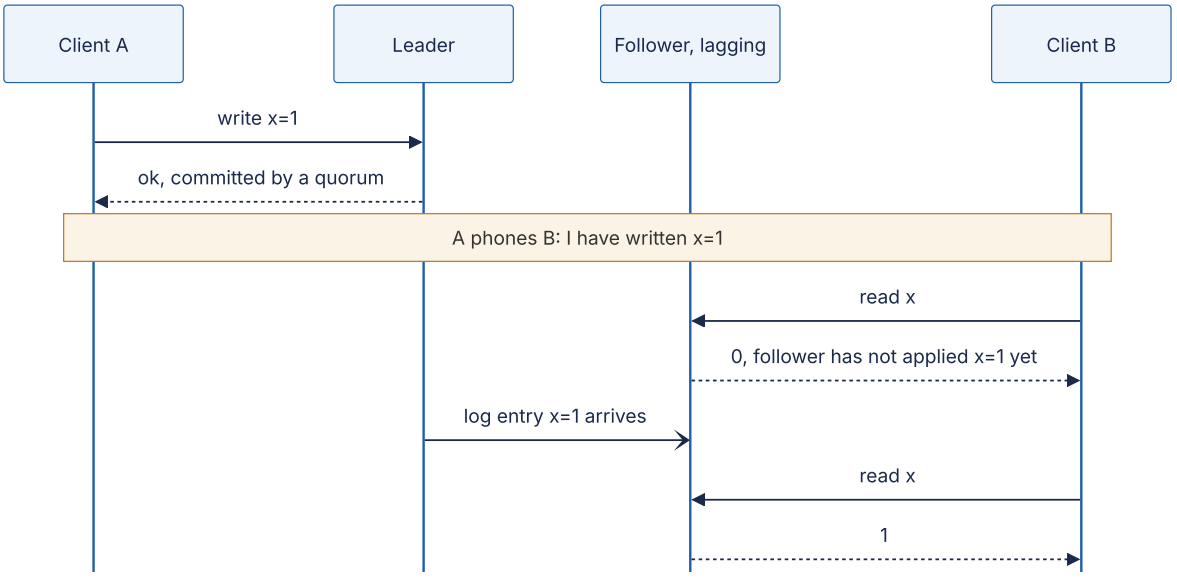


Figure 25.5 — A causality violation across regions.

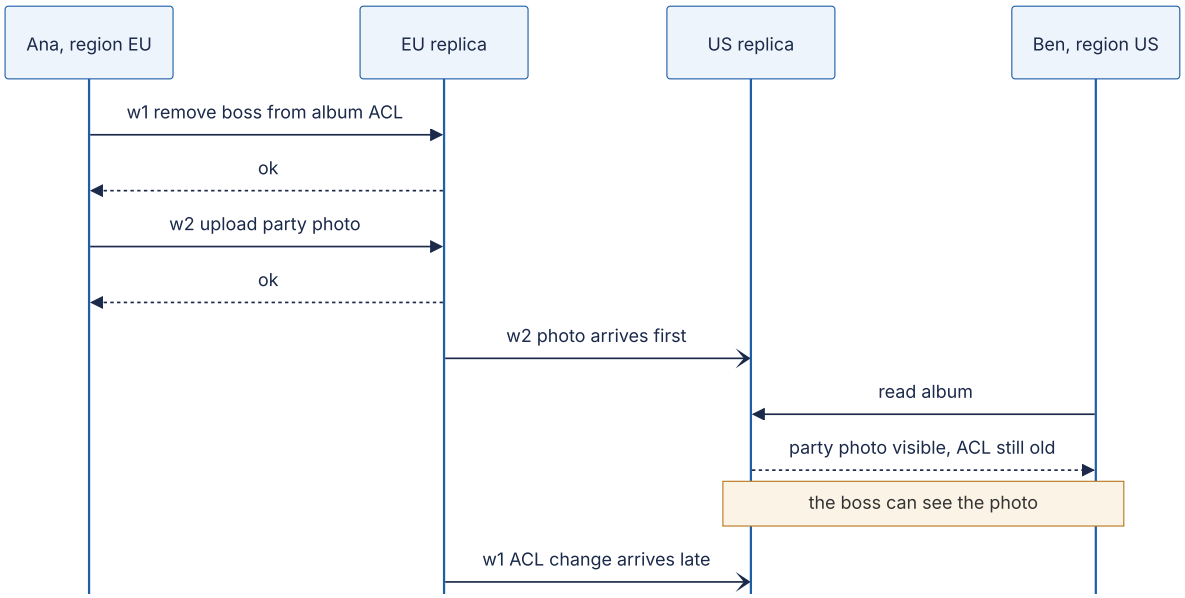


Figure 25.6 — COPS: local writes, asynchronous replication, dependency checks.

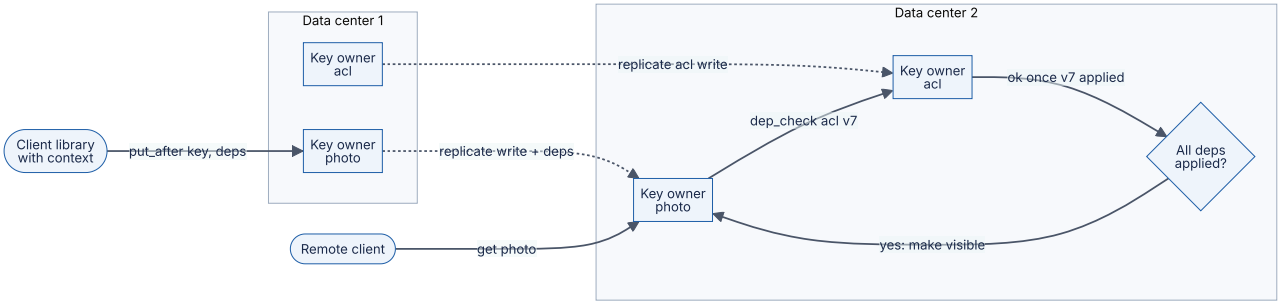


Figure 25.7 — Read-your-writes and monotonic-reads violations behind a load balancer.

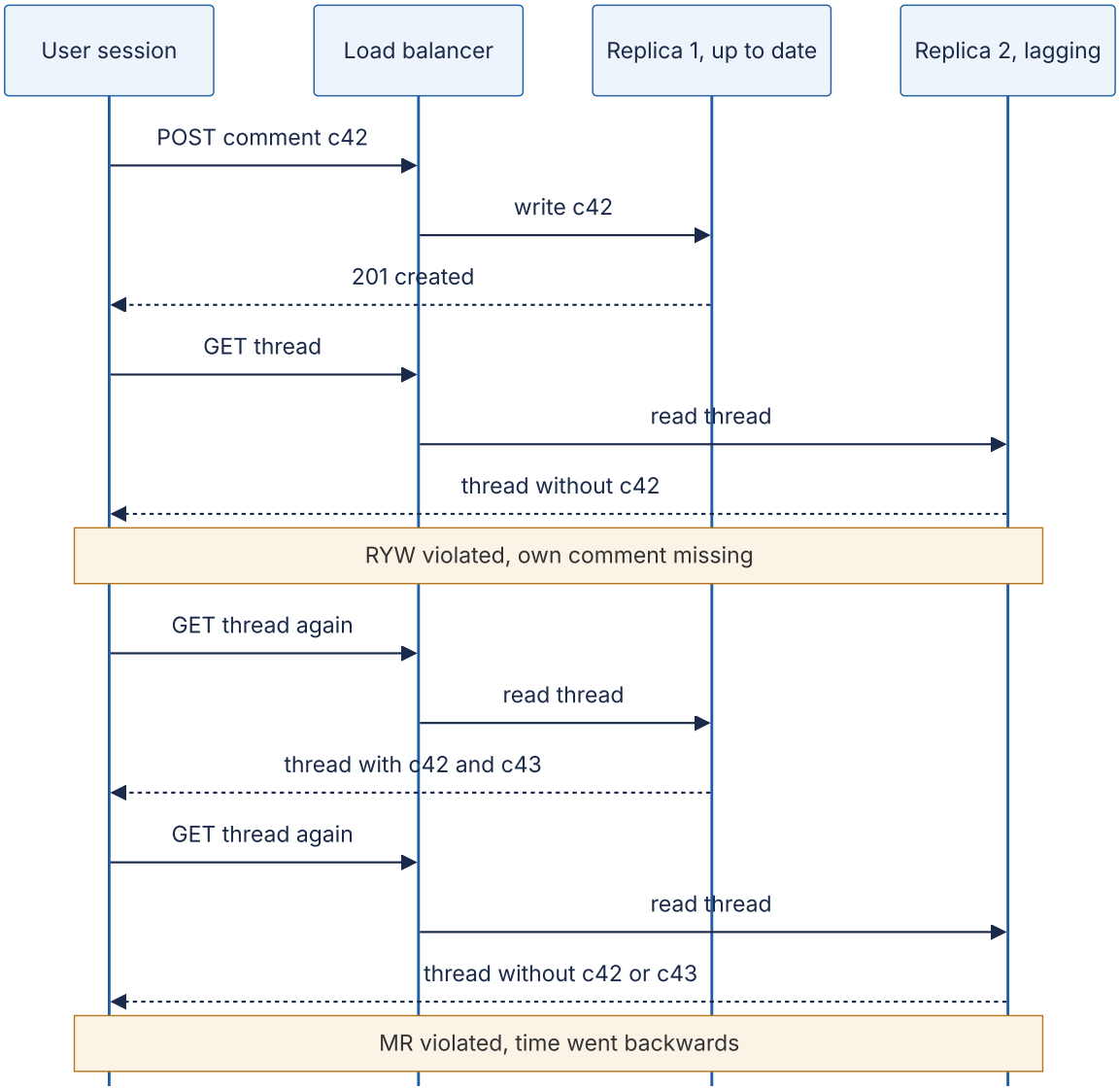


Figure 25.8 — Monotonic-writes and writes-follow-reads violations.

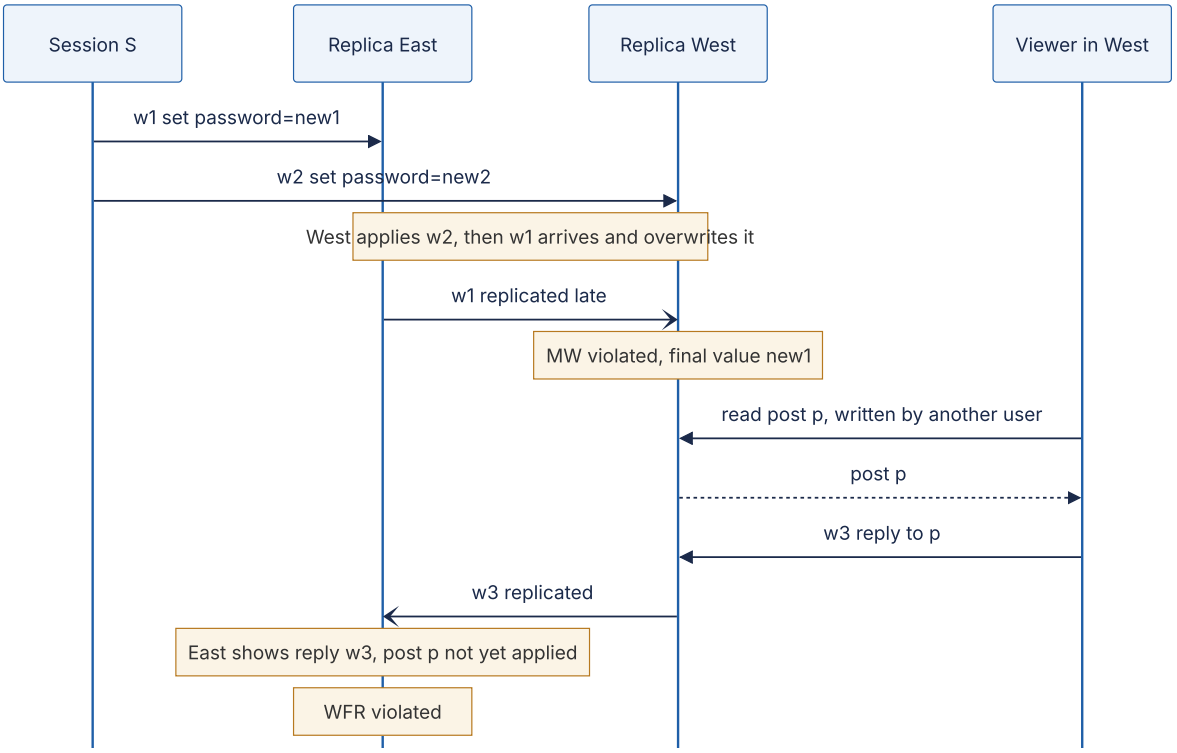


Figure 25.9 — The WARS model of a partial-quorum write and read (N = 3, W = 1, R = 1).

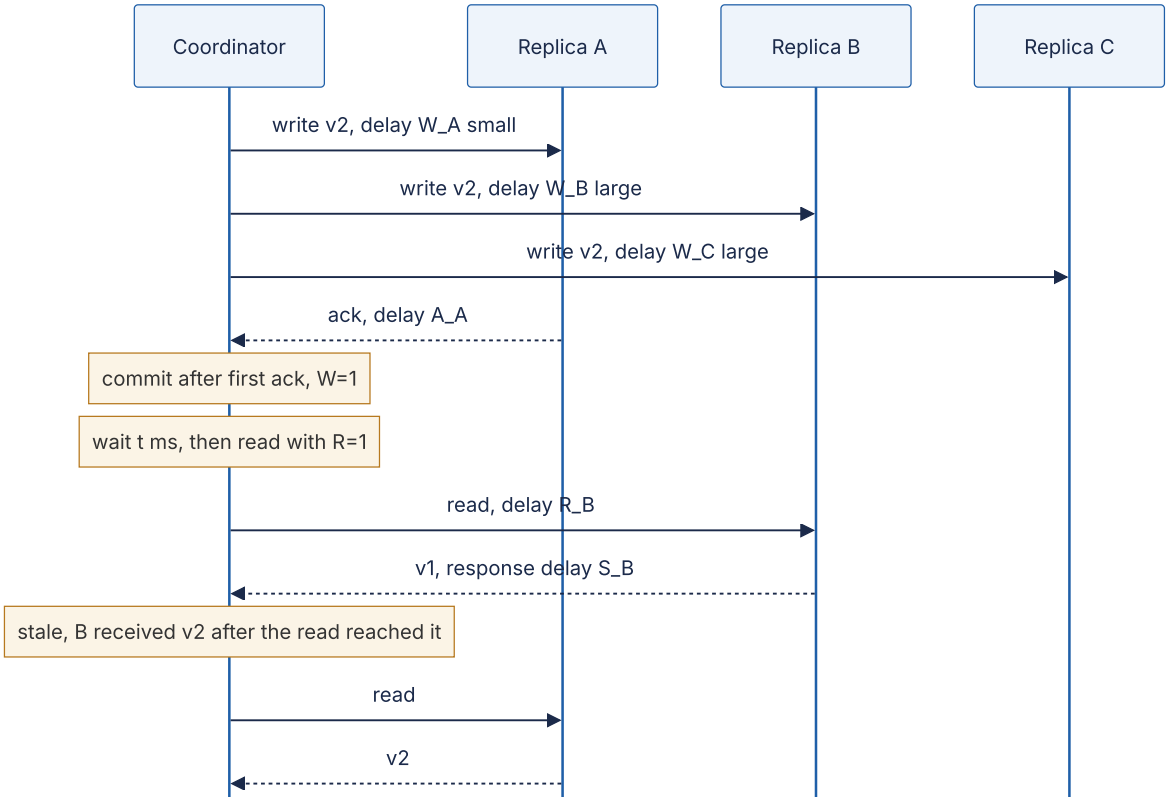


Figure 25.10 — The consistency hierarchy: transactional isolation (left) and single-object consistency (right).

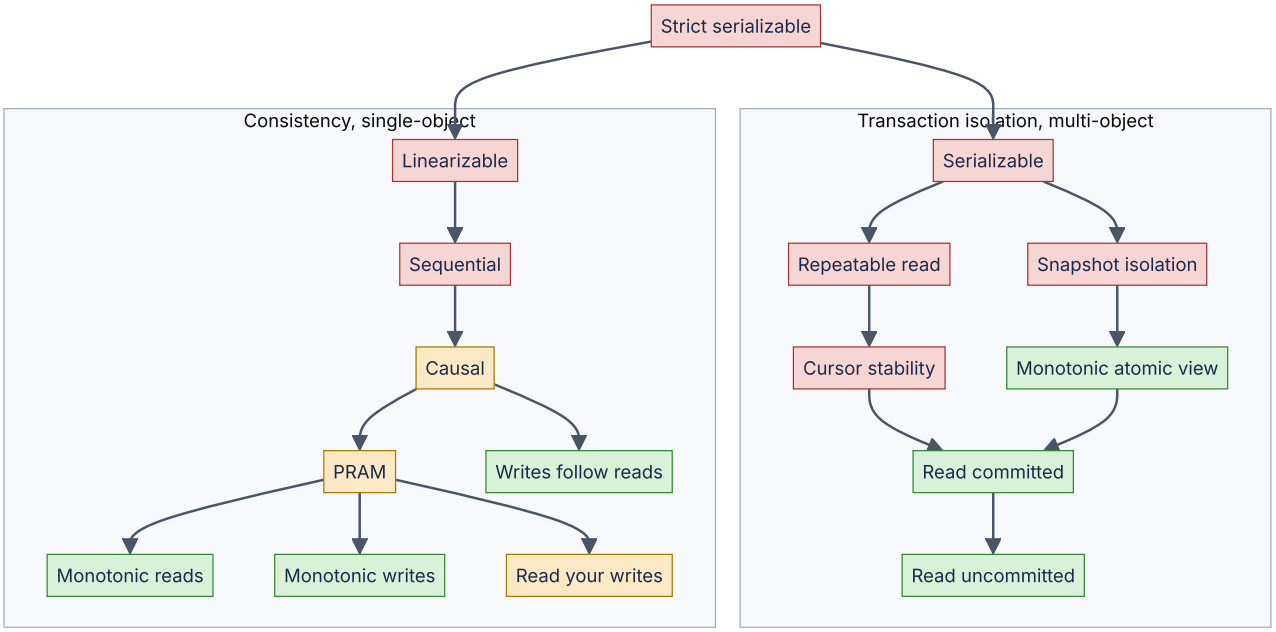


Figure 25.11 — The CAP proof scenario.

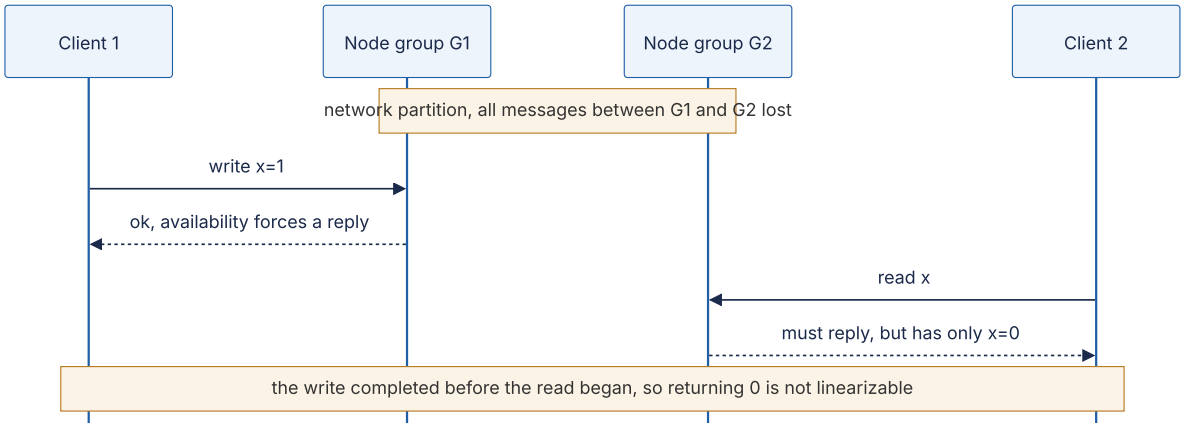


Figure 25.12 — The PACELC decision for one operation.

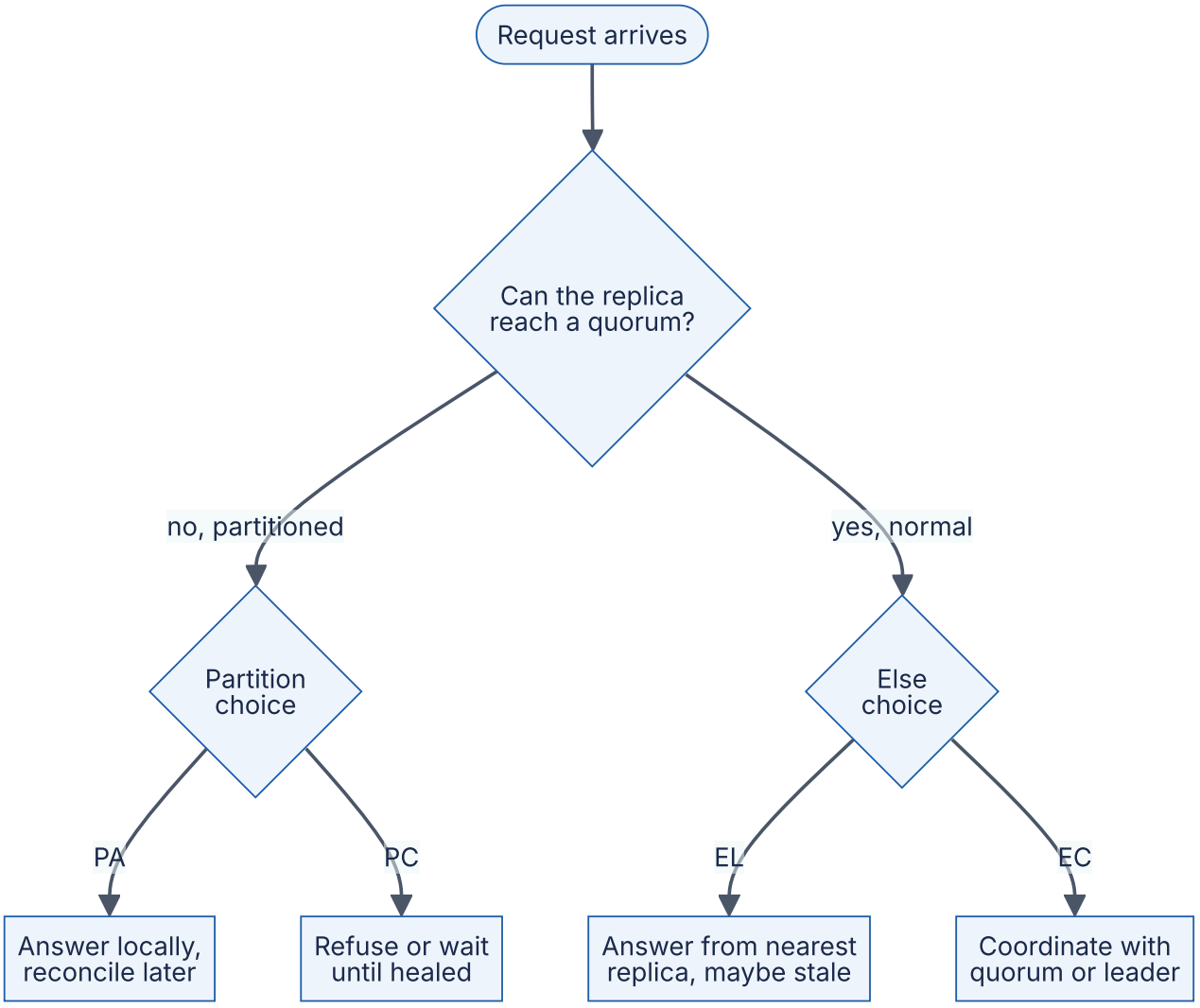
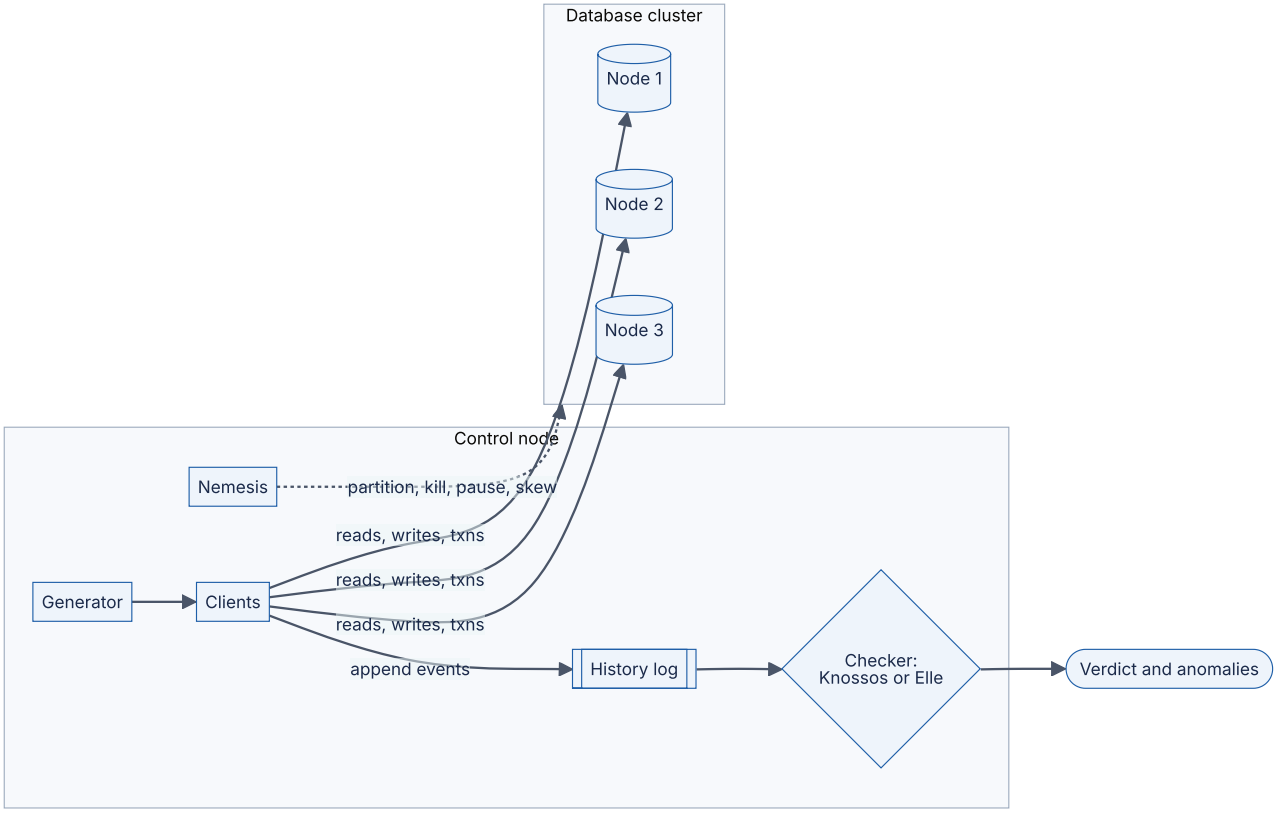


Figure 25.13 — Architecture of a Jepsen test.



Chapter 26 — Consensus and Coordination

Figure 26.1 — A replicated state machine built on a consensus log.

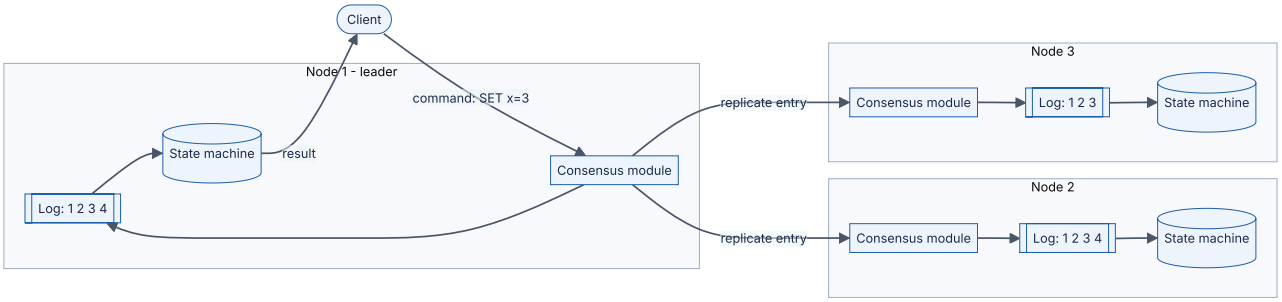


Figure 26.2 — Two majority quorums in a five-node cluster always intersect.

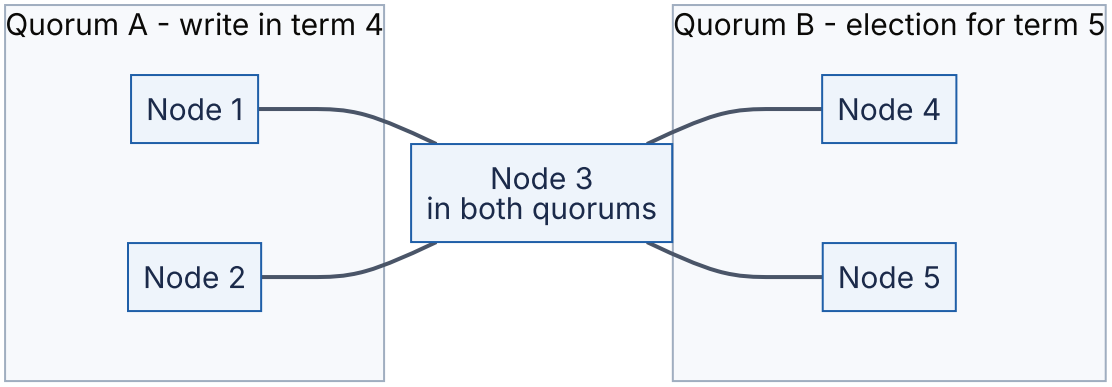


Figure 26.3 — Lease-based leader election through a coordination service.

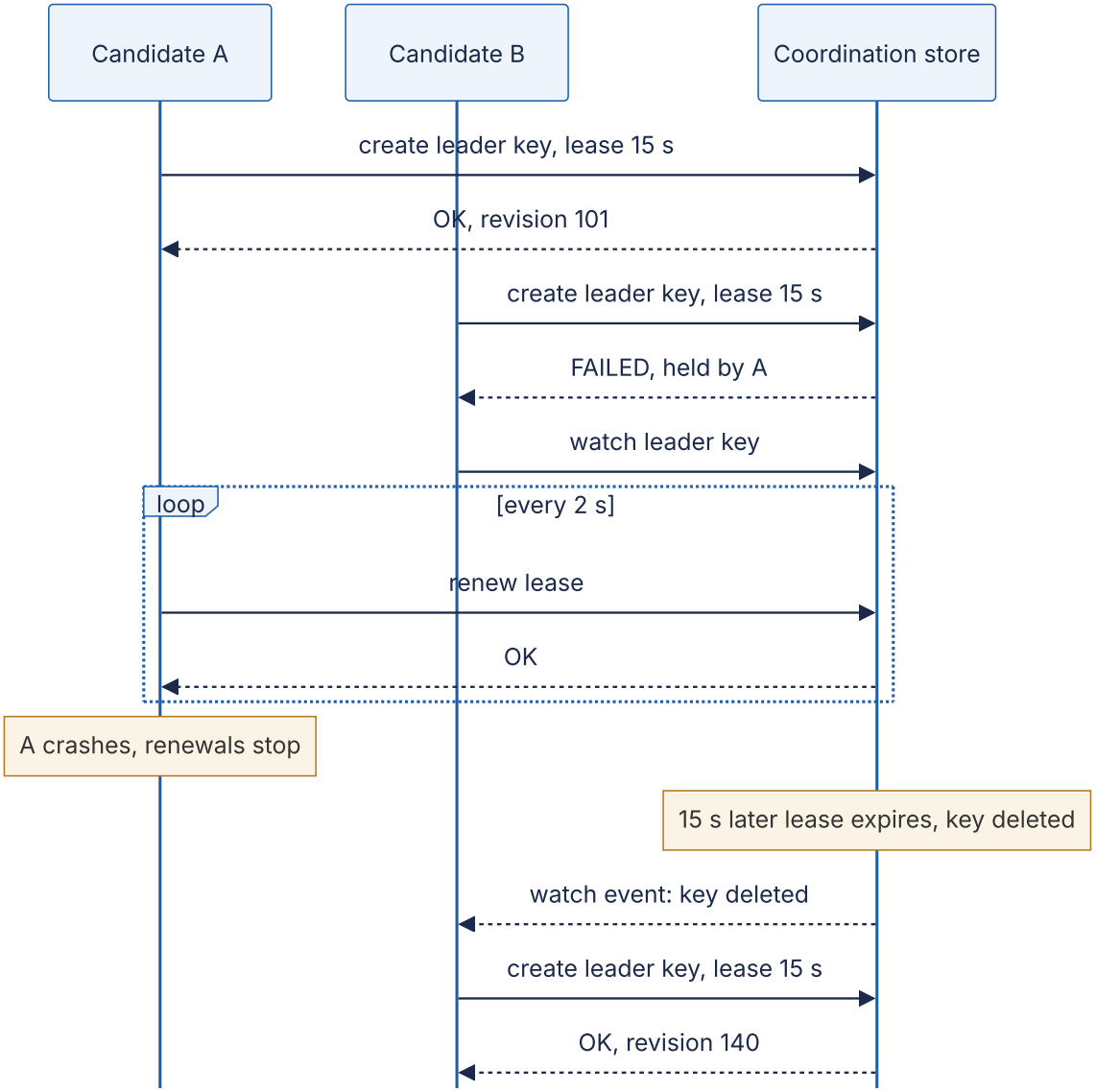


Figure 26.4 — Single-decree Paxos without contention.

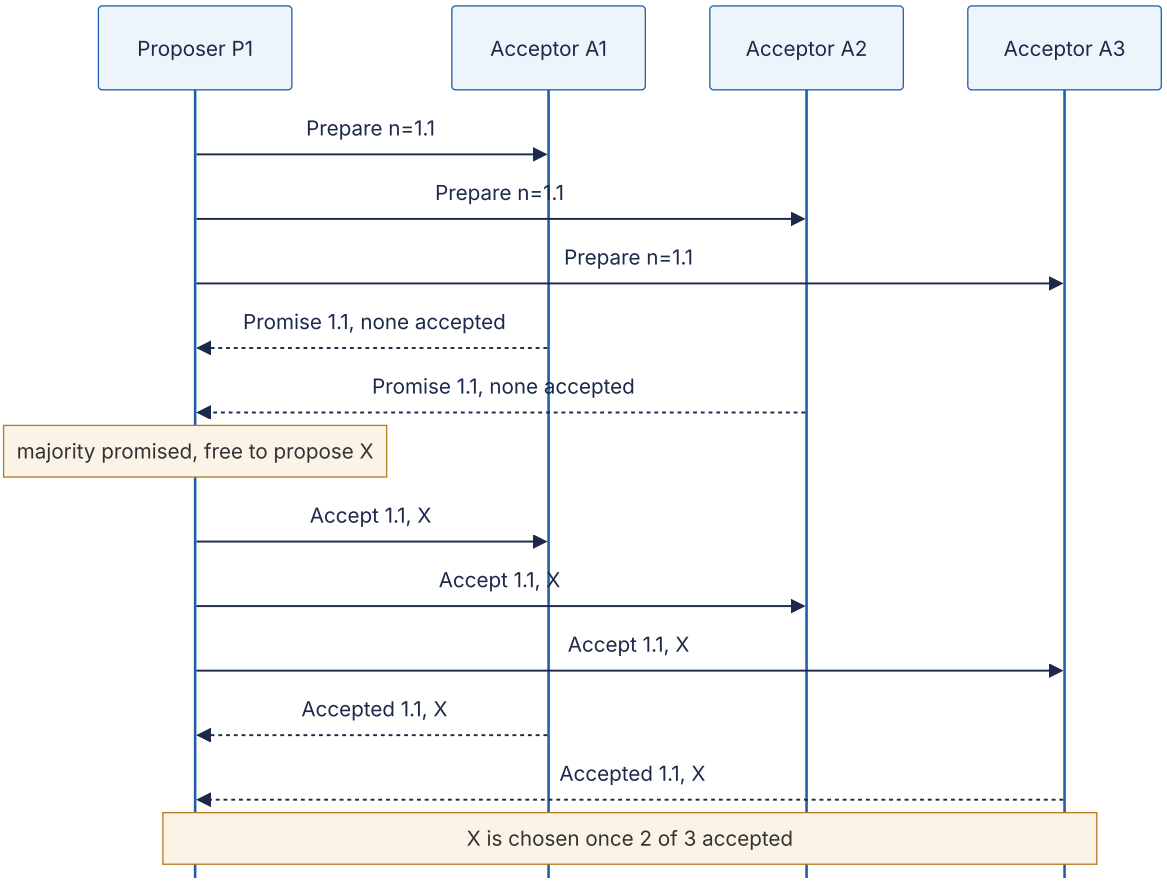


Figure 26.5 — Dueling proposers: safe but not live.

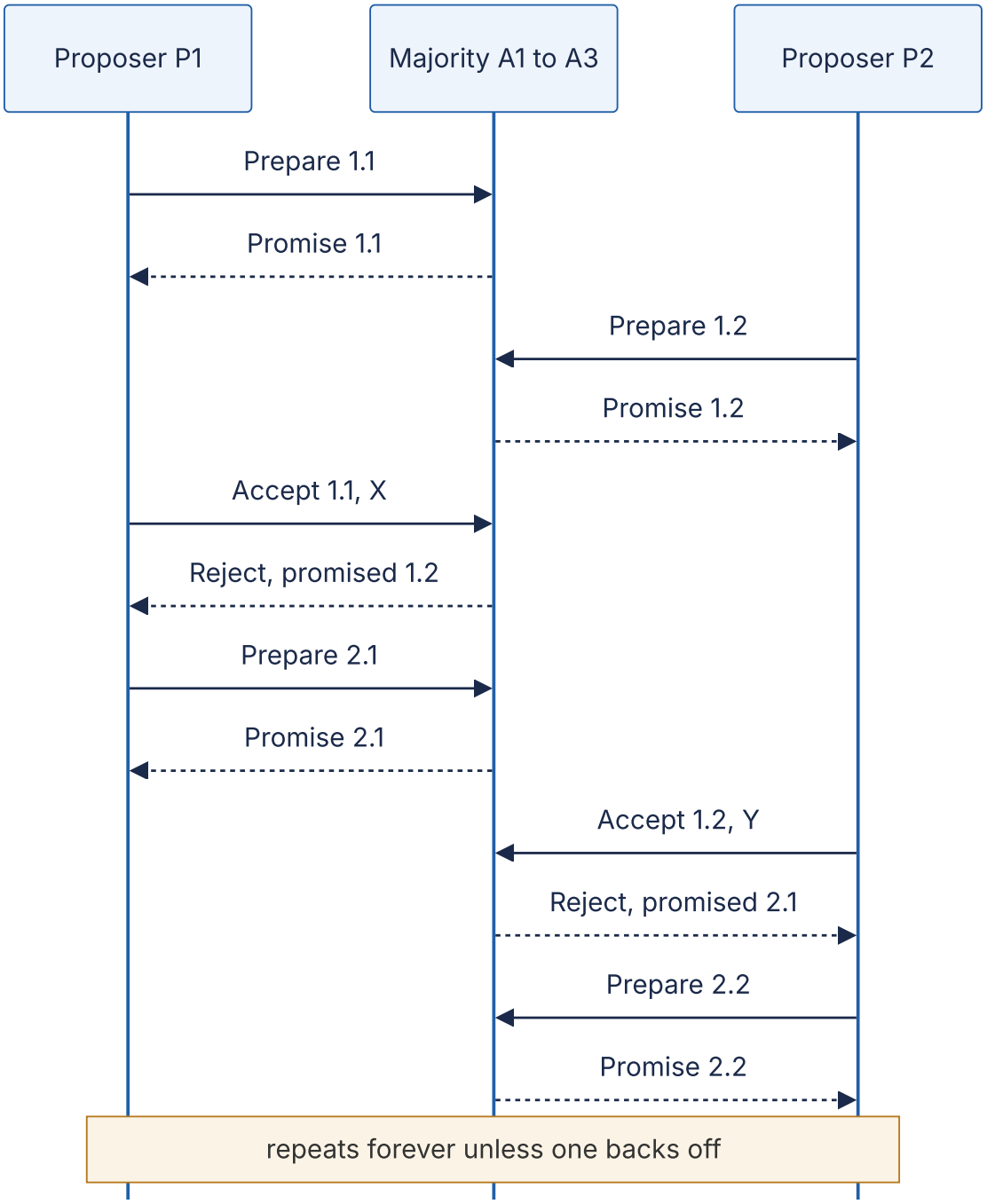


Figure 26.6 — Multi-Paxos: phase 1 once, then one round trip per command.

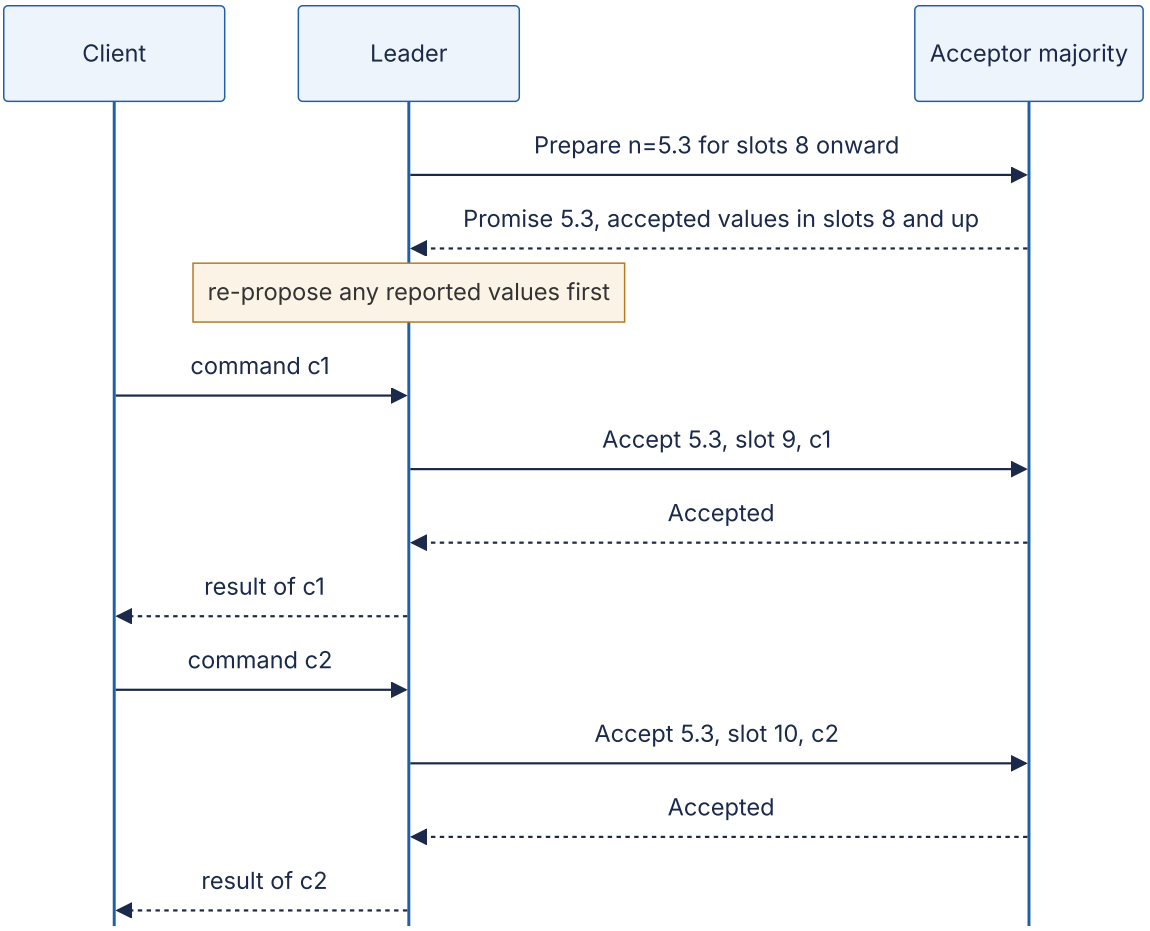


Figure 26.7 — Raft role transitions.

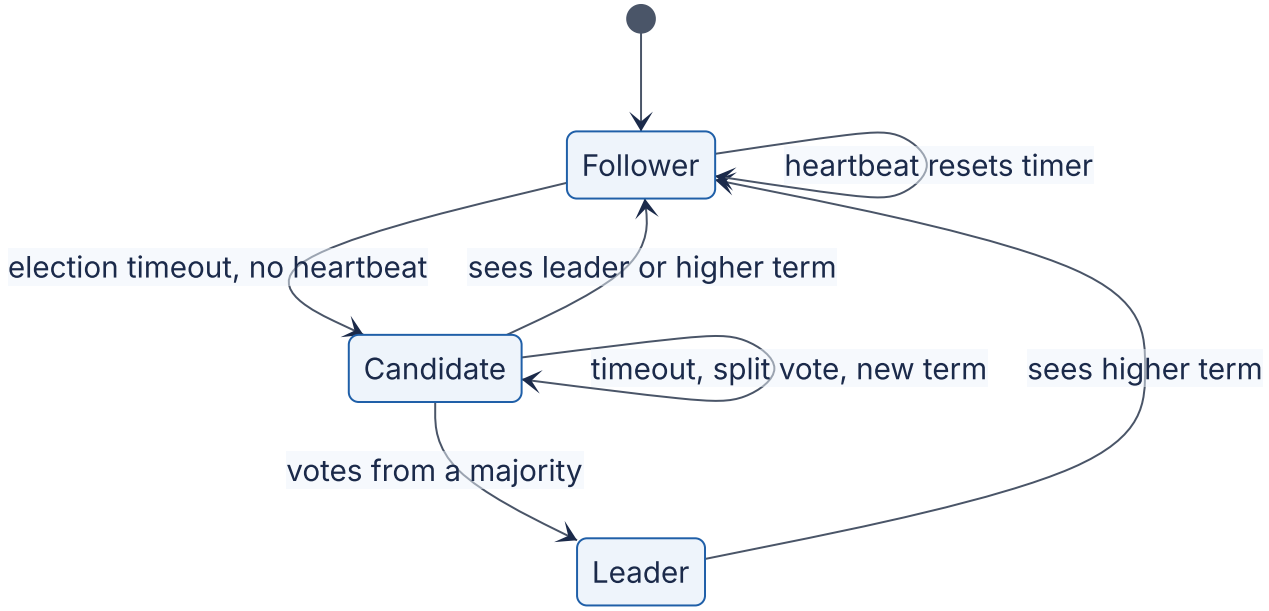


Figure 26.8 — Replicating and committing one entry.

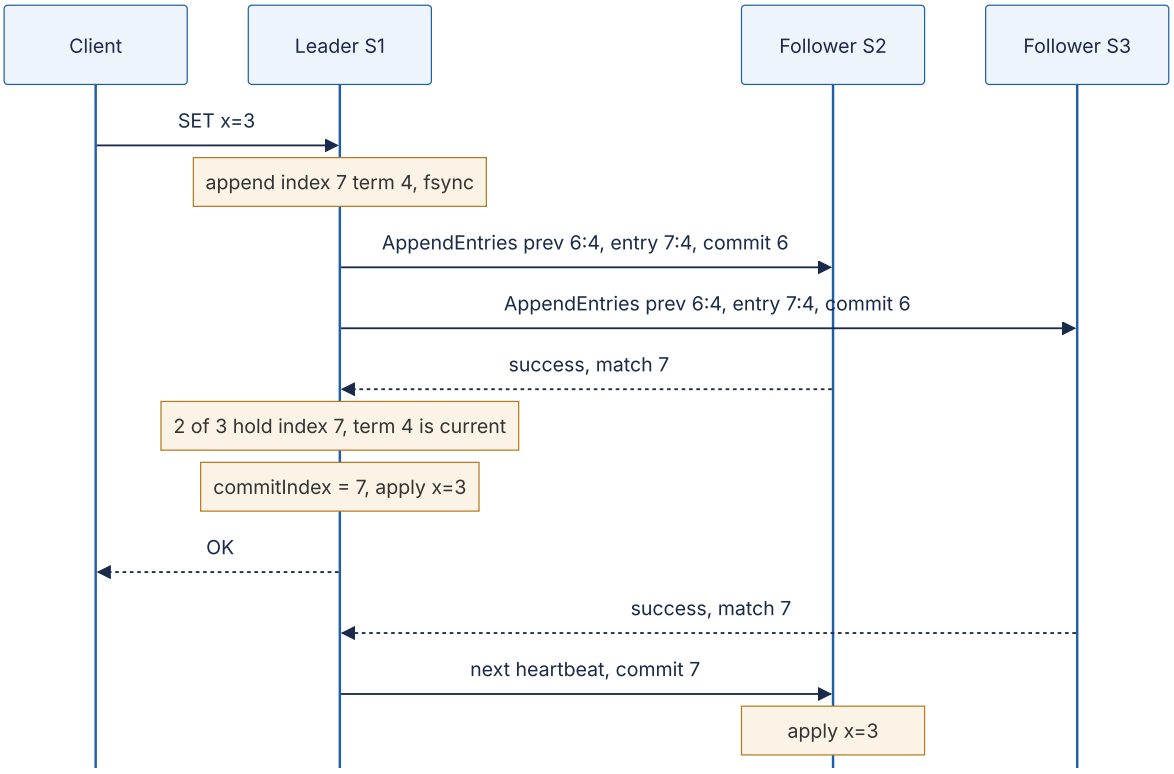


Figure 26.9 — Repairing a divergent follower log.

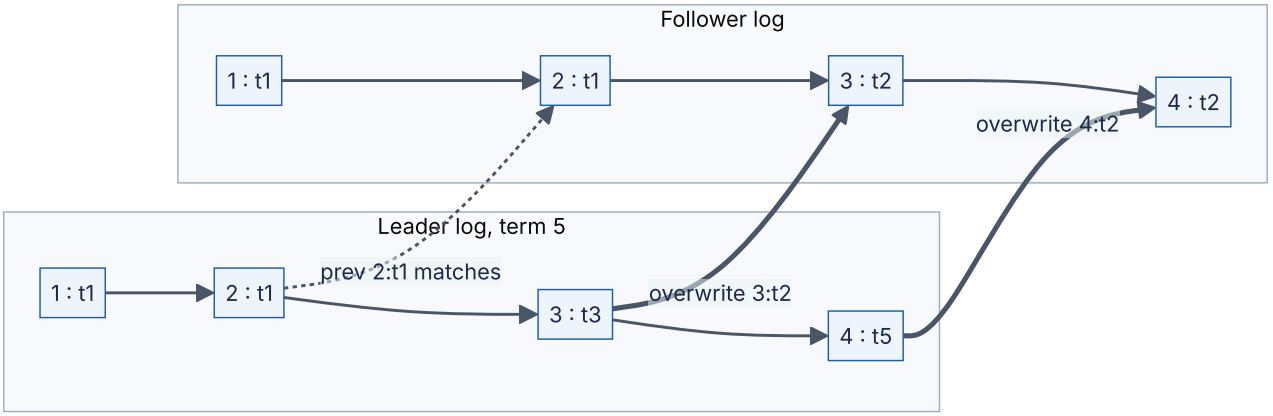


Figure 26.10 — Why a leader must not commit old-term entries by counting replicas.

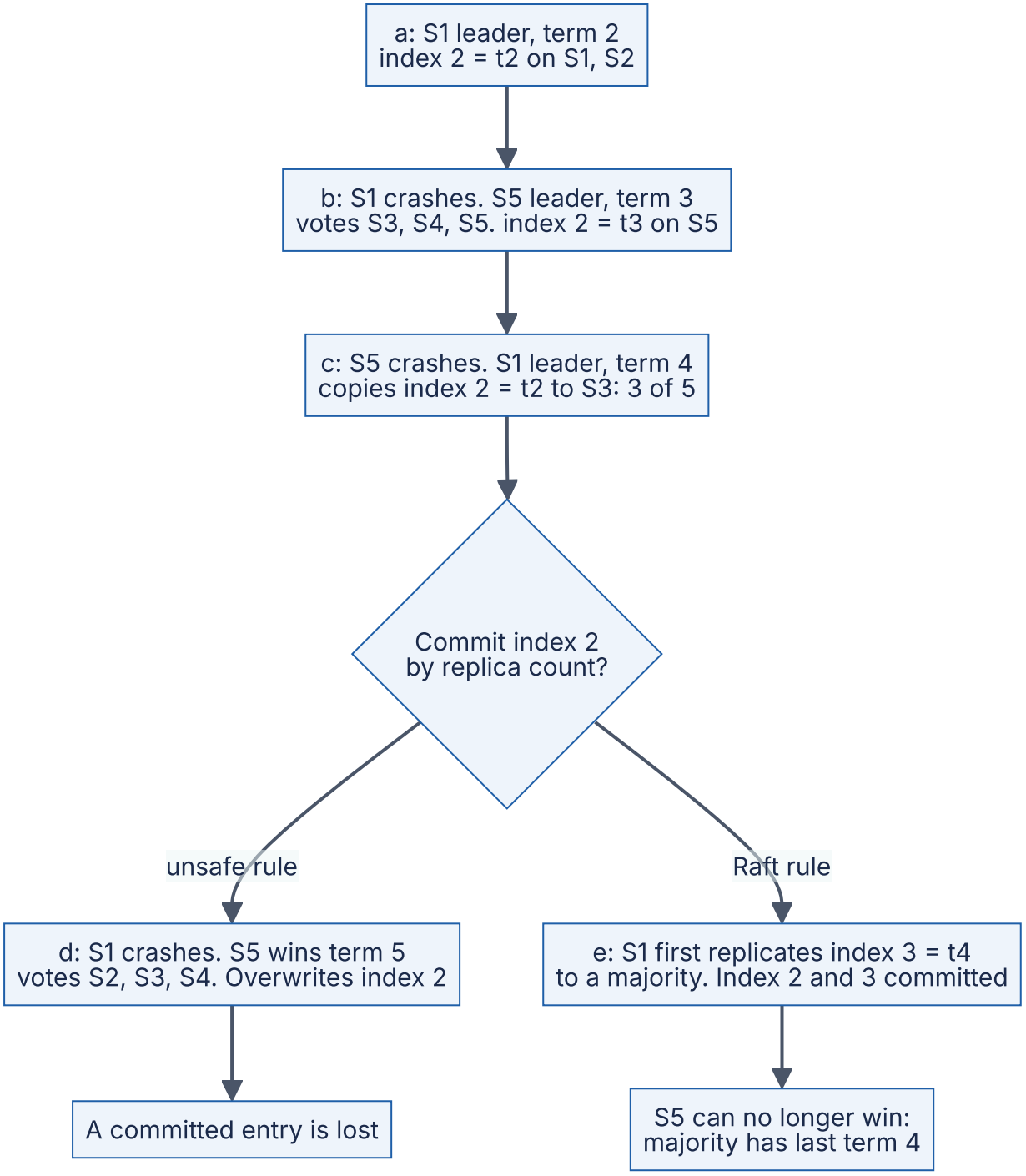


Figure 26.11 — ReadIndex: linearizable reads without writing to the log.

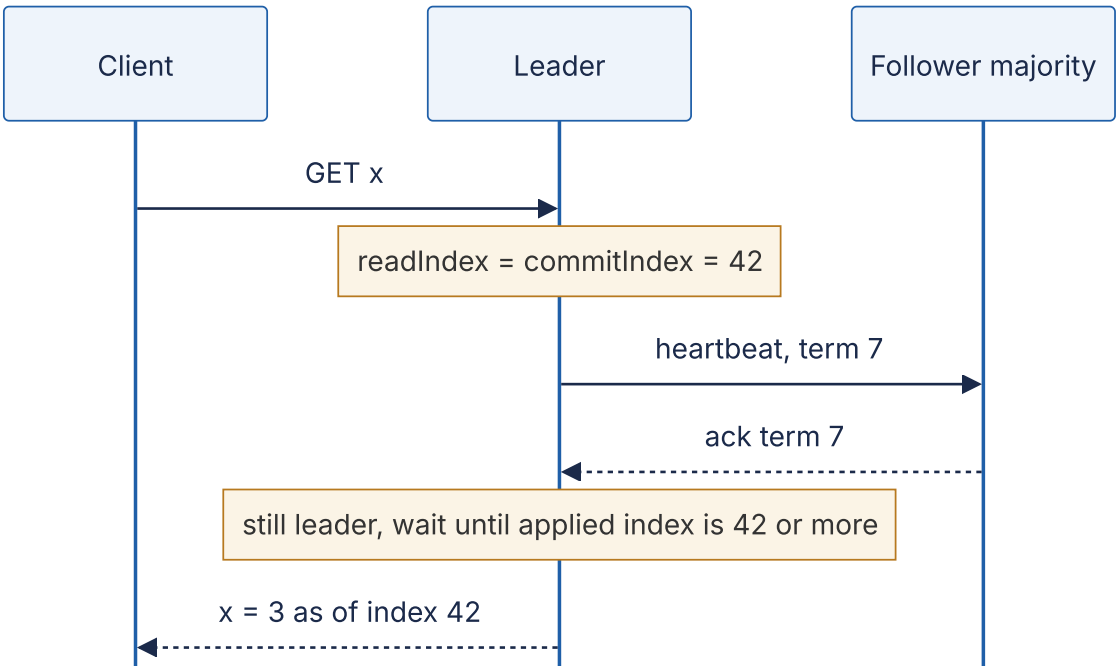


Figure 26.12 — The ZooKeeper lock recipe with sequential ephemeral znodes.

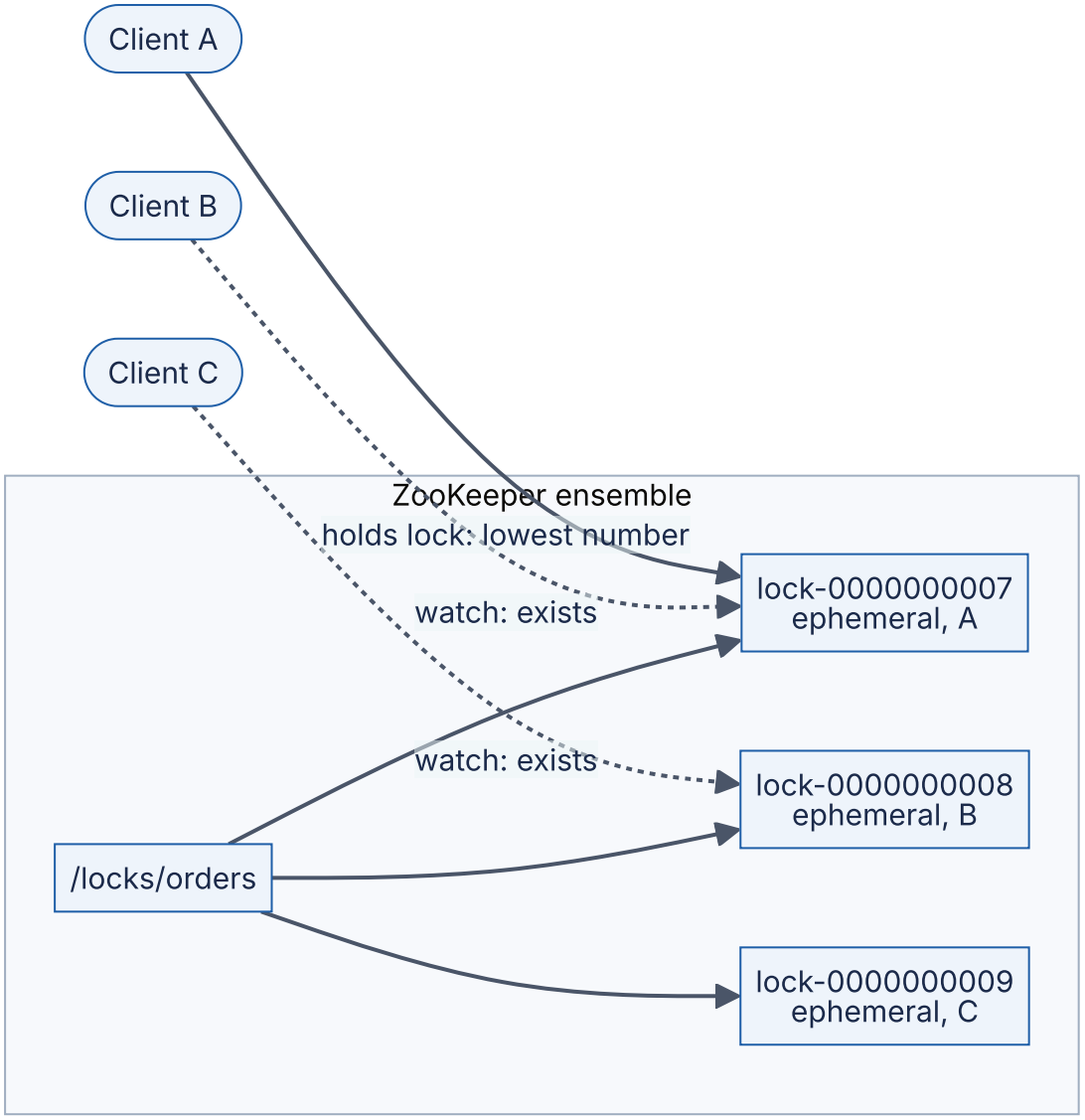


Figure 26.13 — Without fencing: a paused client corrupts storage.

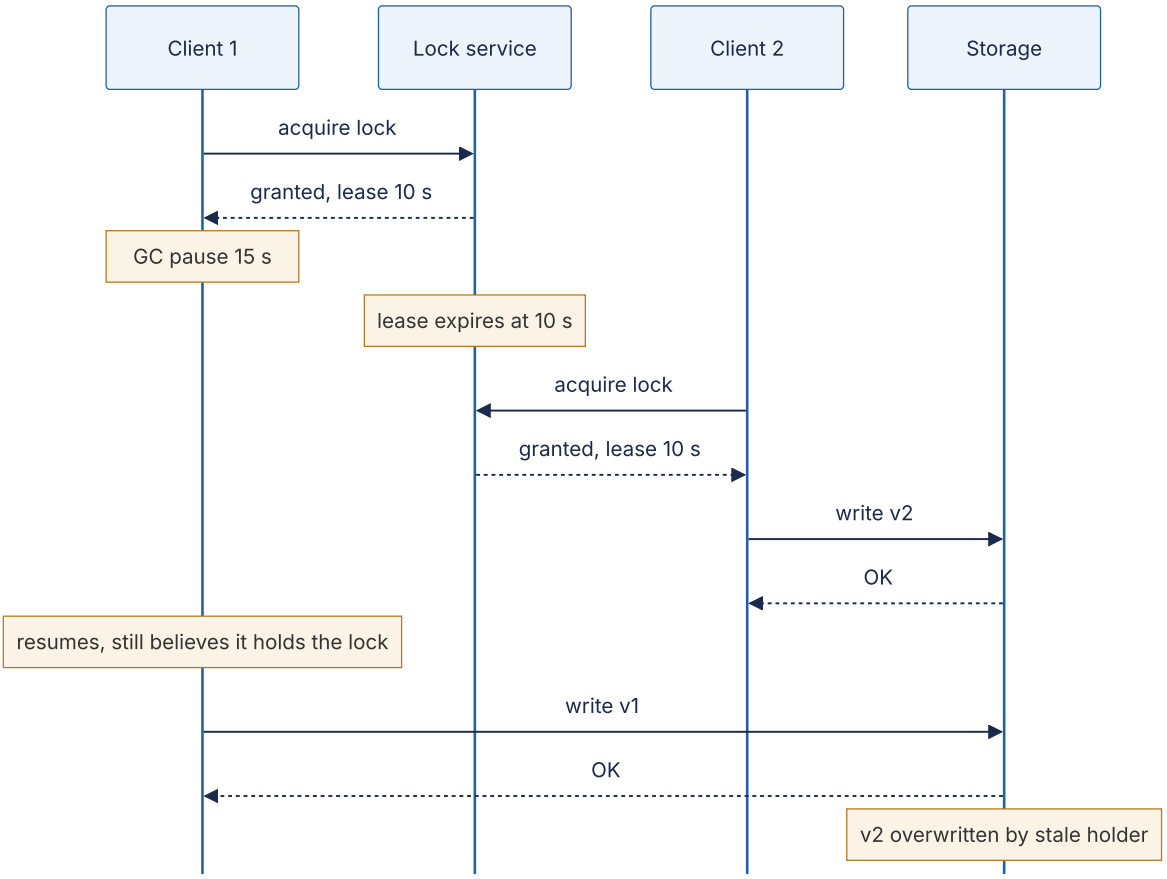


Figure 26.14 — With fencing: the storage service rejects the stale holder.

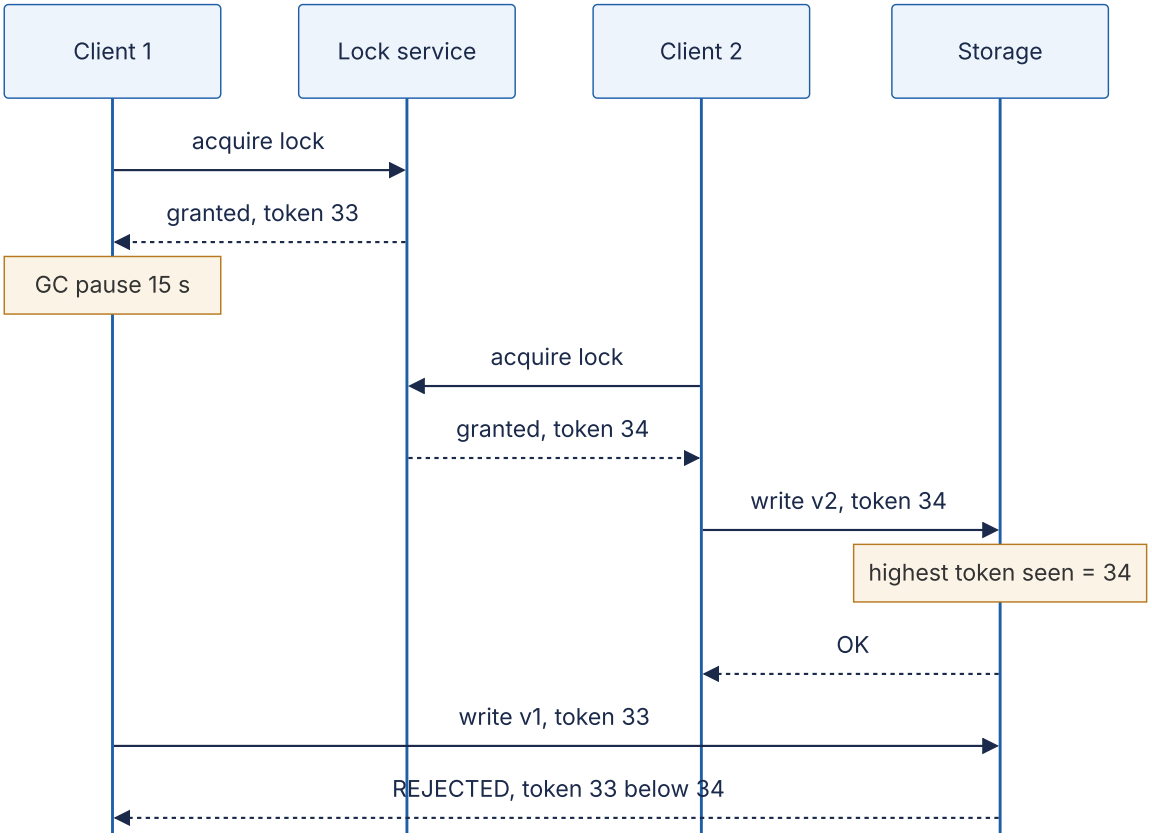
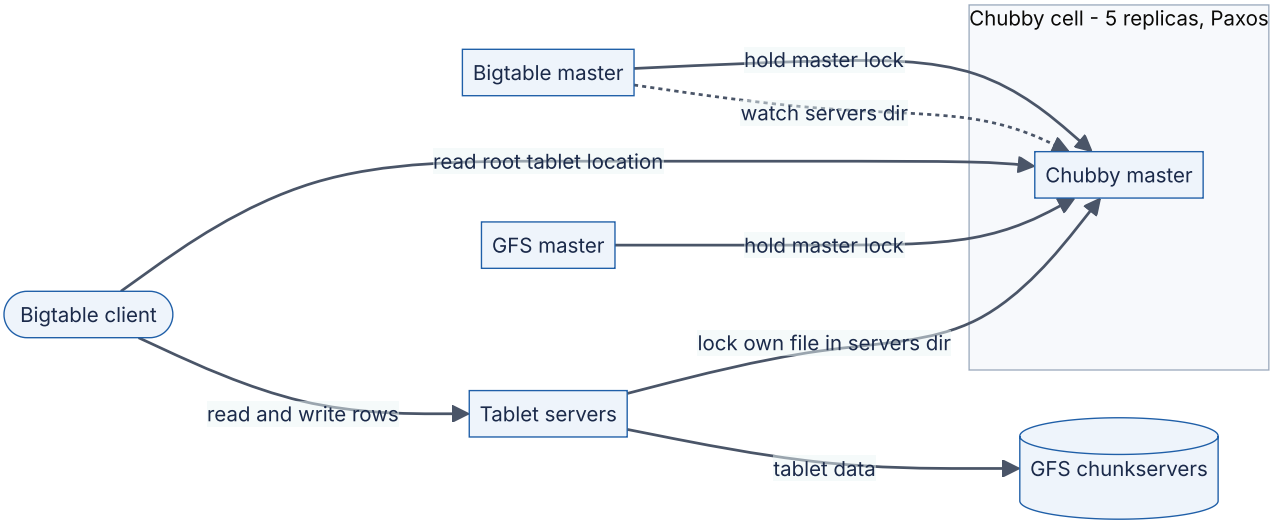


Figure 26.15 — How Bigtable and GFS use Chubby.



Chapter 27 — Replication and Fault Tolerance

Figure 27.1 — Choosing a replication strategy.

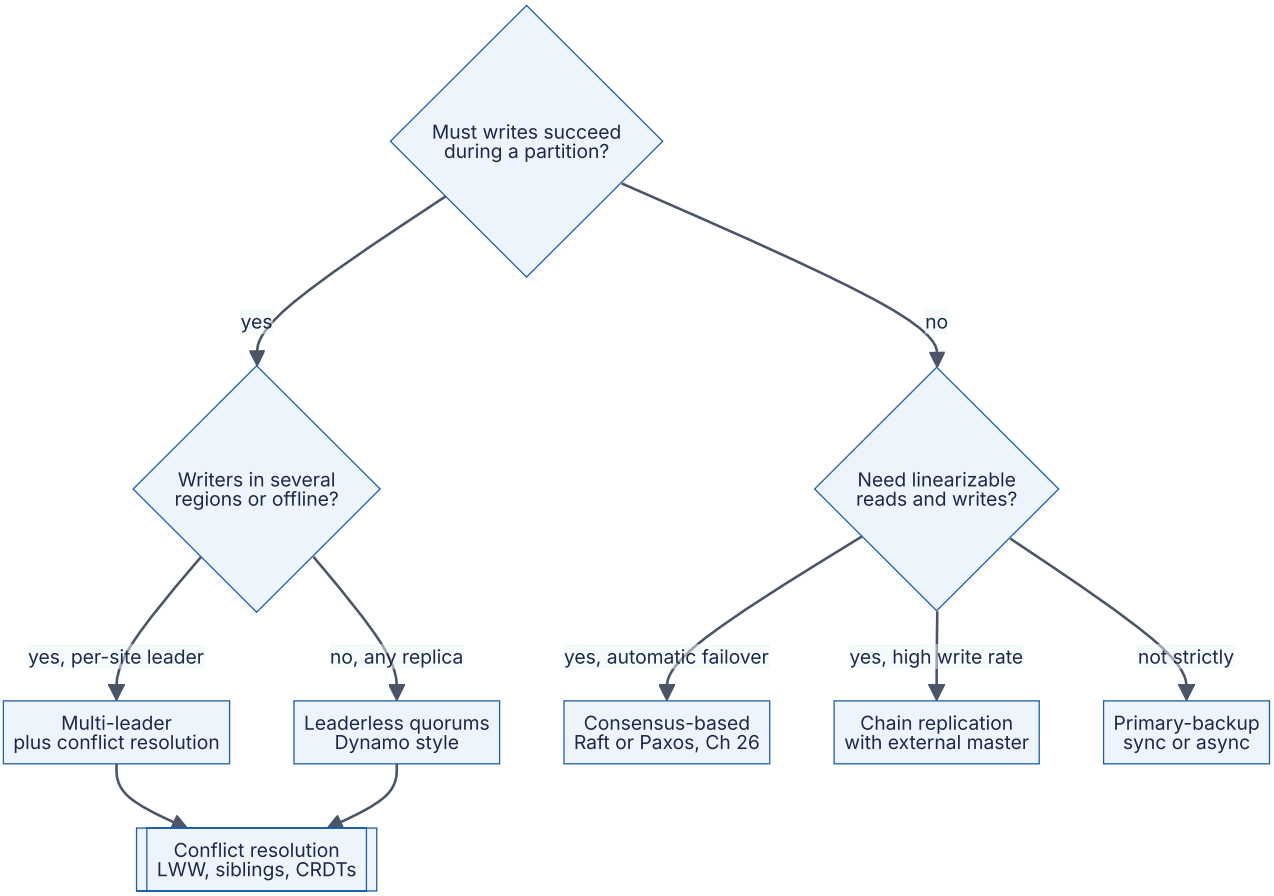


Figure 27.2 — Primary-backup with a synchronous backup and the failover window.

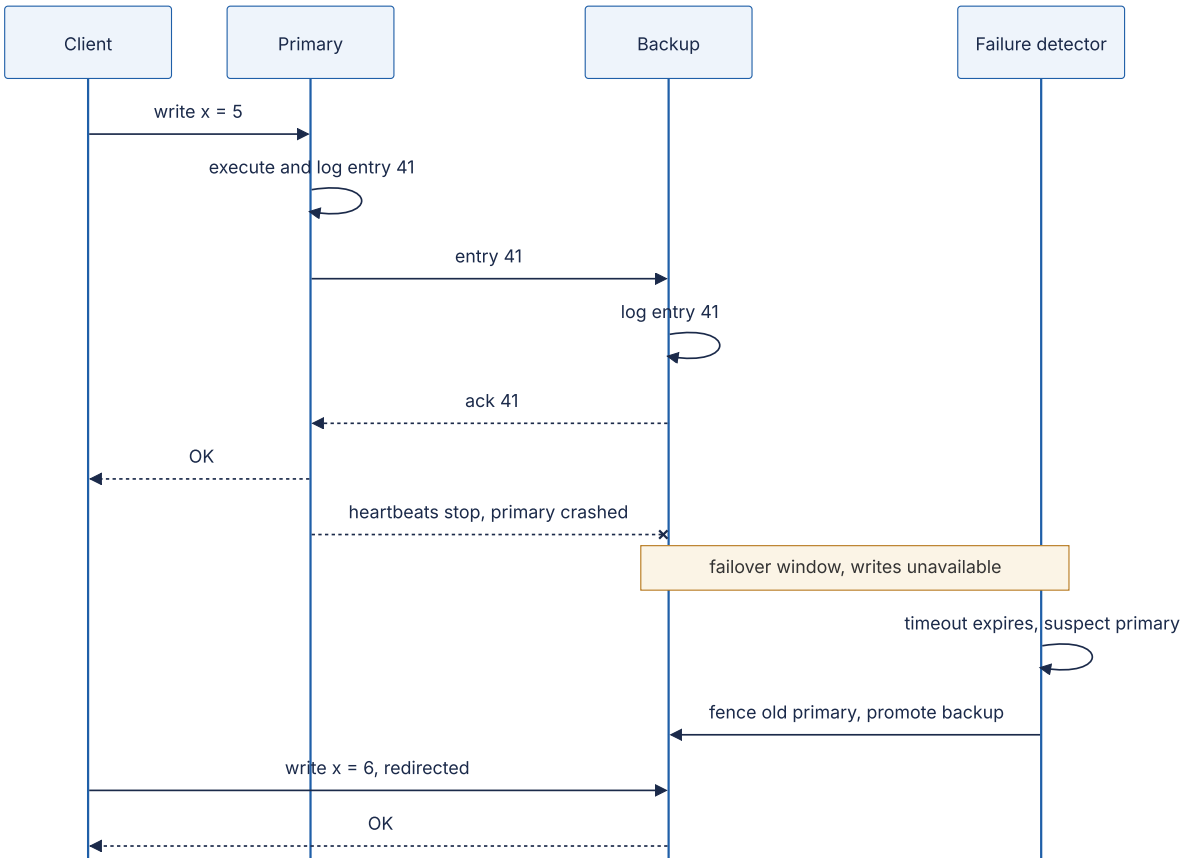


Figure 27.3 — Chain replication with an external master.

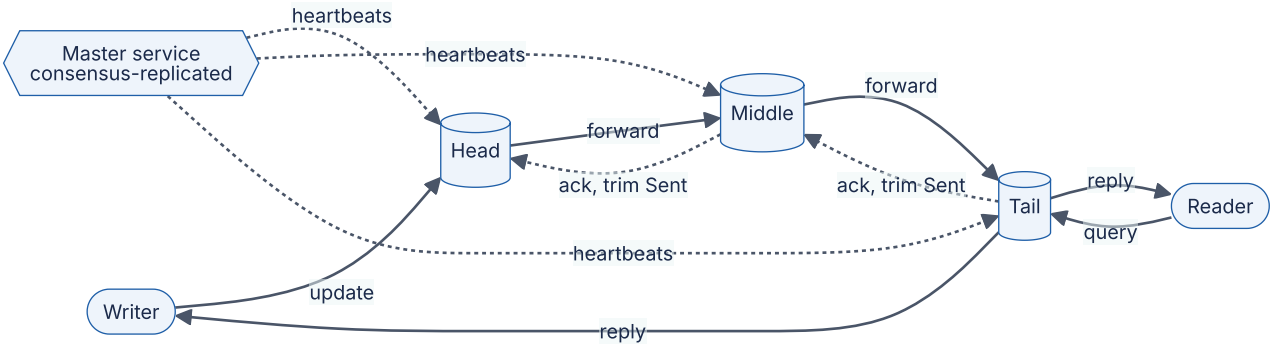


Figure 27.4 — A CRAQ read that hits a dirty object.

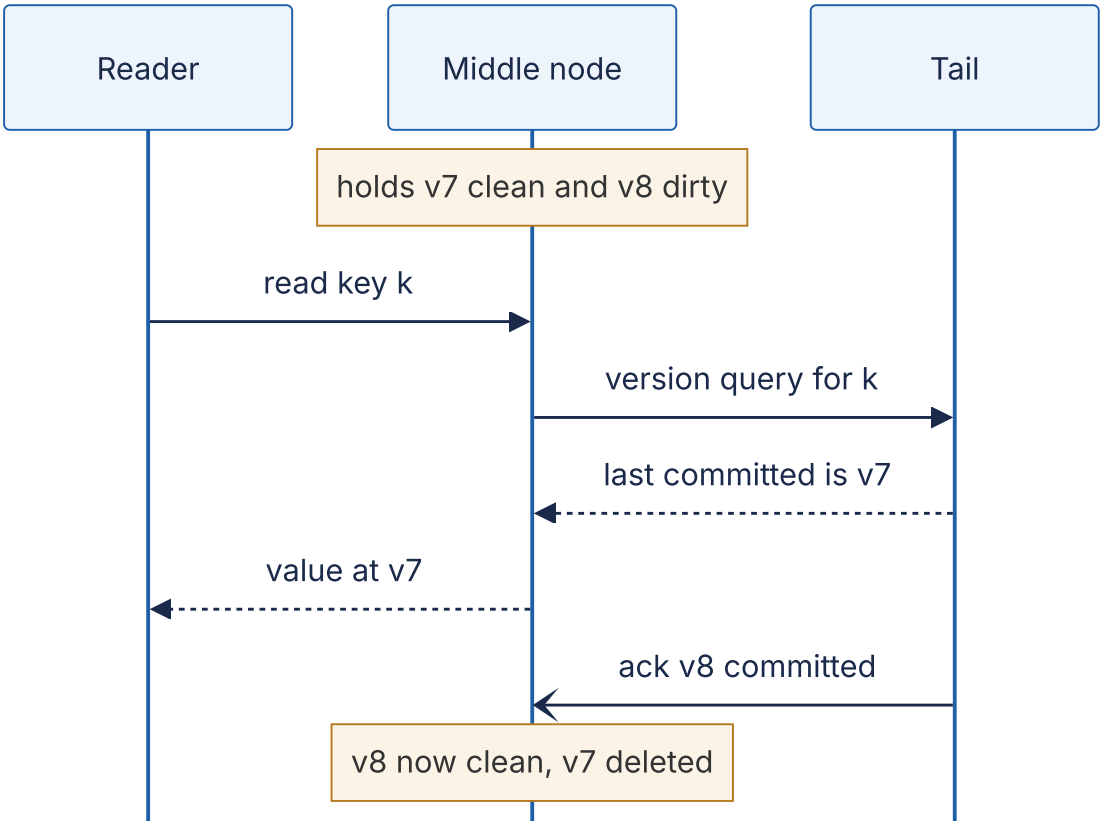
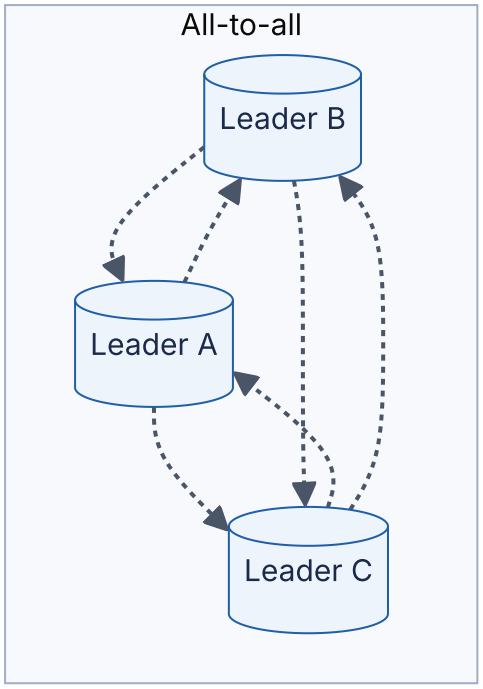
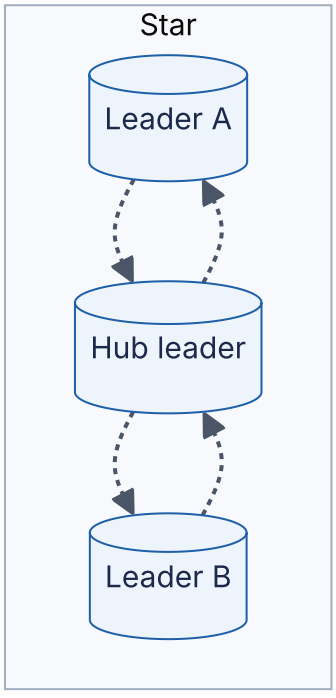
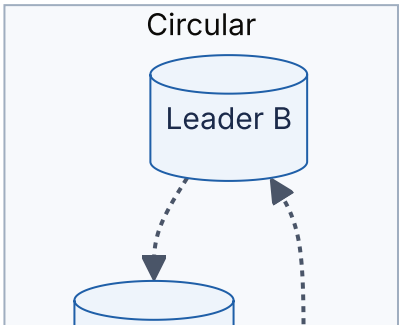


Figure 27.5 — Multi-leader topologies.


All-to-all
Leader B
Leader A
Leader C


Star
Leader A
Hub leader
Leader B


Circular
Leader B

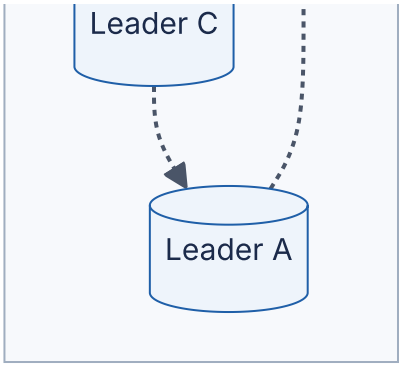


Figure 27.6 — A Dynamo-style write with N = 3 and W = 2.

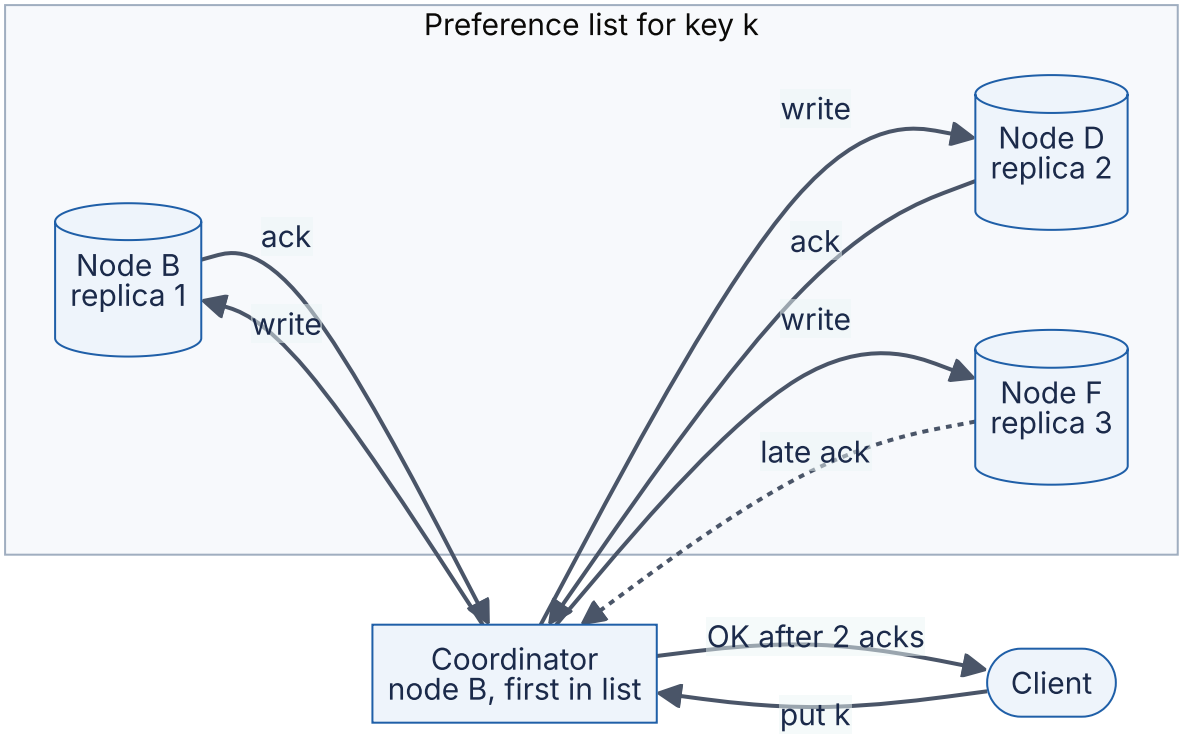


Figure 27.7 — R = W = 2, N = 3, and a read that goes back in time.

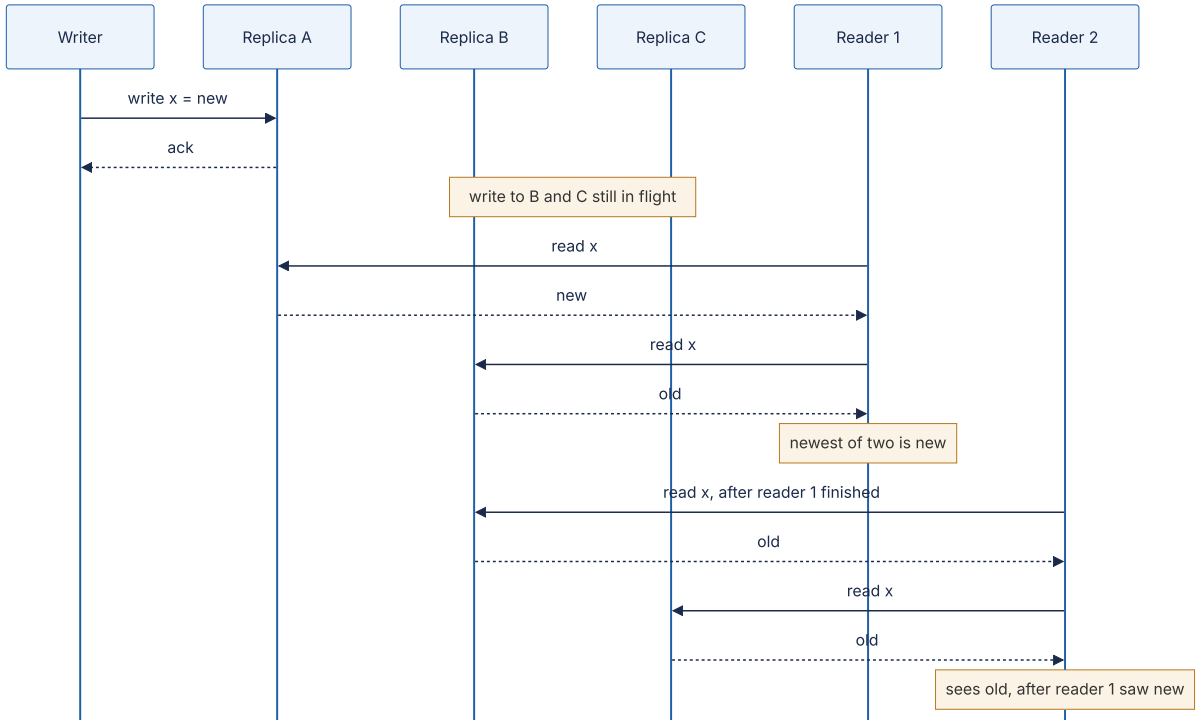


Figure 27.8 — A sloppy quorum write and later handoff.

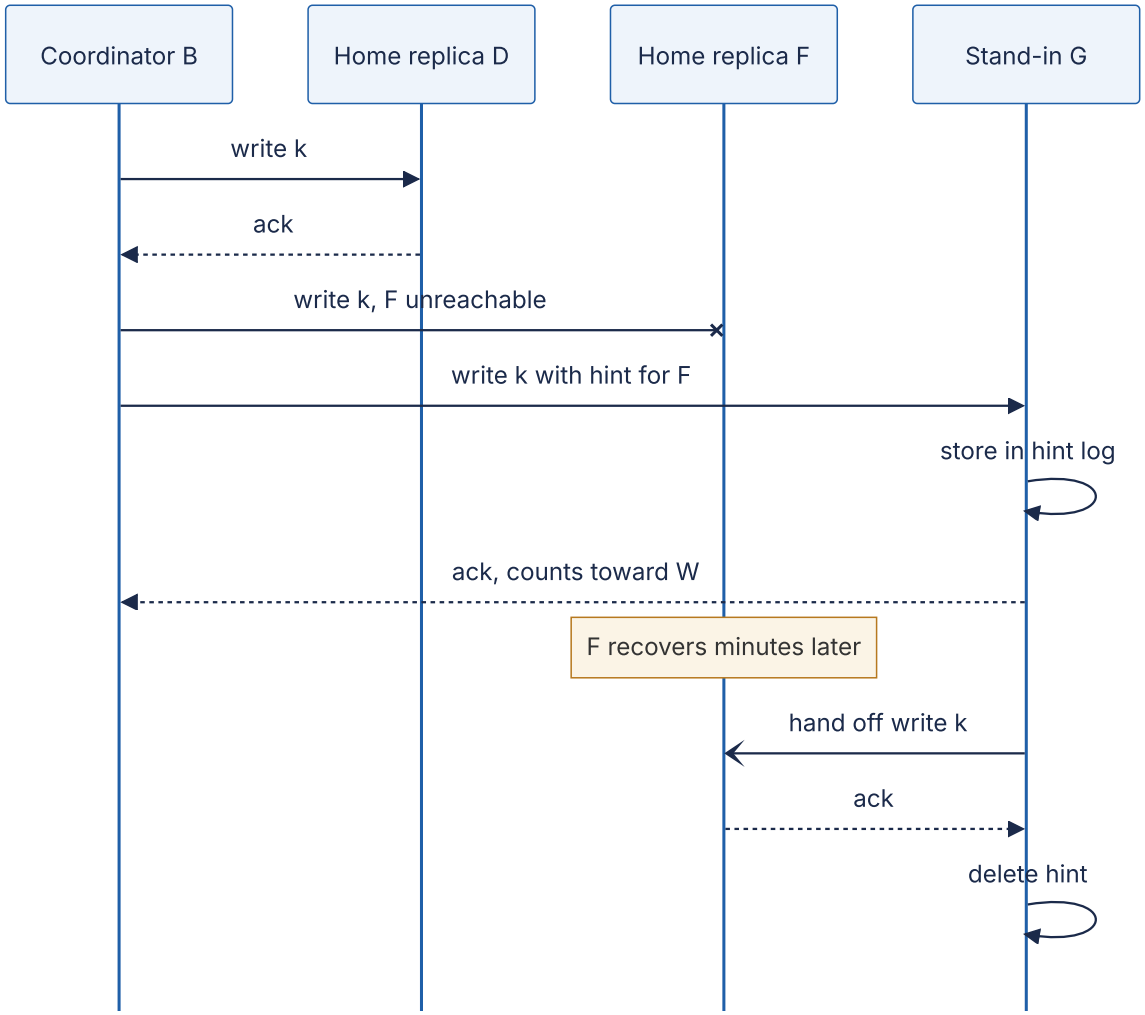


Figure 27.9 — A SWIM protocol period with an indirect probe.

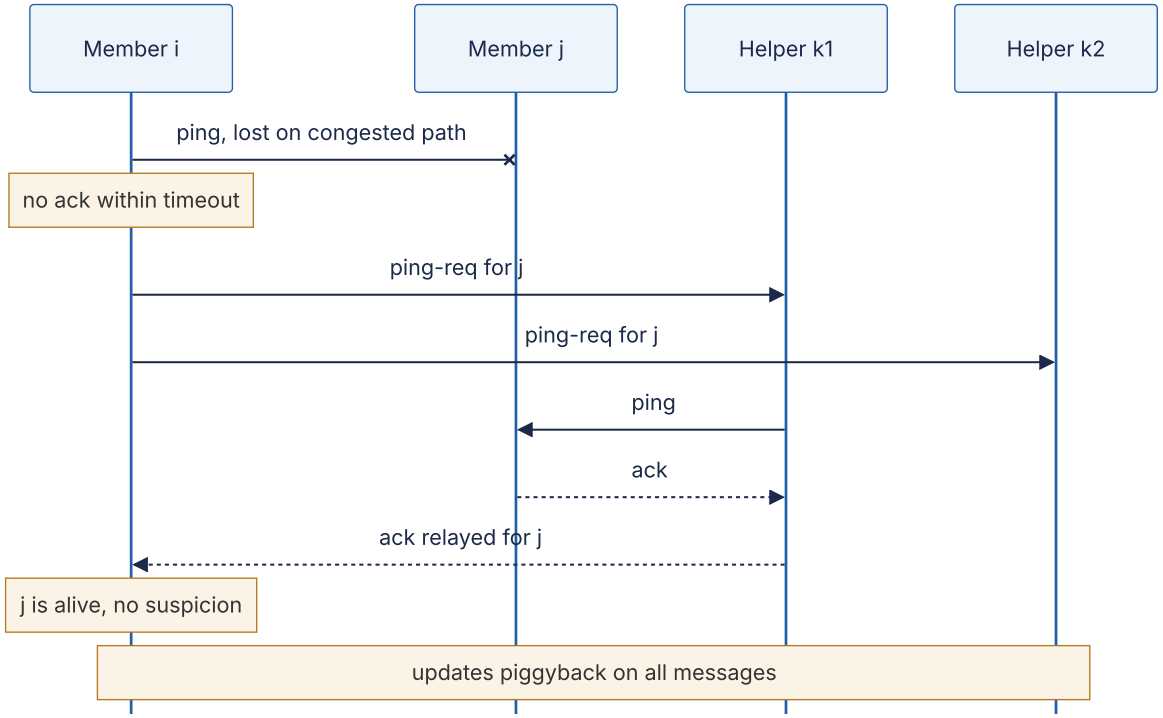


Figure 27.10 — A failover time budget with DNS redirection.

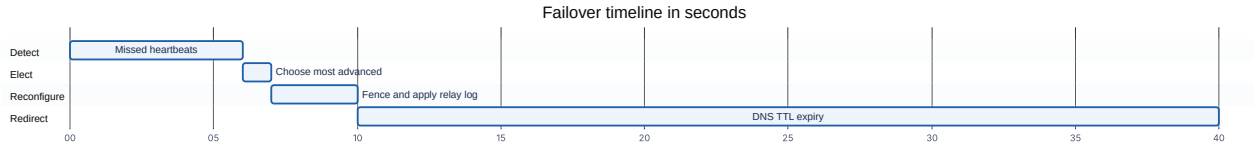


Figure 27.11 — Choosing a recovery path.

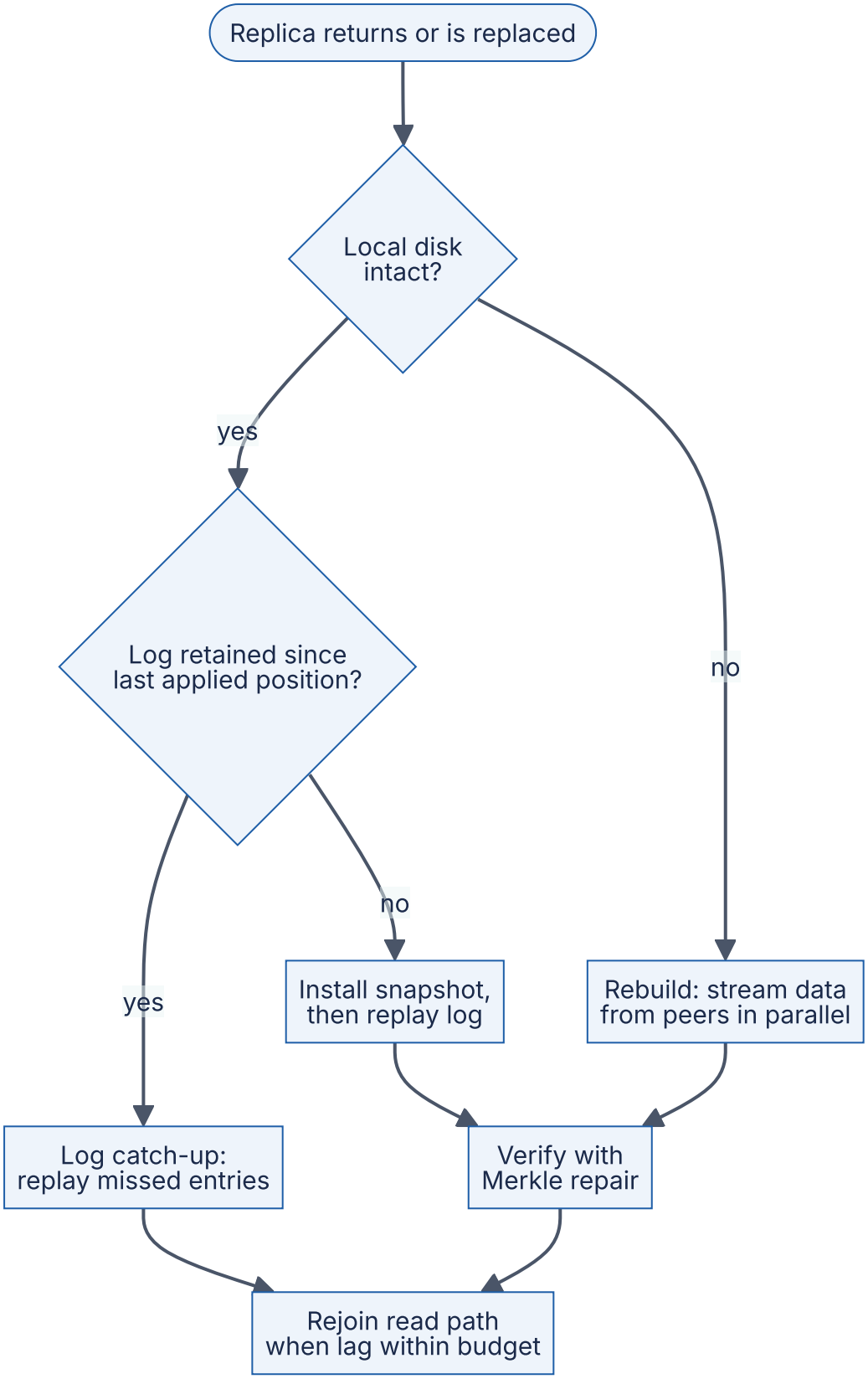


Figure 27.12 — Blocking read repair with R = 2.

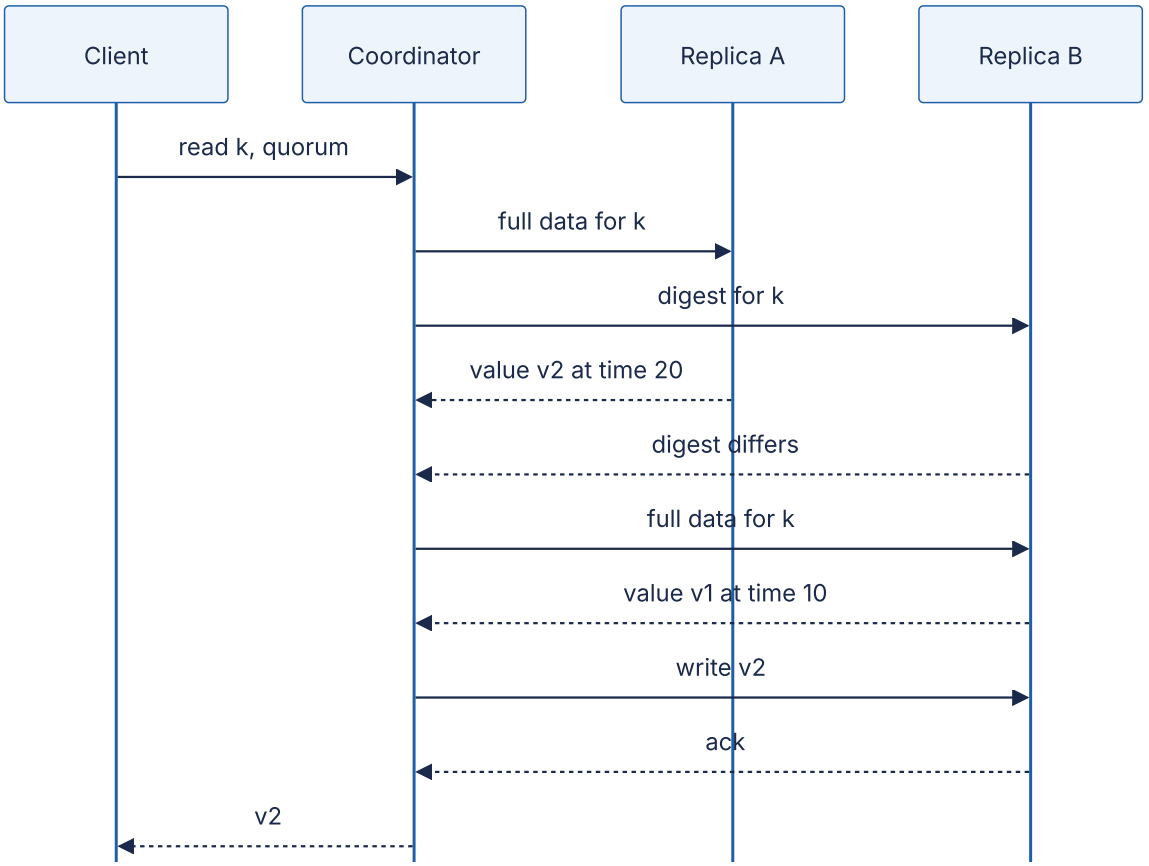
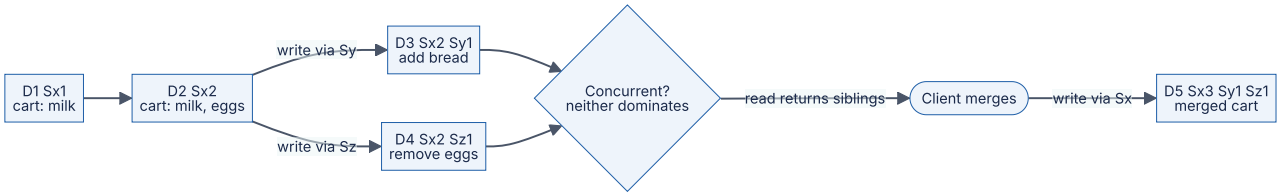


Figure 27.13 — Version vectors creating and resolving siblings.



Chapter 28 — Distributed Transactions

Figure 28.1 — A naive cross-shard transfer that loses money.

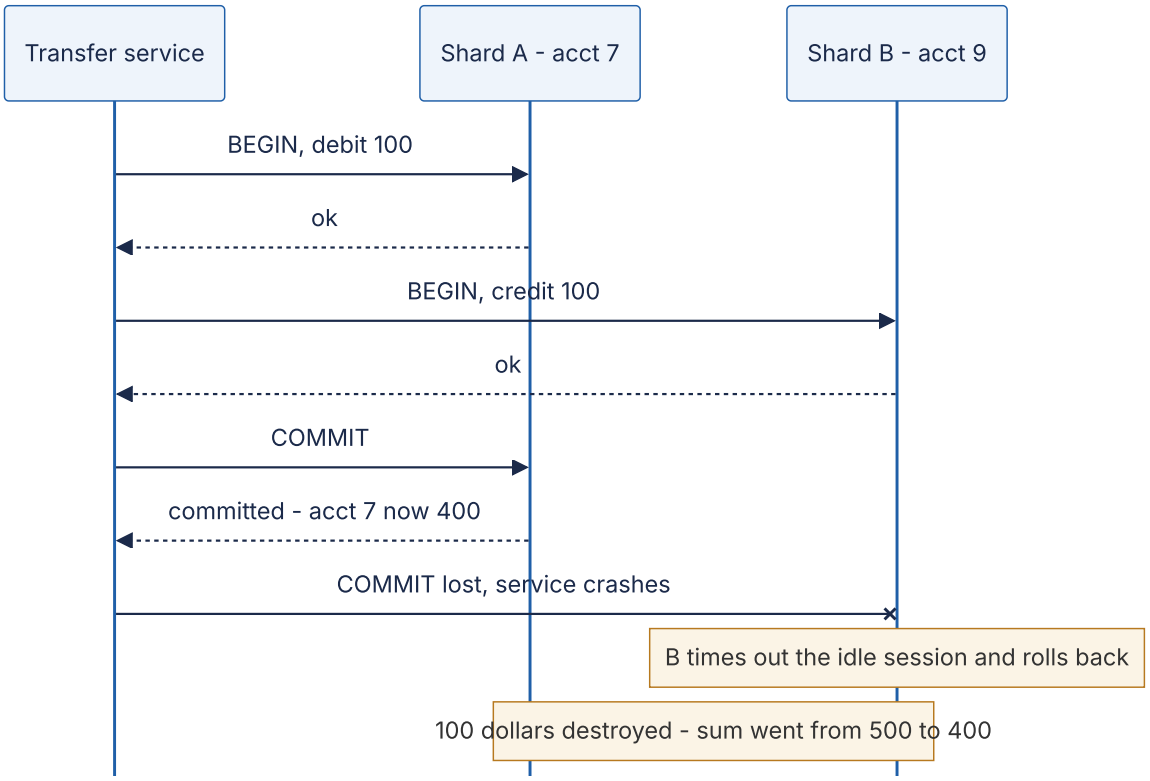


Figure 28.2 — Two-phase commit, happy path.

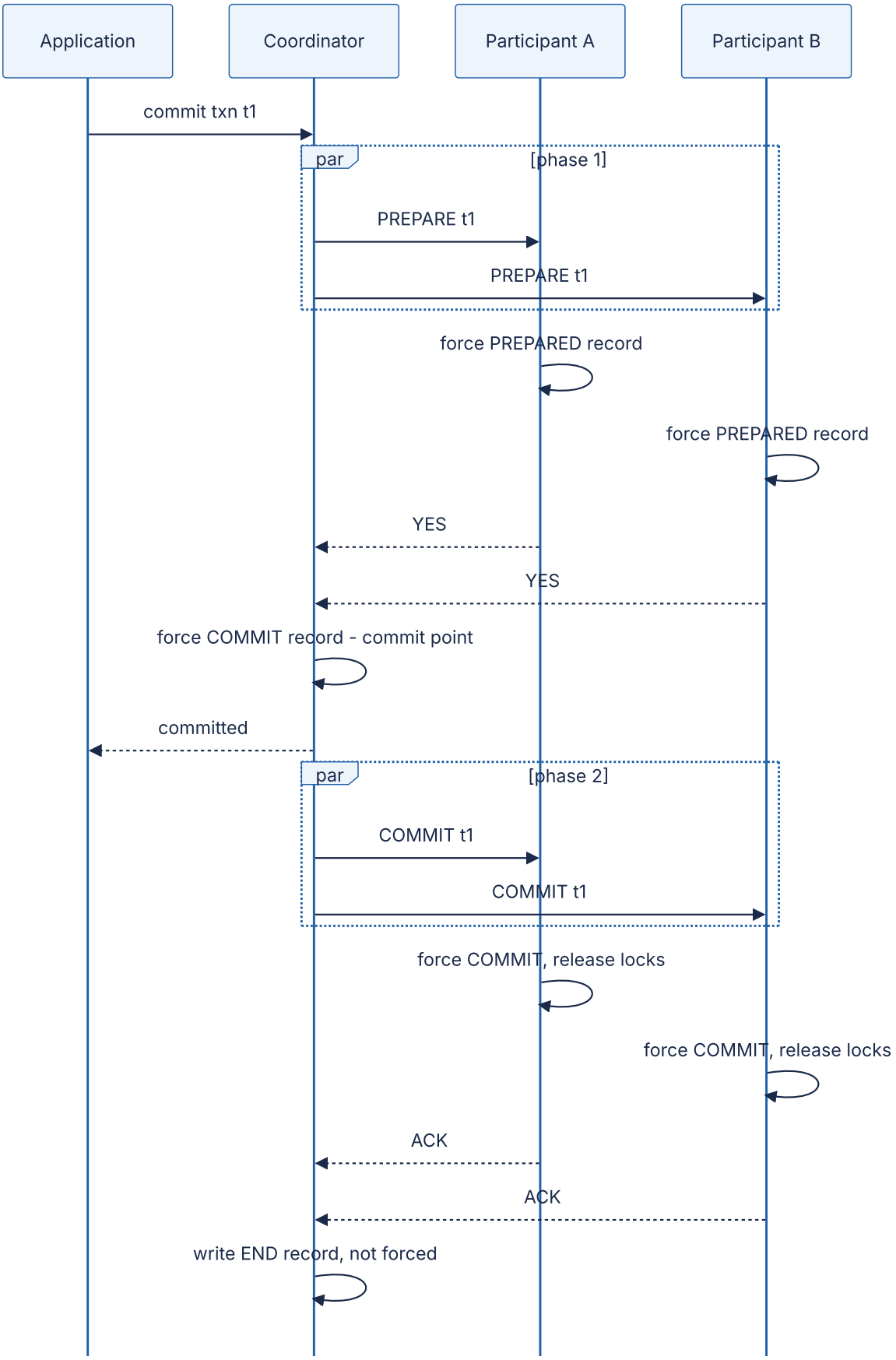


Figure 28.3 — Coordinator and participant state machines.

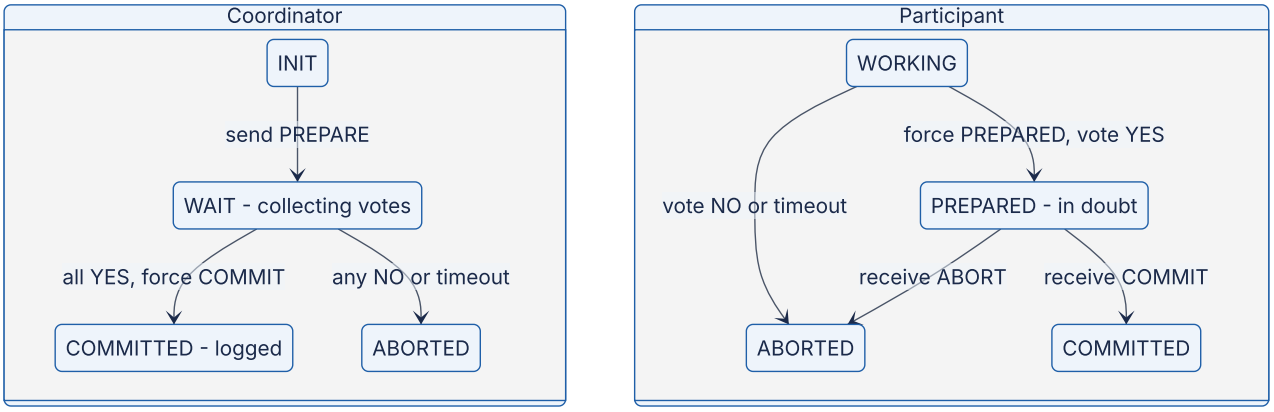


Figure 28.4 — Coordinator failure after the votes: participants block.

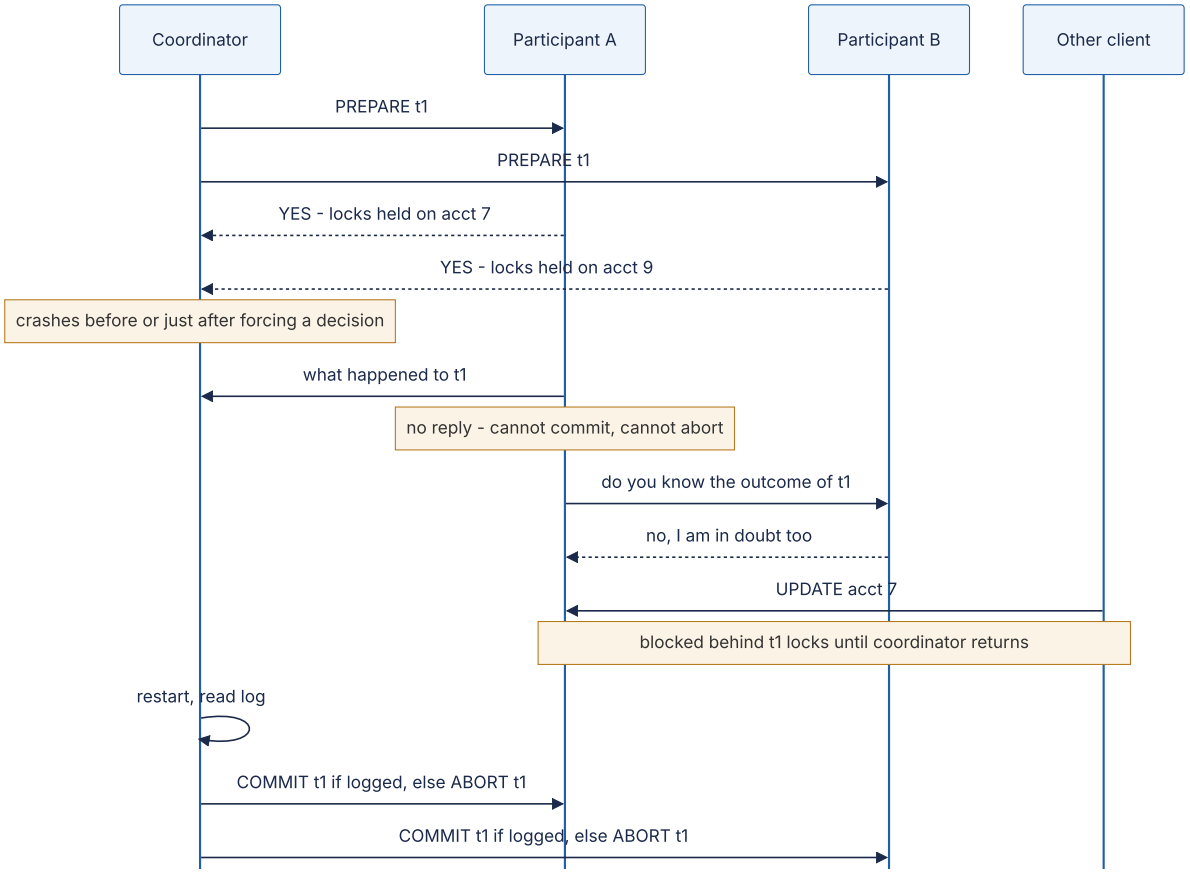


Figure 28.5 — The X/Open DTP model.

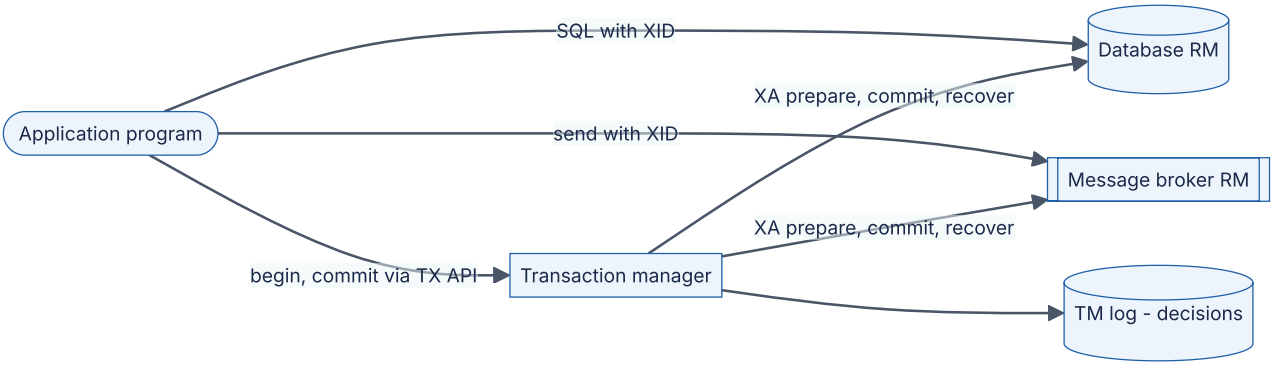


Figure 28.6 — Two-phase commit across consensus groups.

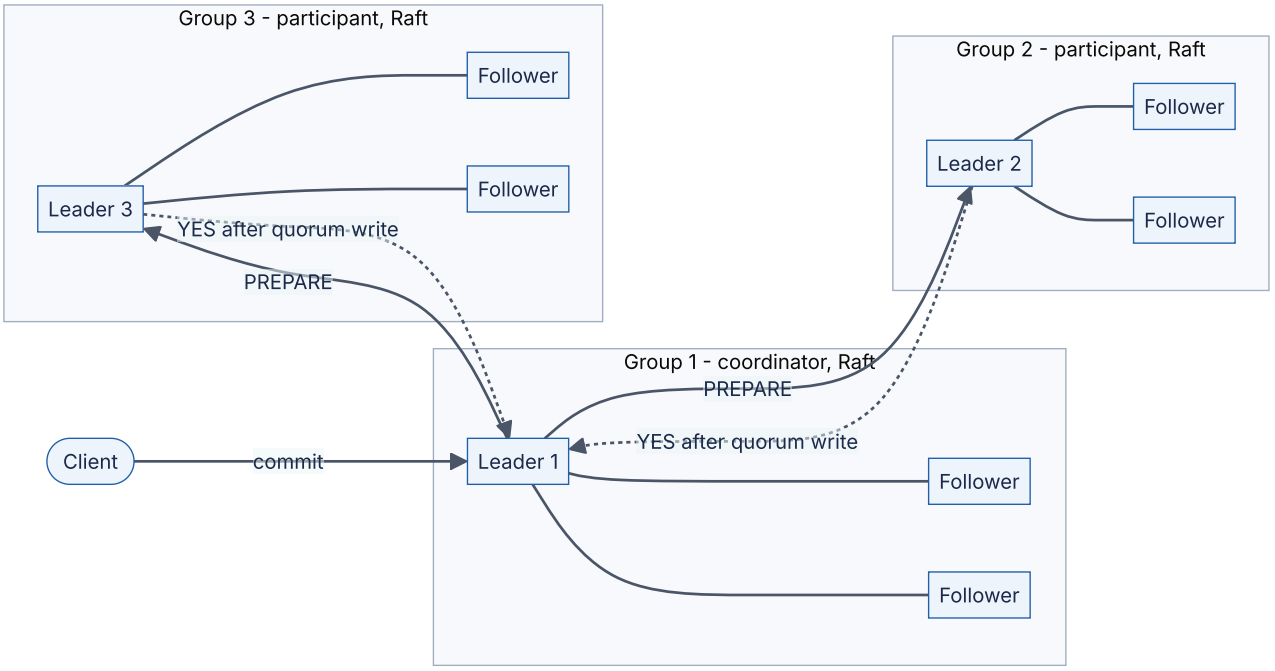


Figure 28.7 — Three-phase commit.

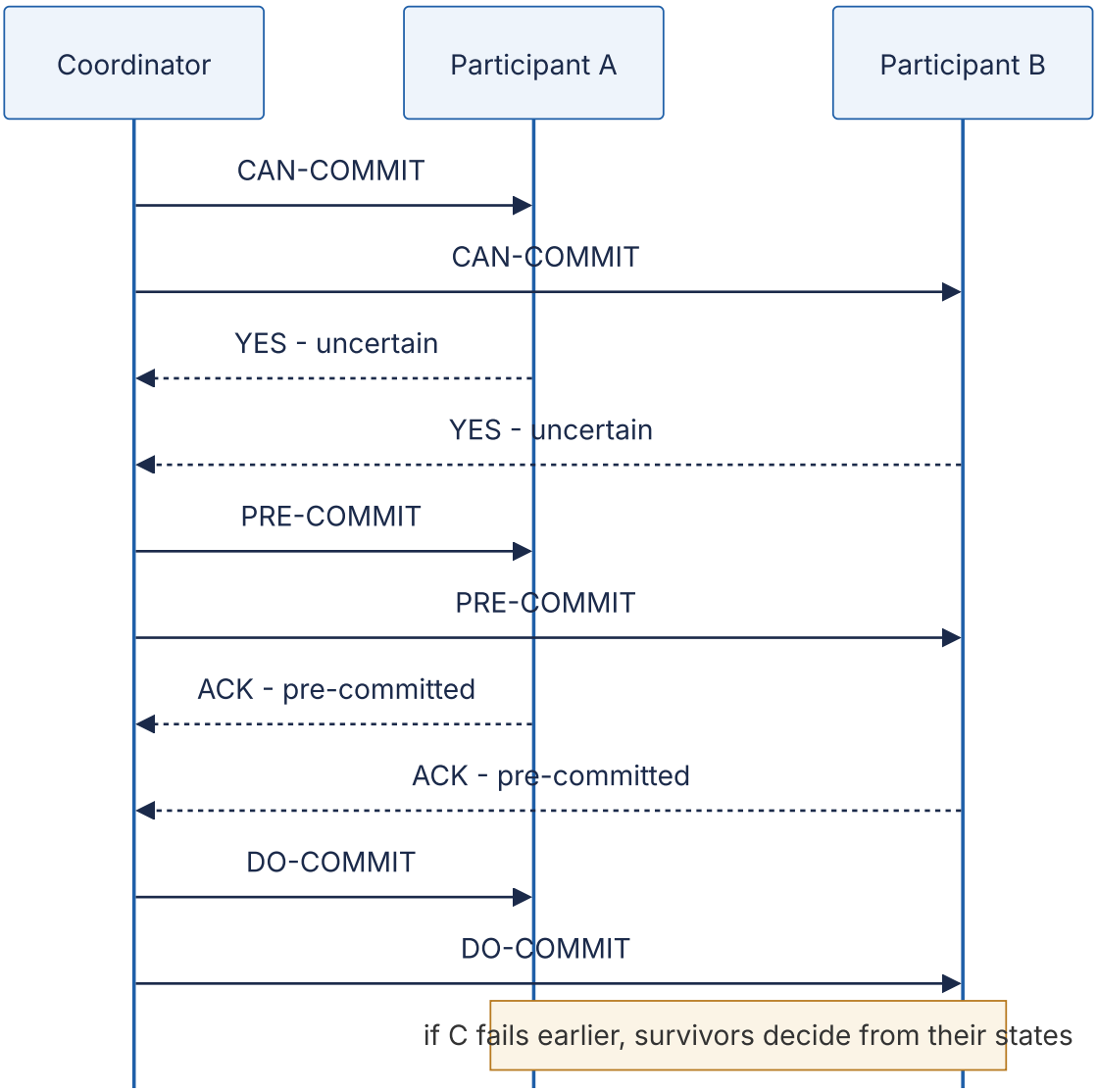


Figure 28.8 — A partition makes 3PC decide both ways.

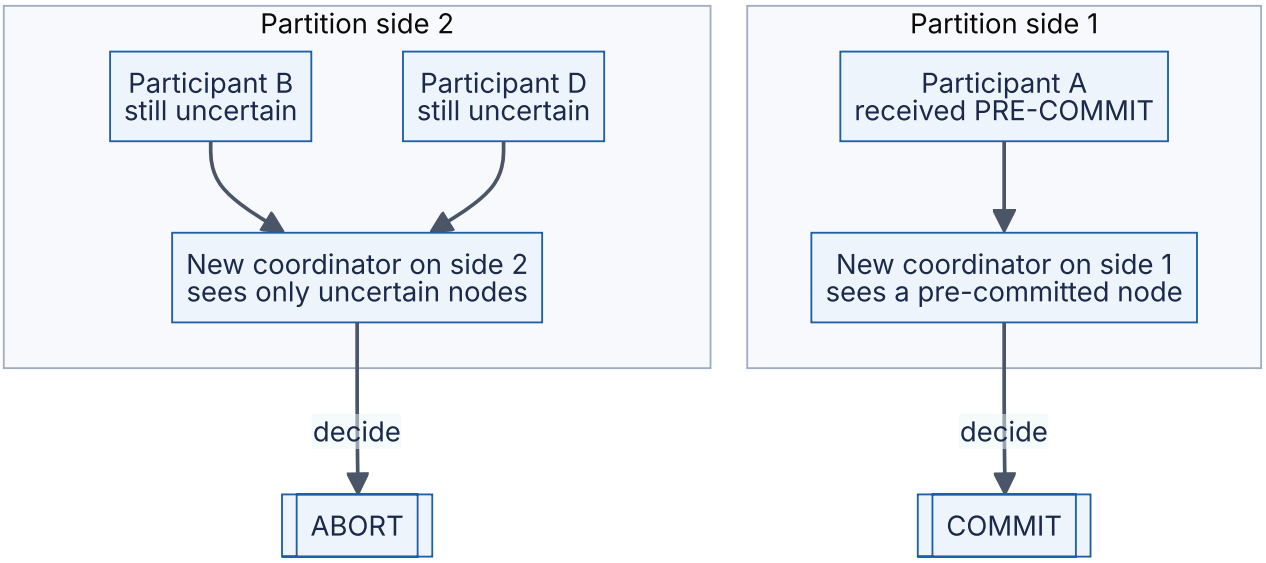


Figure 28.9 — A transfer saga with compensation.

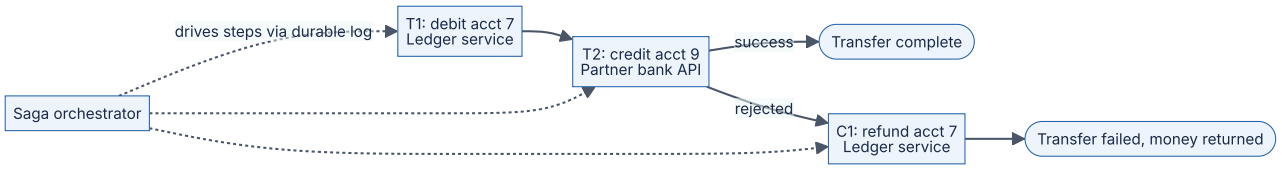


Figure 28.10 — A Percolator write of two cells.

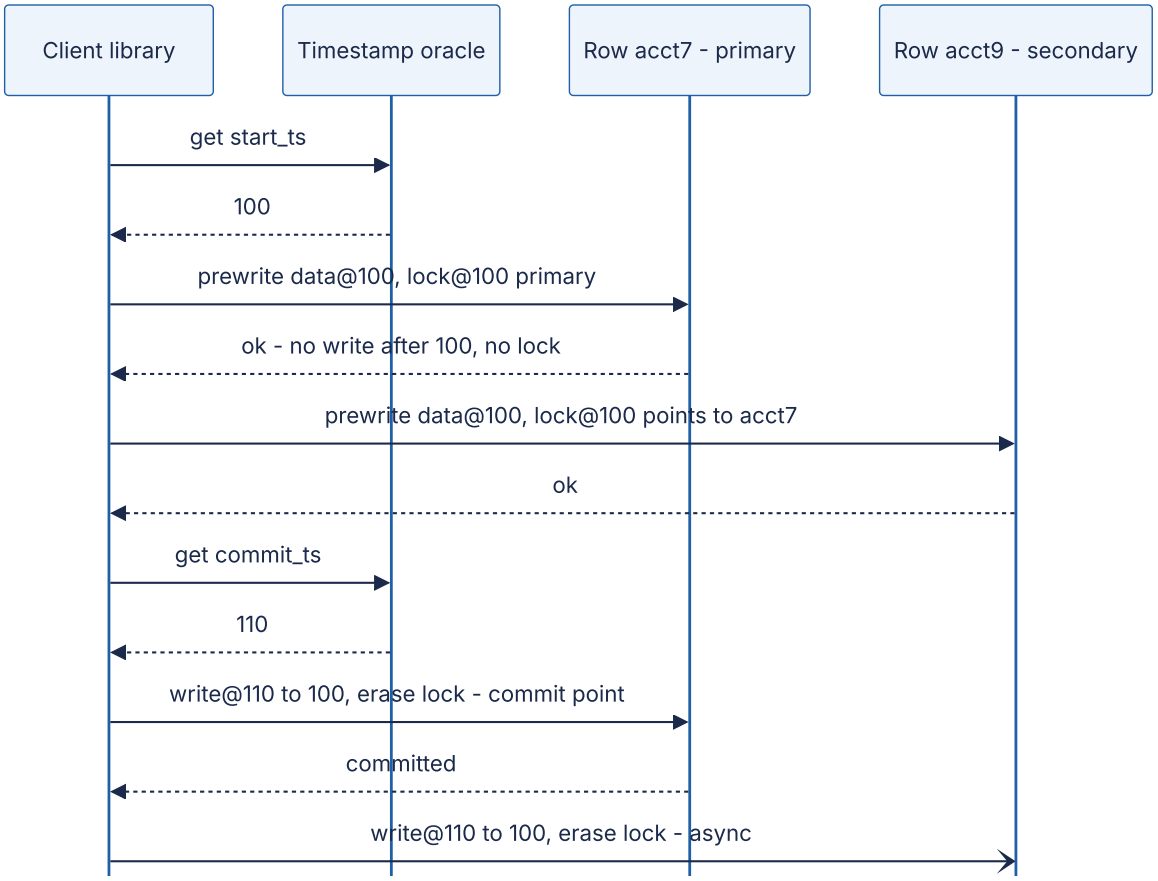


Figure 28.11 — Spanner: transactions across Paxos groups.

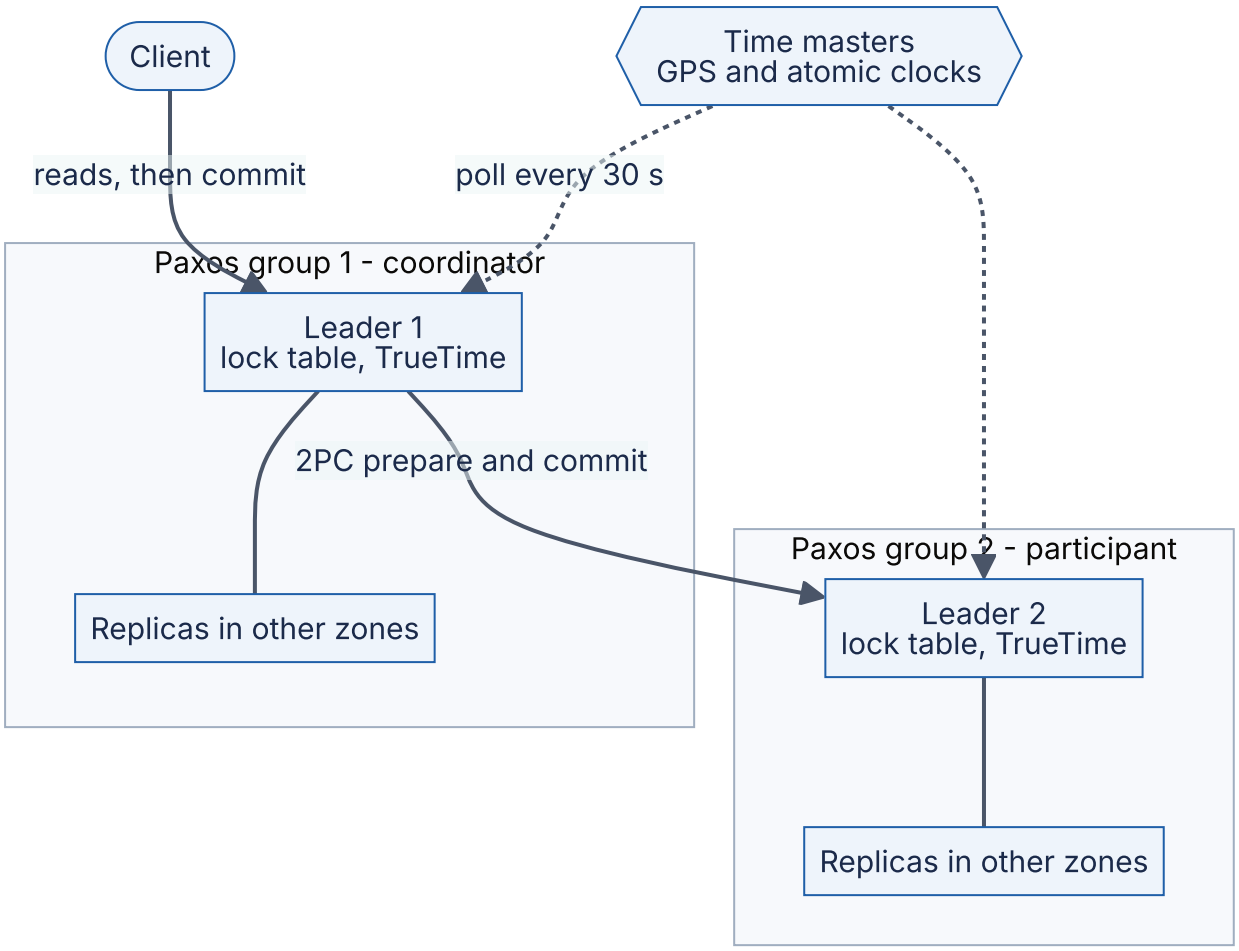


Figure 28.12 — Commit wait timeline at the coordinator leader.

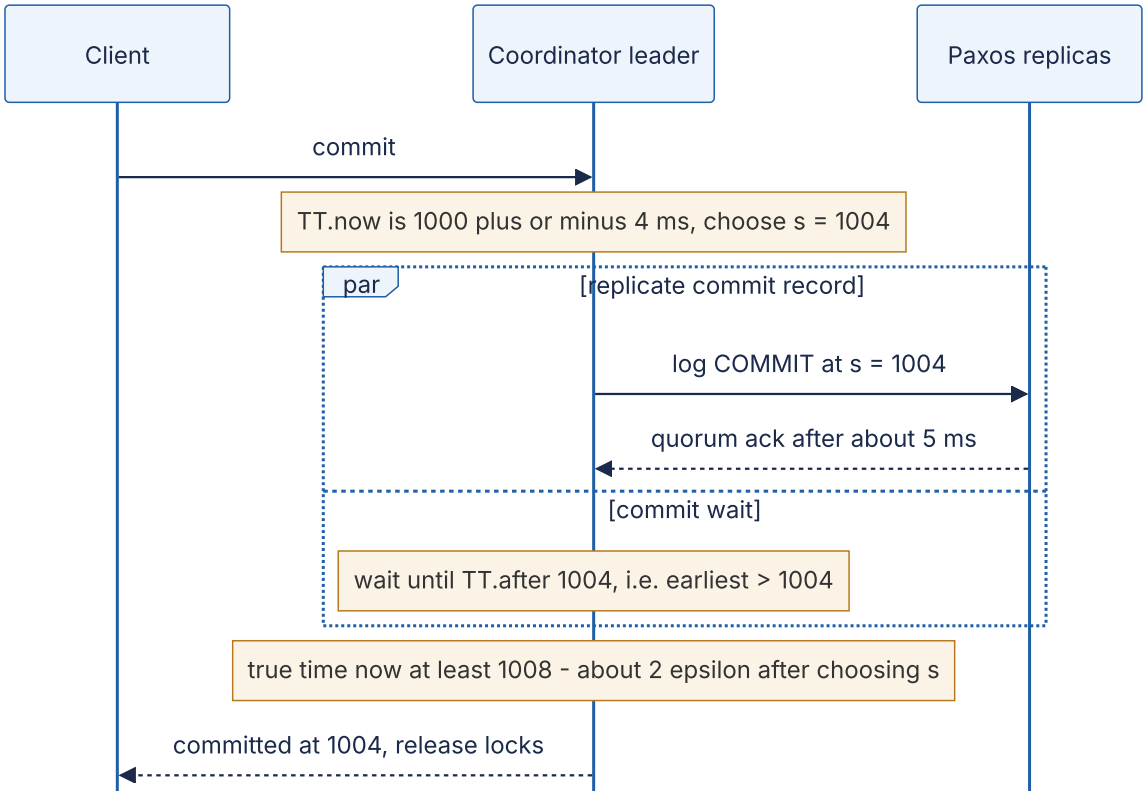


Figure 28.13 — Calvin’s architecture.

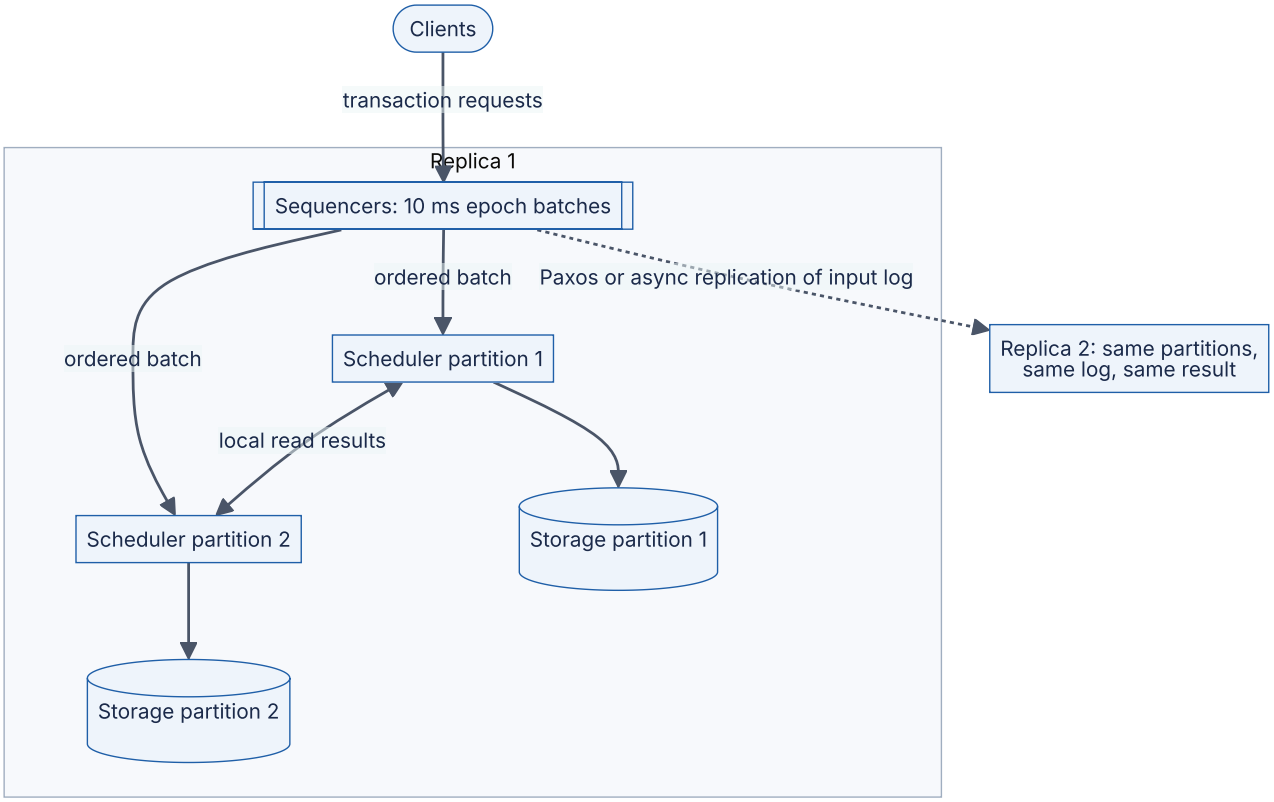


Figure 28.14 — CockroachDB parallel commits.

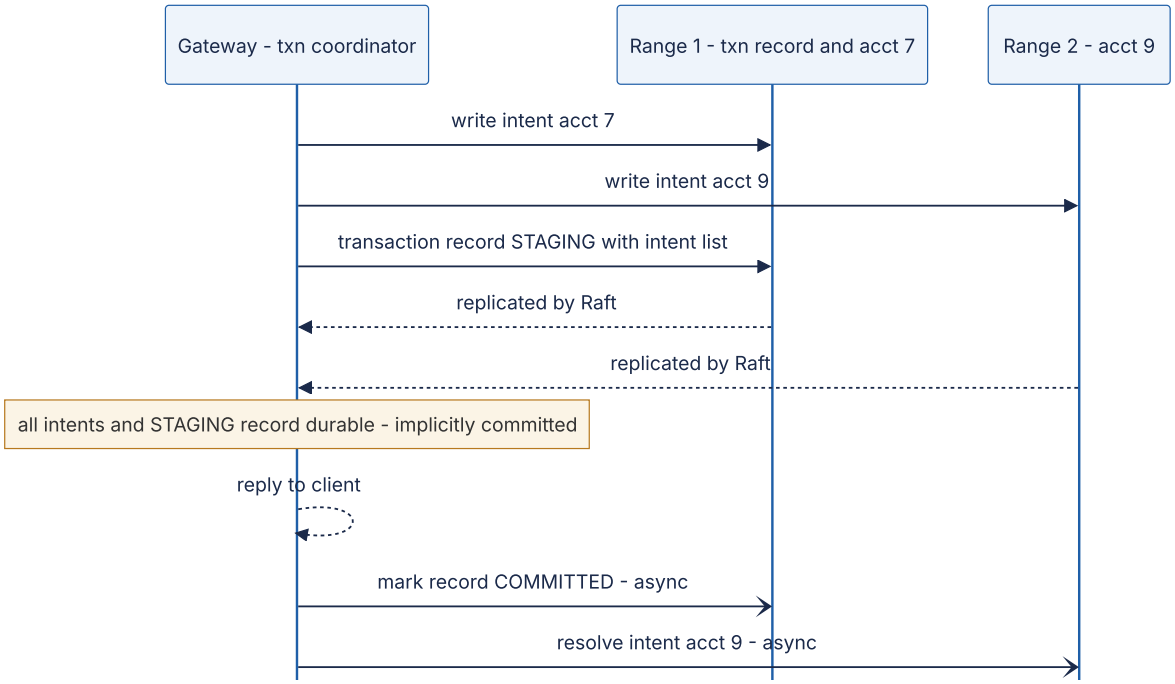
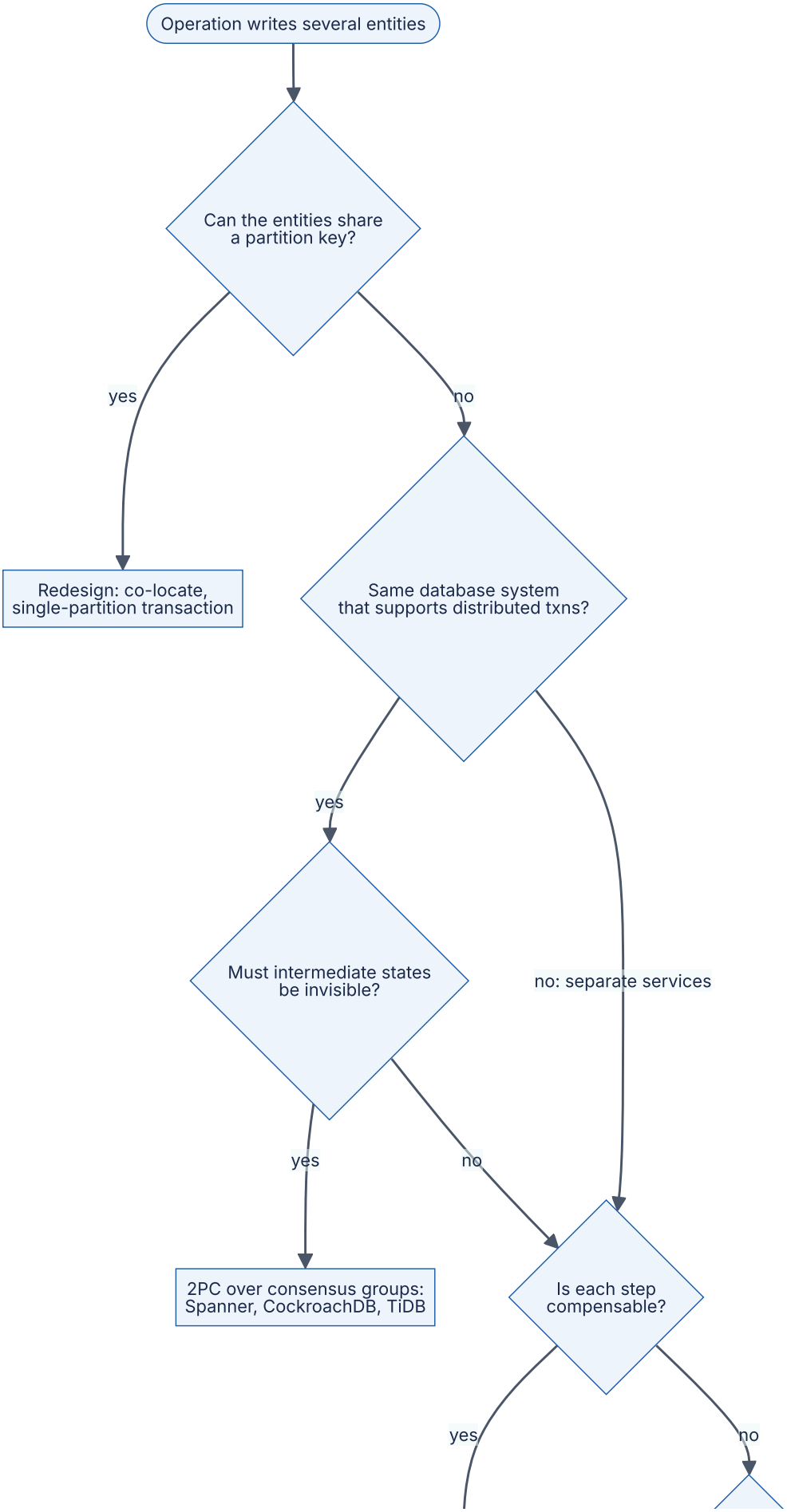
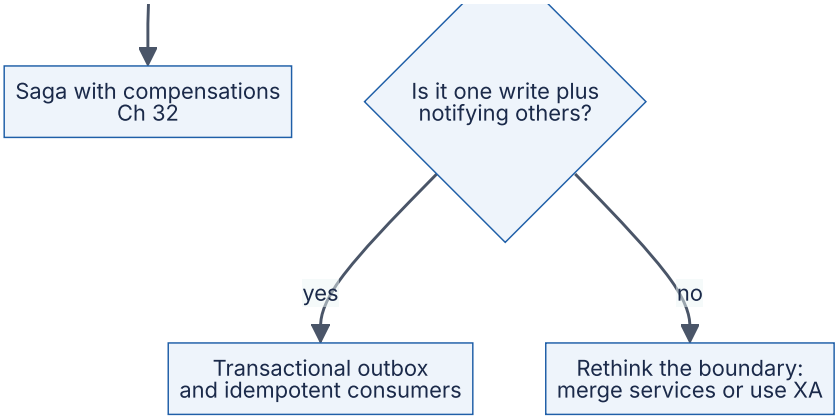


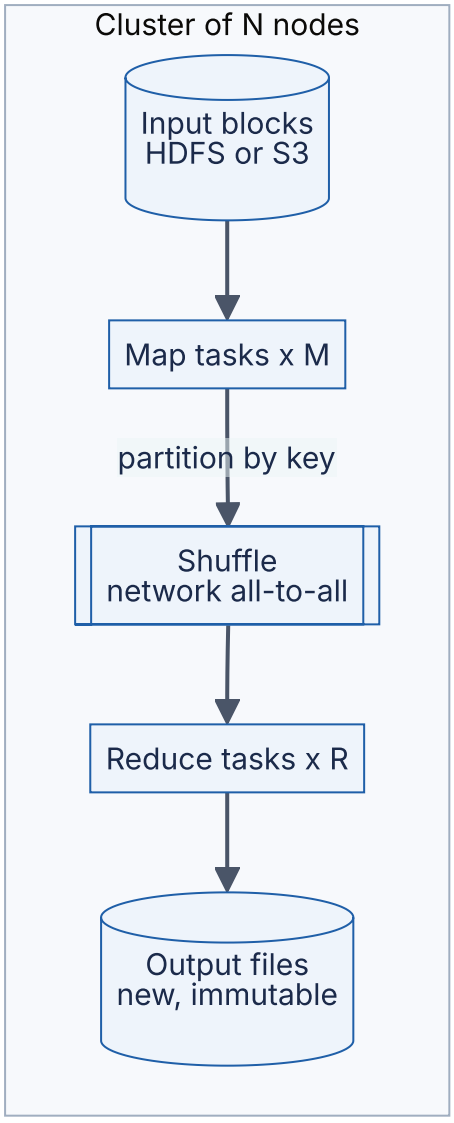
Figure 28.15 — Choosing a transaction strategy.

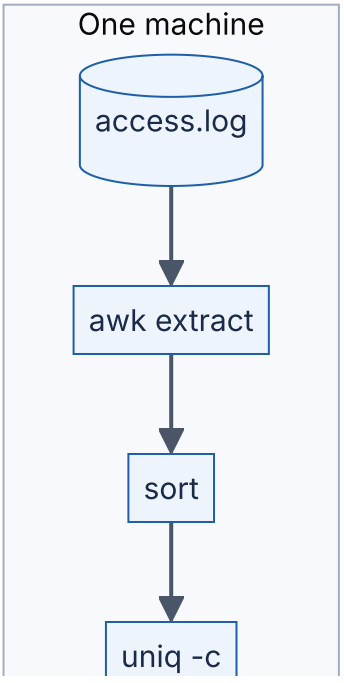

Operation writes several entities
Can the entities share a partition key?
yes
no
Redesign: co-locate, single-partition transaction
Same database system that supports distributed txns?
yes
no: separate services
Must intermediate states be invisible?
yes
no
2PC over consensus groups: Spanner, CockroachDB, TiDB
Is each step compensable?
yes
no



Chapter 29 — Distributed Data Processing

Figure 29.1 — A Unix pipeline and the distributed batch job that generalizes it.


Cluster of N nodes
Input blocks
HDFS or S3
Map tasks x M
partition by key
Shuffle
network all-to-all
Reduce tasks x R
Output files
new, immutable


One machine
access.log
awk extract
sort
uniq -c

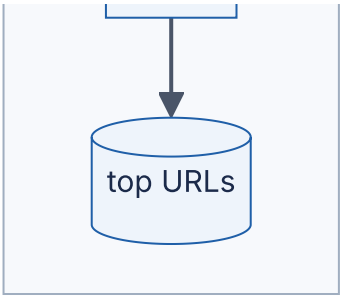


Figure 29.2 — MapReduce job execution.

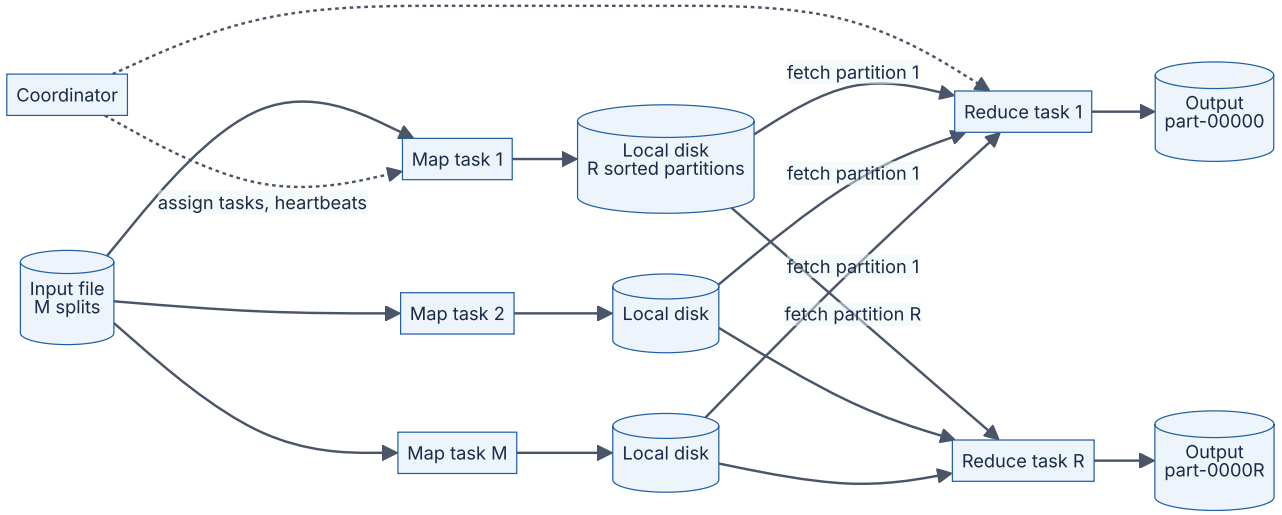


Figure 29.3 — Inside a map task and a reduce task.

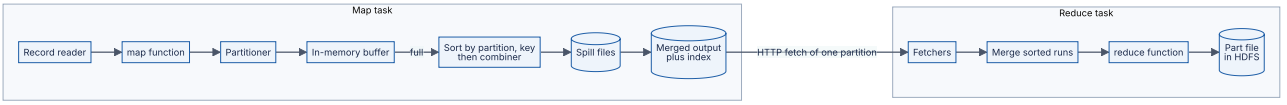


Figure 29.4 — Lifecycle of a task under re-execution and speculation.

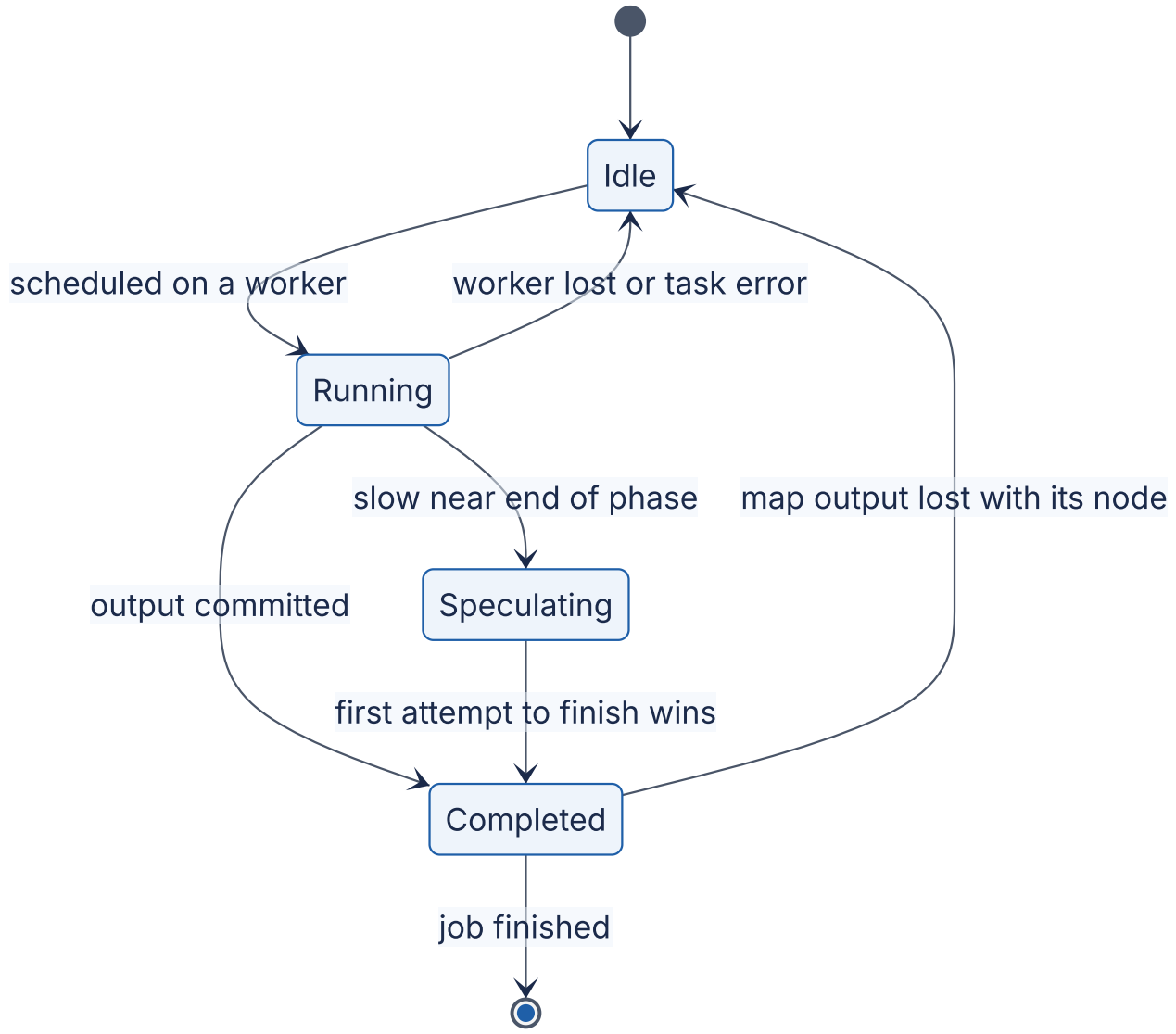


Figure 29.5 — Reduce-side join versus broadcast join.

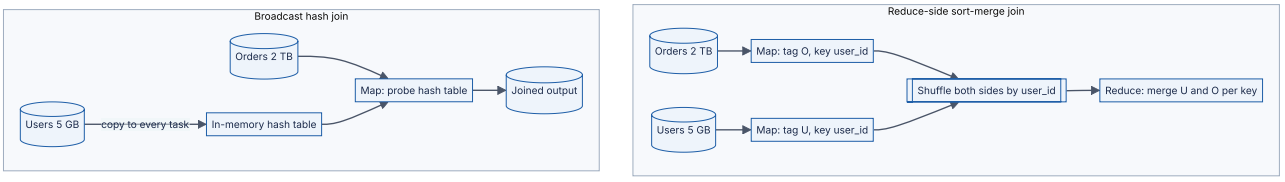


Figure 29.6 — Spark runtime: driver, cluster manager and executors.

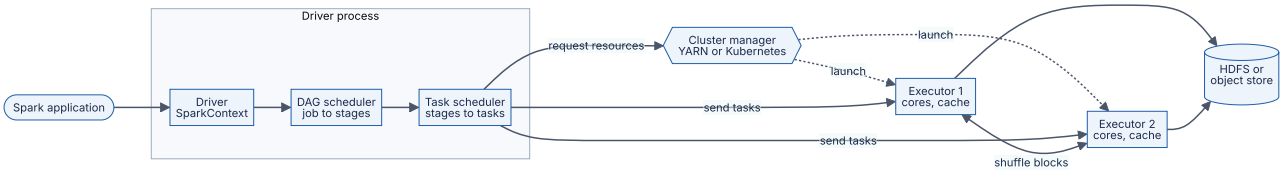


Figure 29.7 — Lineage of a word-count job cut into stages at the wide dependency.

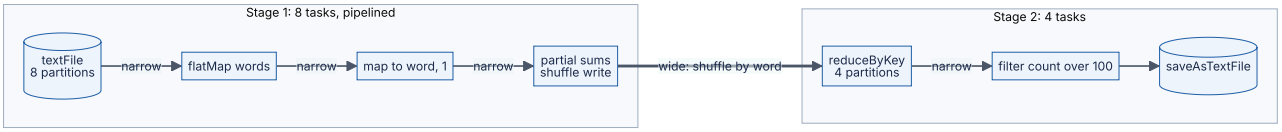


Figure 29.8 — Catalyst query planning with Adaptive Query Execution.

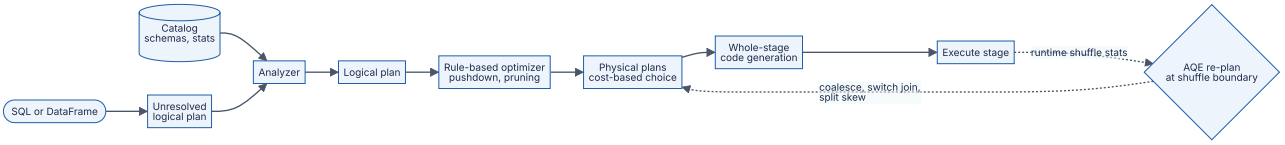


Figure 29.9 — A shuffle crossing rack uplinks.

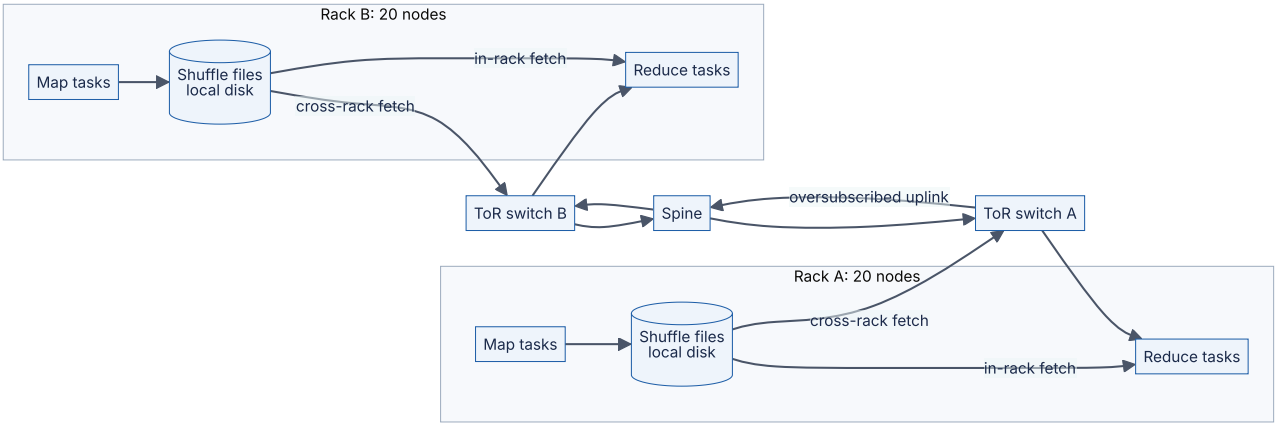


Figure 29.10 — Two-stage aggregation with a salted hot key.

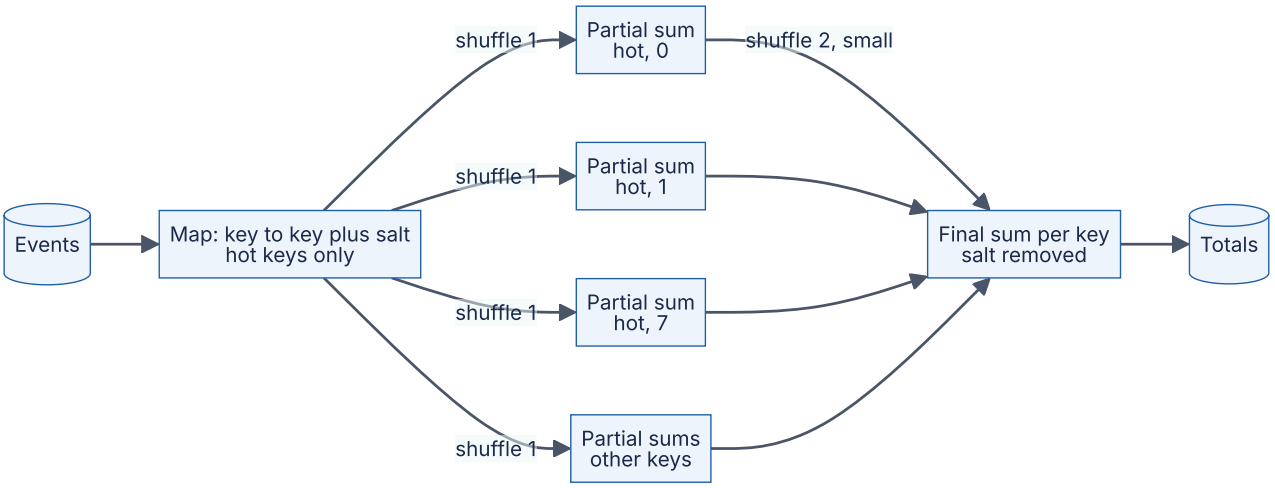


Figure 29.11 — Tumbling, sliding and session windows over the same events.

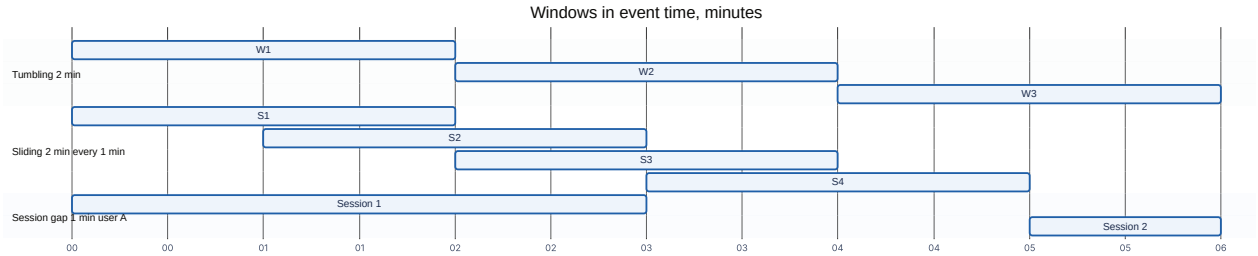


Figure 29.12 — A 10 s window with a watermark, a late update and a dropped event.

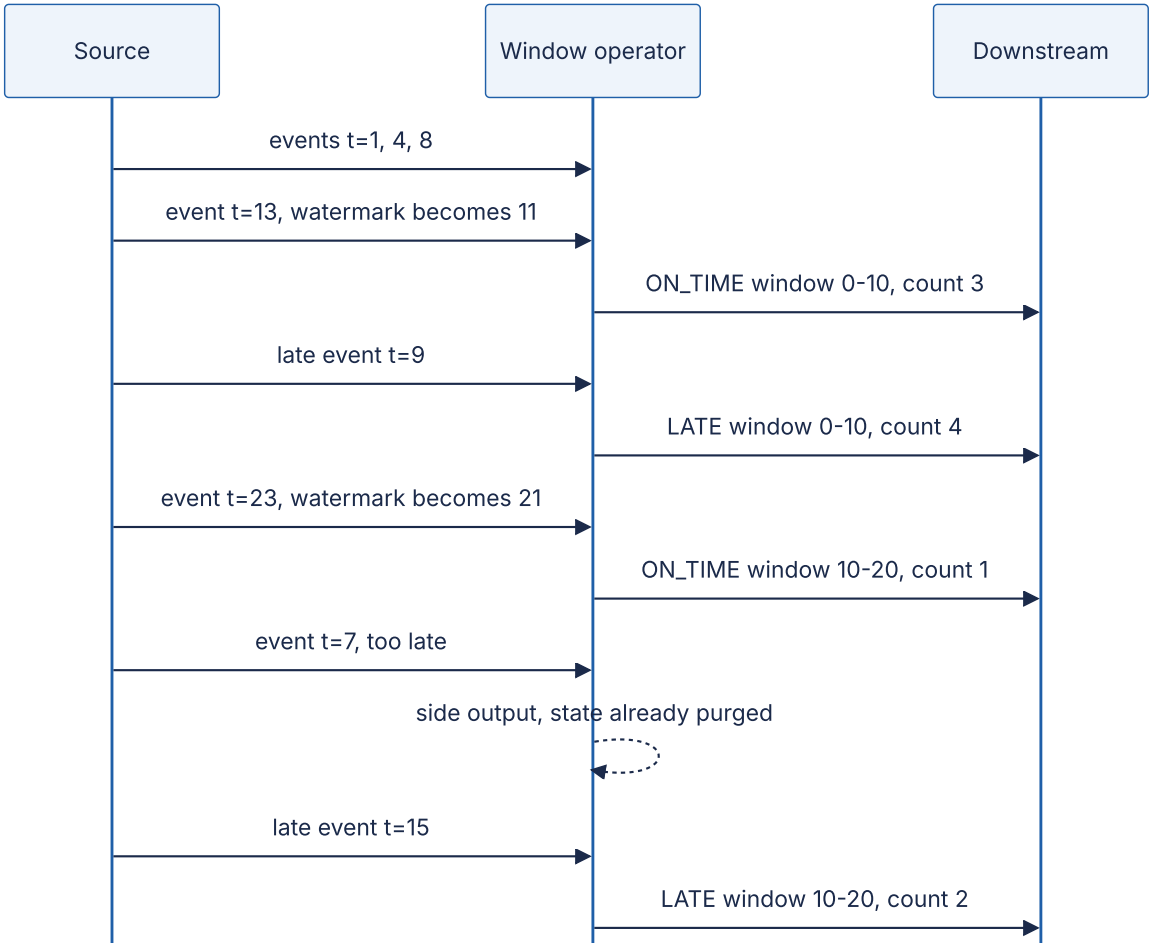


Figure 29.13 — Flink cluster architecture.

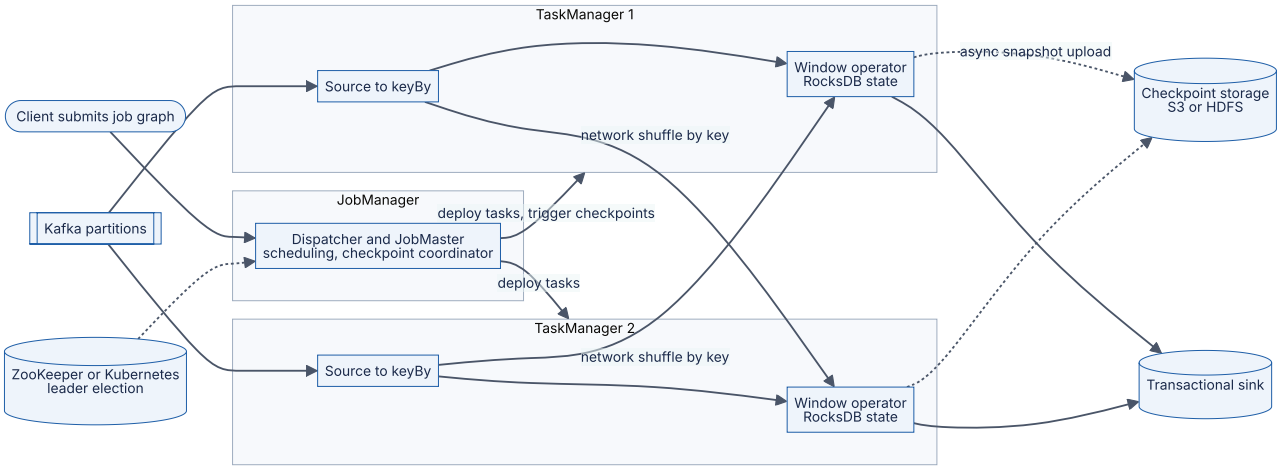


Figure 29.14 — Barrier alignment in a two-input operator.

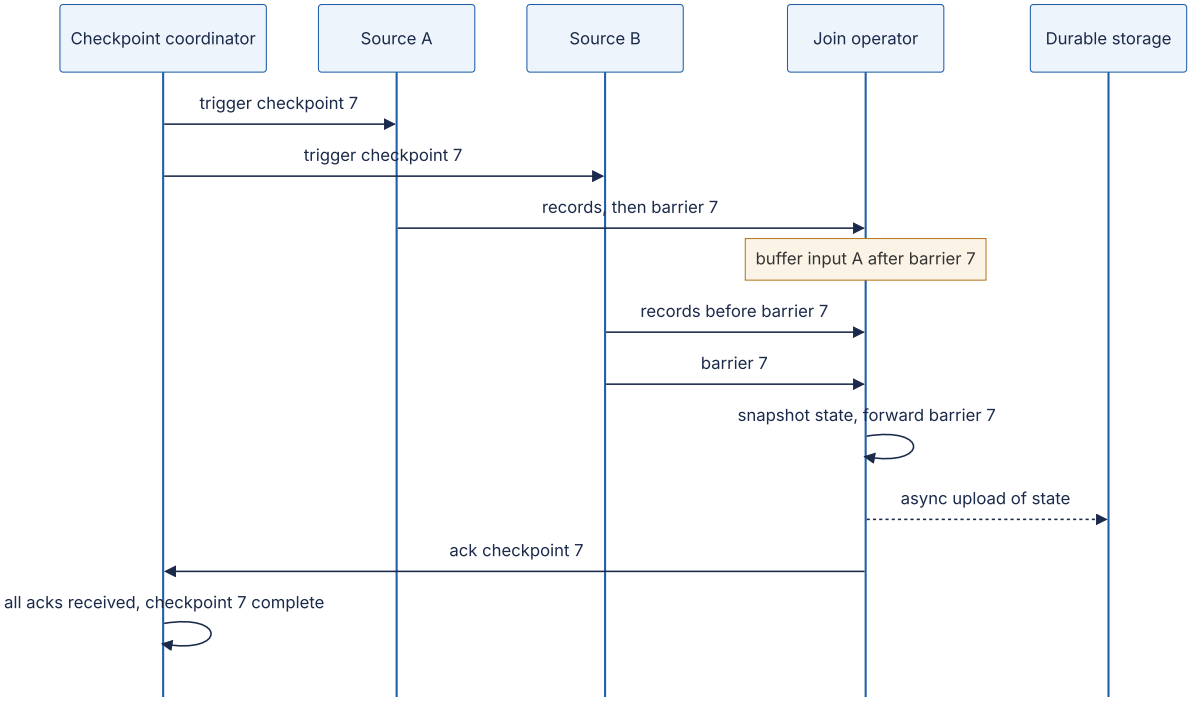
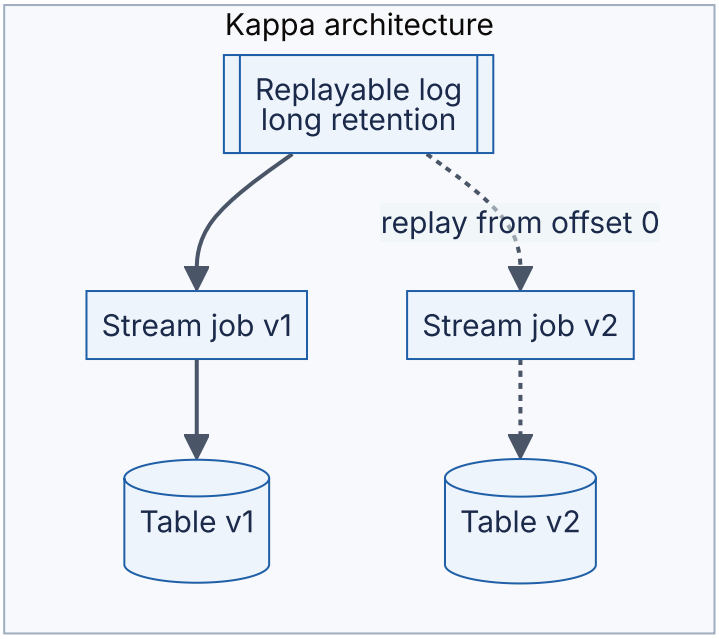
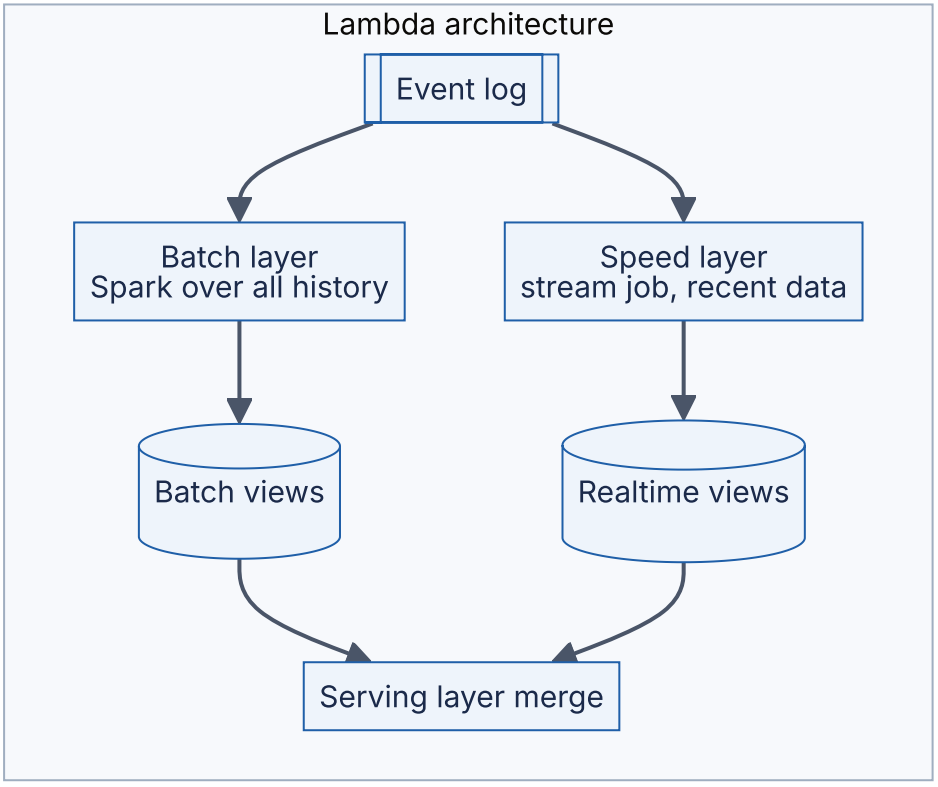


Figure 29.15 — Lambda versus Kappa.


Kappa architecture
Replayable log
long retention
replay from offset 0
Stream job v1
Stream job v2
Table v1
Table v2


Lambda architecture
Event log
Batch layer
Spark over all history
Speed layer
stream job, recent data
Batch views
Realtime views
Serving layer merge